

인공지능과 소프트웨어 테스트

신뢰할 수 있는 시스템 구축

Adam Leon Smith · Rex Black · James Harold Davenport
Jeremias Rößler · Joanna Isabelle Olszewska · Jonathon Wright

원문 의미 보존 한국어 번역본

번역 기준

원문의 의미와 구조를 가능한 한 그대로 유지하여 번역했으며, 제목과 기술 용어를 임의로 요약하거나 새로운 의미로 재구성하지 않았다. 참고문헌의 저자명·문헌 제목·학술지/학회명·URL은 인용 정확성을 위해 원문 표기를 유지했다. 색인의 페이지 번호는 원서 기준이다.

원문 PDF 2 페이지

독자에게 새로운 AI 개념과 과제를 소개하는 데 초점을 둔 훌륭한 참고서이다. AI와 소프트웨어 테스트에서 위험한 점은 사람들이 모든 것을 이해하고 있다고 잘못 믿는 데 있다. 이 책은 유스케이스, 모델 및 참고자료를 활용하여 풍부한 참고자료와 유용한 개념을 독자에게 제공함으로써 이 문제를 다루며, 이 광범위한 주제에 대한 독자 자신의 지식을 자극하는 동시에 도전하게 한다. 적극 추천한다.

Paul Mowat MBCS CITP, BCS SIGiST, 소셜 미디어 서기 및 위원회 위원, Deloitte UK 품질 테스트 엔지니어 디렉터

AI 기반 시스템은 우리 일상생활의 점점 더 많은 영역을 차지하고 있다. 사람들은 이러한 시스템을 신뢰할 수 있는지 우려한다. 『인공지능과 소프트웨어 테스트』은 이 문제를 다루며, AI 품질과 그것이 기존 소프트웨어 품질과 어떻게 다른지, 그리고 머신러닝 시스템을 테스트할 때 어려움과 과제가 어디에 있는지에 대한 통찰을 제공한다. 이 주제에 대한 훌륭한 입문서이며, 신뢰할 수 있는 AI 기반 시스템 구축에 관심이 있는 모든 사람이 반드시 읽어야 할 책이다.

Klaudia Dussa-Zieger, GTB 의장 및 ISTQB® 부회장, ISTQB® Certified Tester AI Testing(CT-AI) 태스크포스 책임자

끊임없이 확장되고 진화하는 가상 영역에서 AI의 중요성은 점점 더 커지고 있으며, 이러한 진화에 과제가 따르지 않을 수는 없다. 이 책은 메타버스의 출현과 함께 가상, 인지 및 물리 공간이 더욱 상호 연결됨에 따라 이 진화하는 분야에서 직면할 수 있는 잠재적 딜레마에 관한 훌륭한 자료를 제공한다. AI의 성장, 개발 및 테스트에서 인간이 담당하는 역할도 논의한다. 저자들의 풍부한 경험, 연구 및 근거를 바탕으로 이 책은 AI와 소프트웨어 테스트에 대한 훌륭한 소개와 이해를 제공한다. 이 분야에 관심이 있는 모든 사람에게 적극 추천한다.

Jonathan Miles MBA BSc(Hons) FCMI, Mimecast 전략 인텔리전스 책임자

Shift Right! ‘테스팅의 7가지 원칙’에서는 찾아볼 수 없는 개념이다. 『인공지능과 소프트웨어 테스트』은 이러한 원칙을 관점에 맞게 바라보게 해준다. 이 책은 조기 테스트를 살펴볼 뿐만 아니라 완전 테스트(exhaustive testing)의 개념도 철저하고 효과적으로 살펴본다. 소프트웨어 테스트 강사로서 나는 이 책이 제공하는 모든 내용을 분명히 활용할 것이다.

차세대 테스터들이 머신러닝의 복잡한 세부사항에 의문을 제기하도록 이끌어 줄 것이다. 소프트웨어 테스터뿐만 아니라 기술 분야에 있는 누구에게나 필독서이다.

Rachel Hurley MBCS TAP.dip, 기술 강사(소프트웨어 테스트)

제목이 설명하듯이, 이 책은 AI 및 ML 테스트를 탄탄하게 탐구하면서 AI 시스템의 신뢰성과 편향이 서로 대비되는 문제도 깊이 다룬다. 온톨로지의 기초와 AI 기반 시스템의 테스트 및 모니터링이 상당한 영향을 미치도록 하는 방법을 다룬다. 이 책을 읽고 나면 중요한 과제 하나에 답할 수 있게 될 것이다. 즉, 자신의 AI 시스템이 광범위하게 테스트되었는지를 어떻게 판단할 것인가 하는 문제이다.

Dina Dede, AI/ML 및 클라우드 아키텍트 리드, 영국

이 책은 판도를 바꾸는 인공지능(AI)의 복잡성과 전통적인 소프트웨어 품질 관리 분야를 훌륭하게 담아낸다. 적절한 속도감과 뚜렷한 미래-현재의 맥락 속에서 두 분야의 난제와 매력적인 가능성을 다루는 종합적인 안내서이다. 워터폴과 DevOps는 잊어라. 우리는 OpsDev, AIOps, 그리고 메타버스의 디지털 트윈으로 오른쪽으로 이동하고 있으며, 이제 상황은 훨씬 더 흥미로워질 것이다. 훌륭한 노력이며, 진정한 전문가들이 이 주제를 매우 필요하게 다룬 책이다.

Jude Umeh FBCS CITP, Salesforce 수석 프로그램 아키텍트

『인공지능과 소프트웨어 테스트』은 훌륭한 읽을거리이다. 저자들의 방대한 경험은 AI를 테스트에 적용하는 것뿐만 아니라 AI 소프트웨어 자체를 테스트하는 것의 과제와 이점을 포괄적으로 설명하는 데서 분명하게 드러난다. 나는 시프트 라이트 접근법과 테스트 및 추적 애플리케이션 개발 중 그 적용에 관한 통찰이 매우 흥미로웠다. 테스트에 대한 ROI를 빠르게 높이려고 하는 모든 테스트/QA 전문가뿐만 아니라 모든 최고경영진이 반드시 읽어야 할 책이다.

Anil Pande, TestPro Consulting Ltd 매니징 파트너

이 책은 소프트웨어 테스트에서 AI를 사용하는 방법과 AI 시스템을 테스트하는 방법 모두에 대한 매우 좋은 입문서이며, 사회적 위험, 편향, 윤리적 행동, 품질, 신뢰성, 그리고 AI/ML 시스템과 관련된 문제 등 여러 관련 주제를 다룬다. 나는 특히 AI/ML 시스템과 관련된 문제를 자세히 설명한 점이 마음에 들었다. AI/ML 시스템과 관련하여 소프트웨어 테스트를 공부하기 시작하는 누구에게나 이 책을 추천한다.

Venkat Ramakrishnan, 소프트웨어 품질 리더 및 소프트웨어 테스트 기술 전문가

『인공지능과 소프트웨어 테스트』은 AI 모델이 우리의 일상생활 속으로 확대되어 가는 상황에서 이를 어떻게 테스트해야 하는지 궁금한 사람이라면 누구에게나 가치 있는 자료이다. 명확하고 유익하게 읽을 수 있는 책으로, 각 장에서 AI 소프트웨어와 관련된 서로 다른 테스트 과제와 이를 효과적으로 다루는 방법에 관한 조언을 논의한다. 테스터와 학생 모두에게 적극 추천할 수 있다.

Katy Hannath BSc(Hons), 인공지능 및 데이터 과학 MSc 학생, VISR Dynamics 품질 보증 테스터

이 책은 매우 실용적인 자료이자, AI 분야에서 경력을 쌓거나 테스트 접근법을 정의하려는 사람이라면 누구나 AI와 ML의 기초를 이해하는 데 활용할 수 있는 훌륭한 참고 안내서이다. AI, ML, 윤리, 온톨로지, 품질, 편향, 과제, 테스트 자동화, 그리고 ‘시프트 라이트’ 테스트의 중요성을 명확하고 직접적이며 간결하게 설명한다. 이 분야의 전문가이자 권위자인 저자들의 풍부한 경험을 한데 모아, 철저하고 데이터에 기반한 실제 사례를 제공한다.

Boby Jose BSc MBA MBCS, BCS 출판물 『Test Automation: A manager's guide』 저자

얼마나 흥미롭고 시의적절한 출판물인가! AI가 제공하는 판도를 바꾸는 긍정적인 사회적 이점에 더해, AI는 소프트웨어 테스트를 포함한 소프트웨어 공학의 모든 측면에도 동일하게 큰 변화를 일으키는 것으로 입증되었다. 이 책은 우리의 전통적인 사고를 보완할 새로운 구축 및 테스트 설계 기법에 대한 훌륭한 통찰을 제공한다. 오늘날의 복잡하고 흔히 예측하기 어려운 AI 시스템을 구축하고 테스트하는 중대한 문제에 어떻게 접근해야 하는지 이해하고자 하는 기술 리더와 테스트 전문가 모두에게 필수적인 안내서이다.

Jack Mortassagne, Cigniti Technologies 디렉터 및 TMMI 공인 심사원

AI가 소프트웨어 테스트 직업에 어떤 영향을 미칠지 더 깊이 이해하고자 하는 사람들에게 훌륭한 책이다. 저자들은 AI의 과제를 이해하기 쉬운 방식으로 소개하며, 제시된 사례 연구는 매우 흥미롭고 현대적이어서 제시된 주제를 명확하게 보여준다. 훌륭한 책이며 적극 추천한다!

Dr Diana Hintea BEng(Hons) PhD SFHEA, Coventry University 컴퓨팅·전자·수학 학부 부학부장

인공지능이 약속해 온 패러다임 전환이 현실 세계에 영향을 미치기 시작하는 시점에서, 이 책은 매우 중요하다.

원문 PDF 5 페이지

이 책은 AI를 둘러싼 사회적, 윤리적, 기술적 우려를 이해하기 쉬운 방식으로 설명하여 복잡한 주제에 쉽게 접근할 수 있게 한다. 비즈니스, 기술, 윤리 또는 품질의 관점에서든 IT에 관여하는 모든 사람은 AI의 영향을 받게 될 가능성이 높으므로, 이 책은 IT 분야의 모든 사람에게 매우 귀중한 자료가 될 것이다. 테스트 및 품질 전문가인 나에게 이 책은 새롭게 등장하는 AI 세계에서 테스트와 품질을 다루는 데 도움을 주는 실용적이고 현실적인 참고서로서 내 책장에서 가장 중요한 자리를 차지할 것이다.

Bryan Jones MBCS, Sopra Steria Private Sector 테스트 프랙티스 디렉터

공공의 신뢰를 얻어야 하는 AI 기술의 품질 보증을 책임지는 소프트웨어 테스트 분야의 사람이라면 누구나 반드시 읽어야 할 책이다. 익숙하면서도 도전적으로 느껴지는 주제들을 통해 저자들은 다양한 주제를 자신 있게 탐구하면서 AI와 테스트의 교차 영역에 대한 독자의 이해를 넓히고 깊게 한다.

Bronia Anderson-Kelly, Sabiduria Ltd IT 변화 컨설턴트

원문 PDF 6 페이지

인공지능과 소프트웨어 테스트

BCS, THE CHARTERED INSTITUTE FOR IT

BCS, The Chartered Institute for IT 는 IT 가 사회에 이롭게 기여하도록 하는 데 전념하고 있다. 우리는 네트워크의 힘을 활용하여 긍정적이고 실질적인 변화를 만들어낸다. 우리는 모두의 이익을 위해 전 세계 IT 전문직과 그 직업에 종사하는 개인들의 이익을 옹호한다.

IT 전문지식과 지식의 교류 연구소는 새로운 사고, 교육 및 지식 공유를 촉진하기 위해 산업계, 학계 및 비즈니스 분야의 전문가들 간의 연계를 장려한다.

실무자 지원 지속적인 전문성 개발과 권위 있는 일련의 IT 자격을 통해 연구소는 비즈니스의 요구에 부합하는 전문적 실무를 촉진하고자 한다. 또한 전 세계의 회원 및 자원봉사 커뮤니티에 실질적인 지원과 정보 서비스를 제공한다.

표준 및 프레임워크 수립 연구소는 정부, 산업계 및 관련 기관과 협력하여 우수한 업무 관행, 행동 강령, 역량 프레임워크 및 공통 표준을 수립한다. 또한 고용주가 모범 사례를 채택하도록 지원하기 위한 다양한 컨설팅 서비스도 제공한다.

회원 가입 학생, 교사, 전문가 및 실무자를 포함한 70,000 명 이상의 사람들이 BCS 회원 자격의 혜택을 누리고 있다. 여기에는 국제적인 커뮤니티에 대한 접근, 다양한 지역 및 전국 행사 초대, 경력 개발 도구, 그리고 분기별로 발행되는 사상 선도(thought-leadership) 잡지가 포함된다. 자세한 내용은 www.bcs.org/membership 을 방문하기 바란다.

추가 정보 BCS, The Chartered Institute for IT, 3 Newbridge Square, Swindon, SN1 1BY, United Kingdom.

전화 +44 (0) 1793 417 417 (월요일~금요일, 영국 시간 09:00~17:00) www.bcs.org/contact

<http://shop.bcs.org/>

인공지능과 소프트웨어 테스트 신뢰할 수 있는 시스템 구축

Adam Leon Smith, Rex Black, James Harold Davenport, Jeremias Rößler, Joanna Isabelle Olszewska, Jonathon Wright

편집: Adam Leon Smith

BCS The Chartered Institute for IT

© BCS Learning and Development Ltd 2022

Adam Leon Smith, Rex Black, James Harold Davenport, Jeremias Rößler, Joanna Isabelle Olszewska 및 Jonathon Wright 는 1988 년 저작권, 디자인 및 특허법(Copyright, Designs and Patents Act 1988) 제 77 조 및 제 78 조에 따라 이 저작물의 저자로서 식별될 권리를 주장하였다.

모든 권리 보유. 1988 년 저작권, 디자인 및 특허법에서 허용하는 연구 또는 사적 학습, 비평 또는 리뷰 목적의 공정 이용을 제외하고, 출판사의 사전 서면 허가 없이는 이 출판물의 어떠한 부분도 어떠한 형태나 수단으로도 복제, 저장 또는 전송할 수 없다. 단, 복사 방식으로 재생산하는 경우에는 Copyright Licensing Agency 가 발행한 라이선스 조건에 따라야 한다. 이러한 조건의 범위를 벗어나 자료를 복제하기 위한 허가 문의는 출판사에 해야 한다.

이 출판물에서 인정된 모든 상표, 등록 명칭 등은 각각의 소유자에게 귀속된다. BCS 및 BCS 로고는 자선단체 번호 292786 인 British Computer Society(BCS)의 등록 상표이다.

영국 표준(British Standards)의 발췌문을 복제할 수 있는 허가는 BSI 가 부여한다. 영국 표준은 BSI 온라인 상점 (www.bsigroup.com/Shop)에서 PDF 또는 인쇄본 형식으로 구할 수 있으며, 인쇄본만 필요한 경우 BSI 고객 서비스(Tel: +44 (0)20 8996 9001, Email: cservices@bsigroup.com)에 문의할 수 있다.

BCS, The Chartered Institute for IT 의 완전 자회사인 BCS Learning and Development Ltd, 3 Newbridge Square, Swindon, SN1 1BY, UK 에서 출판.

www.bcs.org

Paperback ISBN: 978-1-78017-5768 PDF ISBN: 978-1-78017-5775 ePUB ISBN: 978-1-78017-5782

영국 출판물 목록 데이터(British Cataloguing in Publication Data). 이 책의 CIP 목록 기록은 British Library 에서 확인할 수 있다.

면책조항: 이 책에서 표현된 견해는 저자들의 견해이며, 명시적으로 그렇게 명시된 경우를 제외하고 연구소 또는 BCS Learning and Development Ltd 의 견해를 반드시 반영하는 것은 아니다. 저자 및 BCS Learning and Development Ltd 는 출판물 준비 과정에서 모든 주의를 기울였으나, 저자 또는 출판사인 BCS Learning and Development Ltd 는 이 출판물에 포함된 정보의 정확성이나 완전성에 대해 어떠한 보증도 하지 않으며, 그러한 정보나 이 출판물에 포함된 지침 또는 조언, 또는 앞서 언급된 사항으로 인해 발생하는 어떠한 손실이나 손해에 대해서도 책임을 지지 않는다.

모든 URL 은 출판 당시 정확하였다.

출판사 감사의 글

원문 PDF 11 페이지

검토자: Andrew Lowe 출판인: Ian Borthwick 기획 편집자: Rebecca Youé 제작 관리자: Florence Leroy 프로젝트 관리자: Sunrise Setting Ltd 교정 편집자: Gillian Bourn 교정자: Barbara Eastman 색인 작성자:

Matthew Gale 표지 디자인: Alex Wright 표지 이미지: Istock/oonal 조판: Lapiz Digital Services, Chennai, India

원문 PDF 12 페이지

목차

그림 및 표 목록

저자 소개

약어

용어집

서문

1. 서론

AI 테스트의 과제

요약

2. AI 신뢰성과 품질

신뢰성

AI 품질 문제

AI 품질 측정을 위한 모델

AI 품질 규제

요약

3. 품질과 편향

편향 정의의 결과

일상생활 속의 편향

의도하지 않은 편향

심슨의 역설

요약

4. 머신러닝 시스템 테스트

원문 PDF 13 페이지

테스터의 역할

ML의 특성

테스팅 메트릭

테스팅 기법

AI 고유 특성 테스트

비결정적 시스템

투명성, 설명가능성 및 해석가능성

요약

5. AI 기반 테스트 자동화

품질 보증

테스팅과 테스트 자동화

단위 테스트 자동화에서의 AI

UI 수준 테스트 자동화에서의 AI

소프트웨어 품질 보증의 다른 작업에 AI 적용

테스팅을 위한 도구 지원 평가

AI에게 계속 어려울 가능성이 높은 작업

요약

6. 소프트웨어 테스트를 위한 온톨로지

온톨로지에 관하여

소프트웨어 테스트에 온톨로지 사용

온톨로지 주도 소프트웨어 테스트의 동향

요약

7. 디지털 트윈 테스트를 활용한 메타버스의 시프트 라이트

테스트의 시프트 라이트 접근법

인지 공학 원칙

시프트 라이트에서의 디지털 트윈 개념

사례 연구: 팬데믹 동안 지역사회가 안전을 유지하도록 돕기

사례 연구: 스마트 시티 데이터 교환 - 메타버스에서의 테스트

메타버스로 시프트 라이트하기

혁명보다 진화

요약

원문 PDF 14 페이지

참고문헌

색인

원문 PDF 15 페이지

그림 및 표 목록

그림 3.1 심슨의 역설에 대한 간단한 예시

그림 4.1 탐색적 데이터 분석과 모델 테스트 범위

그림 4.2 혼동 행렬

그림 5.1 테스트 피라미드의 개략적 예시

그림 6.1 소프트웨어 테스트를 위한 AI-T 온톨로지의 보기

그림 6.2 소프트웨어 테스트를 위한 차세대 온톨로지의 UML 유스케이스 다이어그램

그림 7.1 시프트 레프트와 시프트 라이트

그림 7.2 인지 공학

그림 7.3 인지적 사고: ‘디지털 선언문(The Digital Manifesto)’

그림 7.4 인지 학습 - ‘디지털 혁명보다 진화(Digital Evolution, Over Revolution)’

그림 7.5 COVID-19 패턴 및 추세

그림 7.6 MIT 프로젝트 Safe Paths App: 디지털 트윈 모델 보기

그림 7.7 MIT 프로젝트 Safe Places: 비식별화 도구(Redaction Tool)

그림 7.8 Google Takeout: 미국 매사추세츠주 보스턴 주변의 과거 데이터

그림 7.9 MIT 프로젝트 Safe Places: 비식별화 도구(Redaction Tool)

그림 7.10 코펜하겐 스마트 시티 데이터 교환

그림 7.11 TED 강연: 인지 학습 - ‘디지털 혁명보다 진화(Digital Evolution, over revolution)’

그림 7.12 Pokémon Go 를 활용한 모델 기반 테스트

그림 7.13 Agile Requirements Designer(ArD), 2016, 런던 지하철의 MBT

원문 PDF 16 페이지

표 3.1 이미지의 출신 국가

표 3.2 색상 및 청결도별 카드

표 3.3 색상별 카드

표 3.4 성별 및 치료 상태별 생존

표 3.5 가을학기 University of California 입학

표 4.1 MSE 의 예

표 4.2 활성화 함수로서 AND 연산자의 초기 예

표 4.3 활성화 함수로서 AND 연산자의 예

원문 PDF 17 페이지

저자 소개

ADAM LEON SMITH

Adam Leon Smith FBCS 는 AI 와 품질의 교차 영역을 전문으로 하는 유럽의 컨설팅, 교육 및 제품 기업

Dragonfly 의 최고기술책임자(CTO)이다. Adam 은 테스트, 테스트 자동화 및 AI 시스템과 관련된 테스트 문제에 대한 정기적인 교육 과정도 운영한다. 또한 그는 현재 영국 최대의 비영리 테스트 그룹인 BCS 소프트웨어 테스트 특별관심그룹(SIGiST)의 의장이다. Adam 은 다국적 금융 서비스 기관뿐 아니라 야심찬 스타트업에서도 고위직을 맡아 왔으며, 복잡한 시스템과 신기술에 검증 및 확인 기법을 적용하는 데 주력하고 있다. 그는 ISO/IEC 의 인공지능 표준화 커뮤니티에서 매우 활발하게 활동하며, AI 편향에 관한 ISO/IEC 프로젝트와 AI 를 포괄하도록 시스템 품질 모델을 확장하기 위한 표준을 이끌고 있다. Adam 은 BCS 펠로우이며, 신기술 사용을 감사하는 독립 감독 기관 ForHumanity 의 이사이다. 그는 컨퍼런스의 정기 연사이자 Ministry of Testing 과 품질, 테스트 및 AI 에 전념하

는 팟캐스트 Fuzzy Quality 의 상주 팟캐스터이다. Twitter(@adamleonsmith) 및 이메일 adam@disc0tech.com 으로 Adam 에게 연락할 수 있다.

REX BLACK

40 년의 소프트웨어 및 시스템 엔지니어링 경험을 보유한 Rex Black 은 컨설팅, 교육 및 전문가 서비스를 제공하는 소프트웨어 및 시스템 엔지니어링·테스팅 선도 기업 RBCS(www.rbc-us.com)의 사장이다. 1994 년 설립 이후 RBCS 는 6 개 대륙에서 1,000 개 이상의 고객에게 서비스를 제공해 왔으며, 임베디드 시스템에서 게임 소프트웨어, 은행 및 보험, 국방 시스템, 제약 및 의료에 이르기까지 소프트웨어 및 시스템 개발의 모든 분야를 포괄한다.

원문 PDF 18 페이지

Rex 는 소규모 스타트업부터 Fortune 20 글로벌 기업에 이르기까지 다양한 고객과 일해 왔다. 그는 Kanban, Scrum, DevOps, Waterfall 및 Spiral 을 포함한 매우 다양한 개발 수명주기에서 고객이 소프트웨어 및 시스템 엔지니어링과 테스트 모범 사례를 적용하도록 지원한 경험이 있다. Rex 는 감사관실(Office of Inspector General) 산하 소프트웨어 품질 감독 조직의 일원으로 전문가 서비스를 제공했으며, 1 억 8 천만 달러 규모의 계약 위반 소송을 포함한 여러 소송에서 전문가 증인으로 활동했다. 그는 또한 현재 소프트웨어 테스트 분야에서 활동하는 가장 다작의 저자 가운데 한 명으로, 지난 20 년 동안 14 권의 책과 수십 편의 기사를 집필했다. 그는 ISTQB 와 ASTQB 의 창립자이자 전 회장이며, TMMi America 의 창립자이자 회장이다. Twitter(@RBCS), 이메일 rex_black@rbc-us.com, LinkedIn(www.linkedin.com/in/rex-black), 또는 웹사이트(www.rbc-us.com)를 통해 Rex 에게 연락할 수 있다.

DR JAMES HAROLD DAVENPORT

James Davenport TD MMath MA PhD DSc (Hon.) FIMA CMath FBCS CITP NTF. University of Cambridge 에서 공부하고 IBM Research 에서 근무하며 Cambridge 와 Grenoble 에서 가르친 후, James Davenport 는 1983 년 University of Bath 에 합류했다. 처음에는 강사로, 이후에는 교수, 학과장, 정보기술 디렉터, 고성능 컴퓨팅 챔피언을 맡았으며, 가장 최근에는 Institute of Coding 의 테마 리드를 맡았다. 그는 또한 Cambridge, Sweden, China, Italy 및 여러 프랑스 대학과 Grandes Écoles 에서 가르쳤고, 독일 GMD, 캐나다 Western University 와 University of Waterloo, New York University 뿐만 아니라 IBM Research 와 Microsoft Research 에서 연구 안식년을 보냈다. 그의 주요 연구 관심 분야는 컴퓨터 대수와 암호학/보안이다. 그러나 두 분야 모두 많은 소프트웨어 개발자들의 낙관론만을 반영하기보다는 반례와 최악의 경우에 매우 큰 관심을 두기 때문에, 이러한 사고방식은 자연스럽게 AI 윤리, 더 정확히 말하면 AI 윤리가 결여될 가능성을 살펴보는 데 이어진다. BCS 의 전 부회장(Academy)이자 오랫동안 전문 인증 업무를 수행해 온 그는 컴퓨팅 교육에서 법적, 사회적, 윤리적 및 전문적 문제의 역할에 오랫동안 관심을 가져 왔다. 그는 AI 가 세부사항은 바꾸었지만 원칙은 바꾸지 않았다고 믿는다. James 에게는 masjhd@bath.ac.uk 로 연락할 수 있다.

원문 PDF 19 페이지

DR JOANNA ISABELLE OLSZEWSKA

Dr Joanna Isabelle Olszewska BSc(Hons) MSc(EPFL) PhD(UCL) CEng CSci FBCS FHEA SMIEEE 는 영국의 컴퓨터 과학자이다. 그녀는 영국 UWS 의 강사(조교수)이며 지능형 비전 시스템을 위한 알고리즘 및 소프트웨어 연구를 이끌고 있다. IEEE 시니어 멤버인 그녀는 IEEE 인공지능 표준위원회에서 활동하며, IEEE Global Initiative for Ethical Considerations in Artificial Intelligence and Autonomous Systems 의 일원이다. 그녀는 University of Edinburgh Union 에서 ‘AI 의 미래(The Future of AI)’에 관한 패널 연사로 초청되었다. ENS Paris 를 포함해 ICRA 같은 컨퍼런스, ACM Sacramento FWS 같은 심포지엄, Canadian Mathematical Society(CMS) 기념 회의 같은 행사에서 강연했으며, BBC Lunch Time ‘Women in Engineering’ 프로그램을 포함한 인터뷰에도 참여했다. 그녀는 IJCAI 를 비롯한 100 개 이상의 국제 컨퍼런스에서 TPC 위원으로 활동했고, IROS 등을 포함하여 70 개가 넘는 컨퍼런스/워크숍 세션의 의장을 맡았다. ISO/IEC/IEEE P41062 부의장을 포함한 다양한 역할로 11 개의 ISO/IEC/IEEE 표준에 기여했으며, 100 편 이상의 동료심사 논문과 3 개의 책 챕터를 저술했다. ESWA Outstanding Reviewer Award 를 포함한 여러 상을 수상했고 ACM Distinguished Speaker 를 비롯한 여러 명예를 보유하고 있으며, GCHQ Prize 를 수상한 최우수 BSc 논문과 AGD Prize 를 수상한 최우수 MSc 학위논문 지도를 포함하여 60 명의 BSc(Hons)/MSc/PhD 학생을 지도했다. Joanna 에게는 joanna.olszewska@ieee.org 로 연락할 수 있다.

DR JEREMIAS RÖßLER

Dr Jeremias Rößler 는 Saarland University 에서 컴퓨터 과학 박사학위를 받았으며 소프트웨어 개발자 및 테스터로 10 년 이상의 경험을 보유하고 있다. 그는 AI 를 테스트 자동화에 도입한 독일 기반 스타트업 @retest_en(<https://retest.de>)의 창립자이자 CEO 이다. 현재는 SAP 에서 근무하고 있다. 그의 참신하고 색다른 테스트 자동화 접근법(차이 테스트, 미국 및 유럽 특허 출원 중)은 기존 테스트 자동화에 비해 많은 장점이 있으며, 그는 이를 AI 와 결합하여 오라클 문제를 극복하는 방법을 보여준다. 차이 테스트는 웹용 오픈소스(<https://github.com/retest/recheck-web>)이며 ‘깨지지 않는(unbreakable)’ 테스트와 자동 치유(autohealing) 테스트를 가능하게 한다. 그는 A4Q ‘AI and Software Testing’

원문 PDF 20 페이지

인증 실라버스의 저자 중 한 명이다. 그는 학계와 산업계 모두에서 많은 국제 컨퍼런스의 연사로 활동했다. 참석자들은 그의 강연을 선견지명 있고 재미있다고 평가한다. 그는 작가, 블로거(<https://dev.to/roesslerj/>), 개발자 및 컴퓨터 과학자이다. 그는 결혼했으며 두 자녀가 있고 독일에 거주한다. 실제로 그를 만나게 된다면 그는 매우 친근하고 대화하는 것을 좋아하므로 주저하지 말고 말을 걸어도 된다. Twitter(@roesslerj) 또는 LinkedIn(<https://www.linkedin.com/in/roesslerj/>)을 통해 쉽게 연락할 수 있다.

JONATHON WRIGHT

Jonathon Wright 는 전략적 사상 리더이자 저명한 기술 에반젤리스트이다. 그는 신기술, 혁신 및 자동화를 전문으로 하며, 20 년 이상 글로벌 조직에서 국제적인 상업 경험을 갖춘 실무자로 활동해 왔다. 현재 그는 Silicon Valley 에 기반을 둔 Keysight Technologies 의 최고 기술 에반젤리스트(Chief Technology Evangelist)이다.

Jonathon은 광범위한 경험과 리더십을 인지 공학(Cognitive Engineering: Enterprise AI 및 AIOps)의 실제 도입에 대한 통찰과 결합한다. 이에 따라 TEDx, Gartner, IDC, Oracle, AI Summit, ITWeb, EuroSTAR, STAREast, STARWest, UKSTAR, Guild Conferences, Swiss Testing Days, Unicom, DevOps Summit, TestExpo 및 Vivit Community 같은 국제 컨퍼런스에서 연사 요청을 자주 받고 있다. 그는 현재 125 개국에서 80,000 명 이상의 회원을 보유한 가장 큰 독립 소프트웨어 커뮤니티인 Vivit Community 의 회장이기도 하다. 코로나바이러스 팬데믹 동안 그는 COVID Paths Check 재단을 위해 MIT 의 품질 엔지니어링 리드로 활동했다. 또한 Harvard Business Council 과 European Commission 의 AI Alliance 회원이며, BCS SIGiST 위원으로서 ISO-IEC 29119 Part 8 ‘Model-Based Testing’ 및 Part 11 ‘Testing of A.I. based systems’ 검토위원회에 참여하고 있다. Jonathon은 캐나다 기반의 QAlead.com 팟캐스트를 진행하며, 여러 수상 경력이 있는 책(2010~2021)과 온라인 과정의 저자이기도 하다. Jonathon에게는 jonathon.wright@keysight.com 으로 연락할 수 있다.

원문 PDF 21 페이지

약어

ACM	Association for Computing Machinery - 미국컴퓨터학회
AI	artificial intelligence - 인공지능
AI-T	artificial intelligence-based system testing ontology - 인공지능 기반 시스템 테스트 온톨로지
ANN	artificial neural network - 인공신경망
API	application programming interface - 애플리케이션 프로그래밍 인터페이스
APM	application performance management - 애플리케이션 성능 관리
AR/MR	augmented and mixed reality - 증강현실 및 혼합현실
BDD	behaviour-driven development - 행위 주도 개발
BLO	Basic Linux Ontology - 기본 Linux 온톨로지
BoK	body of knowledge - 지식 체계
CI/CD	continuous integration/continuous deployment - 지속적 통합/지속적 배포
CORA	core ontology for robotics and automation - 로봇틱스 및 자동화를 위한 핵심 온톨로지
CQ	competency question - 역량 질문
CRS	cloud robotics systems - 클라우드 로봇틱스 시스템
CSV	comma separated values - 쉼표로 구분된 값
DL	description logic - 기술 논리
DOLCE	descriptive ontology for linguistic and cognitive engineering - 언어 및 인지 공학을 위한 기술 온톨로지
DX	digital experiences - 디지털 경험
EDA	exploratory data analysis - 탐색적 데이터 분석
EO	enterprise ontology - 엔터프라이즈 온톨로지
ETL	extract, transform and load - 추출, 변환 및 적재
EU	European Union - 유럽연합
GAEN	Google Apple Exposure Notifications - Google Apple 노출 알림
GDPR	General Data Protection Regulation - 일반개인정보보호규정

원문 PDF 22 페이지

GPS	global positioning system - 위성위치확인시스템
GPX	GPS exchange format - GPS 교환 형식

GUI	graphical user interface - 그래픽 사용자 인터페이스
HAL	hardware abstraction layer - 하드웨어 추상화 계층
HIPAA	Health Insurance Portability and Accountability Act - 건강보험 이동성 및 책임법
HRPA	human resource participation - 인적 자원 참여
I4.0	Industry 4.0 - 산업 4.0
IA	intelligent agent - 지능형 에이전트
IEC	International Electrotechnical Commission - 국제전기기술위원회
IEEE	Institute of Electrical and Electronics Engineers - 전기전자공학회
IoT	Internet of Things - 사물인터넷
ISO	International Organization for Standardization - 국제표준화기구
ISTQB	International Software Testing Qualifications Board - 국제 소프트웨어 테스트 자격위원회
IT	information technology - 정보기술
ITOM	IT operations management - IT 운영 관리
IVS	intelligent vision system - 지능형 비전 시스템
KR	knowledge representation - 지식 표현
LTP	Linux Test Project - Linux 테스트 프로젝트
MAS	multi-agent system - 멀티에이전트 시스템
MBT	model-based testing - 모델 기반 테스트
ML	machine learning - 머신러닝
MSE	mean square error - 평균제곱오차
NFT	non-fungible token - 대체 불가능 토큰
NIST	National Institute of Science and Technology (US) - 미국 국립표준기술연구소
NLP	natural language processing - 자연어 처리
ODE	ontology-based software development environment - 온톨로지 기반 소프트웨어 개발 환경
OSOnto	operating system ontology - 운영체제 온톨로지
OWL	Web Ontology Language - 웹 온톨로지 언어
PAE	process and activity execution - 프로세스 및 활동 실행
PRPA	procedure participation - 절차 참여
RDF	Resource Description Framework - 자원 기술 프레임워크
ROCO	cloud-robotic-system domain ontology - 클라우드 로봇 시스템 도메인 온톨로지
ROI	return on investment - 투자수익률
ROoST	Reference Ontology on Software Testing - 소프트웨어 테스트 참조 온톨로지
RPA	resource participation - 자원 참여

원문 PDF 23 페이지

SDLC	software development life cycle - 소프트웨어 개발 수명주기
SoS	System of Systems - 시스템 오브 시스템즈
SQL	Structured Query Language - 구조화 질의 언어
SRT	site reliability testing - 사이트 신뢰성 테스트
STOWS	Software Testing Ontology for Web Service - 웹 서비스를 위한 소프트웨어 테스트 온톨로지
STVO	spatio-temporal, visual domain ontology - 시간·공간·시각 도메인 온톨로지
SUMO	suggested upper merged ontology - 제안된 상위 병합 온톨로지
SWEBOK	Software Engineering Body of Knowledge - 소프트웨어 공학 지식 체계
SwTOI	Software Test Ontology Integrated - 통합 소프트웨어 테스트 온톨로지
TaaS	Testing-as-a-Service - 서비스형 테스트
TOVE	Toronto virtual enterprise - 토론토 가상 기업

UFO	unified foundational ontology - 통합 기초 온톨로지
UI	user interface - 사용자 인터페이스
UK	United Kingdom - 영국
UML	Unified Modeling Language - 통합 모델링 언어
US	United States - 미국
UX	user experiences - 사용자 경험
W3C	World Wide Web Consortium - 월드 와이드 웹 컨소시엄
WPPA	work product participation - 작업 산출물 참여
WWW	World Wide Web - 월드 와이드 웹
XAI	explainable artificial intelligence - 설명가능한 인공지능
XML	Extensible Markup Language - 확장성 마크업 언어

원문 PDF 24 페이지

용어집

인수 테스트(Acceptance testing): 시스템을 인수할 것인지 여부를 결정하는 데 초점을 맞추는 테스트 레벨.*

적대적 공격(Adversarial attacks): 공격자가 모델의 예측에 영향을 주기 위해 모델의 입력을 약간 교란(변경)하는 것.

AIOps: IT 운영의 분석 능력을 향상시키는 머신러닝 분석 기술을 위한 산업 범주.

자동 추론(Automated reasoning): 컴퓨팅 시스템에 논리 형태의 추론을 적용하여 자동으로 추론 결과를 도출하는 것과 관련된 컴퓨터 과학의 고급 분야.

자동화 편향(Automation bias): 인간 의사결정자가 다른 입력보다 자동화된 의사결정 시스템의 권고를 더 선호할 때 발생하는 인지 편향.

블랙박스 테스트 기법(Black-box test technique): 컴포넌트 또는 시스템의 명세 분석을 기반으로 하는 테스트 기법.*

카오스 엔지니어링(Chaos engineering): 시스템이 혼란스럽고 예상하지 못한 조건을 견딜 수 있는 능력에 대한 신뢰를 구축하기 위해 운영 환경의 소프트웨어 시스템을 대상으로 실험하는 분야.

분류(Classification): 주어진 입력에 대해 출력 클래스를 예측하는 머신러닝(ML) 기능.

원문 PDF 25 페이지

컴포넌트 테스트(Component testing): 개별 하드웨어 또는 소프트웨어 컴포넌트에 초점을 맞추는 테스트 레벨.*

개념 드리프트(Concept drift): 동일한 입력 데이터가 제시된 모델에 대해 사용자가 기대하는 예측이 변화하는 것.

데이터 파이프라인(Data pipeline): ML 알고리즘을 지원하는 인프라. 데이터 획득, 전처리 및 준비, 하나 이상의 모델 학습, 그리고 모델을 운영 환경으로 내보내는 과정을 포함한다.

디지털 트윈(Digital twin): 물리적 객체를 나타내는 디지털 데이터를 생성하거나 수집하는 것.

하이퍼파라미터(Hyper-parameter): 모델을 학습하기 전에 사람이 ML 모델에 설정하는 변수.

통합 테스트(Integration testing): 컴포넌트 또는 시스템 간의 상호작용에 초점을 맞추는 테스트 레벨.

지식 표현(Knowledge representation): 컴퓨터가 추론할 수 있는 형태로 지식을 표현하는 방법에 대한 연구.

신경망(Neural network): 조정 가능한 가중치를 가진 가중 링크로 연결된 두 개 이상의 뉴런 계층으로 이루어진 네트워크로, 입력 데이터를 받아 출력을 생성한다. 인공신경망이라고도 한다.

뉴런(Neuron): 입력값을 받아 이를 결합하고 그 결과에 활성화 함수를 적용하여 출력값을 생성하는 신경망의 노드.

온톨로지(Ontology): 개념화에 대한 명시적 명세.

추론(Reasoning): 논리적 기법을 사용하여 이용 가능한 정보로부터 결론을 생성하는 AI의 한 형태.

회귀(Regression): 연속적인 값(일반적으로 부동소수점 값)을 출력하는 ML 기능.

강화학습(Reinforcement learning): 시행착오 과정을 사용하여 목표를 최대화하는 의사결정을 내리는 모델을 학습시키는 작업.

원문 PDF 26 페이지

시프트 레프트(Shift left): 테스트 노력의 초점을 초기 설계 및 테스트 단계로 이동하는 것, 즉 시스템 또는 소프트웨어 개발 주기의 왼쪽으로 이동하는 것.

시프트 라이트(Shift right): 테스트 노력의 초점을 운영 환경에서 사용자를 대상으로 하는 테스트로 이동하는 것.

하위 기호적 AI(Sub-symbolic AI): 수치 표현과 암묵적 정보 인코딩을 사용하는 기법 및 모델에 기반한 시스템.

기호적 AI 시스템(Symbolic AI system): 기호와 구조를 사용하는 기법 및 모델에 기반한 시스템.

시스템 테스트(System testing): 시스템 전체가 명시된 요구사항을 충족하는지 검증하는 데 초점을 맞추는 테스트 레벨.*

테스트 커버리지(Test coverage): 명시된 커버리지 항목이 테스트 스위트에 의해 결정되었거나 실행된 정도를 백분율로 나타낸 것.*

테스트 레벨(Test level): 테스트 프로세스의 구체적인 인스턴스.*

테스트 오라클(Test oracle): 테스트 대상 시스템의 실제 결과와 비교할 기대 결과를 결정하기 위한 출처.*

사용자 인수 테스트(User acceptance testing): 의도된 사용자가 시스템을 인수하는지 결정하기 위해 수행하는 인수 테스트의 한 유형.

확인(Validation): 특정한 의도된 사용 또는 적용에 대한 특정 요구사항이 충족되었음을 객관적 증거의 제공을 통해 확인하는 것.*

검증(Verification): 명시된 요구사항이 충족되었음을 객관적 증거의 제공을 통해 확인하는 것.*

화이트박스 테스트 기법(White-box test technique): 컴포넌트 또는 시스템의 내부 구조만을 기반으로 하는 테스트 기법.*

XPath: XML 에서 노드를 선택하기 위한 질의 언어.

원문 PDF 27 페이지

*Copyright © International Software Testing Qualifications Board(이하 ISTQB®).

원문 PDF 28 페이지

서문

이 책은 AI와 품질에 관련된 여러 주제를 둘러보는 여정이다. 이는 실무자뿐 아니라 정책 입안자와 학계에서도 뜨거운 주제이다.

AI 품질은 기존 소프트웨어 품질과 다르다. 이 책의 제 2 장에서는 편향부터 드리프트에 이르는 주제를 다루며 그 이유를 설명한다. 우리는 이러한 문제를 품질의 관점에서 이야기하지만, 그중 많은 문제는 일반적으로 자신이 품질에 관심이 있다고 생각하지 않는 실무자들에게도 큰 관심사이다. 예를 들어 AI의 윤리적 적용이나 AI를 둘러싼 법적 문제에 더 초점을 맞추는 사람들도 편향과 드리프트를 우려할 가능성이 높다. 실제로 이 책을 집필할 당시 제안된 유럽연합(EU) AI 법안은 대부분 AI를 위한 기술적 품질 관리 시스템의 관리에 초점을 맞추고 있다.

더 복잡한 문제 중 하나는 의도하지 않은 편향이다. 이는 서로 다른 출처에서 발생할 수 있으며 상당한 법적·사회적 영향을 미칠 수 있다. 제 3 장에서는 이를 개념적 관점뿐 아니라 수학적 관점에서도 자세히 설명한다.

제 4 장에서는 오늘날 AI 배포에서 가장 큰 비중을 차지하는 머신러닝 시스템을 테스트할 때의 복잡성과 과제로 넘어간다. 이 장은 특히 ISTQB Certified AI Tester 자격 취득을 계획하는 실무자에게 유용하다.

그다음 제 5 장에서는 방향을 전환하여 AI가 소프트웨어 테스트 프로세스를 지원하는 데 어떻게 사용될 수 있는지 다룬다. 이는 AI를 테스트하는 것과 거의 반대이지만, 편향과 드리프트 같은 동일한 문제 중 많은 부분을 고려해야 한다.

원문 PDF 29 페이지

다음으로 제 6 장에서는 보다 고급 주제인 온톨로지를 살펴본다. 온톨로지는 기본적으로 우리가 가지고 있는 지식, 이 경우에는 테스트에 관한 지식을 형식적으로 표현한 것이다. 온톨로지를 수립하면 AI를 통한 자동 추론을 가능하게 하여 테스트를 더욱 지원할 수 있다.

마지막으로, 앞의 장들에서 AI는 품질 문제에 대한 지속적인 모니터링이 필요하다는 점을 설명했으므로, 제 7 장에서는 시프트 라이트 테스트와 AIOps의 몇 가지 사례를 살펴본다.

1 서론

Rex Black

나는 언제나 바다를 좋아했다. 바다가 너무 춥거나 날씨가 너무 껏지만 않다면, 바다 가까이에 있을 때 수영이나 스쿠버 다이빙을 하는 것을 좋아한다. 하지만 바다에는 파도와 해류가 있다. 어떤 것은 잔잔하고, 어떤 것은 조금 더 크며, 어떤 것은 거대하다. 거대한 파도는 무엇을 하고 있는지 모른다면 위험할 수 있지만, 진지한 서퍼들은 이런 거대한 파도를 찾아 나서 그 속에서 인생 최고의 시간을 보낸다. 나는 거대한 파도의 경사면을 타고 내려오는 그런 서퍼들을 늘 부러워했지만, 정작 나는 그것을 배운 적이 없다.

소프트웨어 산업은 바다와 같다. 크기가 서로 다른 변화의 파도가 언제나 밀려온다. 큰 파도는 그것을 탈 수 있는 기술이 있다면 흥미로울 수 있지만, Agile 이라는 파도를 대수롭지 않게 여겼던 많은 소프트웨어 테스트 전문가들이 고통스럽게 배웠듯이, 그 파도는 당신의 경력을 집어삼킬 수도 있다.

또 하나의 큰 파도는 - 이상하게도 수십 년에 걸쳐 다가온 - 인공지능(AI)이다. 나는 UCLA 4 학년 때 AI 수업을 들었고, 한 교수를 위해 AI 를 주식 거래에 사용하는 개념 증명 프로젝트를 수행했다. 1988 년의 일이다. 느리게 온 파도였지만, 이제 마침내 도착하여 현실 세계에 실제 변화를 가져오고 있다.

따라서 이 책에서는 테스트 전문가로서 이 파도를 타기 위해 필요한 기술을 읽게 될 것이다. 이미 테스트 전문가라면, 자신의 업무에서 핵심적인 부분 중 하나가 질문을 하고 위험을 이해하는 것임을 아마 알고 있을 것이다. 그렇다면 이 책이 제기하고 강화하게 될 질문, 위험 및 기술에는 어떤 것들이 있을까?

원문 PDF 31 페이지

한 가지 핵심 질문은 우리가 AI 를 신뢰할 수 있는가 하는 것이다. 특히 AI 가 담당하게 될 중요한 역할들(예: 자율주행 자동차)을 생각하면 더욱 그렇다. 실리콘과 회로 속에서 전자가 빠르게 움직이는 것에 그치지 않고 물리적 세계에서 물체가 움직이는 경우에는 언제나 손상, 부상 또는 사망의 가능성이 존재한다. 물론 소프트웨어는 오랫동안 현실 세계에서 잠재적으로 위험한 물체를 움직이게 하는 데 관여해 왔다. 항공전자 소프트웨어나 체내 이식형 의료기기를 생각해 보라. 그러나 AI 는 은둔자로 살기로 선택한 사람들을 제외하면, 소프트웨어가 구동하는 움직이는 물체와 마주치는 일을 매일, 아니 어쩌면 매시간 경험하게 만들 가능성이 있다. 소프트웨어 테스트 전문가로서 우리는 사회가 이러한 시스템을 위험한 것보다 이로운 것으로 신뢰할 수 있도록 어떻게 도울 수 있을까?

물론 우리는 AI 시스템을 테스트함으로써 그렇게 할 것이다. 하지만 AI 시스템, 특히 가장 일반적인 형태인 머신러닝이 우리의 테스트에 반응하여 행위를 바꾼다면 어떻게 테스트할 수 있을까? 이는 대부분의 상황에서 소프트웨어가 변경되지 않는 한 동일한 입력에 대해 계속 동일한 출력을 제공하는 전통적인 소프트웨어와 뚜렷하게 다르다. 이 책은 이러한 과제에 어떻게 접근할 수 있는지에 대한 몇 가지 아이디어를 제공한다.

테스트를 실행하는 목적은 물론 무언가를 배우고 결과를 얻는 것이다. 우리는 언제나 그 결과가 명확하기를 바란다. ‘음, 아마 제대로 된 것 같다’라고 말하는 것이 아니라 테스트가 통과했는지 실패했는지를 말할 수 있기를 원한다. 하지만 AI에서 ‘테스트를 통과한다’는 것은 도대체 무엇을 의미할까? 올바른 행위에 대한 명확한 명세가 없을 수도 있고, 올바른 행위가 시간에 따라 변할 수도 있으며, 소프트웨어가 알려주는 것 외에는 무엇이 올바른 행위인지 우리 자신도 모를 수 있다. 한때 이러한 종류의 문제에 대한 해결책은 병렬 시스템을 만드는 것이었다. 이 접근법은 과거 특정 고위험도 시스템에서 선호되었지만 지금은 널리 사용되지 않는다. 이 책에서는 이러한 과제에 접근하는 몇 가지 방법도 읽게 될 것이다.

테스트 결과의 성격이 이렇게 달라진다는 것은 테스트 메트릭도 달라진다는 뜻이다. 예를 들어 전통적인 시스템의 기능 테스트에서는 흔히 통과한 테스트와 실패한 테스트의 비율을 살펴본다. AI 시스템의 기능 테스트에서는 올바른 질문이 ‘각 테스트는 얼마나 자주 올바른 것으로 보이는 결과를 내는가?’ 또는 ‘각 테스트의 결과는 기대 결과 값에서 얼마나 벗어나는가?’와 같은 것이 될 가능성이 높다.

원문 PDF 32 페이지

AI 테스트의 과제

어느 정도는, AI 시스템 테스트가 더 어려울 것이다. 테스트 전문가로서 우리 중 많은 사람이 익숙하게 다뤄 온 문제 공간보다 문제 공간 자체가 더 어렵기 때문이다.

AI 시스템은 복잡하고 혼란스럽고 정돈되지 않은 현실을 다루는 데 사용되고 있으며, 가능한 경우의 수는 기존 소프트웨어와 비교해도 엄청나다. 예를 들어 체스 프로그램에는 약 10^{123} 개의 수가 있고, 바둑에는 10^{360} 개가 넘는 수가 있다(Koch, 2016 참조). 이 두 숫자 모두 우주에 있는 원자의 수보다 크므로, 우리가 지금까지 단지 사소한 문제만 해결하는 데 익숙했던 것은 아니다. 그러나 자율주행 자동차를 생각할 때는 영국(UK), 독일 또는 미국 내 임의의 한 위치에서 같은 나라의 다른 임의의 위치로 가는 가능한 주행 경로의 수를 생각해 보라. 분명히 이것은 체스판이나 바둑판의 수보다 훨씬 덜 제한된 가능성의 집합이므로 가능한 결과의 수는 훨씬 더 많다.

문제가 더 어려울 뿐만 아니라 AI 시스템 자체도 다르다. 의도적으로 업데이트할 때만 변경되는 다른 소프트웨어와 달리, AI 시스템은 자극에 반응하여 변화한다. AI의 행위 변화는 다른 소프트웨어 업데이트처럼 미리 결정된 것이 아니라 자극에 의해 유도된다. 따라서 테스트 자체가 시스템이 다음에 어떻게 행동할지에 영향을 미치게 된다.

역사적으로 우리는 인간이 잘하지 못하는 활동(똑같은 일을 정확히 같은 방식으로 계속 반복하는 것)이나 수작업으로 하기에는 너무 오래 걸리는 활동(복잡한 수학 또는 회계)을 자동화하기 위해 컴퓨터를 사용해 왔다. 이제 우리는 수학 공식으로 쉽게 표현하기 어려운 복잡성을 가지지만 인간은 어린 시절에 배우는 활동을 컴퓨터로 자동화하려고 있다.

사람과 데이터에는 편향이 있으며, 이러한 편향은 AI 시스템에 내재될 수 있다. 예를 들어 IT 분야에서 여성의 상대적 수가 남성보다 적다는 사실은, 누가 IT 직무에서 성공할 가능성이 더 높은지를 판단할 때 AI가 남성에게 유리한

편향을 갖게 할 수 있다. AI 시스템의 광범위한 사용으로 이러한 편향이 굳어지고 강화되는 것은 반드시 다루어야 할 중대한 사회적 위험이며, 테스트 전문가는 그 위험을 인식하고 관리하는 데 대응해야 한다.

원문 PDF 33 페이지

사람들이 컴퓨터를 지나치게 신뢰한다는 사실은 이러한 위험을 더욱 키운다. 당신과 나에게는 이상하게 들릴 수 있다. 테스트 전문가로서 우리는 소프트웨어에 대해 매우 회의적인 태도를 배웠기 때문이다. 우리는 소프트웨어가 실패할 것이라고 예상하는 경향이 있다. 하지만 많은 사람은 그런 관점을 갖고 있지 않으며, 오히려 ‘컴퓨터가 그렇게 말한다면 틀림없이 맞겠지’라고 생각한다.

테스트 전문가가 직면하는 또 다른 과제는 세상이 끊임없이 변하고, 따라서 AI 시스템도 변한다는 사실에서 생긴다. COVID-19 팬데믹을 생각해 보자. 중국에서 이상한 호흡기 질환이 발생했다는 소식을 처음 들었을 때 나는 ‘그래, SARS 나 MERS 처럼 통제될 것이고 조금 이상하긴 하겠지만 크게 문제되지는 않겠지’라고 생각했다. 하지만 나는 완전히 틀렸다. AI 시스템을 테스트할 때는 작고 점진적인 변화뿐 아니라 팬데믹처럼 크고 빠르며 파괴적인 변화도 생각해야 한다. 그렇지 않으면 중요한 테스트를 놓칠 위험이 있다.

조금 물러나 테스트의 목적을 생각해 보면, 일반적인 목적 중 하나는 소프트웨어 품질에 대한 위험을 허용 가능한 수준으로 줄이는 것이다. 그러나 전통적인 품질 모델은 AI 시스템의 품질을 충분히 포착하지 못한다. 예를 들어 전통적인 소프트웨어는 주어진 입력 집합에 대해 올바른 답을 주거나 그렇지 않다. AI 시스템은 동일한 입력에 대해 어떤 때는 올바른 답을 주고 어떤 때는 그렇지 않을 수 있으며, 기대한 것과 다르지만 여전히 올바른 답을 줄 수도 있고, 동일한 동등 분할에 속하여 전통적으로는 같은 방식으로 처리될 것으로 기대되는 입력에 대해 서로 다른 결과를 줄 수도 있다. 이는 우리의 테스트 기법을 다시 생각해야 한다는 의미이다.

테스트 커버리지를 어떻게 측정할지도 다시 생각해야 한다. 지금까지 말한 내용만으로도 요구사항, 설계 및 기타 명세 커버리지 측정이 분명 제대로 작동하지 않을 것이라고 짐작했을 것이다. 잠시 후 AI 시스템의 결정론적 행위가 아닌 확률적 행위를 다룰 때 이 점은 더욱 분명해질 것이다. 위험 커버리지와 지원 구성 같은 다른 일반적인 커버리지 차원은 관련이 있을 수 있지만, 그것들이 명세 커버리지를 대신할 수 있을까? 또한 전통적인 프로그래밍을 사용하지 않으므로, 어떤 경우에도 테스트의 완전성을 측정하는 데 사용 범위가 제한적이었던 코드 커버리지는 더욱 유용하지 않으며, 어쩌면 전혀 쓸모가 없을 수도 있다. 이 책은 AI 시스템 테스트에서 이 중요한 문제에 어떻게 접근할지를 이해하는 데 도움을 줄 것이다.¹

원문 PDF 34 페이지

AI 시스템은 입력에 반응하여 자주 변화하기 때문에, 그러한 변화가 바람직한 변화인지가 중요하다. 예를 들어 고의적인 악의 또는 단순히 잘못된 데이터에 노출되는 것만으로도 AI 가 인종차별적으로 변할 수 있다. 실제로 한 Twitter 봇에서 그런 일이 발생했다(Vincent, 2016). 물론 인공지능이 인종차별적으로 변한다는 발상 자체는 여러 측면에서 초현실적이고 거의 카프카적이다. ‘튜링 테스트를 통과할 만큼 지능적이지는 않지만 튜링 테스트를 통과할 수 있는 다른 사람들에게 혐오 발언을 할 수는 있다’라는 설명은 어쩌면 인종차별 봇뿐 아니라 우리가 불행히도 마주칠 수 있는 일부 사람들에게도 해당할 수 있겠지만, 그것은 이 책의 범위를 벗어나는 문제이다.

우리는 윤리적 행위와 비윤리적 행위를 어떻게 테스트할 것인가? 회색 영역이나 잘 알려진 윤리적 딜레마가 존재하는 상황에서는 어떻게 처리해야 할까? 예를 들어 통제 불능의 전차가 군중 속으로 돌진해 12 명이 넘는 사람을 죽일 수 있다고 하자. 다만 한 사람(또는 이 경우에는 AI)이 개입해 전차를 다른 선로로 전환하면 한 사람만 치게 되지만 그 사람은 확실히 죽게 된다. 이것이 단순히 보인다면, 12 명의 부주의한 자전거 운전자들이 무모하게 자율주행 자동차 앞을 가로막는 경우를 생각해 보라.² 자동차는 그들을 치지 않기 위해 인도로 방향을 틀어 법을 잘 지키던 보행자 한 명을 치어야 하는가? 여기에서 올바른 행위란 무엇인가? 윤리적인 행위란 무엇인가? 그런 행위를 어떻게 프로그래밍할 것인가? 그것을 어떻게 테스트할 것인가?

최근 몇 년 동안 영국, EU 및 미국(US)의 규제기관은 데이터 프라이버시 문제를 다루느라 고심해 왔다. 그러나 한 입력 집합에서 다른 입력 집합으로 바뀔 때 행위가 달라질 수 있다면, 데이터 프라이버시 같은 규정 준수를 어떻게 테스트할 수 있을까? 예를 들어 미국에서는 환자 정보에 대한 접근이 Health Insurance Portability and Accountability Act(HIPAA)라는 법률의 규제를 받는다. 테스터는 이러한 법률의 준수 여부를 테스트할 수 있어야 하지만, AI가 진화함에 따라 우리의 테스트 결과가 바뀌지 않을 것이라고 확신할 수 있을까?

그렇다면 사회적 문제에서 품질 전문가로서 우리의 역할은 무엇인가? 물론 개인으로서 우리는 어떤 대의에 기부하거나, 어떤 사안이나 사람을 지지하거나 반대하는 시위에 참여할 수 있다. 하지만 그것은 개인적인 선택이다. AI 시스템에서는 우리의 업무가 매우 까다로운 문제 한가운데로 밀려들어갈 수 있다. 예를 들어 미국의 주택 소유 시장은 인종을 둘러싼 매우 복잡하고 고통스러운 사회적 역사를 갖고 있다.³ 은행 분야에서 소프트웨어 테스터, 소프트웨어 엔지니어, 비즈니스 분석가 또는 다른 소프트웨어 전문가로 일한 경험이 있다면, 미국에서 주택대출과 관련해 체계적으로 불이익을 받은 소수 인종에게 가해졌던 피해를 은행이 더 이상 지속하지 않도록 보장하기 위한 규정을 알고 있을 수도 있다.

원문 PDF 35 페이지

그 분야 밖에서는 이 문제에 전문적으로 관여할 일이 제한적이었을 수 있다. 이제 AI 시스템에서는, 이 책이 설명하듯이, 여러분이 작업하는 시스템이 사회적 결과에 영향을 미칠 수 있는 정도만큼(좋은 방향이든 나쁜 방향이든) 그 시스템이 악영향을 미치고 있는지 평가하는 업무에 전문적으로 관여하게 될 수 있다. 그러한 영향은 의도하지 않은 것일 수도 있고 매우 미묘할 수도 있다.

이 책에서 더 설명하겠지만, AI 시스템 테스트 업무가 사회적 문제와 맞닿는 정도만큼 다양한 편향 때문에 상황은 복잡해질 것이다. 여러분은 ‘나는 편향되어 있지 않다’고 항의할 수 있고 실제로 그럴 수도 있다. 하지만 AI를 학습시키는 데 사용한 데이터는 편향되어 있는가? AI가 다른 어떤 방식으로 편향되어 있는가? 어떤 방식인가? 그러한 편향이 존재하는지 확인하기 위해 어떻게 테스트할 수 있는가?

이러한 AI 시스템을 테스트할 때, 우리가 수년에 걸쳐 어렵게 축적해 온 테스트 설계 기법, 특히 Glenford Myers와 Boris Beizer 같은 선구자들의 작업에서 발전한 동등 분할, 경계값 분석, 결정 테이블, 상태 다이어그램 및 조합 테스트 같은 기법은 그 힘의 일부를 잃을 수 있다. 각 기법 뒤에 Beizer가 ‘버그 가정(bug assumption)’이라고 부른 것이 있기 때문이다.⁴ 버그 가정이란 각 기법이 특히 잘 찾아내는 버그의 유형을 말하며, 그러한 버그 유형은 전통적

인 절차형 및 객체지향 프로그래밍에서 발생하는 버그 유형이다. AI 시스템에는 다른 유형의 버그가 존재하며, 전통적인 프로그램에서 찾는 일부 버그는 발생 가능성이 더 낮다. 이 책에서는 AI 시스템을 테스트할 때 전통적인 기법을 보완할 새로운 테스트 설계 기법에 대한 통찰을 얻을 것이다.

우리는 동일한 입력 집합으로 문제를 해결할 때 소프트웨어가 매번 같은 방식으로(적어도 기능적으로는) 작동하는데 익숙하다. 그러나 신뢰성과 성능 같은 비기능적 행위에서는 흔히 확률적 행위를 보게 된다. 예를 들어 신뢰성은 주어진 부하 수준에서 시스템이 실패할 확률의 백분율 또는 주어진 부하 수준에서 정해진 시간 목표 내에 수신되는 응답의 백분율로 표현할 수 있다. AI 시스템에서는 시간이 지남에 따라 진화하는 것에 더해 기능적 행위 역시 확률적일 수 있다. 이것은 AI 시스템의 기능 테스트에서 신뢰할 수 있는 테스트 오라클을 찾기 어렵게 만드는 또 하나의 요인이다.

원문 PDF 36 페이지

물론 수십 년 동안 테스트 분야에서 눈길을 끄는 화려한 대상 중 하나는 테스트 자동화였다. 도구 공급업체들은 유행하는 전문용어를 사용하고 손쉬운 성공과 빠른 투자수익률(ROI)을 약속하면서 많은 돈을 벌어 왔다. 하지만 나와 내 고객들은 장기적으로 보았을 때 테스트 자동화가 성공할 가능성이 심장 절개 수술보다 낮다는 사실을 발견했다. 심장 절개 수술을 받은 다른 면에서는 건강한 70 세 이상 사람들의 80% 이상이 5 년 후에도 생존해 있지만(Khan et al., 2000), 내가 고객들과 함께 본 대규모 테스트 자동화 활동 중 5 년 뒤에도 같은 전략과 기술을 사용하면서 긍정적인 ROI 를 달성하고 있는 것은 절반도 되지 않는다. 이 책에서는 AI 가 테스트 자동화에 어떤 영향을 미칠지 배우게 될 것이다. 그와 똑같이 중요하게, AI 가 테스트 자동화에 어떤 영향을 미치지 않을지도 배우게 되며, 단기적으로 테스트 자동화에서 특정 AI 이점을 가로막는 장애물도 알게 될 것이다. 따라서 도구 공급업체의 유행어 가득한 영업 문구에 현혹될 가능성이 줄어들 것이다.

자동화된 테스트는 여러 레벨에서 다양한 인터페이스를 통해 작동할 수 있다. 테스트 레벨과 자동화 인터페이스에 따라 테스트 자동화 문제에 AI 시스템을 적용할 때의 과제가 달라진다. 그러나 각 레벨과 특정 인터페이스에서 테스트 자동화가 가진 근본적인 과제는 AI 를 적용했다는 이유만으로는 흔히 바뀌지 않는다. 물론 테스트 자동화 도구의 열성적인 홍보자들에게는 그 반대를 주장하는 것이 매우 유리하지만 말이다. 이 책은 테스트 자동화 도구 평가를 위한 올바른 기준을 정의하도록 도울 것이며, 이는 어떤 테스트 자동화 프로젝트에서도 매우 중요하다. 언제나 그렇듯이 확실한 비즈니스 사례와 입증 가능한 ROI 는 모든 중대한 활동에 필수적이며, 테스트 자동화는 AI 를 사용하든 사용하지 않든 거의 언제나 중대한 활동이 될 것이다. 또한 투자수익률은 명확히 정의된 목표에 대해 측정해야 하며, 그 목표는 올바른 목표여야 한다는 점도 기억해야 한다.

지금쯤 분명해졌겠지만, AI 시스템 테스트와 관련된 이러한 모든 과제와 차이는 역량에도 영향을 미친다. 예를 들어 고빈도 주식 거래를 지원하는 AI 시스템을 테스트한다고 가정해 보자. 금융 시스템, 금융시장 및 금융 규제에 관한 상당한 도메인 전문지식 - 전통적인 기술로 구현된 그러한 시스템을 테스트할 때도 모두 필요한 역량 - 외에도, 시스템의 확률적 특성 때문에 상당한 데이터 과학 및 수학 역량이 필요할 수 있다.

이러한 역량은 AI 기반 고빈도 거래 시스템과 관련된 더 복잡한 테스트 오라클 문제를 다루는 데뿐만 아니라,

까다로운 테스트 데이터 설계 문제를 다루는 데도 필요하다. AI 기반 고빈도 거래 시스템의 테스트 오라클 문제는 전통적인 구현에서도 이미 충분히 어려운 문제이다. 실제로 테스트 데이터 설계는 매우 복잡하기 때문에 테스트 전문가가 그 프로세스를 이해해야 하기는 하지만, 흔히 전문 데이터 과학자가 수행해야 한다. 테스트 전문가가 테스트 데이터 설계 문제를 이해하는 주된 목적은 이와 관련된 데이터 과학자의 작업을 검토하는 역할을 수행하고, 그 작업의 타당성에 영향을 줄 수 있는 다양한 편향을 다루기 위한 것이다.

AI 시스템 테스트에 새로운 과제, 새로운 위험 및 새로운 역량 요구사항이 많다고 해서 기존 위험과 역량이 더 이상 필요 없다는 뜻은 아니다. 예를 들어 시스템 오브 시스템즈(System of Systems)를 구성할 때 - 그리고 이 사물인터넷 세계에서는 거의 모든 것이 다른 모든 것과 통신하므로 거의 모든 것이 시스템 오브 시스템즈라고 할 수 있다 - 각 컴포넌트 시스템이 실제 사용 방식에 맞는 형태로 적절하게 테스트되었는지 보장해야 한다. 인터페이스를 가로지르는 데이터 흐름의 테스트는 동등 분할, 경계값 분석 및 조합 테스트라는 전통적인 테스트 설계 역량이 필요한 영역이다.

변하지 않을 다른 것들도 있다. 결함이 오래 존재할수록 프로젝트, 시스템 및 사용자에게 미치는 영향과 결함을 제거하는 비용 모두가 증가한다는 사실은 오래전부터 확립되어 있다. 이러한 비용은 주로 몇 줄의 코드를 변경하는 개별 행위 자체와 관련된 것이 아니다. 결국 코드에서 결함을 찾은 뒤에는 코드를 변경하는 노력은 그리 크게 달라지지 않는다. 오히려 처음부터 결함을 찾아내는 일과 그 결함이 일으키는 실패의 결과를 완화하는 일과 관련된다. 이를 해결하기 위해 영리한 소프트웨어 엔지니어들은 수십 년 동안 시프트 레프트를 이야기해 왔다. 이는 결함이 유입된 지점에 가능한 한 가까운 시점에서 결함을 발견하고 제거하는 것을 의미한다.

하지만 결함이 유입된 시점에서 발견되는 시점까지의 시간을 줄이는 또 다른 방법에는 시프트 라이트가 포함될 수 있다. 시프트 라이트는 운영 환경으로 빠져나간 결함을 가능한 한 빠르고, 고통 없이, 저렴하게 발견하는 것과 관련된다. AI 시스템은 우리가 이를 수행하는 데 어떻게 도움을 줄 수 있을까? 계속 읽어보면 알 수 있다.

요약

이제 여러분은 손에 - 또는 자신이 좋아하는 전자 독서기기에 - AI 시스템의 테스트 및 품질과 관련된 여러 새로운 개념과 새로운 과제를 소개하는 책을 들고 있다. 이 책은 여러분이 여정을 시작하도록 할 수는 있지만, 모든 과제에 쉬운 해결책을 제공할 수는 없다. 과제에는 항상 쉬운 답이 있는 것이 아니기 때문이다.

하지만 위에서 언급했듯이 검증된 기존의 방법이 여전히 적용되는 영역도 있다. ‘이것이 모든 것을 바꾼다’고 말하며 다가오는 사람을 경계하고, 본질적인 회의감을 가지고 그 말을 들어라. 그러나 AI에 관해 이야기할 때는 AI가 소프트웨어 테스트를 포함한 소프트웨어 공학의 모든 측면에 매우 큰 파괴적 변화를 일으키지 않는 상황을 상상하기 어렵다. 결국 어떤 발명은 정말로 모든 것을 바꾸거나, 적어도 많은 것을 바꾼다. 이 지구상에는 거의 아무도 들어본 적 없

는 인물인 Fritz Haber 의 획기적인 발견이 없었다면 존재하지 못했을 사람들이 많다. 그의 아이디어는 전 세계 인구의 약 절반이 매일 음식을 먹을 때마다 영향을 받는다.⁵

이 책을 읽으면 여러분에게 다가오고 있는 - 이미 바로 눈앞에 와 있지 않다면 - AI 주도의 파괴적 변화에 대한 통찰을 얻을 수 있을 것이다. 그리고 문장 중간에 비유를 조금 바꾸는 것을 허락한다면, 그 파괴적 변화에 휩쓸려 가기보다 그 파도를 타는 방법에 대한 아이디어도 얻을 것이다. AI 가 변화시킬 세계로 향하는 파도를 타는 소프트웨어 테스트 전문가로서 성공하기를 바란다. 어쩌면 AI 는 Haber 의 발명이 그랬던 만큼 세상을 바꿀지도 모른다. Haber 의 발명과는 달리 우리가 AI - 인공지능, 즉 인간이 아닌 지능 - 를 서로에게 비인간적인 행동을 할 수 있는 새롭고 더 효율적인 방법을 만드는 데 사용하지 않기를 바란다. 그것의 일부는 우리 각자에게 달려 있다.

¹ 테스트 커버리지에 대해 생각할 수 있는 가능한 방법에 대한 자세한 내용은 필자의 발표 ‘Dimensions of Test Coverage’(Black, 2015)를 참조한다. ² 이는 Malcolm Gladwell(2021)이 팟캐스트 Revisionist History 의 한 에피소드에서 논의한다. ³ 이 역사에 대한 한 가지 설명은 Richard Rothstein 의 『The Color of Law』 (2017)를 참조한다. ⁴ 각 기법의 버그 가정에 대한 자세한 내용은 Boris Beizer 의 『Software Testing Techniques』 (1990)를 참조한다. ⁵ Fritz Haber 는 화학 비료 제조에 필수적인 암모니아를 만드는 데 사용되는 공정을 발명했다. 안타깝게도 Haber 는 화학전의 발명에도 핵심적인 역할을 했기 때문에 변화에 관한 그의 유산은 엇갈린 평가를 받는다. 비극적이게도 제 1 차 세계대전 당시 독일의 전쟁 수행에 Haber 가 기여했음에도, 그의 유대계 혈통 때문에 가족 구성원 일부가 제 2 차 세계대전 중 강제수용소에서 사망했다. Haber 의 복잡한 삶, 경력 및 과학적 공헌에 대한 간단한 요약은 다음에서 확인할 수 있다: https://en.wikipedia.org/wiki/Fritz_Haber

원문 PDF 39 페이지

2 AI 신뢰성과 품질

Adam Leon Smith

인공지능(AI)은 1950 년대부터 존재해 왔으며 이제는 새로운 전기와 같은 존재가 되었고 많은 기업에 ‘반드시 갖추어야 할 것’이 되었다. AI 의 도입이 증가하면서 이 기술의 진정한 잠재력이 열리고 여러 산업에서 효율성이 향상되고 있다. 이는 언론에서 접하는 최첨단 응용 분야에만 해당하는 것이 아니라, 가정의 기기, 휴대전화 및 직장에서도 보다 일상적이고 평범한 방식으로 나타나고 있다.

AI 는 정의하기 어렵고, 그 정의는 기술만큼이나 빠르게 진화한다. 아마 가장 폭넓은 합의가 이루어진 설명은 획득한 지식을 적용하여 의사결정을 내리는 것으로, 일반적으로 보다 명시적인 논리(전통적인 프로그래밍)를 사용하는 것과 대비된다. 이는 소프트웨어 및 시스템 품질 전문가들에게 새로운 기회와 새로운 문제를 동시에 제시한다.

AI 는 품질 및 테스트 관련 활동을 자동화하는 데 도움을 줄 수 있지만, 많은 엔지니어들은 AI 시스템의 품질을 평가하고 설명하는 데 수반되는 과제와 AI 컴포넌트를 통합하면서 발생하는 새로운 위험을 관리하는 데 어려움을 겪고 있다. 가장 어려운 영역 중 하나는 통계적 시스템에 내재하는 불완전성과 결과를 재현하고 설명하기 어렵다는 점이

다. 또 다른 과제는 중요한 의사결정을 AI 시스템에 맡겨도 신뢰할 수 있다고 모두를 납득시키는 것이다. 이 장에서는 이러한 문제를 살펴보고 AI 품질을 정의하는 데 사용할 수 있는 모델을 논의한다.

원문 PDF 40 페이지

신뢰성

AI 기술의 구매자, 규제기관 및 최종 사용자는 실제 사용 시점에서 AI 기술의 ‘신뢰성’에 대해 우려한다. 이러한 우려는 연구실 안에만 있는 것이 아니라 공공 및 민간 부문의 더 광범위한 디지털 전환 과정에서 점점 더 중요한 일부가 되고 있기 때문에 실제 사용 시점에서 이를 고려하는 것이 중요하다.

BCS가 수행한 조사(BCS, 2020a)에 따르면 영국 성인의 53%는 자신에 대해 판단할 때 어떤 기관이 알고리즘을 사용하는 것도 신뢰하지 않았다. 이와 유사하게 미국의 대규모 조사에서는 응답자의 대다수가 개인 금융 점수를 포함한 특정 알고리즘 사용 사례에 우려를 나타냈다(Smith, 2018). EU 28 개 회원국 전체를 대상으로 정규화되고 대표성을 갖춘 연구에서도 응답자의 74%가 알고리즘 사용에 대한 더욱 엄격한 통제를 원한다고 나타났다(Grzymek and Puntschuh, 2019).

신뢰성의 정의는 맥락에 크게 좌우되는 것으로 보이며, 단순히 사람들이 시스템을 신뢰하고 싶게 만드는 시스템의 특성을 의미할 수도 있다.

그렇다면 사람들은 왜 신뢰성이라는 용어를 사용할까? 신뢰는 매우 인간적인 특성이다. 어떤 사람들은 분명 시스템이 인간과 거의 비슷한 형태를 갖추고 인간처럼 믿을 수 있고 의지할 수 있는 모습을 떠올릴 것이다. 어떤 사람들은 불편함을 느끼게 하는 속성을 기준으로 사람들을 다르게 대하는 것으로 드러난 시스템의 정확성 및 편향 문제를 더 많이 생각할 것이다. 또 어떤 사람들은 확률을 사용하여 논리적 결론을 추론할 때 존재하는 본질적인 통계적 정확성 문제를 걱정할 것이다.

품질과 신뢰성의 정의는 서로 다르지만 연결되어 있으며, 신뢰성에도 여러 정의가 있다. EU 인공지능 고위급 전문가 그룹은 신뢰할 수 있는 AI를 ‘합법적(적용 가능한 모든 법률 및 규정을 준수), 윤리적(윤리적 원칙과 가치를 존중), 강건한(기술적 관점과 사회적 관점 모두에서)’ AI로 정의한다(High-Level Expert Group on Artificial Intelligence, 2020).

이 정의를 ISO/IEC가 ‘고유한 특성의 집합이 요구사항을 충족하는 정도’로 정의한 품질과 비교해 볼 수 있다(ISO, 2005). 서로 연결된 이 두 정의의 한 가지 차이는 품질에는 이해관계자가 명시한 요구사항이 필요하다는 점이다.

원문 PDF 41 페이지

EU의 관점을 고려하면, 시스템은 명시적인 요구사항뿐 아니라 암묵적인 법적, 윤리적 및 사회적 요구사항도 충족해야 한다. 이 책을 집필할 당시 국제 표준화 커뮤니티에서 개발 중이던 두 번째 정의는 기술 신뢰성을 ‘안전, 보안, 프라이버시, 신뢰도 및 회복탄력성 등의 특성을 근거로, 일반적인 조건 집합에서 시스템이 설계된 행위에 따라 수행할 작업을 입증할 수 있는 가능성’으로 정의한다.

이 정의에는 특히 ‘설계된(designed)’이라는 단어가 포함되어 있으며, 이는 품질의 정의에서 요구되는 것처럼 요구 사항의 명세를 전제로 한다. 또한 미국 국립표준기술연구소(NIST)의 정의(Griffor et al., 2017)를 기반으로 하며, 집필 당시 미국에서는 NIST 지침을 기반으로 신뢰할 수 있는 AI 를 요구하는 2020 년 국방수권법(National Defense Authorization Act)이 막 통과된 상태였다.

이렇게 진화하는 정의들 사이의 의미론적 차이는 잠시 제쳐 두면, AI 시스템 개발자는 법적, 윤리적 및 사회적 문제를 고려하여 검증 가능한 품질 요구사항을 명세함으로써 시스템을 신뢰할 수 있게 만들 수 있다. 개발자든 테스터든 엔지니어는 요구사항을 구현하고 검증함으로써 신뢰할 수 있는 시스템을 구축할 수 있다. 그러면 이해관계자는 시스템이 객관적으로 신뢰할 수 있기 때문에 그 시스템을 신뢰할 수 있기를 기대할 수 있다. 물론 실제 신뢰는 이보다 훨씬 주관적이다. 사람들은 단지 시스템을 만든 회사와 연관된 브랜드 때문에 시스템을 신뢰하거나 불신할 수도 있다. 신뢰성을 어떻게 정의하든, 신뢰성은 측정 가능한 특정 품질 특성 집합을 포함하는 상위 개념임이 분명하다. 연구자들은(Kuleshov, 2018) 기능 특성 벡터를 적절한 가중치를 적용한 품질 측정값으로 구성하고, 이를 특정 AI 작업의 표준과 비교할 수 있다고 제안했다. 이러한 표준에는 측정 방법, 평가에 사용되는 특정 데이터 양, 특정 작업에서 허용되는 최소 관찰 품질 특성 등이 포함될 수 있다.

AI 품질 문제

인공지능에는 ‘기호’로 인코딩된 지식을 사용하는 기호적 시스템과, 이전에 본 데이터를 사용하여 미래 데이터 포인트를 예측하는 하위 기호적(머신러닝) 시스템이 포함될 수 있다. 머신러닝(ML)은 가장 널리 사용되는 방식이며,

원문 PDF 42 페이지

품질 목표를 논의하고 명세하며 이를 어떻게 평가할지 분석하는 것이 어렵다. 그렇게 생각하는 것은 품질 전문가만이 아니다. 일본에서 ML 엔지니어 278 명을 대상으로 한 연구에서는 ML 통합 과정에서 직면하는 가장 큰 새로운 과제로 고객과의 의사결정 및 테스트/품질 보증을 확인했다. 이들은 테스트 ‘오라클’의 부재와 근본적인 불완전성을 테스트 문제의 가장 큰 원인으로 지적했다(Ishikawa and Yoshioka, 2019).

오라클은 답으로 해결될 수 있는 의사결정 문제를 해결하거나 단순히 연구하는 데 도움을 주는 추상적인 개념이다. 특히 시스템 테스트에서 테스트 오라클(ISTQB, 2021)은 개별 테스트가 통과했는지 실패했는지를 결정하기 위해 ‘테스트 대상 시스템의 실제 결과와 비교할 기대 결과를 결정하는 출처’이다.

기호적 AI 는 ML 을 사용하지 않지만, 그렇다고 인기 있는 방법이 아니라는 뜻은 아니다. 전문가 시스템은 사용자나 전문가가 명시한 명시적 규칙과 함께 연역을 사용할 수 있다. 비교적 흔하게, 검색 문제, 즉 텍스트를 검색하는 것이 아니라 문제에 대한 가능한 해결책을 탐색하는 문제에서는 기호적 접근법을 사용한다. 이러한 접근법은 ML 과 완전히 동일한 모든 문제를 겪지는 않지만, 그중 일부 문제는 분명 겪는다.

기호적 AI 는 기호와 구조를 사용하는 반면, 하위 기호적 AI(ML 포함)는 수치 표현과 암묵적 정보 인코딩을 사용한다(ISO/IEC, 2021a).

그렇다면 AI 품질과 관련된 문제는 왜 이렇게 많을까? 하나씩 살펴보자.

문제 1. 명세 없는 자동화

원문 PDF 43 페이지

첫 번째 문제는 AI에서 가장 일반적인 접근법인 ML을 명세가 거의 없거나 전혀 없는 자동화라고도 정의할 수 있다는 점이다. 시스템이 어떻게 작동하는지를 설명하는 논리를 명세하는 대신, 데이터셋을 사용해 학습시킨다. 이 접근법의 인기는 수집되는 데이터의 양이 증가한 것과 함께 지난 20년 동안 AI 사용이 크게 성장한 이유를 상당 부분 설명한다.

이해하기 쉬운 예는 전자상거래 사이트에서 사용하는 추천 엔진이다. 장바구니에 상품 하나를 추가하면 관심이 있을 만한 다른 상품을 제안받을 가능성이 높다. 개발자가 관련 상품을 식별하는 코드를 작성하는 것이 아니라, 과거 구매 조합을 이용해 ML 모델을 학습시켜 추천할 가능성이 높은 후보를 식별할 수 있다.

머신러닝은 계산 기법을 통해 모델 파라미터를 최적화하여 모델의 행위가 데이터 또는 경험을 반영하도록 하는 과정이다(ISO/IEC, 2021a).

ML 모델은 입력 데이터를 기반으로 추론 또는 예측을 생성하는 수학적 구조이다(ISO/IEC, 2021a).

그렇다면 이것은 어떻게 작동할까? 적어도 최근에는 주로 신경망을 사용한다. 인공신경망(ANN)은 뇌가 작동하는 방식에서 영감을 받았다. 어떤 사건들 사이의 연관성을 인식하면 우리의 뇌는 특정 경로를 따라 신호를 보내고, 그 경로를 더 자주 사용할수록 그 경로는 더 강해진다. ANN도 유사한 원리로 작동한다. 어떤 경험(학습 데이터)을 관찰하고, 우리가 예측하려는 대상과 특정 관찰 가능한 데이터 항목(특성) 간의 상관관계가 클수록 결과 ANN에서 해당 예측 경로가 더 강해진다. 이는 동일한 경로를 새롭고 이전에 보지 못한 데이터에 사용하여 무언가를 예측할 수 있다는 의미이다.

원문 PDF 44 페이지

신경망(인공신경망이라고도 함)은 조정 가능한 가중치를 가진 가중 링크로 연결된 두 개 이상의 뉴런 계층으로 이루어진 네트워크로, 입력 데이터를 받아 출력을 생성한다(ISO/IEC, 2021a).

뉴런은 입력값을 받아 입력값을 결합하고 그 결과에 활성화 함수를 적용하여 출력값을 생성하는 신경망의 노드이다(ISO/IEC, 2021a).

여기에는 본질적인 품질 평가 문제가 있다. 시스템을 만들기 위해 필요한 명세가 적다면, 그것이 제대로 작동하는지 어떻게 알 수 있을까? 전자상거래 추천에서는 직관적으로 판단할 수 있을지 모르지만, 더 복잡한 응용에서는 흔히 그렇게 명확하지 않다. 사람들이 관심을 가질 만한 뉴스 기사를 추천하는 ML 알고리즘을 생각해 보자. 올바른 기사를 추천하고 있다는 것을 어떻게 알 수 있을까? 사람들이 그 기사를 클릭하는지는 확인할 수 있을지 모르지만, 정말 가장 좋은 기사를 추천하고 있는가?

문제 2. 알 수 없는 정답

두 번째 문제는 어떤 특정 문제의 정답을 우리가 알지 못하고, 바로 그 알지 못하는 정답을 알아내기 위해 시스템을 작성하는 경우 그것이 맞는지 어떻게 알 수 있느냐는 것이다.

현재 AI 개발 단계에서는 AI 시스템이 전문가 판단을 대체하는 경우에 이 문제가 발생한다. 예를 들어 질병 진단에는 의료 전문지식이 필요하다. 물론 전문가에게 시스템이 올바른지 평가해 달라고 요청할 수 있지만, 여기에도 문제가 있다. 예를 들어 전문가들끼리 의견이 다를 수 있고, 전문가들은 시스템이 할 수 없는 방식으로 답변에 단서를 달거나 조건을 붙일 수 있다. AI가 더 발전하여 인간 전문가도 확인할 수 없는 문제를 해결하기 시작하면 이 문제는 점점 더 커질 것이다.

단순한 ML 모델의 행위를 검증하는 경우에도 이 문제는 어떤 형태로든 나타난다. 모든 ML 동작은 예측이며, 각 예측이 정확한지를 관찰하려면 ‘그라운드 트루스(ground truth)’, 즉 실제 정답을 알아야 한다.

원문 PDF 45 페이지

여기에는 명백한 실무적 한계가 있으므로 평가는 흔히 편의 표본에 제한된다.

문제 3. 데이터의 복잡성

세 번째 문제는 입력의 복잡성이다. 시스템에 매우 다양한 현실 세계의 센서가 있다고 할 때, 모든 입력 범위를 어떻게 예측할 수 있을까?

자율주행 자동차의 입력 복잡성을 상상해 보라. 자율주행 자동차를 학습시키는 데 사용할 수 있는 입력 데이터로 구성된 한 오픈소스 테스트 데이터셋(Binas et al., 2017)은 카메라 입력 데이터를 표현하기 위해 단지 340 x 260 픽셀의 저해상도 카메라를 사용하여 준비되었다. 이 데이터셋에는 스티어링 휠의 각도, 가속 페달 위치, 브레이크 페달 상태 같은 자동차 제어 데이터도 포함되어 있었다. 엔진 회전속도, 차량 속도, 헤드램프 상태, 토크 및 기어 위치 등 차량 내부 작동에 관한 더 많은 데이터도 이용할 수 있다. 사용 가능한 연료와 소비된 연료, 지리적 위치 같은 맥락도 관련이 있다. 저해상도 카메라를 사용했음에도 초당 7~10 메가바이트의 데이터가 기록되었다.

이 문제는 AI에만 고유한 것은 아니지만, AI의 데이터 주도적 특성과 결합되면 품질을 분석하는 일이 복잡해진다. 다른 방식으로 설명해 보자. 코너에 접근할 때 속도를 줄이도록 명시적 논리를 사용해 설계한 자율주행 시스템은 실제로 속도를 줄이는지 확인함으로써 검증할 수 있다. 반면 여러 시간의 학습 데이터를 기반으로 속도를 변경하도록 설계된 ML 시스템은 검증하기가 훨씬 더 어렵다. 도로 상태나 기압계 측정값까지 고려한다면 어떨까? 또는 다른, 덜 직관적인 입력을 고려한다면? 기대 행위가 코너에 접근할 때 속도를 줄이는 것처럼 잘 정의된 것이 아니라, 제공된 테스트 및 학습 데이터셋에 있는 운전자들의 속도를 단순히 모방하는 것이라면 어떨까? 실제 자율주행 자동차는 단순한 ML 모델보다 훨씬 더 복잡하며, 이는 다음 문제로 이어진다.

문제 4. 시스템 복잡성

AI 시스템은 매우 복잡할 수 있다. 예를 들어 딥러닝 시스템을 생각해 보자. 딥러닝 시스템은 여러 계층의 신경망으로 구성될 수 있으며, 각 신경망은 학습 데이터를 기반으로 만들어진다.

이들의 내부 구조는 매우 복잡하지만, 중요한 점은 인간이 이해할 수 있는 방식으로 기능을 표현하지 않는다는 것이다. 전통적인 시스템은 기술자가 이해할 수 있는 코드로 표현된 논리를 포함하고 있으며, 본질적으로 사람이 정의하고 일반적으로 요구사항, 기능 또는 입력·출력 데이터와 어떤 방식으로든 관련된 클래스와 메서드로 논리적으로 구성되기 때문에 더 이해하기 쉽다.

ML 모델 자체의 복잡성뿐만 아니라 AI 시스템은 흔히 여러 모델을 결합하거나, 모델과 비 ML AI 컴포넌트를 결합한다. 이러한 모델의 상호운용성은 중요하지만 이해하기 어려울 수 있다. 확률적 불완전성을 또 다른 확률적 불완전성 안에 감싸면 분석하고 이해하기 어려운 품질 문제가 발생한다.

이러한 복잡성은 시스템이 특정 의사결정에 어떻게 도달했는지에 대한 투명성 부족으로도 이어지며, 책임성과 설명 가능성을 감소시킨다. 이는 진정으로 신뢰할 수 있는 AI에 대한 일부 우려를 더욱 키운다.

문제 5. 자기 최적화

문제 목록에 추가할 또 다른 문제는 시스템이 스스로를 어떻게 최적화하는지 이해하는 것이다. 시스템을 테스트하는 것이 시스템을 어떻게 변화시킬까? 무엇이 변경되었는지 어떻게 이해할 수 있을까? 시스템이 적응함에 따라 품질 역시 진화하는 것이 된다.

어떤 ML 시스템은 한 번 학습한 뒤 배포하고 다시 다루지 않는다. 어떤 시스템은 그 과정을 자동화하여 예를 들어 매일 주기적으로 재학습한다. 또 다른 시스템은 자기학습 기법을 사용한다. 한 예가 강화학습으로, 시스템이 수행하는 모든 동작이 ‘벌(punishment)’ 또는 ‘보상(reward)’으로 이어지고 시스템은 얻을 수 있는 보상을 최대화하도록 스스로 적응한다.

강화학습은 환경과의 순차적 상호작용을 통해 정책 함수 또는 가치 함수를 최적화하기 위해 보상 함수를 활용하는 것이다(ISO/IEC, 2021a).

시스템을 완전히 통제할 수 있고 어떤 상호작용을 최적화에 사용할지 명세할 수 있다면 자기 최적화를 통제하는 문제는 쉽게 해결할 수 있을지도 모른다. 알려진 상태로 복원하는 것도 쉬울 수 있다. 그러나 조직들은 점점 더 제3자가 서비스 형태로 제공하는 AI를 사용하고 있다.

나는 클라우드에서 제공되는 자연어 처리 시스템을 테스트하면서 처음으로 이를 경험했다. 처음에는 자동화된 테스트에서 어느 정도 변동성이 나타났고 테스트가 항상 통과하지는 않았다. 제3자 시스템을 변경할 수는 없었지만 시간이 지나면서 테스트가 안정되기 시작했고, 이후에는 항상 제대로 작동했다. 잠시 동안은 만족했지만, 곧 입력을 조금만 바꿔도 잘못된 응답이 발생한다는 것을 알아차리기 시작했다. 제3자 공급자가 사용하던 모델이 내가 제공한 테스트 입력에 지나치게 최적화되어 일반 사용자에게는 정확도가 낮아지고 있었던 것이다.

문제 6. 인간의 능력

시스템이 인간의 능력을 모방하도록 의도되었다면 어떨까? 그러한 인간의 능력을 어떻게 명세할 수 있을까? 친절하거나 윤리적인 시스템에 대한 품질 요구사항 목록을 작성해 보라. 몇 가지 아이디어는 떠올릴 수 있겠지만, 그 목록을 완전하다고 할 수 있을까? 가능성은 낮다. 인간의 행위는 복잡하며 요구사항으로 표현하기 어렵다.

인간의 뇌는 극도로 강력하고 복잡한 시스템이며, 사회적 행위도 마찬가지이다. 윤리와 같이 AI 시스템이 보여주기 원하는 인간 행위의 특성은 본질적으로 철학적이며 원하는 기능이라는 관점에서 명세하기가 매우 어렵다.

Google의 자율주행 자동차 활동을 창립한 Sebastian Thrun은 한때 이렇게 말했다. ‘아무도 이런 식으로 표현하지 않지만, 나는 인공지능이 거의 인문학의 한 분야라고 생각한다. 그것은 실제로 인간 지능과 인간 인지를 이해하려는 시도이다’(Thrun, 2018).

현재 세대의 AI는 ‘규칙’을 명시적으로 정의하거나 매우 많은 사례를 제공해야 한다. AI는 이러한 규칙과 사례의 범위를 벗어나 일반화하는 데 매우 취약한 것으로 잘 알려져 있다.

원문 PDF 48 페이지

이 때문에 인간의 행위 같은 일반적인 개념을 인위적으로 모방하려는 시도는 매우 어렵다.

명세하기 가장 어려운 능력 중 하나는 윤리적 행위이다. 윤리 이론 자체에도 서로 경쟁하는 여러 원칙 집합이 존재한다. 현실 세계에서 생명과 자유에 심각한 결과를 초래할 수 있는 AI 시스템에서는 윤리적 의사결정이 중요하지만 이를 명세하기는 어렵다.

문제 7. 편향

AI가 직면한 가장 큰 문제 중 하나는 시스템이 보통 인간이 제공한 지식으로부터 학습하고, 그 데이터 또는 심지어 요구사항에도 대개 편향이 있다는 점이다. 인간의 인지 편향, 사회적·역사적 편향, 데이터 편향 및 통계적 편향은 모두 조금씩 다르며 AI에서 매우 큰 연구 분야를 이룬다. 이 책은 이 주제만을 다루는 장 하나를 별도로 할애하고 있다.

문제 8. 드리프트

또 다른 문제는 현실 세계가 끊임없이 변한다는 것이다. 이 문제를 개념 드리프트라고 하며, 현실 세계에서 입력과 출력 사이의 상관관계가 시간에 따라 변화하는 것을 의미한다. 일부 시스템은 이에 맞추어 조정하고 일부 시스템은 그렇지 않다.

개념 드리프트는 입력과 출력 사이의 실제 상관관계가 시간에 따라 변화하는 것이다(Google, 2021).

때로는 현실 세계의 상관관계는 그대로 유지되지만 관찰되는 데이터가 변한다. 이를 데이터 드리프트라고 한다.

드리프트의 좋은 예는 사기성 신용카드 거래를 탐지하기 위해 사용하는 ML이다. 고객의 습관이 변하고 사기범도 공격 전략을 바꾼다. 이로 인해 시스템에서 드리프트가 발생한다(Dal Pozzolo et al., 2015).

이는 품질이 움직이는 목표라는 의미이다. COVID-19 팬데믹이 시작되기 전과 동일하게 사람들이 행동할 것이라고 계속 예측했던 AI 시스템이 얼마나 많았을지 상상해 보라. 인간의 행위가 더 이상 과거의 행위를 대표하지 않게 되면서, 하룻밤 사이에 예측은 무효가 되었을 것이다.

문제 9. 자동화 편향

마지막 문제는 자동화 편향 또는 안일함 편향(complacency bias)이라고 불리는 다른 종류의 편향이다. AI 시스템을 지나치게 신뢰하여 인간이 내리는 의사결정의 품질이 저하된다면 어떻게 될까?

자동화 편향은 인간이 자동화된 의사결정 시스템의 제안을 선호하고, 자동화를 사용하지 않고 만들어진 상반된 정보가 올바르더라도 이를 무시하는 경향이다(ISO/IEC, 2021b).

이는 인간이 의사결정 루프에 포함되는 모든 시스템에 영향을 미친다. 널리 알려진 사례는 2018 년 자율주행 자동차와 관련된 사고로 보행자가 비극적으로 사망한 사건이다(Krisher, 2019). 자동차는 보행자를 올바르게 식별하지 못했고, 인간 운전자는 시스템을 너무 많이 신뢰하게 되어 텔레비전 프로그램을 보고 있었다.

덜 널리 알려진 사례로, 현재 많은 기업은 스캔한 문서에서 데이터를 추출하거나 데이터 항목을 계산하여 작업자가 확인할 수 있도록 표시하는 시스템을 사용한다. 이러한 시스템은 일반적으로 인간 의사결정의 품질을 낮추는 결과를 낳는다.

AI 품질 측정을 위한 모델

물론 품질은 단 하나의 메트릭으로 정량화할 수 없다. 품질 요구사항을 명세하고 평가할 때 사용할 수 있는 특성과 용어를 정의해야 한다.

ISO/IEC 25010(ISO/IEC, 2011)은 SQuaRE 표준 시리즈의 일부로서 전통적인 소프트웨어 품질 모델을 사용하는 품질 관리에 자주 활용되는 국제 표준이다. 전문가와 실무자들은 ISO/IEC 25010 을 검토했으며(Kuwajima and Ishikawa, 2019), AI 시스템의 품질을 충분히 설명하려면 이를 보강해야 한다고 본다. ISO/IEC 는 이를 확장하여 새로운 모델을 만들 계획이며(ISO/IEC, 2021c), 이는 2023 년에 발행될 가능성이 있다.

ISO/IEC 25010 에서 정의한 제품 품질 특성은 이 장의 나머지 부분에서 논의하며, 궁극적으로 AI 를 완전히 포괄하려면 어떻게 확장할 수 있을지에 대한 논의도 함께 다룬다.

기능 적합성

기능 적합성은 품질 관리에서 가장 많은 초점이 맞춰지는 영역이지만, 그것이 가장 중요한 품질 특성인지는 맥락에 따라 달라진다. 여기에는 완전성, 정확성 및 적절성 같은 하위 특성이 포함된다. 이는 시스템이 필요한 특성과 기능을 갖추고 있는지를 측정한다.

AI 시스템의 정확성과 완전성을 측정하기는 어려울 수 있다. ML을 사용하는 경우에는 통계적 접근법을 취해야 한다. 전통적인 시스템은 일반적으로 작동하거나 작동하지 않는 것으로 구분되는 반면, AI 시스템에서는 시스템이 얼마나 자주 틀리는지를 고려해야 한다.

테스팅에 사용한 데이터가 학습에 사용한 데이터셋과 동일한 통계적 분포를 가진다면 AI 시스템은 올바르게 작동할 것으로 기대할 수 있다. 문제는 시스템이 새롭고 이전에 보지 못한 데이터를 경험할 때 발생한다.

분류 문제는 AI의 일반적인 응용 분야이다. 이러한 문제는 어떤 것이 특정 범주에 속하는지 결정하는 것과 관련된다. 흔한 예는 사진이 고양이인지 판단하는 이미지 인식 모델이다. 이러한 시스템의 기능 정확성은 일반적으로 제 1종 오류와 제 2종 오류로 측정한다. 제 1종 오류는 ‘거짓 양성(false positive)’으로, 모델이 기린 사진을 고양이라고 잘못 판단하는 경우이다. 제 2종 오류는 ‘거짓 음성(false negative)’으로, 모델이 고양이 사진을 고양이가 아니라고 잘못 판단하는 경우이다.

원문 PDF 51 페이지

분류는 주어진 입력에 대해 출력 클래스를 예측하는 ML 기능이다(ISO/IEC, 2020).

회귀 문제는 모델의 출력이 숫자인 경우이다. 숫자의 정확성을 측정하려면 평균 오차를 측정하는 것과 같은 다른 측정 방법이 필요하다. 평균 오차는 올바른 숫자와 모델이 계산한 숫자 사이 거리의 평균이며, 평균절대오차(mean absolute error)라고 한다. 그러나 평균절대오차만으로는 성능의 방향성을 알 수 없기 때문에 사용할 수 있는 다른 측정값도 많이 있다.

회귀는 연속적인 값(일반적으로 부동소수점 값)을 출력하는 ML 기능이다.

기능 정확성의 맥락에서 매우 중요한 주제는 편향이다. 편향은 복잡하며 여러 의미로 사용되는 용어로, 서로 다른 사람들이 다르게 사용한다. 데이터 과학자들은 편향을 주어진 맥락에서 양수 또는 음수가 될 수 있는 통계적 개념으로 설명하는 경우가 많지만, 철학자, 정책 입안자 및 윤리학자는 편향을 결과의 불공정성으로 이어지는 것으로 생각할 수 있다. AI 시스템에서는 둘 다 해당한다. 편향은 데이터와 알고리즘에 관련되지만 인지적인 것이기도 하다. 시스템을 만드는 팀에도 인지 편향이 존재할 수 있다. 예를 들어 팀이 모델의 기능을 결정할 때 다른 사람이나 집단에 대한 고정관념을 사용할 수 있다. 인간이 가진 믿음이 정확하지 않을 수 있지만, 개발자가 그 믿음을 확인하도록 모델을 최적화하면 모델 개발 과정에서 그 믿음이 전파될 수 있다.

모델이 편향될 수 있는 또 다른 방식은 모델을 학습시키는 데 사용하는 데이터의 출처를 통해서이다. 예를 들어 인터넷에는 다른 사람들의 사진보다 백인 서구인의 사진이 몇 배나 더 많다.

원문 PDF 52 페이지

따라서 인터넷에서 가져온 사람 사진을 기반으로 모델을 학습하면 백인 서구인과 관련된 모든 기능에서 더 정확할 것이다. 고품질 얼굴 인식을 진지하게 개발하는 기업은 이를 보정하기 위해 상당한 노력을 투자해야 한다.

편향은 일반적인 정확성을 측정하는 것과 동일한 방식으로 측정할 수 있는 정확성 문제이지만, 결과를 특정 집단(인구통계학적 집단 또는 그 밖의 집단)별로 묶으면 전체 결과와 해당 집단의 결과를 비교할 수 있다. 순위가 매겨진 출력을 평가할 때는 다른 측정값이 필요하지만, 분석 대상 집단을 일반 모집단과 비교한다는 원칙은 동일하다.

맥락 커버리지와 완전성

전통적인 시스템에서는 환경과 예상 입력 및 출력의 범위가 더 제한적이다. 예를 들어 단순한 챗봇을 생각해 보자. 챗봇의 핵심 목적은 영어를 올바르게 처리할 수 있어야 한다는 것이다. 단일 제품에 대한 지원을 제공하는 것과 같이 제한된 맥락에서는 질문에 답하는 데 상당히 효과적일 수 있다. 그러나 챗봇이 받을 수 있는 입력의 범위는 매우 다양할 수 있으며, 개발자가 가능한 모든 입력을 예상하는 것은 불가능하다.

AI 시스템이 필요한 입력을 처리할 수 있는 정도는 관련 맥락에 대한 시스템의 커버리지 및 완전성과 관련된다. 실제로 이는 AI 연구에서 가장 크고 중요한 영역 중 하나이다. 한 맥락에서 잘 작동하는 시스템을 완전히 다른 맥락으로 옮기면서도 계속 올바르게 기능하게 하려면 어떻게 해야 할까? 이것이 오늘날 사용되는 ‘협의 AI(narrow AI)’와 미래에 기대되는 범용 인공지능(artificial general intelligence)의 차이이다.

범용 인공지능(또는 강한 AI)은 일반적으로 AI가 인류와 동등한 지능을 얻을 수 있는 지점을 의미하는 것으로 이해된다.

원문 PDF 53 페이지

맥락 커버리지는 이전에 보지 못한 데이터셋 또는 입력에서도 성능을 유지하는 능력이라고 설명할 수도 있다. 이것과 편향 사이에는 복잡한 관계가 있다. 서로 다른 맥락에서 시스템의 유연성이 클수록 편향의 위험도 커진다.

유연성과 적응성

적응성은 ISO/IEC 25010에서 ‘제품 또는 시스템이 서로 다르거나 진화하는 하드웨어, 소프트웨어 또는 기타 운영·사용 환경에 효과적이고 효율적으로 적응할 수 있는 정도’로 정의된 제품 품질 하위 특성이다(ISO/IEC, 2011). 현재는 이식성 특성의 일부이지만, AI 맥락에서 이를 설명하는 방식은 보다 기능적이다. 전통적으로 이는 시스템이 작동하는 환경, 즉 운영체제, 지원 소프트웨어 환경 또는 하드웨어와 관련된 것으로 이해된다. 그러나 AI 시스템은 강화 학습을 통해 실시간으로 변경될 수 있고, 새로운 데이터로 모델을 재학습한 뒤에도 변경될 수 있다.

AI 시스템의 적응성은 관찰 데이터의 변화에 따라 시스템이 다르게 반응하기까지 걸리는 시간 또는 시스템을 얼마나 쉽게 재학습할 수 있는지로도 측정할 수 있다.

강건성

AI가 작동하는 환경은 입력과 출력 사이 상관관계의 드리프트 등을 통해 자연스럽게 변할 수 있다. 악의적인 활동으로 인해 변할 수도 있다. 강건성은 환경이 변화하더라도 AI 시스템이 성능을 유지하는 정도를 설명한다. 이는 맥락 완전성과는 다르지만, 어느 정도는 ‘맥락 커버리지’라는 용어에 포함된다. AI 시스템을 특히 악의적인 공격에 대해 강건하게 만들려면 매우 어려운 문제들을 해결해야 한다.

AI 시스템에 대한 공격은 비교적 오랜 역사를 갖고 있다. 스팸 필터를 우회하려는 시도는 초기 사례이다(Grosse et al., 2017). 최근에는 심층 신경망이 AI 시스템에 매우 큰 복잡성을 추가하기 시작하면서 시스템을 속일 수 있는 방법을 식별하기가 매우 어려워졌다.

원문 PDF 54 페이지

컴퓨터 비전 시스템에는 조금 변경된 이미지를 입력할 수 있다. 단 몇 개의 픽셀만 변경해도 인간은 속지 않을 정도의 변화만으로 기계에는 한 사람의 사진이 다른 사람처럼 보이게 만들 수 있다(Kurakin et al., 2017).

적대적 공격은 공격자가 모델의 예측에 영향을 주기 위해 모델의 입력을 약간 교란(변경)하는 것이다(ISO/IEC, 2021a).

AI 시스템을 속이는 새로운 방법은 계속 발견되고 있다. 자연어 시스템에서도 마찬가지이며(Alzantot et al., 2018), 의미는 바꾸지 않으면서 텍스트에 단어를 추가해 AI 시스템의 결과를 바꾸는 방식 등이 있다.

투명성과 설명가능성

투명성과 설명가능성은 AI 시스템에서 매우 큰 문제이며, 그 안에는 여러 개념이 서로 얽혀 있다. 한 가지 문제는 사용자나 다른 이해관계자가 AI 시스템 출력의 이유를 이해할 수 있는 정도이다. 이를 일반적으로 설명가능성이라고 한다. 그러나 투명성은 설명가능성보다 더 넓은 주제이다. 시스템이 투명하려면 레이블, 주석 및 규칙을 포함하여 입력 데이터의 출처와 품질을 이해할 필요가 있다.

한 연구자 Creel(2020)은 투명성을 세 가지 방식으로 설명한다.

- 기능적 투명성(Functional transparency): 시스템 전체의 작동에 관한 정보를 이용할 수 있는 정도.
- 구조적 투명성(Structural transparency): 구현에 관한 정보를 이용할 수 있는 정도.
- 실행 투명성(Run transparency): 특정 사례에서 시스템이 어떻게 실행되었는지에 관한 정보를 이용할 수 있는 정도.

기능적 투명성은 이해관계자가 특정 사례에서 시스템이 어떻게 행동할지를 예측할 수 있을 정도로 시스템을 이해할 수 있음을 의미한다. 이는 전통적인 시스템에서는 가능하다.

원문 PDF 55 페이지

코드에 있는 명시적 논리를 따라갈 수 있기 때문이다. 신경망의 여러 계층과 상호작용하는 여러 AI 컴포넌트로 이루어진 복잡한 알고리즘에서는 이것이 거의 불가능해질 수 있다.

구조적 투명성은 코드, 알고리즘 및 학습 데이터를 이해함으로써 얻을 수 있다. 이러한 투명성이 있으면 학습 데이터에 통계적 분석을 수행하여 정확성과 편향에 대한 우려를 줄일 수 있다.

실행 투명성은 설명가능성과 같다. 연구자들(Arya et al., 2019)은 사용자가 본질적으로 이해할 수 있는 직접 해석 가능한 시스템과, 특정 예측에 대한 설명을 얻기 위해 별도의 활동을 수행해야 하는 시스템에는 차이가 있다고 설명했다. 또한 사용자가 시스템의 의사결정을 더 깊이 탐색하고 질의할 수 있는 상호작용형 설명가능성도 논의한다. 로보틱스 맥락에서 상정한 한 사례는 로봇에 있는 버튼을 누르면 로봇이 자신의 직전 동작을 설명하는 것이다.

어떤 방법을 사용하든, 설명가능성은 설명의 이용 가능성, 설명의 정확성, 그리고 운영자가 설명을 얻고 필요하다면 전달하는 데 걸리는 시간의 관점에서 측정할 수 있다.

사회적 및 윤리적 위험 완화

AI 시스템은 일반적으로 모든 사람을 서로 다르게 대하는 것을 목적으로 한다. 그것이 그 목적의 일부이기 때문이다. 그러나 바로 그 이유 때문에 윤리적 문제가 발생할 위험도 있다. 우리는 이미 AI 시스템의 편향과 관련된 문제를 논의했다. 이는 사회에 이미 존재하는 불공정성을 전파하면서 발생할 수도 있고, 시스템 구현으로 인해 불공정한 결과가 발생하면서 생길 수도 있다.

불공정성은 측정하기 어렵고 맥락과 문화에 따라 크게 달라진다.

위험으로부터의 자유에 관한 측정은 AI 시스템의 품질이 사용자, 조직 및 더 넓은 공동체에 대한 잠재적 위험을 완화하는 정도를 평가한다. 전통적인 품질 모델은 건강, 안전 및 환경에 더 초점을 맞춘다. 그러나 AI 시스템에서는 사회 구성원의 권리와 자유에 대한 위험도 고려할 필요가 있다.

예를 들어 건강과 안전에 대한 측정은 일반적으로 보고된 부상이나

원문 PDF 56 페이지

위험과 관련된다. AI 시스템에서는 공개적인 조사가 이루어지지 않는 한 불공정성이 보고되지 않을 수 있다. 한 사례가 미국에서 사용되는 가석방 시스템에 관한 2016 년 ProPublica 조사(Larson et al., 2016)이다. 연구자들은 수감자가 재범할 가능성이 있는지를 다양한 데이터 항목을 기반으로 판단하는 알고리즘을 분석했는데, 인종은 데이터에 포함되지 않았다. 연구 결과 흑인 범죄자는 데이터에 내재한 사회적 편향 때문에 재범할 것으로 예측될 가능성이 훨씬 높았다. 즉, 흑인 인구에서 범죄를 저지른 사람의 비율이 더 높았기 때문에 시스템도 이들이 재범할 가능성이 더 높다고 예측한 것이다. 인종을 기록하지 않았음에도 우편번호 같은 다른 변수는 흔히 인종과 상관관계를 가진다.

윤리적 위험 완화를 고려해야 하는 방식이 편향뿐인 것은 아니다. 매우 널리 논의되는 한 가지 주제가 ‘트롤리 문제(trolley problem)’이다. 이는 자율주행 자동차에 적용할 수 있는 유명한 사고실험이다(Wikipedia, 2020). 자동차가 충돌을 피할 수 없으며 어떤 행동을 취하더라도 사망자가 발생할 것이라고 판단한다면 누가 죽어야 하는가? 대상에는 운전자(그리고 시스템의 소유자)와 서로 다른 여러 보행자가 포함될 수 있다. 시스템은 어떻게 의사결정을 내려야 하는가? 이 맥락에서 무엇이 공정한가?

제어가능성

시스템을 제어할 수 있는 정도는 새로운 개념이 아니며(PK et al., 2021), 일반적으로 시스템의 기능 특성이다. 그럼에도 AI 시스템은 인간의 개입이나 제어 없이 자율적으로 작동할 수 있는 능력을 점점 더 갖추고 있다. 인간의 상호작용이 선택 사항이 되거나 불가능해진다면, 인간 운영자 또는 사용자 입장에서 AI 시스템을 얼마나 제어할 수 있는지를 고려해야 한다.

제어가능성은 제어 기능(예: 로봇 뒤쪽의 버튼)을 통해 시스템을 특정 초기 상태에서 다른 상태로 이동시킬 수 있는 능력일 수 있다. 이 과정의 품질은 운영자가 수행해야 하는 단계의 수, 결과의 신뢰도, 그리고 시스템이 상태 사이를 이동하는 데 걸리는 시간과 관련될 수 있다.

사용 중 품질

ISO/IEC 25010은 제품 품질 모델과 사용 중 품질 모델이라는 두 가지 모델을 정의한다. 첫 번째는 일반적으로 출시 시점의 시스템 품질로 간주되고,

원문 PDF 57 페이지

두 번째는 실제 사용 중인 시스템의 품질이다.

이 장에서 논의한 일부 문제는 시스템의 제품 품질과 사용 중 품질 모두에 걸쳐 있다. 이는 앞서 맥락 커버리지를 논의할 때 언급한 문제 때문이다. 예를 들어 한 국가의 얼굴 데이터로 학습한 시스템을 다른 국가에 배포하면 결과가 크게 달라질 수 있다.

AI 품질 규제

품질은 일반적으로 시스템에서 규제되는 측면이 아니다. 다만 특정 분야, 대개 안전이 중요한 분야에서는 예외이다. 그러나 사람들에 관한 의사결정을 내리는 시스템에서 AI 사용이 증가하면서, 품질과 관련하여 조식을 법적으로 제한하는 여러 규제가 이미 시행되었거나 진행 중이다.

이들 중 일부는 알고리즘에서 개인정보를 사용하는 것과 구체적으로 관련된다(Chaudhuri et al., 2018).

일반개인정보보호규정(GDPR)

GDPR(European Parliament, 2016)은 일부 자동화된 의사결정을 수행하는 시스템에서 개인정보를 사용함으로써 발생하는 위험을 완화하도록 요구한다. 일부 사람들은 이를 의외라고 생각하지만, 데이터 보호의 핵심 원칙 중 하나는 사람들에 대해 보유한 데이터가 실제로 정확함을 보장하는 것이다. 예를 들어 사용자를 서로 다른 유형으로 군집화하기 위해 AI를 사용해 개인정보를 처리하면, 그 추론 결과 역시 개인정보가 될 수 있다. 데이터 보호에서의 정확성은 AI 시스템의 출력에도 관련된다. 예를 들어 AI 시스템이 사람들에 대해 어떤 종류든 자동화된 의사결정을 내리는 경우이다.

GDPR 제 22 조의 첫 문장은 다음과 같다. ‘정보주체는 프로파일링을 포함하여 오직 자동화된 처리에만 근거하고 자신에게 법적 효력을 미치거나 이와 유사하게 자신에게 중대한 영향을 미치는 결정의 대상이 되지 않을 권리를 가진다.’ 이어 몇 가지 예외와 사용할 수 있는 몇 가지 메커니즘을 열거하지만, 해당 조항을 준수하는 핵심 요소는 정확성의 원칙을 유지하는 것이다.

원문 PDF 58 페이지

제안된 EU AI 규정

GDPR은 자동화된 의사결정에 대해 일부 규제를 제공하지만 AI를 구체적으로 언급하지는 않는다. 더 나은 규제가 필요하다는 신호가 한동안 제시된 뒤, 이 책을 집필할 당시 유럽의회는 제안된 EU AI 규정(European Parliament, 2021)을 검토하고 있었다. 이것이 통과되면 광범위한 입법 변화가 될 것이다. 여러 해 동안 테스트와 품질 관리에 영향을 미친 분야 횡단적 규제 중 가장 중요한 규제가 될 가능성이 높다.

규정의 세부사항은 진화하겠지만, 초기 문안은 소셜 스코어링 및 잠재의식적 조작 같은 특정 AI 사용을 금지한다. 고용 관련 시스템과 같은 다른 사용 사례는 고위험으로 간주하고, 조직에 기술적 품질 관리 시스템을 수립할 의무를 부과한다. 이 절에서는 그러한 시스템의 범위와 예상 요구사항을 살펴본다.

첫째, AI의 정의는 그 범위가 매우 넓다는 점에서 주목할 만하다. ML만을 규제하는 것이 아니라 더 많은 기호적 AI 기법을 규제하며, 놀랍게도 모든 통계 기법까지 포함한다. 이는 평균값을 사용하는 스프레드시트조차 기술적 범위 안에 들어갈 가능성이 있다는 뜻일 수 있다. 많은 사람은 이것이 지나치게 넓다고 생각할 것이지만, 다른 사람들은 ML에 영향을 주는 품질 문제(예: 편향)가 이러한 통계 기법에서도 동일한 정도로 나타날 수 있다고 말할 것이다.

한 예로 2020년 COVID-19 팬데믹 동안 많은 학생이 시험을 치를 수 없었다. 영국 정부는 학생이 치르지 못한 시험의 결과를 예측하고 이를 최종 성적으로 사용하는 시스템(BCS, 2020b)을 도입했다. 계산은 학교의 과거 성과, 해당 학교의 특정 과목 성과, 교사가 매긴 학생 순위 및 교사가 예측한 성적을 기반으로 했다.

알고리즘은 일반적으로 교사가 정한 순위에 특히 큰 가중치를 두고 이를 해당 과목과 학교의 과거 성적 분포에 적용했다. 그 결과 알고리즘이 예측한 성적은 교사가 예측한 성적과 크게 다른 경우가 많았고, 많은 학생이 불만을 가지게 되었다. 또한 어떤 과목을 공부하는 학생 수가 적은 경우에는 교사의 순위보다 교사가 예측한 성적에 더 의존했다. 이는 특정 유형의 학교가 더 다양한 과목을 제공하고

원문 PDF 59 페이지

학급 규모가 더 작은 경향이 있었기 때문에 논란을 일으켰다. 이러한 학교는 비용이 더 비쌌고, 성적이 낮은 학생에게 계속 다니게 하기보다 학교를 떠나도록 요구하는 경향이 있었으며, 사회의 특정 집단이 다닐 가능성도 더 낮았다. 다니는 학교가 비쌀수록 알고리즘이 학생에게 더 좋은 성적을 부여한다는 인식이 널리 퍼졌다. 대중의 항의는 거셌고, 알고리즘은 폐기되었으며 관련 정부 부처의 책임자가 사임했다.

엄밀히 말하면 이는 AI 시스템의 사례가 아닐 수 있지만, 통계적 기법을 사용함으로써 이 책에서 논의하는 여러 문제가 어떻게 발생할 수 있는지를 보여주는 사례이다. 또한 신용도 평가나 채용 사용 사례와 함께 AI 규정에서 정의하는 고위험 사용 사례의 예이기도 하다. 대부분의 신용도 평가에서 통계적 기법을 사용한다고 보는 것은 합리적이며, 채용에서도 ML 사용이 시작되고 있다.

제안된 규정은 이러한 사용 사례를 금지하지 않는다. 대신 이를 관리하기 위한 규제 생태계, 수립해야 하는 통제 집합, 그리고 규정을 준수하지 않을 경우 부과되는 큰 벌금을 마련하는 데 초점을 맞춘다.

이 규정은 AI 시스템 개발자에게 시스템이 적절한 수준의 정확성, 강건성 및 보안을 갖추도록 할 의무를 부과한다. 테스트를 통해 결정되는 정확성 수준과 사용되는 구체적인 메트릭을 정해야 한다. 기능적 또는 비기능적 결함으로

발생한 것이든 실패에 대한 회복력도 갖추어야 한다. 보안과 관련해서는 데이터 포이즈닝 공격이 구체적으로 언급된다. 이는 공격자가 시스템의 예측에 영향을 주기 위해 학습 데이터 또는 기타 입력 데이터를 오염시킬 수 있는 경우이다.

통제 항목은 우리에게 가장 흥미로운 부분이다. 두 가지, 즉 위험의 식별 및 관리와 반드시 수행해야 하는 품질 관리 및 테스트 활동에 초점을 맞추기 때문이다.

규정은 대표성 있는 테스트 데이터, 지속적인 운영 모니터링, 그리고 테스트와 관련된 상당한 양의 문서화 등을 요구한다. 테스트에 구체적인 요구사항을 부과하며, 어떤 경우에는 학습 및 테스트 데이터, 코드 및 테스트 결과에 대한 독립적인 감사까지 요구할 것이다. 또한 정확성에 관한 설명(아마 테스트 결과에서 얻은 것)을 제 3자 시스템의 배포자에게 제공하도록 요구한다. 이러한 모든 테스트의 결과로 CE 마크가 부여된다. CE 마크는 화장품에서 가구에 이르기까지 유럽에서 폭넓게 사용되는 품질 표시로, 제품의 판매, 수입 및 수출을 허용한다.

원문 PDF 60 페이지

EU 밖에서 AI와 관련된 일을 하고 있더라도 이 규정은 영향을 미칠 것이다. GDPR과 마찬가지로 범위가 ‘역외적(extraterritorial)’이기 때문이다. 즉 EU에서 개발된 시스템뿐 아니라 EU 시민에게 영향을 미칠 수 있는 모든 시스템에 적용된다.

법률 초안의 문제는 요구되는 많은 통제 및 테스트 목표를 위한 세부 기법이 아직 초기 단계에 있다는 것이다. 모범 사례는 계속 진화하고 있으며 국제 표준도 아직 상당한 시간이 필요하다. 예를 들어 신경망에서 코드 커버리지 대신 뉴런 커버리지를 측정할 수 있다는 것은 알고 있지만, 이 기법의 효과에 관한 연구는 아직 초기 단계이다.

국제 소프트웨어 테스트 자격위원회(ISTQB)는 AI Testing을 위한 전문 인증 체계를 발표했으며, 이는 큰 진전이 될 것이다. 그러나 EU의 관점이 앞으로 수개월과 수년에 걸쳐 발전함에 따라 이를 반영하도록 업데이트해야 할 가능성이 높다. 우리에게 어느 정도 시간이 있다는 점은 주목할 만하다. GDPR과 마찬가지로 세부사항을 정리하는 동안 긴 구현 기간이 있을 것으로 예상된다. 최종 시행은 2023년 또는 2024년에 이루어질 가능성이 높다.

요약

요약하면, AI 시스템에 고유하게 적용되는 다양한 품질 고려사항이 있다. 용어와 모델은 여전히 진화하고 있으며, 규제 압력으로 인해 향후 몇 년 동안 AI와 품질에 대한 관심이 점점 더 높아질 것으로 보인다. 이 여러 영역을 가로지르는 가장 중요한 우려 중 하나는 편향이며, 다음 장에서 이를 더 깊이 살펴본다.

원문 PDF 61 페이지

3 품질과 편향

James Harold Davenport

시스템, 특히 AI 시스템이나 AI 를 포함한 시스템이 ‘편향되어 있다’ 또는 ‘불공정하다’고 비난하는 것은 흔한 일이다. 이 두 단어는 흔히 동의어처럼 사용되지만, 여기서는 국제표준화기구(ISO)의 패러다임을 따라 둘을 구분한다. ‘편향(bias)’은 정확하게 정의할 수 있는 통계적 현상이며, ‘공정성(fairness)’은 예를 들어 ‘확립된 규범’에 따라 해석이 달라지는 윤리적 개념이므로 공정성에 대한 인식은 달라질 수 있다. 보다 정확히 말하면, 이 문제를 검토하는 표준 개발 조직 중 하나인 ISO/IEC SC 42 에서 진행 중인 작업을 바탕으로 다음 정의를 채택한다.

‘편향(Bias)’은 특정 객체, 사람 또는 집단을 다른 대상과 비교하여 체계적으로 다르게 대우하는 것이다(ISO/IEC, 2021a, 정의 3.2.2; ISO/IEC, 2021b).

‘공정성(Fairness)’은 확립된 사실, 신념 및 규범을 존중하고 편애나 부당한 차별에 의해 결정되거나 영향을 받지 않는 대우, 행위 또는 결과이다(ISO/IEC, 2022).

원문 PDF 62 페이지

이는 중요한 구분이지만 마케팅과 정치의 소음 속에서 흔히 잊힌다. 예를 들어 Facebook 에는 ‘Fairness Flow’라는 도구가 있으며, ‘모두에게 더 잘 작동하는 AI 를 구축하는 데 Fairness Flow 를 사용하는 방법’이라고 크게 홍보한다(Kloumann and Tanner, 2021). 그러나 기술적 설명은 훨씬 더 절제되어 있다(원한다면 ‘현실적’이라고 할 수도 있다).

“Fairness Flow 는 일부 유형의 AI 모델과 레이블이 서로 다른 집단에서 어떻게 수행되는지 우리 팀이 분석할 수 있게 하는 기술 툴킷이다. Fairness Flow 는 진단 도구이므로 공정성에 관한 우려를 스스로 해결할 수는 없다. 이를 위해서는 윤리학자와 다른 이해관계자의 의견뿐 아니라 맥락별 연구가 필요하다. 그러나 Fairness Flow 는 제품 내 일부 시스템이 사용자 집단별로 어떻게 수행되는지를 이해하는 데 필요한 통찰을 제공할 수 있다.” (Kloumann and Tanner, 2021)

여기에는 흥미로운 대비가 있다. 이 메시지는 이해관계자를 언급하지만, 이어서 더 넓은 이해관계자가 아니라 사용자 집단별 성능을 묻는다. 이 장의 뒤쪽 ‘타겟 광고는 편향되어 있다’ 절을 참조하라. 사실 어느 쪽도 완전히 올바른 설명은 아니다. 자동화 채용 도구의 성별 편향에 관해 Fairness Flow 에 질문한다면, 우리는 아마 사용자(인사 담당자)에 대한 성별 편향이 아니라 데이터 주체(채용 대상으로 고려되는 사람)에 대한 성별 편향에 관심이 있을 것이다.

이 장에서는 편향의 정의를 뒷받침하는 몇 가지 기본 통계와, 일상생활에서 의도된 편향과 의도하지 않은 편향의 몇 가지 사례를 살펴본다. 한 예는 성별에 따라 편향된 자동차 보험 가격 책정이다. 원시 데이터는 편향을 보여준다(남성이 사고를 더 많이 내고 더 심한 사고를 낸다). 그다음 질문은 가격을 책정할 때 이 차이를 고려하는 것이 공정한가 하는 것이다.

편향 정의의 결과

사용자 집단별 성능을 측정하면 여러 결과가 따른다.

(i) 살펴볼 의사결정의 집합이 있어야 한다. 하나의 개별 의사결정만을 두고 ‘그 결정은 편향되었다’고 말할 수는 없다.

(ii) 적어도 두 개의 객체, 사람 또는 집단 집합이 있어야 하며, 그래야 ‘이러한 대우는 B 보다 A 에게 유리하게 편향되었다’고 말할 수 있다.

원문 PDF 63 페이지

(iii) 각 집합에 어떤 차이의 대우가 적용되었는지 알아야 한다.

(iv) 편향되지 않은, 또는 적어도 덜 편향된 의사결정을 내리는 것이 가능할 정도로 집합의 크기가 충분히 커야 한다. 예를 들어 많은 사람 중 한 명을 선택하는 경우(예: 채용 심사위원회)에는 선택된 개인이 속한 모든 집단에 유리하게 의사결정이 편향된다. 이는 결과 (i)의 사례이기도 하다. 마찬가지로 1,000 명 중 10 명을 선택한다면, 그 1,000 명의 모집단에서 구성원이 100 명 미만인 모든 범주에 대해서는 해당 범주에서 사람을 선택했는지 여부에 따라 의사결정이 유리하게 또는 불리하게 편향될 수밖에 없다.

실제로는 집합과 편향의 크기가 충분히 커서 관찰된 편향이 우연히 발생했을 가능성이 낮아야 한다. A 와 B 가 같은 수로 구성된 모집단이 있고, A 의 구성원과 B 의 구성원을 구분하지 않는 선택 시스템, 즉 A 와 B 사이에서 편향되지 않은 시스템이 있다고 가정해 보자. 통계학자는 이 시스템에서 A 와 B 사이에 차이가 없다는 귀무가설(null hypothesis)을 세울 것이다. 이제 모집단 크기가 서로 다를 때 시스템이 서로 다른 수의 구성원을 선택할 확률을 살펴보자.

- 4 명의 모집단에서 2 명 선택. A 두 명을 모두 선택할 확률은 $1/6$ 이다. 처음에 A 를 뽑을 확률은 $1/2$ 이고, 이후 A 한 명과 B 두 명이 남으므로 두 번째 A 를 뽑을 확률은 $1/3$ 이기 때문이다. B 두 명을 뽑을 확률도 동일하며, 각 집단에서 한 명씩(AB 또는 BA) 뽑을 확률은 $2/3$ 이다.
- 6 명의 모집단에서 3 명 선택. AAA 의 확률은 $1/20$ 이고 BBB 도 동일하며, AAB 와 ABB 의 확률은 각각 $9/20$ 이다.
- 8 명의 모집단에서 4 명 선택. AAAA(또는 BBBB)의 확률은 $1/70$, AAAB(또는 BBBA)는 $8/35$, AABB 는 $18/35$ 이다.
- 10 명의 모집단에서 5 명 선택. AAAAA(또는BBBBB)의 확률은 252 분의 1 이고, AAAAB(또는 BBBBA)는 252 분의 25, AAABB(또는 BBBA)는 252 분의 100 이다.
- 20 명의 모집단에서 10 명 선택. 이제 확률은

원문 PDF 64 페이지

$1/184,756 \approx 0.0000054$, $25/46,189 \approx 0.00054$, $2,025/184,756 \approx 0.011$, $3,600/46,189 \approx 0.078$, $11,025/46,189 \approx 0.24$, 정확히 반반으로 나뉘는 경우 $15,876/46,189 \approx 0.34$ 이며, 이후에는 반대 방향으로 동일하다.

- 100 명의 모집단에서 50 명 선택. 정확히 절반씩 나눌 확률은 약 5%이며 다른 모든 확률은 이보다 낮다. 따라서 편향되지 않은 시스템조차 완벽하게 편향되지 않은 결과를 낼 것으로 기대할 수 없다.

통계학자들은 이 문제에 익숙하며, 정확히 어떤 값이 나올 확률이 얼마인지가 아니라 적어도(또는 많아야) 이 값이 나올 확률이 얼마인지를 묻는 방식으로 접근한다. 보다 형식적으로, T 를 우리가 관찰하는 값(여기에서는 선택된 A 의 수)이라고 하자. 그러면 임계값 t 에 대해 ‘귀무가설 하에서 $T > t$ 일 확률은 얼마인가?’라고 묻는다(‘ A 가 너무 많이 선택되고 있는가?’라는 질문에 관심이 있다고 가정한다). 이 확률이 매우 작은 사건 t 를 관찰했다면 다음 중 하나를 결론으로 생각할 수 있다.

또는: 귀무가설이 거짓이다(여기에서는 선택 시스템이 A 와 B 를 구분한다).

또는: 우리가 가능성이 낮은 사건을 관찰했다.

통계학자는 어떤 유의수준 α 를 선택하고, t 를 관찰하며, 귀무가설 하에서 $T > t$ 일 확률 p (‘적어도 이만큼 놀라운 결과를 보게 될 확률은 얼마인가?’)를 계산하여 p 와 α 를 비교한다. p 가 더 작으면 통계학에서는 일반적으로 ‘귀무가설을 기각한다’고 말한다. 그러나 이는 귀무가설이 거짓이라는 뜻은 아니다. 언제든지 가능성이 낮은 사건을 관찰했을 수도 있기 때문이다.

이제 핵심 질문에 도달한다. α 는 얼마여야 하는가? 과학에서 흔히 사용하는 값은 $\alpha = 0.05$ 이지만, 이는 ‘추가 실험이 필요하다’는 결론과 함께 해석해야 한다. 20분의 1은 그렇게까지 드문 일이 아니기 때문이다. 흔히 사용하는 다른 값은 0.01 또는 0.001이지만 훨씬 더 작은 값도 사용된다. 예를 들어 힉스 보손 발견(CMS Collaboration, 2012)은 $\alpha = 1/3,500,000$ (통계 이론에서 5σ 에 해당)으로 발표되었으며, 유전체 분석에서 인정되는 표준(G. M. Clarke et al., 2011)은 5×10^{-8} (2 천만 분의 1)이다.

원문 PDF 65 페이지

여기에는 두 가지 경고를 덧붙여야 한다.

지금까지는 편향의 한 방향(A 보다 B 가 아니라 A 를 선택하는 것)에 대해 이야기했다. 그러나 가능한 방향은 여러 개일 수 있고 이분법적이지 않을 수도 있다. 예를 들어 영국의 Equality Act 2010은 9개의 보호 특성을 정의하는데, 일부는 이분법적이지만 나이처럼 많은 특성은 그렇지 않다. 모두 이분법적이라고 해도, 그리고 시스템이 편향되지 않았다고 해도, $\alpha = 0.05$ 에서 특정 하나의 특성에 대해 실패할 확률은 정의상 $1/20$ 이지만 9개 중 하나에서 그 수준으로 실패할 확률은 37%이다. 따라서 단순한 테스트는 실제로 편향되지 않은 시스템을 이 특성 중 하나에 따라 편향된 시스템이라고 선언할 가능성이 상당히 높다. 즉 실험을 먼저 수행한 뒤 특성을 선택하는 경우가 그렇다. 이에 대한 간단한 해결책은 Bonferroni-Dunn 보정(Bonferroni, 1936; Dunn, 1961)으로 알려져 있으며, 서로 다른 편향 방향이 n 개라면 α 를 $\alpha^* = \alpha/n$ 으로 대체한다. 조금 더 정교한 보정은 Šidák 보정(Šidák, 1968)으로 $\alpha^* = 1 - (1 - \alpha)^{1/n}$ 을 사용한다. 하지만 실제 차이는 매우 작으며 그 차이가 중요하다면 통계학자와 상의하는 것이 좋을 것이다. 따라서 편향되지 않은 시스템을 편향되었을 가능성이 있다고 선언할 확률을 0.05로 유지하려면 9개의 편향 방향 각각에 대해 α^* 를 0.0056(Šidák 방식을 사용하면 0.0057)으로 봐야 한다.

이들은 통계적 유의성을 위한 테스트이지 실질적 유의성을 위한 테스트가 아니다. 이 구분은 특히 의학에서 중요하며, 의학에서는 효과가 실제로 중요하다는 것을 나타내기 위해 ‘임상적 유의성(clinical significance)’이라는 표현을 사용한다.

일상생활 속의 편향

우리가 살아가는 삶은 실제로 편향으로 가득하다. 어떤 것은 명시적이고 어떤 것은 암묵적이며, 어떤 것은 인간이 만들었고 어떤 것은 그렇지 않다. 인간이 만든 명시적 성별 편향의 사례는 장자 상속에서 남성을 우선하는 규칙으로, 2015년까지 영국에서는 가장 나이가 많은 남자아이가 왕위를 계승했고 남자아이가 없는 경우에만 여자아이가 왕위를 계승했다. 역사적으로 인간이 만든 이런 명시적인 성별 편향은 매우 많았으며, 많은 것이 법적으로 폐지되었지만 그 영향은 여전히 남아 있다.

원문 PDF 66 페이지

이 예를 계속하면, 남성 우선 장자 상속은 영국 군주를 무작위로 한 명 선택할 때 여왕보다 왕을 선택할 가능성이 훨씬 높다는 뜻이다(‘William and Mary’를 제외하면 16 분의 13).

자연이 만들어낸 것으로 보이는 또 다른 편향은 여성이 남성보다 더 오래 사는 경향이 있다는 것이다. 젊은 남성을 더 많이 죽게 하는 전쟁 같은 문제를 제외하더라도 이는 사실이다. 영국의 65 세 기대수명은 여성이 평균적으로 남성보다 2 년 더 살 것으로 기대할 수 있음을 보여준다(Institute and Faculty of Actuaries, 2018). 수백 년 동안 보험회사와 연금 제공자는 기대 지급액의 편향을 없애기 위해 남성과 여성에게 서로 다른 요율을 제시하는 방식으로 이를 처리했다. 그러나 이 접근법은 유럽연합에서 ‘Test-Achats 판결’(European Court of Justice, 2011)에 따라 불법으로 판결되었고, 모든 보험 요율은 성별에 따른 편향 없이 제시해야 한다. 이는 요율은 편향되지 않게 되었지만 지급액에는 이제 편향이 생긴다는 뜻이다. 여성은 남성보다 더 많은 금액을 받을 것으로 기대할 수 있다. 이는 일반적인 원칙의 사례이다. 자연으로 인해 편향이 존재한다면, 우리는 그것을 없앨 수 없고 단지 어디에서 나타나는지만 바꿀 수 있다.

많은 사회에서, 그리고 분명 영국에서는 남성, 특히 젊은 남성이 여성보다 교통사고를 더 많이 내고 더 심한 사고를 낸다. The Guardian(2004)은 사망이나 신체 상해가 발생한 사고의 94%를 남성이 일으켰다고 언급한다. 따라서 보험회사는 과거 남성 운전자에게 더 높은 보험료를 제시했다. 이 역시 현재 유럽연합에서는 불법이다. 그러나 자동차 유형이나 운전자의 직업 같은 다른 요인은 사용할 수 있다. McDonald(2015)는 영국 자동차 보험 시장의 맥락에서 이를 연구했다. 그는 해당 판결이 직접적인 성별 차별을 없애는 데 효과적이었다고 관찰했다. 하지만 영국에서는 많은 직업에 강한 성별 편향이 있다. 그의 예로 미장공(plasterer)은 96%가 남성이었고 치과 간호사(dental nurse)는 100%가 여성이었다. 실제로 그는 치과 간호사가 자동차 보험 평균보다 9% 적게, 미장공이 9% 더 많이 청구받고 있음을 확인했다.

McDonald의 결론은 신중하게 해석해야 한다.

“이 판결은 성별에 의한 직접 차별을 중단시키는 데 효과적이었던 것으로 나타났다. 그러나 남성과 여성 사이 위험 차이가 가장 큰 젊은 운전자에서는 기업이 직업을 성별의 대리변수로 사용하여 간접 차별을 하고 있다는 증거가 있으며, 여성(남성) 비율이 높은 직업의 보험 가격이 상대적으로 낮아지는(높아지는) 것으로 나타났다. 이러한 영향은 자동차 보험 시장을 넘어선다. 예를 들어 연금과 같이 이 판결의 영향을 받는 다른 시장에서도 관찰될 가능성이 있다.”

원문 PDF 67 페이지

마지막 문장은 분명 사실이며 중요한 의미를 가진다. 앞 문장들을 읽을 때는 의인화하지 않도록 주의해야 한다. 이러한 가격은 여러 보험회사의 가격 책정 엔진에 질의하는 비교 웹사이트에서 나온다. 세부적인 가격 책정 의사결정에 사람이 관여하지 않는다. 의사결정은 많은 양의 데이터 분석을 바탕으로 이루어진다. 실제로 이러한 분석 프로그램은 변경되지 않았을 수도 있으며(저자는 적어도 한 사례에서는 변경되지 않았다는 말을 들었다), 단지 입력 변수에서 ‘성별’이 제거되었을 뿐이다. 흔히 ‘직업이 성별의 대리변수이다’라고 말하지만, 이 역시 의인화된 표현이다. 실제로 일어나는 일은 통계 엔진이 단순히 상관관계를 계산하거나, 달리 말하면 ‘데이터를 따르는 것’이다.

차등 가격 책정은 본질적으로 편향되어 있다

위의 자동차 보험 사례는 많은 사람이 무시하려 하는 일반적인 사실을 보여준다. 모든 차등 가격 책정은 편향되어 있으며, 더 많은 금액을 부과받는 사람에게 불리하게 편향되어 있다. 질문은 편향되어 있는가가 아니라, 그 편향이 사회적으로 또는 법적으로 허용 가능한가이다. 성별에 따라 편향된 자동차 보험은 EU에서 오랫동안 사회적으로 허용되었고(법적으로 허용되지 않음에도 실제로는 여전히 어느 정도 그럴 가능성이 있다), 오늘날 세계 여러 지역에서는 사회적으로도 법적으로도 허용된다.

차등 가격 책정의 한 예는 기차 요금이다. 영국에서는 ‘비혼잡 시간대(off-peak)’에 여행하는지 여부에 따라 매우 다른 가격을 지불하는 데 익숙하다.¹ 예를 들어 Bath Spa에서 London까지의 왕복 요금은 집필 당시 £214.20(‘peak’)에서 £86.80(‘off-peak’), £63.00(‘super off-peak’)까지 다양하다. 절대 금액과 가격 차이의 크기에 대한 불만은 있지만, 가격 차이 자체의 존재는 대부분 문제 삼지 않는다. 항공 요금도 대체로 마찬가지이지만 더 많은 변수에 좌우되고 규제가 덜하기 때문에 그 효과를 측정하기 더 어렵다.

그러나 동일한 원칙이 휴가 비용과 학교 학기·방학 일정의 관계에 적용되면 큰 분노가 생긴다. The Telegraph(Spocchia and Morris, 2020)는 학교 방학 가격 인상을 ‘대낮의 강도질(daylight robbery)’이라고 표현하며, 이와 유사한 기사를 많이 찾을 수 있다. 실제로 인용된 가격 차이인 ‘거의 3 배’는 기차 요금의 차이보다 작지만,

원문 PDF 68 페이지

대중의 분노는 훨씬 더 크다.

이들은 모두 사전(a priori) 가격 책정의 사례이다. 고객은 얼마를 지불해야 하는지 미리 알고, 직장 업무나 학교 방학 같은 다른 제약의 영향을 받기는 하지만 다른 서비스를 선택하여 더 적은 금액을 지불할 수 있다. 사후(a posteriori) 가격 책정의 전형적인 사례는 먼저 서비스를 이용하고 나중에 가격이 결정되는 택시 요금이다. 전통적

인 미터기 택시에서는 가격이 사후 미터기 측정값(이동 거리와 대기 시간)과 여러 공개된 파라미터, 일반적으로 시간대와 요일 또는 공휴일 여부를 함수로 하여 결정된다. 그러나 Uber와 Lyft 같은 서비스에서는 가격 책정이 더 동적이며 항공 요금과 비슷하다. Pandey와 Caliskan(2020)이 이를 조사했고, Lu(2020)가 그 결과를 대중적으로 소개했다. 이들은 특히 Chicago의 차량 호출 서비스를 분석했으며 다음과 같은 결과를 제시했다.

“결과는 40세 이상 인구 비율이 낮거나, 중간 주택가격 이하의 주택 비율이 낮거나, 고등학교 졸업 이하 학력자의 비율이 낮거나, 비백인 인구 비율이 높은 지역일수록 요금이 증가함을 보여준다.”

마지막 항목은 보호 특성이므로 여기에서의 차별은 불법일 수 있다. 처음 세 특성의 지도를 보면 이 세 특성이 상당히 상관되어 있음을 알 수 있으며, 이는 합리적으로 보인다. 안타깝게도 민족성에 대한 대응 지도는 없으며 이것이 얼마나 상관되어 있는지 아는 것은 흥미로운 것이다.

단순하게 생각하면 편향되지 않았을 것이라고 기대하는 수요 기반 가격 알고리즘에서 인종 편향이 나타난 사례는 이것뿐이 아니다. Angwin et al.(2015)은 Princeton Review가 온라인 SAT 과외 서비스에 청구한 가격이 ZIP 코드에 따라 2015년 기준 \$6,600에서 \$8,400까지 달랐다고 보고한다. 그들은 다음을 발견했다.

“아시아인은 비아시아인보다 더 높은 가격을 제시받을 가능성이 거의 두 배 높다. 이러한 차이는 저소득 지역의 아시아인에게서도 유지된다. New York Queens의 Flushing에 있는 한 ZIP 코드를 생각해 보자. 이 ZIP 코드 인구의 70.5%가 아시아인이다. 미국 인구조사에 따르면 이 ZIP 코드의 가구 중위소득은 \$41,884로 대부분의 지역보다 낮지만, 그곳의 The Princeton Review 고객들은 가장 높은 가격을 제시받는다.”

처음 이 가격 차이를 발견한 학생은 핵심적인 점을 지적했다. ‘개인 한 명의 삶에는 매우 작은 영향을 미치는 일이지만 큰 집단에는 큰 영향을 줄 수 있는 일이다’(Angwin et al., 2015). 이는 중요한 지적이다. 한 민족 집단이 교육 지원의 가격 때문에 불균형적으로 배제된다면,

원문 PDF 69 페이지

교육 격차에 연쇄적인 영향이 생길 것이다.

타겟 광고는 편향되어 있다

생각해 보면 너무나 명백한 말이다. 타겟팅은 선택적으로 편향시키는 것이다. 잡지에 광고를 게재하는 것을 생각해 보자. 콤팩트 수확기 제조업체는 Interior Decorators Weekly보다 Cereals Weekly에 광고할 가능성이 높다. 이에 반대할 사람은 거의 없을 것이며, Interior Decorators Weekly 독자들보다 아마 콤팩트 수확기 광고가 많이 실리는 것을 원하지 않을 것이다. 그러나 다른 것, 예를 들어 COVID-19 백신 접종소 보조원 채용 광고라면 어느 쪽 잡지나 비슷한 확률로 광고할 것이다.

그러나 특히 Facebook과 많은 다른 업체가 실행하는 온라인 광고 타겟팅은 다르다. Cereals Weekly 구독과 같은 명시적인 선택에 의해 좌우되는 것이 아니라, 대행사(Facebook 등)의 이윤 추구에 의해 좌우된다. 이윤은 일반적으로 사람들이 광고를 클릭할 때 발생한다. 따라서 대행사는 곡물 가격 등에 관한 기사 옆에 콤팩트 수확기 광고를 배치할 가능성이 높다. 하지만 그런 기사는 많지 않으므로, 예를 들어 Facebook 픽셀(Newberry, 2019; Venkatadri

et al., 2018)로 추적한 행위가 콤파인 수확기에 관심이 있을 수 있음을 나타내는 독자에게 광고를 보여줄 가능성이 높다.

단지 콤파인 수확기 판매에만 관심이 있다면 이것은 큰 문제가 아닐 수 있다. 그러나 소셜 미디어에서 광고되는 것 중 하나는 주택이며, 미국 법률에서는 주택 분야를 인종 차별로부터 보호한다. 소셜 미디어가 타겟 광고를 허용하는 것에는 많은 문제가 있었고, 관련 주요 기업은 광고주가 그러한 명시적 타겟팅을 하지 못하도록 하는 데 동의했다 (ACLU, 2019). 그러나 Ali et al.(2019)이 발견했듯이 이것만으로 편향을 피하기에는 충분하지 않다. 콤파인 수확기 광고를 시스템이 구매 가능성이 높다고 판단한 사람에게 보여주는 동일한 메커니즘은 예를 들어 구매용 주택(임대가 아니라) 광고를 백인에게 우선적으로 보여주게 된다.

소셜 미디어에서 광고되는 또 다른 사회적으로 중요한 상품은 일자리이다. 이는 Ali et al.(2019), 더 최근에는 Imana et al.(2021)이 조사했다. 이들은 미국 법률상 ‘광고는 자격을 기준으로 타겟팅할 수 있지만 보호 범주를 기준으로 타겟팅할 수 없다’고 지적한다.

원문 PDF 70 페이지

관심 가능성을 기준으로 타겟팅하는 것이 합법인지는 명확하지 않다. 단순히 생각하면 콤파인 수확기 운전원 채용 광고도 콤파인 수확기 광고와 같은 원리를 따를 것으로 예상할 수 있으며, 잡지 광고주라면 분명 Interior Decorators Weekly 보다 Cereals Weekly 를 사용할 가능성이 높다.

그들의 방법론(Imana et al., 2021)은 ‘실제 직원의 성별 분포가 서로 다른 두 회사의 유사한 일자리에 대해 두 개의 광고를 동시에 게재’하는 것이었다. 그들의 결론은 성별에 따른 편향이 이러한 프로세스에 본질적으로 내재하지 않을 수도 있음을 보여준다는 점에서 특히 흥미롭다.

“우리는 Facebook 광고 전달에서 성별에 따른 편향을 확인했으며, 이것이 자격 차이로 정당화될 수 없음을 보여준다. LinkedIn 광고 전달에서는 편향을 발견하지 못했다.”

Princeton Review 의 가격 책정과 마찬가지로, 사회에 미치는 영향은 개인에게 미치는 영향보다 훨씬 더 크다. 성별 불균형이 계속 유지되기 때문이다.

이 모든 내용은 우리가 의도하는 편향(예: 더 안전한 운전자)이 우리가 의도하지 않는 편향(예: 성별)과 상관될 수 있으며, 그러한 편향을 허용해서는 안 될 수도 있음을 보여준다. 이 경우 원래 의도한 편향 자체를 추구하지 않거나(보험회사에서는 가능성이 낮다), 원치 않는 결과적 편향을 의도적으로 보정하는 것 중 하나를 선택해야 한다. 그러나 미국의 대출 사례에서 논의된 것처럼(Hao, 2019) 그러한 보정 자체가 불법일 수도 있고, 필요한 데이터를 수집하는 것이 불법인 경우(예: 프랑스에서 인종 데이터를 수집하는 경우)에는 불가능할 수도 있다.

이력서를 사람이 직접 처리하고 1차 선발하던 시절에는 인사부서(당시에는 ‘인적 자원’이라고 부르기 전)가 인구통계 데이터를 별도 용지에 기록하고, 이 정보는 1차 선발 심사위원회에 전달하지 않고 편향 모니터링에만 사용할 것이라고 명시하는 것이 흔했다. 이 메커니즘은 이제 낡았으며, 사람들이 종이 처리 과정에 가졌던 것과 같은 신뢰를 완전

자동화 프로세스에도 가질지는 분명하지 않다. 그러나 그 취지는 기억해야 한다. 원치 않는 편향이 존재하지 않는다고 단순히 가정할 것이 아니라 실제로 확인해야 한다.

의도하지 않은 편향

원문 PDF 71 페이지

우리는 특정 종류의 편향(자격 등)을 위해 AI 시스템을 구축하고 있을 수 있고, 그 결과로 발생하는 편향도 보정했을 수 있지만 다른 종류의 편향이 스며들 수 있다. 여기에는 많은 원인이 있다.

데이터 편향

우리가 가진 데이터는 현실 세계의 모든 상황 범위를 제대로 대표하지 못할 수 있다. 실제로 세계 전체가 데이터셋에 포함되지 않는 한 완전히 대표할 수는 없으므로, 일반적으로는 ‘대표성 있는 부분집합’을 확보하기를 기대한다. 데이터 편향에는 많은 변형이 있으며, ISO/IEC는 상세 보고서인 ISO/IEC TR 24027(ISO/IEC, 2021b)에서 현재 이를 열거하고 있다. 매우 중요한 한 가지는 샘플링 편향으로, 표본으로 추출한 데이터셋이 현실 세계의 인구통계 또는 기타 중요한 특성을 대표하지 못할 때 발생한다. 여론조사 전문가들이 잘 알고 있듯이 무작위로 선택된 사람들도 일반적으로 대표성을 갖는 것은 아니다. 선택 방법과 사람들이 질문에 답하려는 의지에 따라 달라진다.

많은 양의 데이터를 수집하는 일은 매우 비싸며, 데이터에 레이블을 붙여야 한다면 특히 그렇다. 따라서 ‘잘 알려진’ 데이터셋에 의존하려는 경향이 매우 강하다. 그러나 ‘잘 알려진’ 데이터셋이라고 해도 그 자체가 대표성이 없을 수 있다. Shankar et al.(2017)은 위치 데이터를 확보할 수 있었던 이미지(OpenImages의 22%, ImageNet은 이보다 다소 낮은 비율이며 일정한 오류 가능성도 있음)에 대해 OpenImages는 미국 출처가 32.1%, 영국 출처가 12.9%로 총 45%였고, ImageNet은 미국 45.4%, 영국 7.6%로 총 53%였다고 보고한다. 평균 순위를 기준으로 상위 14개 국가(두 데이터셋에서 상위 14개 국가는 동일함)는 표 3.1에 제시되어 있으며, 여기에 Wikipedia의 인구 순위를 추가했다. 이 14개 국가 중 인구 순위 상위 14개에 포함되는 국가는 6개뿐이다. 특히 눈에 띄게 빠진 국가는 Indonesia와 Pakistan으로, Wikipedia 표에서는 각각 4위와 5위이다.

이 논문에는 그 밖에도 유용한 사례가 많다. 연구 중 하나에서는 ‘신랑(bridgroom)’ 이미지가 올바르게 분류될 가능성을 살펴보았다. 표 3.1을 고려하면 놀랍지 않게도 미국(및 Australia)의 이미지는 Ethiopia나 Pakistan의 이미지보다 올바르게 분류될 가능성이 훨씬 높았다. 이러한 관찰과 다른 유사한 관찰은 ‘데이터셋을 위한 데이터시트(Datasheets for datasets)’ 운동으로 이어졌다(Gebru et al.,

원문 PDF 72 페이지

2021). ‘[전자 컴포넌트의 데이터시트와] 유사하게, 모든 데이터셋에 그 동기, 구성, 수집 과정, 권장 용도 등을 문서화한 데이터시트를 함께 제공할 것을 제안한다.’

다른 많은 ‘잘 알려진’ 데이터셋에도 오류가 포함되어 있다. 예를 들어 Inside Airbnb 데이터셋이 그렇다(Alsudais, 2021). 그러나 ImageNet의 문제는 ImageNet이 시작점으로 널리 사용된다는 점 때문에 특히 알아차리기 어렵고 위험하다. ‘먼저 ImageNet에서 이미지 분류를 수행하도록 CNN을 학습시키고(즉, 사전학습), 그다음 이러한 특성

을 새로운 대상 작업에 맞게 적응시키는(즉, 미세조정) 방식은 광범위한 컴퓨터 비전 문제를 해결하는 사실상의 표준이 되었다'(Huh et al., 2016). 따라서 미세조정 데이터가 완벽하더라도 ImageNet(또는 다른 시작점)의 오류와 편향이 '미세조정'을 거쳐 계속 남을 위험이 있다. 이것이 단지 이론적 위험이 아니라는 점은 Steed와 Caliskan(2021)이 보여준다. 이들은 많은 결과를 제시했으며 그중 하나는 다음과 같다. '[ImageNet으로 사전학습된] iGPT와 SimCLRv2 임베딩 모두 고전적 및 현대화된 버전의 Weapon IAT[암묵적 연관 검사]에서 백인을 도구와, 흑인을 무기와 연관시킨다.'

표 3.1 이미지의 출신 국가 (출처: Shankar et al., 2017)

국가	ImageNet %	ImageNet 순위	OpenImages %	OpenImages 순위	평균 %	평균 순위	Wikipedia 인구 순위
USA	45.43	1	32.10	1	38.76	1	3
UK	7.58	2	12.90	2	10.24	2	21
Italy	6.23	3	2.75	9	4.49	3	25
Canada	3.01	4	3.73	5	3.37	4	37
Spain	2.46	6	4.09	4	3.27	5	30
France	2.06	8	4.29	3	3.18	6	20
Australia	2.80	5	2.88	8	2.84	7	53
Germany	1.63	9	3.16	6	2.40	8	19
Japan	1.18	11	2.96	7	2.07	9	11
India	2.08	7	1.52	11	1.80	10	2
Brazil	0.96	14	2.03	10	1.49	11	6
Netherlands	1.40	9	1.47	12	1.44	12	67
Mexico	1.11	12	1.09	13	1.10	13	10
China	1.05	13	1.00	14	1.03	14	1

원문 PDF 73 페이지

역사적 편향

수집된 모든 데이터는 분명 과거의 데이터이다. 얼마나 먼 과거인지는 일반적으로 알 수 없으며, 실제로 많은 인터넷 자료에는 날짜가 명확히 표시되어 있지 않다. 앞서 영국 군주의 성별을 살펴볼 때 한 사례를 보았지만, 그 밖에도 많은 사례가 있다.

과거가 지속되는 문제, 과거의 고정관념, 또는 현재의 고정관념은 특히 언어 처리를 괴롭힌다. Caliskan et al. (2017)이 이를 지적했다. 그들의 사례는 Google 이 터키어(성중립 대명사를 사용하는 언어)를 영어(성별 대명사를 사용하는 언어)로 번역하는 경우이다. ‘O bir doktor. O bir hemşire’가 ‘He is a doctor. She is a nurse’가 되던 것이 ‘She is a doctor. She is a nurse’로 ‘수정’되었다(그 사이에는 사용자에게 성중립 대명사에 관한 대화창을 보여주는 과도기가 있었다). 그러나 인간에게는 사소한 변경인 ‘O bir matematikçi. O bir hemşire’로 바꾸면

원문 PDF 74 페이지

‘He’s a mathematician. She is a nurse’가 된다. 성별이 있는 명사에서도 같은 문제가 발생할 수 있다. 루마니아어 ‘profesor’와 ‘profesoara’는 영어로 각각 ‘professor’와 ‘teacher’로 번역된다. 이러한 문제에 단순한 해결책은 없다는 점을 유의해야 한다. 좋은 언어 번역은 때로 매우 많은 맥락을 필요로 하는 숙련된 작업이다.

레이블링 편향

많은 응용 분야에서는 레이블이 있는 데이터가 필요하며, 일반적으로 사람이 레이블을 붙인다. 이 레이블링 과정에서 오류나 인간 편향(아래 참조)이 유입될 수 있다. 오류가 작고 체계적이지 않기를 기대할 수는 있지만, 이를 가정할 것이 아니라 검증해야 한다. 레이블링 과정에서 체계적인 오류가 유입될 수도 있다. 19 페이지에서 언급한 ProPublica 연구는 미국의 보석 결정 소프트웨어에서 다음 세 개념이 혼동되었다고도 지적했다(Angwin et al., 2016).

(i) ‘재범했는가(Has reoffended)’: 모든 범죄가 해결되는 것은 아니므로 측정할 수 없지만 원하는 메트릭이며, 더 나쁘게도 소프트웨어가 사용한다고 설명된 개념이다.

(ii) ‘재범으로 유죄 판결을 받았는가(Has been convicted of re-offending)’: 치안 시스템의 편향에 영향을 받으며 사법 시스템에서 보정될 수도 있고 그렇지 않을 수도 있다. 인간 판사가 사용하는 기준이기 때문에 대부분의 사람은 아마 소프트웨어도 이것을 사용한다고 생각했을 것이다.

(iii) ‘재범 혐의를 받았는가(Has been accused of re-offending)’: 이 역시 치안 시스템의 편향에 영향을 받으며, 실제로 사용된 기준이다.

여기에는 복잡한 사회학적·방법론적 문제가 있다. 특히 보석이 거부된 범죄자는 구금되어 있는 동안 재범할 가능성이 매우 낮다는 점도 있다. 따라서 우리는 ‘거짓 양성(false positives)’, 즉 보석이 거부되었지만 보석이 허용되었다면 재범 혐의를 받지 않았을 사람들을 측정할 수 없다. 더 단순한 레이블 혼동의 사례는 San Diego의 자전거 계수에 관한 보고(Perry, 2019)이다. 원래 자전거 이용자를 세려던 시스템이 자동차 뒤에 매달린 자전거까지 세었다.

데이터 처리 편향

데이터 처리 과정, 그리고 일반적으로 복잡한 데이터 처리 파이프라인을 갖는 ML 과정에서 오류가 유입될 수 있다. 현실 세계의 데이터에는 흔히 빈칸, 또는 통계학자들이 말하는 ‘결측값(missing values)’이 포함된다.

원문 PDF 75 페이지

저자가 오래전에 발견한 사소한 사례는 남성=1, 여성=2, 알 수 없음=9로 코딩한 뒤 평균을 계산한 것이었다. 이 때문에 표본의 성별 구성에 관한 잘못된 설명이 만들어졌으며, ‘알 수 없음’이 충분히 많아 평균이 2.1이 되고 연구자가 이 ‘버그’에 대해 저자에게 도움을 요청하고 나서야 문제가 발견되었다. 복잡한 파이프라인에서 많은 참여자가 데이터 파이프라인에 관여하면 이보다 더 미묘한 변형이 그대로 통과할 수 있다.

인간 편향

인간은 의식적이든 무의식적이든 편향되어 있다(Gilvoich and Griffin, 2010). 이러한 편향은 현재에 작용할 수도 있고 과거에 작용했을 수도 있으며, 그 결과 ML 알고리즘이 미래를 예측하기 위해 흔히 의존하는 과거 데이터가 편향될 수 있다. 이는 채용 과정에서도 나타나며(Bertrand and Mullainathan, 2004; Stokel-Walker, 2021), 이러한 응용에서는 특히 주의해야 함을 보여준다.

주거 역시 많은 지역에서 차별이 불법일 수 있는 영역이며 Oregon에서도 그렇다(Serrano, 2022). 그러나 적어도 일부 차별은 컴퓨팅(AI에 국한되지는 않음)이 가능하게 한 인간 차별이다(Edelman et al., 2016). Airbnb 호스트

는 일반적인 호텔 예약보다 더 많은 정보를 갖고 있기 때문에 더 많이 차별할 수도 있다. ‘뚜렷하게 아프리카계 미국인으로 인식되는 이름을 가진 투숙객의 요청은, 그 외 조건이 동일하면서 뚜렷하게 백인 이름으로 인식되는 투숙객보다 수락될 가능성이 약 16% 낮다.’ 그들의 결론은 주목할 만하다. ‘전체적으로 우리의 분석은 주의가 필요함을 시사한다. 정보는 거래를 촉진하지만 차별도 촉진한다.’

인간 편향에 대해서는 일반적 편향, 집단 귀인 편향, 암묵적 편향, 확증 편향, 내집단 편향, 외집단 동질성 편향, ‘보이는 것이 전부다(What you see is all there is)’ 편향 및 사회적 편향 등 많은 설명을 제시할 수 있다. 핵심은 ‘현실 세계’ 데이터 자체가 충분히 편향되어 있을 수 있다는 점이다.

심슨의 역설

‘심슨의 역설(Simpson’s paradox)’은 개별 집단에서 나타나는 추세가 집단을 합쳤을 때 사라지거나 심지어 반대로 바뀌는 현상이다. 통계학 역사에서 더 일찍 관찰된 적도 있지만, 일반적으로 이 역설은 이를 보다 체계적으로 연구한 Simpson(1951)의 이름을 따른다. 간단한 예는 그림 3.1에서 볼 수 있다.

원문 PDF 76 페이지

그림 3.1 심슨의 역설에 대한 간단한 예시

원문 PDF 77 페이지

여기에서는 세 집단 각각을 개별적으로 보면 출력(Output)이 목표(Target)의 감소 함수(기울기 약 -0.4)이지만, 세 집단을 모두 합치면 증가 함수(기울기 약 +0.4)가 되는 것을 볼 수 있다. 이 데이터와 더 많은 사례는 <https://doi.org/10.15125/BATH-01099> (Davenport, 출판 예정)에서 확인할 수 있다.

이 역설의 중요한 사례가 Simpson의 논문에서 제시되어 있다. 먼저 연구자가 일반적인 52 장짜리 프랑스식 카드 덱에서 그림 카드(court card)와 숫자 카드(plain card)의 비율이 검은색 카드와 빨간색 카드 사이에 다른지 궁금해한다고 하자. 다만 어떤 카드는 더럽고 어떤 카드는 깨끗하므로, 연구자는 이것이 영향을 미칠 경우에 대비해 이 정보도 기록한다. 연구자는 표 3.2를 기록한다.

표 3.2 색상 및 청결도별 카드 (출처: Simpson, 1951 © 1951 Royal Statistical Society)

더러움	깨끗함	그림 카드	숫자 카드	그림 카드	숫자 카드	빨간색	4/52	8/52	2/52
12/52	검은색	3/52	5/52	3/52	15/52				

더러운 카드 중 그림 카드의 $4/7=57\%$ 가 빨간색이고, 숫자 카드의 $8/13=61\%$ 가 빨간색이다. 따라서 더러운 숫자 카드는 더러운 그림 카드보다 빨간색일 가능성이 높다. 깨끗한 카드 중 그림 카드의 $2/5=40\%$ 가 빨간색이고, 숫자 카드의 $12/27=44\%$ 가 빨간색이다. 따라서 깨끗한 숫자 카드는 깨끗한 그림 카드보다 빨간색일 가능성이 높다.

하지만 더러움/깨끗함의 구분을 무시하면 표준(프랑스식, 52 장) 카드 덱에 대해 표 3.3을 얻게 된다. 빨간색과 검은색 사이에는 차이가 없다. 둘 다 $20/26=77\%$ 가 숫자 카드이다.

표 3.3 색상별 카드 (출처: Simpson, 1951 © 1951 Royal Statistical Society)

그림 카드 숫자 카드 빨간색 6 20

원문 PDF 78 페이지

검은색 6 20

겉으로 보이는 결론은 명백하다. 깨끗함/더러움의 구분은 무시해야 한다.

이제 표 3.4 를 생각해 보자.

남성 중 치료받지 않은 사람의 $4/7=57\%$ 가 생존하지만, 치료받은 사람의 $8/13=61\%$ 가 생존한다. 따라서 치료받은 남성이 생존할 가능성이 더 높다. 여성 중 치료받지 않은 사람의 $2/5=40\%$ 가 생존하지만, 치료받은 사람의 $12/27=44\%$ 가 생존한다. 따라서 치료받은 여성이 생존할 가능성이 더 높다. 그렇다면 분명 치료가 효과적이라는 결론을 내릴 수 있을 것처럼 보인다. 그러나 수치적으로 표 3.4 는 레이블만 다를 뿐 표 3.2 와 동일하다. 남성과 여성을 합치면 다시 표 3.3 과 같은 결과를 얻어 치료 여부에 차이가 없음을 관찰하게 되므로 치료가 가치 없는 것처럼 보일 수 있다.

Simpson 은 이렇게 말한다. ‘남성과 여성에게 각각 적용했을 때 유익한 치료를 가치 없다고 기각하기는 어렵다.’ 이 상황에서 역설을 대략적으로 설명하면, 남성($13/20=65\%$)보다 훨씬 더 높은 비율의 여성($27/32=84\%$)을 치료했지만 여성은 남성($8/20=40\%$)보다 사망할 가능성($18/32=56\%$)이 더 높았으므로 균형 잡힌 실험을 했어야 한다는 것이다.

표 3.4 성별 및 치료 상태별 생존 (출처: Simpson, 1951 © 1951 Royal Statistical Society)

남성	여성	미치료	치료	미치료	치료	생존	사망
3/52	5/52	3/52	15/52	4/52	8/52	2/52	12/52

현실 세계에서 잠재적으로 심각한 심슨의 역설 사례는 Bickel et al.(1975)이 설명한다. 이들은 1973 년 가을 University of California, Berkeley 대학원 입학 을 살펴보았다. 첫 번째 분석은 표 3.5 와 같았다.

표 3.5 1973 년 가을 University of California 입학

원문 PDF 79 페이지

관찰 데이터	성별	편향이 없을 경우 기대값	합격	불합격	합격	불합격	남성	3738
4704	3460.7	4981.3	여성	1494	2827	1771.3	2549.7	

이는 전체 합격률이 41%인 반면 남성은 44.3%, 여성은 34.6%였음을 보여주며, 통계학자는 χ^2 검정을 수행하여 이것이 우연히 발생할 확률이 극히 작다고 결론 내릴 것이다. 그러나 Berkeley 에서는 개별 학과가 입학 여부를 결정하며, 학과별 합격률과 지원자의 성별 비율은 매우 크게 다르다. 더 나아가 이 둘은 상관되어 있다. ‘여성 지원자의 비율은 입학이 어려운 학과에서 높은 경향이 있고, 입학이 쉬운 학과에서 낮은 경향이 있다.’ 실제로 더 자세히 분석하면 합격한 여성에게 유리한 편향이 존재하는 것으로 나타난다. 저자들은 이렇게 결론 내린다. ‘집계 데이터의 편향은 전반적으로 상당히 공정해 보이는 입학위원회의 차별 패턴에서 비롯된 것이 아니라, 교육 시스템의 더 이전 단계에서 이루어진 사전 선별에서 비롯된 것으로 보인다.’

요약

첫 번째 결론은 편향이 전혀 없는 AI는 무의미하다는 것이다. 우리는 AI가 일부 집단을 다른 집단과 다르게 대우하기를 원한다. 즉, 그러한 집단에 유리하게 편향되기를 원한다. 구분을 하지 않는다면 AI를 사용할 이유가 없기 때문이다. 그러나 우리는 AI가 올바른 구분을 하기를 바라며, 특정한 편향(예: 성별, 민족성, 능력)을 갖지 않도록 하는 윤리적 또는 법적 요구사항이 있을 수 있다. 실제로 이는 매우 어렵거나 불가능한 작업일 수 있다. 젊은 운전자의 경우 많은 문화에서 성별 편향을 갖지 않으면서 사고를 잘 내는 개인에게 완전히 불리하게 편향되는 것은 불가능하다. 그러나 도움이 되기를 바라며 몇 가지 의견을 제시할 수 있다.

(i) ‘나는 성별(또는 민족성 등)을 수집하지 않으므로 그것에 대해 편향될 수 없다’는 말은 사실이 아니다. 자동차 보험 사례(McDonald, 2015)가 보여주듯 대리변수는 매우 많다. 실제로 성별(또는 민족성 등)을 수집하지 않는다면, 자신이 편향되어 있는지 아닌지 모른다고만 말할 수 있다. Uber와 Lyft가 Chicago에서 인종적으로 편향되도록 의도했을 가능성(Pandey

원문 PDF 80 페이지

and Caliskan, 2021)이나 Princeton Review가 가격 책정에서 인종 차별을 하려고 의도했을 가능성(Angwin et al., 2015)은 낮지만, 결과적으로 모두 그렇게 나타났다.

(ii) 자연적인 편향은 제거할 수 없고 단지 위치를 옮길 수 있을 뿐일 수 있다. 따라서 Test-Achats 판결은 연금 요율에 편향이 없도록 요구함으로써 그에 대응하는 편향을 연금 지급액에 도입했다.

(iii) 데이터 선택에 주의를 기울이는 것은 중요하지만, 그것 또한 보장은 아니다. 데이터가 더 많다는 것은 단지 사회에 이미 존재하는 편향이 더 크게 나타난다는 뜻일 수 있다(Metz, 2019).

(iv) 데이터의 레이블이 실제로 무엇을 의미하는지, 자신이 그것이 무엇을 나타내기를 바라는지가 아니라 실제 의미에 대해 정확하게 설명하는 것이 중요하다.

(v) ‘결측값’ 및 데이터의 다른 이상 항목에 주의하라.

(vi) Berkeley 사례가 보여주듯 요약 통계만 보는 것은 매우 오해를 불러일으킬 수 있으며, Bickel et al.(1975)은 다음과 같이 결론 내린다. ‘편견에 따른 대우를 최소화하려면 먼저 그것이 어디에 있는지를 정확히 찾아야 한다.’

이 연구 중 생성된 모든 데이터는 University of Bath Research Data Archive의

<https://doi.org/10.15125/BATH-01099>에서 공개적으로 이용할 수 있다.

¹ 하지만 많은 국가에서는 그렇지 않다. 독일에서 비슷한 여행(Heidelberg에서 Frankfurt)은 €17.90에서 €28.90까지 다양하지만 시간대가 아니라 열차 종류에 따라 달라지며, 실제로 €17.90 열차가 가장 빠른 것으로 보인다.

원문 PDF 81 페이지

4 머신러닝 시스템 테스트

Adam Leon Smith

이 장에서는 ML 시스템의 테스트, 테스트 전문가가 그 프로세스에서 어떤 역할을 하는지, 테스트 레벨, 메트릭 및 기법을 살펴본다. 또한 AI 시스템의 몇 가지 특정 특성을 어떻게 테스트할지 논의한다.

테스터의 역할

ML 시스템을 개발하는 조직에서 일하고 있다면 뛰어난 데이터 과학자와 엔지니어 팀과 함께 일하는 행운을 누리고 있을 수도 있다. 일부 테스트 및 품질 전문가(그리고 개발자도 마찬가지)는 이러한 팀이 사용하는 수학적으로 들리는 언어와 마치 마법처럼 보이는 프로세스에 위축될 수 있다. 테스터는 흔히 그들의 산출물을 충분히 테스트하는 것을 꺼리거나, 평가하지 말라는 분위기를 경험한다. 데이터 과학 전문지식이 없는 사람이 전문가의 작업에서 어떻게 허점을 찾을 수 있을까?

그 작업에 결함이 가득하지 않을 이유는 없다. 데이터 과학자도 자신의 모델에서 문제를 찾는 데 있어 개발자가 자신의 코드에서 문제를 찾는 것보다 더 뛰어나지 않다. 전통적인 개발자는 확증 편향 때문에 자신의 작업에서 문제를 놓친다. 즉, 가정을 하거나 작업에 작은 논리적 오류를 삽입하기 때문이다. 데이터 과학자도 가정을 한다. 때로 이러한 가정은 현실이 아니라 모델이 어떤 예측을 해야 한다고 자신이 기대하는 바를 기반으로 한다. 결과가 올바른지 고려하지 않은 채 자신이 기대하는 결과가 나올 때까지 모델의 하이퍼파라미터를 계속 조정하는 것은 너무나 쉽다.

원문 PDF 82 페이지

하이퍼파라미터는 모델을 학습하기 전에 사람이 ML 모델에 설정하는 변수이다.

테스팅 및 품질 전문가가 가치를 더할 수 있는 첫 번째 영역은 단순히 올바른 질문을 하는 것이다. 이들이 가질 수 있는 가장 중요한 두 가지 자질은 첫째, 작업 산출물을 비판적으로 사고하는 것, 둘째, 위험을 매우 잘 발견하는 것이다. 이는 데이터 수집과 레이블링에서 시작한다. 학습용 데이터는 어떻게 수집되었는가? 현실 세계의 분포를 대표하는가? 어떻게 레이블이 지정되었는가? 레이블에 대해 표본 검사가 수행되었는가? 레이블링 작업자의 동기는 무엇인가? 학습 데이터의 일부 부분집합이 잘못 레이블링되었을 가능성이 있는가? 이러한 질문만으로도 훨씬 나중까지 발견되지 않을 수 있는 잠재적 결함의 근본 원인을 드러낼 수 있다.

이 질문 중 일부는 통계적으로 탐색할 수 있다. 어느 정도 기술 역량이 있다면 학습 데이터를 탐색적 분석 도구에 불러오는 것이 유용할 수 있다. 이는 스프레드시트, Python 라이브러리 또는 상용 비즈니스 인텔리전스 도구일 수 있다. 데이터를 직접 탐색하여 이상치, 패턴, 불일치, 샘플링 문제 및 오류를 찾아볼 수 있다.

다음으로 살펴볼 영역은 데이터가 모델로 들어가고 나오는 과정인 데이터 파이프라인이다. 조직에 데이터 과학자의 요구사항을 받아 데이터를 변환하고 조작하는 훌륭한 데이터 파이프라인을 만드는 숙련된 데이터 엔지니어링 팀이 있을 수도 있고, 데이터 과학자가 직접 만들 수도 있다. 후자는 품질 측면에서 경고 신호이다. 데이터 과학자는 엔지니어가 아니며, 데이터 파이프라인은 엔지니어링 산출물이다. 이 때문에 데이터 파이프라인의 코드 품질은 매우 나쁠 수 있다(Lenarduzzi et al., 2021). 물론 가장 먼저 살펴볼 곳은 컴포넌트 테스트 코드 커버리지이지만, 커버리지가 매우 낮더라도 놀라지 말아야 한다.

데이터 파이프라인은 ML 알고리즘을 지원하는 인프라이다. 데이터 획득, 전처리 및 준비, 하나 이상의 모델 학습, 그리고 모델을 운영 환경으로 내보내는 과정을 포함한다.

데이터 파이프라인을 별도의 팀이 구축하는 경우에는 가정과 구현 제약사항을 놓칠 위험이 있다. 데이터의 출처에서 모델의 예측에 이르는 엔드투엔드 통합은 매우 중요하다. 데이터 과학자가 자신의 모델에서 100 만 개 레코드를 테스트했다고 들을 수 있지만, 대부분 그 레코드는 CSV 또는 다른 형식으로 모델에 직접 입력되어 엔드투엔드 데이터 흐름을 건너뛰었을 가능성이 높다. 통합에서 놓칠 수 있는 잠재적 버그 몇 가지를 생각해 보자.

첫째, 데이터 파이프라인이 모델의 특성인 특정 값을 잘못 변환하여 영향을 받는 레코드와 받지 않는 레코드 모두에서 잘못된 예측을 일으킬 수 있다. 예를 들어 나이(age)가 잘못 변환되어 한 자리 숫자가 된다면, 나이가 예측의 핵심 구성요소인 경우 매우 잘못된 예측으로 이어질 수 있다.

둘째, 데이터 파이프라인이 일부 필드에 null 값을 잘못 전달하여 전체 레코드가 제거될 수 있다. ML은 결측값을 잘 처리하지 못하며, 이는 잘못된 학습 및 출력 누락으로 이어진다. 예를 들어 우편번호를 기반으로 한 조회가 실패하여 도시(city)가 null로 남으면, 학습 또는 운영에서 해당 레코드를 유효한 것으로 고려하지 못할 수 있다.

셋째, 데이터 파이프라인이 일부 레코드를 잘못 중복하여 해당 중복 레코드의 사례가 지나치게 많아지고, 모델이 그 레코드의 값 쪽으로 편향될 수 있다.

테스터의 핵심 역할 중 하나는 각 테스트 레벨에서 테스트 범위와 접근법을 식별하고, 명시된 컴포넌트 및 인수 기준을 정하는 것이다. 통합 시스템과 ML 모델에서 발생하는 제품 위험을 적절하게 식별하고 테스트를 통해 완화하는 것이 중요하다.

ML의 특성

AI가 가질 수 있는 몇 가지 품질 문제를 논의했지만, 테스트의 관점을 완전히 뒤집는 문제가 하나 있다. ML은 통계적 상관관계를 사용하여 예측하며, 이것은 ML의 초능력이자 약점이다. 앞서 설명했듯이 이는 ‘명세 없는 자동화’를 가능하게 하고, 모델이 식별하고 예측에 사용하는 패턴을 인간 엔지니어가 실제로 이해하지 못하더라도 예측을 만들 수 있게 한다.

전통적인 소프트웨어는 대체로 결정론적이다(멀티스레딩 같은 요소가 도입하는 작은 차이는 무시한다고 하자). 그리고 명시적 논리를 사용하여 출력을 결정한다. 우리는 명세를 보고 입력을 설계하며 일반적으로 시스템의 출력 또는 동작을 예측할 수 있다. ML에서는 이것이 훨씬 어렵다. 기대 결과를 계산하기가 너무 어렵기 때문일 수도 있고, ML 시스템이 100% 정확할 것으로 기대하지 않기 때문일 수도 있다.

예를 들어 얼굴 인식 시스템을 생각해 보자. 시스템이 알고 있는 사람들의 얼굴이라는 사실을 알고 있는 100 장의 얼굴 사진을 준비하여 테스트로 시스템에 입력할 수 있다. 99 개의 얼굴은 인식되었지만 하나는 인식되지 않았다면 이것은 테스트 실패인가? 명확하지 않다. 이것이 테스트 오라클 문제의 한 사례이다.

또한 100이라는 숫자는 전 세계 얼굴 모집단의 맥락에서 통계적으로 유의한 수가 아니므로 타당성도 의심스러울 수 있다. 통계적으로 의미 있는 결과를 얻으려면 흔히 수천 개(또는 그 이상)의 입력 집합을 테스트해야 한다. 더 나아가 얼굴 인식에서는 인간의 눈으로 기대 결과(일치 또는 불일치)를 판단하려 할 수 있지만, 많은 예측 시스템에서는 ‘그라운드 트루스’를 결정하는 것이 그렇게 쉽지 않다.

화이트박스 테스트 기법(즉 시스템의 코드 또는 구조에 대한 지식을 활용하는 기법)을 사용하기도 어렵다. ML 모델의 내부 구조를 쉽게 이해할 수 없기 때문이다. 전통적인 소프트웨어는 특정 휴리스틱(루프 경계, 필드 경계, 필드 조합)에 따라 발생하는 결함을 가지며, 이에 따라 그러한 결함을 탐지하도록 설계된 테스트 기법이 발전했다. 시스템을 블랙박스로 테스트할 때(즉 내부 구조에 대한 지식 없이 테스트할 때)조차 이러한 기법을 사용하여 발생 가능성이 높은 문제를 찾을 수 있다. ML은 대체로 그러한 휴리스틱을 따르지 않으므로

원문 PDF 85 페이지

문제를 드러내는 테스트를 계획하기가 더 어렵다. 우리가 알지 못하는 사이 얼굴 인식 시스템이 인간의 눈으로는 구별하기 어려운 어떤 차이를 가진 얼굴 유형에서 완전히 실패할 수도 있다.

ML은 확률적 시스템으로 설명할 수 있다. 확률적 시스템은 확률을 사용하지 않고는 설명하기 어려운 시스템이다. 예를 들어 날씨는 모든 입자, 모든 돌풍, 모든 미세 수준의 온도 변화를 모델링하기가 너무 어렵기 때문에 확률적 시스템이다. 따라서 기상 예보자는 확률을 사용하여 예측한다. ML 모델도 여러 면에서 비슷하다.

이러한 테스트 문제는 극복할 수 없는 것은 아니지만 전통적인 소프트웨어를 테스트할 때와는 다른 사고방식이 필요하다. 또한 여러 요인에 의해 문제가 크게 복잡해질 수 있다. 그중 하나는 여러 ML 컴포넌트를 통합하여 하나의 통합 시스템을 만드는 것이다. 이는 생각보다 흔하다. 예를 들어 얼굴 인식에서는 큰 이미지 안에서 얼굴을 탐지하는 모델 하나와 기존 얼굴 이미지 데이터셋을 대상으로 식별을 수행하는 모델 하나를 사용하는 것이 일반적이다. 여러 확률적 시스템을 결합하면 행위를 예측하기가 더 어려워지고, 그 결과 무엇이 잘못될 수 있는지도 예측하기 어려워진다.

또 다른 문제는 ‘서비스형 AI(AI-as-a-service)’의 사용이다. 서비스형 소프트웨어와 마찬가지로, 이는 예측 결과를 제공하기 위해 호출하는 제 3자 애플리케이션 프로그래밍 인터페이스(API) 뒤로 많은 복잡성을 추상화한다. 개발에는 많은 노력을 절약할 수 있지만 테스트 관점에서는 악몽이 될 수 있다. 시스템의 투명성이 낮아지고 통계적으로 유의한 규모로 테스트하기도 일반적으로 더 어려워진다. 데이터 과학자가 자신의 데스크톱에서 수백만 개 레코드를 모델에 통과시키는 대신, 결과를 얻기 위해 API를 동적으로 호출해야 한다.

이 책 전체에서 테스트 오라클이라는 표현을 보게 될 것이다. 오라클은 테스트가 통과했는지 실패했는지를 결정할 수 있게 하는 무언가이다. 요구사항, 명세, 대체 시스템(의사 오라클, pseudo-oracles) 또는 단순히 우리가 알고 있는 사실(암묵적 오라클)이 될 수 있다.

원문 PDF 86 페이지

테스트 레벨

테스트 레벨은 테스트 프로세스의 구체적인 인스턴스이다. 전통적인 프로젝트에서는 일반적으로 가장 작은 독립 소프트웨어 모듈을 테스트하는 컴포넌트 테스트부터 시작하여, 통합 및 시스템 테스트, 그리고 인수 테스트로 이어진다. 맥락, 기술, 위험 및 사용하는 방법론에 따라 이러한 레벨은 프로젝트마다 다르다. 반복성이 매우 높고 Agile 한 프로젝트에서는 이러한 레벨을 짧은 기간에 걸쳐 작은 증분 단위로 실행한다.

그렇다면 ML 시스템에서는 어떻게 달라질까? 한 관점에서는 많이 달라지지 않는다. 테스트 레벨은 대체로 동일하다. 그러나 기술의 복잡성 때문에 계획에 더 신중한 사고가 필요할 수 있다. 이를 설명하려면 먼저 왜 이러한 레벨이 존재하는지를 생각해야 한다. 본질적으로 검증 범위가 작을수록 실패를 발견하고 조사하기가 더 쉽기 때문이다. 이를 보여주기 위해 마이크로서비스의 단순한 for 루프에서 경계 오류(또는 ‘off-by-one’ 오류)가 발생한 경우를 생각해 보자. 많은 컴포넌트가 하나의 통합 시스템으로 결합된 시스템 또는 인수 테스트 과정에서 이 오류를 찾고 탐지하는 것은 매우 복잡하다. 경계값을 포함하는 엔드투엔드 테스트를 실행하는 데 필요한 노력은 상당할 가능성이 높다. 발생한 실패를 조사하고 근본 원인을 특정하는 데 필요한 노력도 상당할 것이다. 이를 컴포넌트 테스트와 비교해 보라. 컴포넌트 테스트에서는 경계를 확인하는 테스트를 작성하는 것이 거의 직관적이며 실패 원인을 더 빨리 찾을 가능성이 높다.

ML 모델의 예측 실패를 탐지할 때도 같은 논리를 적용할 수 있다. 비용이 많이 드는 통합 엔드투엔드 환경에서 찾는 것보다 모델 테스트 과정에서 많은 양의 테스트를 실행할 때 실패를 훨씬 쉽게 찾고 탐지할 수 있다. 불필요한 컴포넌트를 추가로 도입하지 않으면서 복잡한 실패를 가능한 한 조기에 탐지하도록 테스트를 계획해야 한다.

또 다른 관점에서 보면 전통적인 테스트 프로세스와 상당히 다른 두 가지 테스트 레벨이 있다. 모델 테스트와 입력 데이터 테스트이다. 첫 번째는 이미 언급했다. 모델 테스트는 컴포넌트 테스트의 한 형태이지만 전통적인 컴포넌트 테스트와 명확히 구별된다. 본질적으로 통계적이며 많은 양의 데이터를 사용하고, 테스트 결과가 복잡하여 수학적 분석이 필요하며, 일반적으로 데이터 과학자가 수행하는 반복적 프로세스이다.

원문 PDF 87 페이지

일반적으로 데이터 과학자는 모델을 학습하고 테스트하기 위한 데이터셋을 받는다. 이 데이터의 일부(보통 20~40%)는 검증을 위해 따로 보관한다. 나머지는 모델 학습에 사용한다. 달리 말하면 대부분의 데이터는 사례로 ML 모델에 보여주고, 나머지는 모델이 올바르게 행동하는지 확인하는 데 사용한다. 여기에는 몇 가지 복잡성이 있다. 예를 들어 일부 방법론에서는 데이터 과학자가 서로 다른 데이터 부분집합을 테스트에 사용하여 동적으로 학습을 시도하고, 서로 다른 부분집합으로 여러 레벨의 테스트를 제공할 수 있다. 이는 모델이 특정 학습 데이터셋에 과적합되지 않도록 하는 데 도움을 준다.

이 테스트 레벨은 매우 중요하며, 테스트 전문가가 데이터 과학자로부터 가져와 자신이 대신하려고 해서는 안 된다. 이는 사실상 개발 프로세스의 일부이다(컴포넌트 테스트 레벨에서의 테스트 주도 개발과 유사하다). 다만 주의해야 할 세 가지 함정이 있다. 첫 번째는 실험자 편향(experimenter’s bias)이다. 이는 실험자(여기에서는 데이터 과학자)가 모델의 테스트 결과가 어떠해야 하는지에 대한 선입견을 갖고 있고, 결과가 자신의 기대를 확인해 주기 때문에 방법론의 결함을 알아차리지 못할 때 발생하는 인간의 인지 편향이다. 한 예는 제공받은 데이터셋에 임의의 일부 부

분집합에서 가져온 제한된 사례만 포함되어 있다는 사실을 알아차리지 못하는 것이다. 이는 데이터 모집단이 현실을 대표하지 않는다는 뜻이다. 이런 이유로 일부 입력 데이터 테스트를 수행하는 것이 유용하다(뒤에서 더 설명한다).

두 번째 함정은 모델이 이런 방식으로 철저히 테스트되었기 때문에 후속 단계에서는 모델의 기능 성능에 대한 대량 테스트가 필요하지 않다고 가정하는 것이다. 그렇지 않다. 거의 모든 ML 구현에는 테스트에 맞도록 데이터를 전처리하고 적응시키는 복잡한 데이터 파이프라인이 있다. 카메라 이미지의 대비를 정규화하여 조정하거나 구조화된 데이터 값을 변환하는 과정이 포함될 수 있다. 이러한 데이터 파이프라인은 전통적인 소프트웨어와 동일한 모든 위험을 가지며, 단순히 일부 레코드를 실수로 제거하거나 일부 값을 잘못 변환하는 것만으로도 ML 예측에 통계적 차원에서 극적인 영향을 줄 수 있다. 엔드투엔드 데이터 흐름을 철저히 테스트하고 이후 단계에서 전체 시스템을 통계적으로 테스트해야 한다.

세 번째 함정은 모델 테스트를 아예 하지 않는 것이다. 사내 개발자가 모델을 개발하는 경우에는 이런 일이 발생할 가능성이 낮지만, 소프트웨어 개발자가 제 3자 모델이나 웹서비스 인터페이스 뒤에 숨겨진 제 3자 모델을 활용하는 경우에는 흔히 발생할 수 있다.

원문 PDF 88 페이지

서비스형 AI는 사내 데이터 과학 역량 없이 ML 컴포넌트를 활용하는 점점 더 일반적인 비즈니스 모델이다. 이 사례에서는 모델 테스트를 수행하는 것이 매우 중요하며, 이러한 위험을 지적하는 일이 품질 및 테스트 전문가의 역할이 될 수 있다.

다시 입력 데이터 테스트로 돌아가면, 이것을 하나의 테스트 레벨로 볼 수 있다. 정적 테스트(즉 프로그램을 동적으로 실행하지 않고 수행하는 테스트)의 범주에 속하며 때때로 ‘탐색적 데이터 분석’이라고 한다(그림 4.1). 일반적으로 ML 모델의 학습 데이터를 확보하여 그 통계적 특성을 분석하는 형태로 수행한다.

그림 4.1 탐색적 데이터 분석과 모델 테스트 범위

이를 수행하는 이유는 여러 가지이다. 첫째, ML 예측 실패의 가장 큰 원인 중 하나는 학습 데이터셋의 불균형이다. 데이터 분포를 분석하면 불균형을 발견할 수 있다. 예를 들어 무응답 편향, 즉 특정 사람들이 질문이나 설문에 답하지 않는 경향을 생각해 보자. 데이터를 제공할 시간이나 의향이 더 많은 사람 집단 쪽으로 데이터가 치우쳐 있을 수 있다. 또 다른 예는 얼굴 인식이다. 많은 모델은 인터넷에서 공개적으로 이용 가능한 데이터를 기반으로 학습하며, 이러한 데이터는 백인 남성 서구인 쪽으로 치우쳐 있다. 학습 데이터에서 이러한 불균형을 발견하면 학습된 모델에서 발생할 가능성이 높은 실패(및 편향)를 탐지할 수 있다.

원문 PDF 89 페이지

입력 데이터 테스트에서 탐지할 수 있는 또 다른 문제의 예는 부분 데이터이다. ML 모델은 부분 데이터와 잘 작동하지 않는다. 예를 들어 세 개의 열로 구성된 구조화된 데이터가 있는데 한 열이 선택 사항이라고 하자. 일반적으로 데이터 과학자는 결측값을 대체(impute, 추정)하거나 해당 열이 비어 있는 행을 제거한다. 처음에는 상당히 균형 잡힌 데이터셋이었더라도 일부 레코드를 제거하는 과정에서 불균형이 생길 수 있다.

테스트 레벨은 아니지만, ML 시스템의 기능 성능은 지속적으로 훨씬 더 많은 모니터링을 수행하는 것이 일반적이다. 전통적인 소프트웨어에서는 보통 시스템을 만들고 테스트한 뒤 ‘운영(production)’ 또는 실제 사용 환경으로 넘기고, 자주 다시 살펴보지 않는다. 그러나 ML 시스템에서는 실제 사용 중 시스템의 품질을 지속적으로 검토하고 다시 평가해야 한다. 여기에는 몇 가지 이유가 있다. 첫째, ML의 품질은 학습 데이터가 실제 환경에서 접하게 될 ‘보지 못한’ 데이터를 얼마나 잘 대표하는지에 전적으로 달려 있다. 따라서 운영 시작 이후에도 데이터를 계속 모니터링하여 모델이 강건성을 유지하는지 확인해야 한다. 또한 모델은 개념 드리프트를 겪을 수 있다. 즉 학습 데이터에 존재하기 때문에 존재한다고 가정한 입력과 출력의 상관관계는 고정되어 있지 않고 시간에 따라 변한다. 여러 데이터셋을 기반으로 쇼핑물의 유동인구를 예측하도록 설계된 모델을 생각해 보자. COVID-19 팬데믹이 발생하자마자 서로 다른 입력 특성과 쇼핑물 유동인구 사이의 상관관계가 완전히 뒤집혔을 가능성이 높다. 이는 극단적인 예이지만 현실 세계가 변화하면서 ML 모델 예측의 품질은 향상될 수도 있고 저하될 수도 있다. 운영 환경에서 ML 품질을 모니터링해야 하는 또 다른 이유는 편향 탐지이다. 학습 데이터와 실제 환경에서 모델이 노출되는 데이터 사이에 잠재적 차이가 있기 때문에 테스트 환경에서는 편향을 판단하기 어려울 수 있으며, 지속적인 모니터링이 이에 도움을 줄 수 있다.

요약하면, ML과 관련하여 고려해야 할 테스트에는 전통적인 소프트웨어에서는 필요하지 않거나 적어도 그렇게 중요하지 않은 두 가지 특정 ‘레벨’이 있다. 모델 테스트와 입력 데이터 테스트이다. 그러나 지속적으로 더 많은 테스트가 필요할 수도 있다.

원문 PDF 90 페이지

AI 시스템의 사용자 인수 테스트는 대체로 전통적인 사용자 인수 테스트와 동일하지만 추가적인 초점이 필요할 수 있다. 시스템이 인간의 활동을 대체하도록 설계되었다면 시스템의 행위를 인간과 비교해야 할 수도 있다. 자동화 편향, 즉 기계의 지원을 받거나 기계를 모니터링할 때 인간의 의사결정이 저하되는 정도를 특별히 고려해야 한다.

테스팅 메트릭

테스팅 프로세스는 관리 및 통제 프로세스의 일부로서 또는 테스트 베이스를 어느 정도 ‘커버’하는지 이해하기 위한 활동의 일부로서 자주 측정된다. 후자는 다음 절에서 논의한다. ML 시스템의 테스트에는 이 밖에도 많은 메트릭을 적용할 수 있으며, 흔히 정확성 또는 성능의 관점에서 사용한다. ML이 항상 모든 것을 올바르게 처리할 것으로 기대하지 않기 때문에 이러한 메트릭을 사용한다. 따라서 통계적으로 유의한 수의 테스트를 실행하고 테스트 결과 역시 통계적으로 살펴봐야 한다. 이는 성능 테스트 결과, 즉 특정 데이터 또는 사용자 부하에서 얼마나 빠르게 실행되는지를 분석하는 것과 비슷하다.

소프트웨어 엔지니어링 분야와 AI/ML 분야에서 ‘performance’라는 단어를 사용하는 방식에는 중요한 용어 차이가 있다. 어떤 이유에서인지 엔지니어는 ‘performance’를 시간 행위, 효율성 및 자원 활용의 의미로 사용한다. AI/ML에서는 그렇지 않다. AI/ML에서 ‘performance’는 특정 메트릭에 따라 시스템 또는 모델이 얼마나 잘 행동하는지를 단순히 의미한다. 혼동을 피하기 위해 여기서는 ‘기능 성능(functional performance)’이라는 용어를 사용한다.

ML 모델의 기능 성능은 비즈니스 목표와 출력 유형에 따라 다르게 측정한다. 가장 일반적인 출력 유형 두 가지는 연속적인 숫자와 어떤 것이 특정 클래스에 속하는지에 대한 예측이다. 상상할 수 있듯이 이는 매우 다른 유형의 출력이므로 서로 다른 방식으로 논의한다.

원문 PDF 91 페이지

ML에서 회귀 모델은 연속 변수, 즉 부동산수점 숫자를 출력하는 모델이다. 분류 모델은 어떤 것이 특정 클래스에 속하는지를 예측하는 모델이다.

회귀 모델 결과에는 평균제곱오차(MSE)를 사용할 수 있다. 이는 단순히 평균 오차율(기대값과 예측값의 차이)을 제공한 것이다. 10 개의 테스트를 실행했고 각 테스트의 기대 결과가 0 이어야 하는데 실제 결과가 모두 10 이라면 (산술) 평균 오차는 10 이다. 그러나 실제 결과가 -10 에서 +10 사이로 다양하다면 단순 평균은 0 이 되어 모든 테스트가 통과한 것처럼 잘못 나타낼 수 있다. 그래서 오차를 ‘제곱’한다. 표 4.1 에서 볼 수 있듯이 결과를 더 올바르게 반영한다.

표 4.1 MSE 예

기대 결과	실제 결과	오차	제곱 오차
0	-10	-10	100
0	0	0	0
0	+10	10	100
평균:		0	66.66

분류 모델의 결과를 측정하려면 다른 접근법이 필요하다. 각 예측에는 두 가지 속성이 있다. 예측이 어떤 것이 클래스에 속한다고 했는지 또는 속하지 않는다고 했는지, 그리고 그 예측이 올바른지이다. 이를 참 양성(true positive: 어떤 것이 클래스에 속한다고 올바르게 예측), 참 음성(true negative: 어떤 것이 클래스에 속하지 않는다고 올바르게 예측), 거짓 양성(false positive: 어떤 것이 클래스에 속한다고 잘못 예측), 거짓 음성(false negative: 어떤 것이 클래스에 속하지 않는다고 잘못 예측)이라고 한다. 이는 서로 다른 두 가지 실패 형태가 있다는 뜻일 뿐 아니라, 그러한 실패의 영향도 매우 다를 수 있음을 의미한다. 어떤 사람에게 질병이 있다고 잘못 예측하는 것은 같은 질병이 없다고 잘못 예측하는 것과 영향이 매우 다르다. 전자의 경우 후속 검사와 예측을 통해 사실을 알게 될 수 있다.

원문 PDF 92 페이지

후자의 경우에는 다시 검사를 받지 않을 수도 있다.¹

따라서 우리는 이러한 실패율을 흔히 혼동 행렬(confusion matrix)을 통해 시각화한다. 아마 ‘혼동 행렬’이라고 불리는 이유는 통계학 연구에서 시작되었지만 수년 동안 주로 심리학 연구에서 사용되었기 때문일 것이다. 그림 4.2는 혼동 행렬의 예이다.

테스팅 기법

앞 절에서 MSE 사례로 사용한 10 개의 테스트는 통계적으로 유의한 결과를 얻기에 충분하지 않을 가능성이 높다. 그러면 당연히 ‘몇 개면 충분한가?’라는 질문이 생긴다. 전통적인 시스템 테스팅과 마찬가지로 완전 테스팅은 불가능하다. 전통적으로는 위험 기반 접근법과 함께 일반적인 소프트웨어 오류의 휴리스틱을 활용하여 오류를 발견할 가능성을 최대화하는 것으로 입증된 테스트 기법을 사용한다. 요구사항의 우선순위를 정하고 시스템 구조를 분석하여 기대 결과가 포함된 테스트 절차를 설계한다.

그러나 시스템이 비결정적이라면 유효한 출력이 여러 개일 수 있으며

원문 PDF 93 페이지

시스템의 처리에서는 동일한 입력이 주어지고 동일한 상태에서 시작하더라도 여러 개의 유효한 출력이 나올 수 있다.

그렇다면 ML에서는 이 과제에 어떻게 접근할 수 있을까? 여러 방법이 있으며, 아직 어느 하나도 근본적으로 최선의 접근법이라고 입증되지는 않았다. 그러나 이는 전통적인 테스트 기법에도 어느 정도 해당하는 말이다. 실제로 특정 하나의 테스트 기법에 지나치게 집중하면 테스트 케이스 폭증 같은 문제가 발생할 수 있으므로 언제나 신중하고 균형 있게 사용해야 한다.

조합 테스팅

일부 상황에서는 테스팅에 사용된 데이터를 가능한 모든 입력 데이터 변형과 비교하여 분석할 수 있다. ML 모델은 입력 데이터에 반응하도록 학습되므로, 사용한 입력 데이터의 다양성을 살펴보면 커버리지의 대리 지표를 얻을 수 있다. 예를 들어 입력 특성으로 ‘나이’ 또는 ‘국가’가 있다면 테스트 데이터가 모든 나이와 국가를 통계적으로 유의한 정도까지 커버했는지 살펴볼 수 있다. 이는 유용할 수 있지만 추가 데이터를 확보하거나 만들어내는 데 많은 노력이 필요할 수도 있다. 또한 가장 단순한 사용 사례에만 적용 가능한 기법이다. 얼굴 인식에 이를 적용한다고 상상해 보라. 입력의 양과 다양성은 엄청나다.

테스트 입력 선택을 단순화하는 한 가지 방법은 페어와이즈 테스팅(pairwise testing)의 개념을 사용하는 것이다. 페어와이즈 테스팅(또는 All-pairs testing; Wikipedia, 2021)은 하나의 입력만 조작하는 것보다 두 입력의 쌍을 조작하는 것이 더 많은 결함을 드러낸다는 개념에 기반한다. 전통적인 소프트웨어를 대상으로 수행된 연구(Nie and Leung, 2011)를 기반으로 하지만, 그렇다고 ML에 적용되지 않는다는 뜻은 아니다.

페어와이즈 테스팅을 적용하려면 다양한 입력을 사용할 수 있다. 얼굴 인식을 예로 들면 피부색, 눈 색깔, 코 크기, 배경 및 조명을 사용할 수 있다. 각 입력을 10 개의 가능한 값으로 구간화할 수 있다고 가정하자. 예를 들어 동등 분할을 사용하여 조명을 1~10으로 구간화하고, 10은 매우 밝음, 1은 매우 어두움을 나타내게 할 수 있다. 각 입력을 구간화하면 가능한 값이 각각 10 개인 5 개의 입력이 남는다. 즉 각 입력에는 50 개의 테스트 케이스가 있으며, 가능한 테스

트 케이스는 50^5 개, 즉 약 3 억 1,200 만 개이다. 이러한 입력에 페어와이즈 기법을 적용하면 테스트 케이스 수를 단 100 개로 줄일 수 있다. 이에 대한 수학을 설명하는 것은 이 책의 범위를 다소 벗어나지만,

원문 PDF 94 페이지

이 기법을 더 깊이 조사할 수 있는 온라인 자료, 논문, 심지어 웹 기반 테스트 설계 도구도 많이 있다.

이 기법을 사용한다고 해서 테스트 오라클 문제, 즉 테스트가 통과했는지 실패했는지를 판단하는 문제까지 해결되는 것은 아니다.

뉴런 커버리지

테스트 커버리지는 소프트웨어 엔지니어링에서 널리 논의되는 개념이다. 한쪽 끝에서는 테스트 분석가가 요구사항과 비즈니스 프로세스를 테스트 집합에 매핑하여 상위 수준의 비즈니스 요구사항이 테스트에 의해 어느 정도 커버되었는지 측정할 수 있다. 다른 쪽 끝에서는 자동화 테스트(특히 컴포넌트 테스트)에서 해당 테스트가 테스트 대상 소프트웨어 코드의 어떤 구문과 분기를 커버했는지 측정하는 것이 일반적이다. 이는 코드에 계층을 삽입하거나 Java Virtual Machine 을 감싸는 래퍼를 사용하여 수행하며, 과학적인 테스트 커버리지 측정값을 제공한다. 실제로 도로 차량 시스템의 안전 테스트 같은 일부 도메인에서는 이것이 요구된다.

신경망에서는 이를 어떻게 할 수 있을까? 먼저 신경망이 작동하는 방식을 조금 더 자세히 살펴봐야 한다. 그 전에 불 논리(Boolean logic)를 다시 살펴볼 필요가 있다. 불 논리는 컴퓨터 과학의 기반으로, 하드웨어 프로세서와 저수준 기계어 코드 안에서 1과 0(이진수)을 처리하는 방식이다. 가장 중요한 불 연산자는 OR, AND, NOT 및 XOR 이다.

- 두 이진수를 OR 연산자에 전달하면 어느 하나의 입력이라도 1 이면 1 을 반환하고 둘 다 1 이 아니면 0 을 반환한다.
- 두 이진수를 AND 연산자에 전달하면 두 입력이 모두 1 일 때 1 을 반환하고 그렇지 않으면 0 을 반환한다.
- 하나의 이진수를 NOT 연산자에 전달하면 다른 이진수를 반환한다(즉 0 을 전달하면 1 을, 그 반대로 동일).
- 두 이진수를 XOR 연산자에 전달하면 어느 하나만 1 일 때 1 을 반환하고 둘 다 0 이거나 둘 다 1 이면 0 을 반환한다.

이 강력하지만 단순한 개념이 현대 컴퓨팅의 기반이 되는 방식을 지나치게 깊이 다루지 않고, 매우 단순한 신경망을 사용해 AND 연산자를 어떻게 재현할 수 있는지 살펴보자.

원문 PDF 95 페이지

신경망의 한 계층에는 두 가지 중요한 변수, 즉 가중치(입력 하나당 하나)와 활성화 값이 있다. 가중치는 각 입력이 해당 계층에서 갖는 상대적 중요도를 결정하고, 활성화 값은 출력의 결정 경계를 결정한다. 학습 과정은 학습 데이터와 모델 하이퍼파라미터를 기반으로 올바른 가중치와 활성화 값을 자동으로 결정한다.

신경망 계층의 수학은 다음과 같이 설명할 수 있다.

$$y = 1 \text{ if } \sum_{i=1..n} (w_i * x_i) + w_0 \geq \theta$$

조금 풀어서 설명하면 다음과 같다.

- y 는 출력값이다.
- x_i 는 모든 입력을 나타낸다. AND 예에서는 입력 x_1 과 x_2 두 개가 있다.
- w_i 는 가중치를 나타낸다. AND 예에서는 두 입력의 중요도가 동일하므로 둘 다 1 이다.
- w_0 는 우리가 선택하는 상수값을 나타낸다.
- θ 는 활성화 값(결정 경계)을 나타낸다. 일반적으로 0 보다 작은 값은 0 을 출력하고, 0 이상인 값은 1 을 반환한다.

위 공식을 다루기 쉬운 부분으로 나누면 먼저 각 입력에 대응하는 가중치를 곱한 뒤 결과를 더해야 한다(α). 표 4.2 에 이를 나타낸다.

표 4.2 활성화 함수로서 AND 연산자의 초기 예

x_1	x_2	w_1	w_2	$x_1 * w_1$	$x_2 * w_2$	$\alpha = (x_1 * w_1) + (x_2 * w_2)$
0	0	1	1	0	0	0
0	1	1	1	0	1	1
1	0	1	1	1	0	1
1	1	1	1	1	1	2

AND 연산자는 두 입력이 모두 1 일 때에만 1 을 반환해야 한다는 것을 알고 있으며, 표 4.2 에서 이 경우 α 가 명확히 다르게 2 가 됨을 볼 수 있다. 이 경우에만 공식 결과가 1 이 되게 하려면 α 를 조정하여 이 결과만 0 이상이 되도록 해야 한다. 이를 위해 상수 w_0 를 조정한다. 이 예에서는 이를 -1.5 로 설정할 수 있으며, 이제 표 4.3 과 같이 신경망에 AND 연산자가 구현된다.

원문 PDF 96 페이지

활성화 함수로서 AND 연산자의 초기 예

표 4.3 활성화 함수로서 AND 연산자의 예

x_1	x_2	$\alpha = (x_1 * w_1) + (x_2 * w_2)$	$\alpha - w_0$	$Y(\alpha - w_0 \geq 0)$
0	0	0	-1.5	0
0	1	1	-0.5	0
1	0	1	-0.5	0
1	1	2	0.5	1

물론 이는 재현하려는 논리로서 믿을 수 없을 정도로 단순한 예이다. 또한 데이터 과학자는 수많은 표를 작성하고 이 수준에서 데이터를 분석하여 신경망을 학습시키지 않는다. 단지 사례를 제공하여 모델을 학습시키면 ML 프레임워크

크가 가중치와 상수를 필요한 대로 모두 조정한다. 실제로 대부분의 문제를 해결하려면 여러 계층이 서로 상호작용해야 한다.

다시 테스트 커버리지로 돌아가면, ML 모델도 궁극적으로는 기계어 코드이지만 구문 커버리지 같은 전통적인 코드 커버리지 측정값을 적용하는 것은 적절하지 않다. 일반적으로 각 예측에서 동일한 코드 줄이 실행되기 때문이다. 위에서 설명한 공식을 다룰 수 있도록 측정값을 조정해야 한다. 이러한 측정값은 다음 질문을 중심으로 구성된다.

원문 PDF 97 페이지

- 테스트가 모든 뉴런에서 0 보다 큰 활성화 값을 한 번 이상 발생시켰는가?
- 테스트가 모든 뉴런에서 양의 활성화 값과 음의 활성화 값을 모두 발생시켰는가?
- 각 뉴런이 활성화 값 사이에 일정한 거리가 있는 두 개의 테스트에 의해 커버되었는가?
- 인접한 계층의 뉴런 쌍을 조합하여 테스트하여, 첫 번째 계층의 활성화 값 변화가 두 번째 계층의 활성화 값 변화로 이어지는지를 확인했는가?

뉴런 커버리지를 측정하면 테스트가 모델의 어느 정도를 실행했는지 이해하는 데 도움이 되지만, 테스트의 기대 결과를 알려주거나 올바른 것을 확인하고 있는지를 알려주지는 않는다. 또한 이러한 커버리지 기준을 측정하는 도구는 널리 이용 가능하지 않으며 ML 프레임워크에 테스트 용이성 기능을 추가해야 한다.

경험 기반 테스트

탐색적 테스트 기법은 체크리스트 기반 기법과 마찬가지로 ML 에 적용할 수 있다. 전통적인 소프트웨어에서 버그가 있을 위치를 추측하기 위해 오류 추정 기법을 사용할 수 있는 것처럼 ML 에도 동일하게 적용할 수 있다.

경험 기반 테스트는 암묵적 오라클을 사용한다. 이는 앞 장에서 논의한 품질 위험과 관련된 휴리스틱 또는 표준화된 점검 항목일 수 있다.

예를 들어 경험 기반 입력 데이터 테스트를 수행할 때 다음과 같이 물을 수 있다.

- (i) 특성에 보호 범주 변수가 있는가?
- (ii) 보호 범주 변수와 상관관계가 있는 다른 특성이 있는가?
- (iii) 보호 범주 변수와 관련된 데이터의 양이 대표성을 갖추기에 부족한가?

체크리스트 기반 기법의 또 다른 예는 운영 준비 상태를 판단하기 위한 Google 의 28 개 ML 점검 목록(Breck et al., 2017)을 사용하는 것이다. 여기에는 특성 선택, 아키텍처, 문서 및 테스트 결과에 대한 점검이 포함된다.

원문 PDF 98 페이지

일부 연구자들(Herbold and Haar, 2022)은 경계값 분석 같은 전통적인 테스트 기법을 사용하여 ML 프레임워크 자체에서 근본적인 결함을 찾을 수 있었다. 예를 들어 많은 특성에 '0'을 전달하여 프레임워크를 충돌시킬 수 있었다.

메타모픽 테스트

메타모픽 테스트는 이전 테스트 실행에서 새로운 테스트 케이스를 생성하는 테스트 기법이다. 일반적으로 테스트는 실행 전에 설계할 수 있기 때문에 이 점에서 독특한 테스트 기법이다. 어떤 면에서는 암묵적 오라클을 사용하는 탐색

적 테스트와 조금 비슷하다. 테스터가 어떤 휴리스틱을 설정하고 이를 사용하여 기대 결과를 결정하기 때문이다. 탐색적 테스트에서 이러한 암묵적 오라클은 '제공근의 결과는 절대 음수가 되어서는 안 된다'일 수 있다. 메타모픽 테스트에서는 입력이 더 많이 제공되면 모델이 더 적은 결과를 제공해야 한다거나, 특정 입력이 주어지면 어떤 것을 예측할 가능성이 더 높아야 한다는 식일 수 있다. 이러한 휴리스틱을 메타모픽 관계(metamorphic relations)라고 하며, 시스템에 확률적인 상대 요구사항을 설정하여 후속 테스트의 기대 결과를 만들 수 있게 한다.

이러한 테스트를 간단하게 상상할 수 있는 예는 검색 결과에서 항목을 필터링할 수 있는 온라인 상점이다. 일반적으로 필터를 적용한 검색은 필터가 없는 검색보다 결과가 적어야 한다. 일련의 테스트 케이스를 실행하면서 점진적으로 필터를 적용하면 결과가 줄어드는 것을 볼 수 있다. 매번 정확히 몇 개의 결과가 나올지는 쉽게 예측할 수 없지만 메타모픽 관계가 성립하는지는 확인할 수 있다.

메타모픽 테스트는 테스트의 기대 결과를 사전에 결정하기 어려울 때 유용할 수 있다. 다시 말해 오라클 문제이다.

A/B 테스트

A/B 테스트는 시스템의 두 가지 변형을 사용하여 결과를 비교하는 기법이다. 이 기법의 이름은 마케팅 이메일이나 웹사이트의 작은 변형을 서로 다른 사용자 집단에 제시하여 반응이 달라지는지를 확인하는 마케팅 기법에서 유래했다.

A/B 테스트는 두 모델을 테스트하는 데 사용할 수 있으며,

원문 PDF 99 페이지

'그라운드 트루스'를 알 수 없는 경우 특히 유용할 수 있다. 예를 들어 개별 사용자에게 관심이 있을 만한 뉴스 기사를 추천한다고 하자. 실제로 그들이 관심 있는 것을 추천하고 있다는 사실을 어떻게 알 수 있을까? 한 가지 방법은 사용자가 무엇을 클릭하는지 모니터링하고 두 모델이 제공한 결과의 클릭률을 비교하는 것이다.

전문가 활용

일부 테스트 오라클 문제는 전문가를 활용하여 테스트가 통과했는지 실패했는지 판단하도록 함으로써 완화할 수 있다. 진단 의학 같은 분야의 전문가를 대체하는 시스템을 생각해 보자. 테스트 과정 어느 시점에는 의료 진단 전문가가 반드시 참여해야 한다.

전문가 활용에도 과제가 있다. 전문가의 역량은 자연스럽게 서로 다르고 의견이 일치하지 않을 수 있다. 인간은 모호성을 포함해 표현하거나 조건을 덧붙이기 위해 응답을 조정할 수 있다. 전문가 시스템이 이를 수행하는 것은 더 어렵다.

전문가는 자동화에 대해 특정한 동기를 가질 수도 있다. 전문가가 동료의 출력을 검토하는지 기계의 출력을 검토하는지 알지 못하도록 평가를 이중맹검으로 해야 할 수도 있다.

적대적 테스트

모델은 적대적 공격에 취약할 수 있다. 적대적 테스트는 모델의 행위를 파악하기 위해 모델을 테스트한 뒤 잠재적 취약점을 식별하는 것이다. 공격자가 하는 것과 정확히 같은 일이지만 목표가 다르다. 공격을 방지하기 위한 것이다.

적대적 테스트에는 도구를 사용하여 모델에 화이트박스 분석을 수행하고 성공 가능한 적대적 공격을 식별한 뒤, 이를 학습 데이터셋에 추가하여 모델이 공격에 덜 취약하도록 하는 방법도 포함될 수 있다. 그러나 공격자는 이런 방식으로 하지 않을 것이다. 공격자는 이러한 취약점의 대부분이 실제로 모델 간 전이 가능하다는 사실을 이용한다. 비슷한 모델을 구축하여 그 모델의 취약점을 찾은 뒤 자신이 무력화하려는 모델에 적용해 볼 수 있다.

적대적 테스트는 기능을 변경하려는 공격을 시뮬레이션한다는 점에서 기능 테스트이면서 보안 테스트라고 볼 수 있다.

원문 PDF 100 페이지

제 2 장에서 언급했듯이 적대적 예시는 컴퓨터 비전과 자연어 처리를 포함한 여러 유형의 AI 응용에서 발견할 수 있다. 두 경우 모두 전체 이미지나 텍스트는 인간에게 동일한 정보를 전달하는 것으로 보이지만 AI 는 다르게 인식한다.

백투백 테스트

병렬 테스트 또는 차이 테스트이라고도 하는 백투백 테스트는 오라클 문제가 있을 때 도움을 준다. 기존 구현 또는 다른 대체 구현을 참조 기준, 즉 의사 오라클(pseudo-oracle)로 사용한다.

테스트 대상 시스템에서 실행하는 모든 테스트를 대체 구현에서도 실행하여 결과를 비교할 수 있다. 이 접근법은 테스트에서 오랫동안 사용되어 왔다. 예를 들어 금융 서비스에서는 수백만 개의 계산을 수행하는 두 개의 대규모 배치 시스템의 결과를 비교할 수 있으며, 각 계산의 기대 결과를 지적으로 정의하는 활동을 하지 않아도 된다.

이를 수행하는 한 가지 방법은 일정 수준의 기능 성능을 대표하는 업계 표준 ML 벤치마크를 사용하는 것이다. 이러한 벤치마크에는 레이블이 지정되어 있을 수 있으므로 테스트 오라클 없이 시스템의 기능 성능을 테스트할 수 있다. 그러나 그 레이블이 어떻게 지정되었는지는 고려해야 한다. 2021 년 연구에서는 널리 사용되는 벤치마크 전체에서 3.4%의 레이블 오류율을 발견했다(Northcutt et al., 2021).

AI 고유 특성 테스트

제 2 장에서는 AI 의 고유한 품질 특성을 논의했다. 이제 그중 일부 특성을 어떻게 테스트할지 살펴보자.

자기 최적화 시스템

자신의 경험에서 학습하는 시스템을 테스트하는 것은 테스트 설계와 유지보수에 여러 과제를 제시한다.

출시 시점에는 시스템의 제약조건과 행위가 명확할 수 있지만,

원문 PDF 101 페이지

그 시점 이후 시스템이 수행한 변경은 알 수 없다. 시스템의 기능적 행위가 변경되었을 수 있으며, 이러한 변경은 전체적으로 시스템을 더 좋게 만들더라도 이전에 통과하던 회귀 테스트를 실패하게 할 수 있다. 이것도 테스트 오라클

문제의 한 사례이다. 시스템이 목표를 달성하는 새로운 방법을 찾아냈을 수 있으며, 그 방법을 쉽게 발견하거나 분석하지 못할 수도 있다.

어떤 시스템이든 테스트 설계를 위해 학습 메커니즘을 이해하는 것이 매우 중요하다. 시스템이 구현 기업의 이해관계자뿐 아니라 많은 이해관계자가 함께 사용하는 공유 시스템일 수도 있다. 클라우드 기반 AI 시스템을 사용하면 프로세스의 세부사항이 사용자에게 숨겨질 수 있으므로, 이해관계자가 학습을 이해하거나 모니터링할 수 있는 능력이 일반적으로 제한된다.

내부 시스템을 사용하고 높은 수준의 통제권을 갖고 있더라도 재학습의 적시성을 생각하고 그 자동화를 테스트해야 할 수 있다. 사용되는 자원도 고려해야 할 수 있다. 시스템이 점점 더 많은 시스템 자원을 사용하면서 목표를 달성할 수 있다면 기능적 행위에는 좋을 수 있지만, 결국 자원이 고갈될 때 실패를 일으킬 수 있다.

또 다른 비기능적 우려는 해커 같은 적대적 행위자이다. 시스템이 학습하는 데이터나 기타 입력을 오염시켜 학습 및 이후 행위에 영향을 주는 것이 가능할 수 있다.

자율 시스템

완전 자율 시스템은 운영에 인간의 모니터링이나 개입이 필요하지 않은 시스템이다. 그러나 이러한 시스템 중 많은 시스템에는 위반될 경우 인간의 개입을 요청하도록 하는 운영 파라미터가 있다. 이 방식이 어떻게 작동하는지 확인하고, 특정 시나리오에서 시스템이 개입을 요청하는지(또는 요청하지 않는지) 보장하기 위한 테스트를 설계해야 할 수 있다. 이러한 시나리오를 확인하려면 운영 환경을 상당히 조작해야 할 가능성이 높다.

알고리즘 편향, 표본 편향 및 부적절한 편향

편향은 제 3장에서 다룬 매우 큰 주제이다. 이를 테스트하려면

원문 PDF 102 페이지

학습 및 입력 데이터를 검토하거나(입력 데이터 테스트), 해당 데이터를 확보할 때 사용한 방법과 의사결정을 검토할 수 있다. 또한 데이터 전처리가 정확하게 수행되고 표본 편향을 도입하지 않는지 보장하고, 모델 테스트 출력을 상세하게 분석하는 것도 포함할 수 있다. 더 나아가 설명가능성 도구를 사용하여 모델의 특정 입력이 출력의 변화를 어떻게 일으키는지 이해할 수 있다.

특히 부적절한 편향과 관련해서는 모델이 직접 처리하지 않는 추가 데이터를 확보하는 것이 매우 유용할 때가 있다. 인구통계 또는 다양성과 관련된 데이터일 수도 있고, 단순히 ‘그라운드 트루스’, 즉 모델의 예측이 올바른지에 대한 판단일 수도 있다.

비결정적 시스템

비결정적 시스템은 동일한 초기 상태와 입력에서도 사용할 때마다 서로 다른 출력을 생성할 수 있는 시스템이다. 일반적으로 시스템에서 회귀 테스트를 실행할 때는 이전과 동일한 결과를 기대하지만, 비결정적 시스템에서는 여러 개의 유효한 출력이 존재할 수 있다.

일반적인 해결책은 시스템을 더 깊이 이해하여 출력이 만족해야 할 암묵적 또는 명시적 허용오차를 정의하는 것이다. 통계적으로 유의한 출력을 얻기 위해 테스트를 매우 많이 실행해야 할 수도 있다.

투명성, 설명가능성 및 해석가능성

투명성과 설명가능성은 제 2 장에서 다루었으며 매우 중요한 특성이다. 이는 결함의 조사 및 해결에 영향을 미칠 뿐 아니라 지속적인 개선에도 필요하다. 설명가능성을 위한 방법과 도구는 빠르게 발전하는 분야이지만 SHAP(Shapley Additive exPlanations) 또는 LIME(Local Interpretable Model-agnostic Explanations) 분석이 좋은 시작점이 될 수 있다. 이러한 도구를 사용하면 입력이 출력에 어떻게 영향을 주는지 분석할 수 있고 이상한 문제를 드러낼 수 있다.

예를 들어 자연어 입력을 기반으로 어떤 것을 분류할 때 SHAP

원문 PDF 103 페이지

분석은 전혀 관련 없는 단어가 분류에 지나치게 큰 영향을 미치고 있음을 보여줄 수 있다.

때로는 사용자가 시스템 기능을 통해 직접 어떤 동작에 대한 설명을 얻을 수 있다. 때로는 운영자가 동일한 작업을 수행해야 할 수도 있다. 여기에서 중요한 요소는 그 과정이 얼마나 쉬운지, 얼마나 적시에 이루어지는지, 얼마나 이해하기 쉬운지이다. 이러한 측면은 ‘해석가능성(interpretability)’에 해당하며, 시스템의 사용자 또는 운영자와 함께 테스트를 수행할 수 있다.

요약

이 장에서는 ML 시스템을 어떻게 테스트할지 살펴보았다. 이 분야는 빠르게 발전하고 있으며 아직 명확한 모범 사례로 표준화되지 않았다. 또한 추론 시스템처럼 ML 이 아닌 다른 유형의 AI 시스템도 존재하며, 이 장에서는 그러한 시스템을 다루지 않았다는 점을 기억할 필요가 있다.

¹ 이러한 거짓 양성은 추가 테스트를 통해 해결할 수 있지만, 큰 모집단에서 거짓 양성인 미치는 영향으로 인해 여전히 수백만 명이 잘못된 양성 결과를 받을 수 있으며 상당한 영향을 받을 수 있다.

원문 PDF 104 페이지

5 인공지능 기반 테스트 자동화

Jeremias Rößler

인공지능은 테스트 자동화의 다양한 작업과 활동에 사용될 수 있으며, 실제로 사용되어 왔다. 여기에는 자동화된 테스트의 자가 복구와 자가 적응부터 테스트 생성과 테스트 데이터 생성, 그리고 그 사이의 여러 측면이 포함된다. 테스트 생성과 테스트 데이터 생성은 코드 수준의 화이트박스 테스트(예: 단위 테스트)부터 인터페이스 수준, 사용자 인터페이스(UI) 수준에 이르기까지 모든 수준에서 이루어질 수 있다. 흥미롭게도 테스트 코드는 생성하기 가장 쉬운 코드이다. 동시에 충분히 높은 코드 커버리지는 자동화된 버그 수정과 자동화된 코드 생성을 가능하게 하는 기반이

다. 이 장에서는 인공지능이 사용될 수 있고 실제로 사용되고 있는 모든 테스트 작업을 살펴본 다음, 테스트 생성과 테스트 데이터 생성에 초점을 맞출 것이다. 마지막으로 코드 생성이 이것에 어떻게 의존하게 될 것이며, 여기에서 다음의 논리적 단계가 될 수도 있는지에 대한 전망으로 마무리한다.

‘AI 품질’이라는 주제는 모호하다. 이는 인공지능 시스템의 품질을 가리킬 수도 있지만, 어떤 시스템이나 제품이 인공지능 시스템에 의해 생산되었거나 품질 보증을 받았다는 의미에서 ‘AI’가 일종의 품질 인증표처럼 쓰이는 것을 나타낼 수도 있다. 이 장에서는 후자를 살펴보고자 한다. 즉, 인공지능 시스템에 의해 품질이 보증되는 시스템, 다시 말해 소프트웨어를 다룬다.

품질 보증

품질 보증은 궁극적으로 인간의 영역이다. 많은 상황에서 원하는 결과를, 그 정의가 잘못 해석될 수 있는 모든 가능성을 고려하면서 빈틈없이 정의하기는 어렵다.

원문 PDF 105 페이지

이는 특히 물리적 세계의 많은 ‘자연적인’ 제약이 적용되지 않는 소프트웨어에서 더욱 그러하다.

디지털 영역에서는 사실상 무엇이든 가능하다. 따라서 최종 결과가 정의된 기준을 수용 가능한 방식으로 충족하는지, 또는 암묵적이고 표현되지 않은 어떤 요구사항을 위반했는지에 대해 인간이 최종 품질 보증 점검을 수행하는 것이 매우 중요하다. 앞서 논의했듯이 이를 오라클 문제라고 한다.

오라클 문제

오라클 문제는 옳고 그름을 구분하는 문제이다. 즉, 계산이나 프로세스의 결과에 어떤 방식으로든 결함이 있는지를 판단하는 문제이다. 오라클 문제는 소프트웨어의 개별 기능에도 적용되며, 소프트웨어 전체에도 동일하게 적용된다. 소프트웨어 개발 프로세스의 결과물, 즉 소프트웨어에 어떤 방식으로든 결함이 있는가?

결국 모든 것은 맥락, 의도, 의사소통으로 귀결된다. $12 + 1 = 1$ 이 맞는지는 우리가 일반적인 수학 계산의 맥락에 있는지, 아니면 한 해의 월을 계산하는 날짜 조작의 맥락에 있는지에 달려 있다. 이러한 각각의 판단에는 대개 고려해야 할 더 큰 맥락과, 그 맥락에 대해 전달된 의도가 있다. 누군가에게 단지 사진 공유 앱을 원한다고 말하는 것만으로는 Snapchat 이 만들어질 수도 있고 Instagram 이 만들어질 수도 있으며, 두 앱은 모두 그 목적을 매우 다른 방식으로 수행한다.

실제 소프트웨어 개발에서는 제품 책임자가 일반적인 아이디어를 서로 다른 수준의 구체성으로 정의하고, 테스터와 개발자가 상식을 적용하여 나머지 빈틈을 채우는 경우가 많다. 현재로서는 테스트이나 개발이라는 이 두 작업 중 어느 하나에 인공지능을 적용하는 것도 가능하지 않다. 빈틈을 채우고, 맥락을 추론하고, 의도를 해석하는 데 필요한 상식 수준은 대략 인간 수준의 지능을 필요로 하기 때문이다. 두 작업에 필요한 노력과 발생 비용이 비슷하다는 점을 고려하면, 언젠가 인공지능이 둘 중 하나를 수행할 만큼 강력해진다면 코드를 생성하는 데 사용될 것이라고 보는 것이

타당하다. 코드 자체가 의도된 동작에 대한 형식적 명세이기 때문이다. 앞서 언급했듯이, 인간에게는 암묵적이고 표현되지 않은 요구사항이 위반되었는지를 확인하는 일이 독창적인 코드 해결책을 만들어내는 것보다 훨씬 쉽다.

원문 PDF 106 페이지

오라클 문제를 일반적인 방식으로 해결하는 것이 얼마나 어려운지 진정으로 이해하려면, 애초에 왜 이것을 ‘오라클’ 문제라고 부르는지 생각해 보아야 한다. 모든 문제와 그에 대한 각각의 해결책은 소프트웨어로 표현될 수 있다. 미래를 알아내거나 신이 존재하는지를 판단하는 것과 같은 터무니없어 보이는 작업까지도 그렇다. 여기서 말하는 것은 이러한 문제의 해답을 소프트웨어로 올바르게 구현하는 것이 아니라, 단지 어떤 해결책이든 존재하는 경우를 말한다. 가장 단순한 해결책의 최소 구현은 제시되는 모든 질문에 항상 ‘예’라고 답하는 것일 수 있다. 많은 질문에 대한 그 답의 정확성, 따라서 소프트웨어의 정확성을 판단하려면 예를 들어 미래를 알거나 신이 존재하는지를 알아야 한다. 바로 이 때문에 고대 그리스 신화의 오라클이 필요한 것이다. 그래서 이러한 이름이 붙었다.

테스팅과 테스트 자동화

소프트웨어는 보통 반복적이고 점진적인 방식으로 개발된다. 첫 번째로 동작하는 버전에서 시작하여 버전 번호가 증가함에 따라 소프트웨어가 확장되고 개선된다. 소프트웨어 개발자가 아닌 사람에게는 때때로 의존성이 서로 관련 없어 보이는 소프트웨어 부분들을 알 수 없고 거의 신비로운 방식으로 연결한다는 것을 이해하기 어렵다. 이는 작은 변경이라도 예상하지 못한 위치에 예상하지 못한 부작용을 일으킬 수 있음을 의미한다. 이 문제를 해결하려면 소프트웨어를 지속적으로 테스트해야 한다. 과거에 이미 테스트한 부분과 기능도 다시 테스트해야 한다. 이는 본질적으로 많은 반복적이고 중복된 작업을 요구하므로 자동화가 필요해진다. 자동화에는 테스트 프로세스의 속도를 높여 이상적으로는 모든 변경 후에 테스트를 수행할 수 있게 한다는 추가적인 이점도 있다.

참고: 이 장의 맥락에서 말하는 테스트 자동화는 회귀 테스트 또는 유지보수 테스트를 위한 테스트 자동화를 의미하며, 그 밖의 테스트 자동화 목적은 다르지 않는다.

하지만 자동화에 따라 테스트를 구현하는 방식이 바뀌는 것처럼, 그 목표도 바뀐다. 테스트의 목표는 결함과 결점을 찾는 것이다. 이것은 기술적으로 테스트 자동화에서도 여전히 사실이지만,

원문 PDF 107 페이지

오라클 문제 때문에 일반적인 방식으로 달성할 수는 없다. 또한 대부분의 경우 그럴 필요도 없다. 수동 테스트를 통해 소프트웨어의 정확성이 이미 확인되었다고 가정할 수 있으므로, 테스트 자동화의 목표는 소프트웨어가 여전히 정확한지를 보장하는 것으로 바뀐다. 즉, 테스트 자동화는 소프트웨어의 동작이 의도하지 않았고 원하지 않는 방식으로 변경되는 경우를 찾는 것을 목표로 한다.

이를 일관성 오라클이라고 한다. 소프트웨어는 동일한 소프트웨어의 이전 버전과 일관되어야 한다. 현재 대부분의 상황에서처럼 이 오라클은 명시적으로 정의할 수 있다. 흥미로운 점은 이 오라클을 도출할 수도 있다는 것이다. 비교할 이전 버전의 소프트웨어가 있다면 두 버전을 병렬로 실행하고 실행 결과와 특성을 비교하기만 하면 된다. 또는 이전 버전의 해당 결과와 특성을 기준선으로 저장한 뒤 그 기준선과 비교할 수 있다.

일관성 오라클을 적용하려면 소프트웨어가 대부분 결정론적이어야 한다. 또한 소프트웨어에 작은 변경만 가하고, 동작의 변경을 기준선과 지속적으로 비교하며, 의도된 변경으로 기준선을 갱신하는 것이 매우 유용하다. 그렇지 않으면 너무 많은 변경에 압도되어 많은 노력이 발생하고 접근 방식의 목적이 무색해질 위험이 있다.

테스트 자동화는 막대한 비용 요인이며 추가 노력과 위험의 주요 원인이기 때문에, 인공지능이 이 문제를 해결하도록 하는 것은 매우 가치 있는 목표이다. 다음 절에서는 먼저 이 특정 작업에 초점을 맞춘 뒤, 인공지능이 테스트와 품질 보증 영역에서 수행할 수 있는 다른 일들을 다룬다.

단위 테스트 자동화에서의 인공지능

인공지능은 자동화된 테스트를 생성하는 데 사용할 수 있다. 그러나 어느 수준에서 이를 수행하느냐에 따라 직면하는 문제가 크게 달라진다. 이 장에서는 흔히 코드, 모듈, 컴포넌트 또는 단위 수준이라고 부르는 수준에 구체적으로 초점을 맞춘다.

많은 사람은 대부분의 테스트 자동화를 이 수준에 두기를 원한다. 이러한 바람은 흔히 ‘테스트 피라미드’(Cohn, 2009)로 표현된다(그림 5.1).

원문 PDF 108 페이지

그림 5.1 테스트 피라미드의 개략도(출처: Cohn, 2009 를 바탕으로 함)

기본적인 생각은 UI 수준의 테스트 자동화가 취약한 테스트를 만든다는 것이다. 이 경우 작은 UI 변경 하나만으로도 이러한 테스트 중 다수가 올바르게 실행되지 못할 수 있다. 이는 흔히 테스트를 수정하기 위한 높은 유지보수 비용이나 문제를 피하기 위한 높은 개발 비용, 또는 둘 모두로 이어진다. 이러한 테스트를 실행하려면 일반적으로 전체 시스템을 시작해야 하며, 때로는 여러 번 시작해야 하기 때문에 시간이 많이 걸린다. 게다가 이러한 테스트는 많은 외부 의존성을 가진 복잡한 소프트웨어 스택을 필요로 하므로 실행이 비결정론적인 경우가 많다. 이를 ‘플레이키니스(flakiness)’라고 한다. 이러한 비결정성은 많은 거짓 양성(false positive)을 발생시키며, 변경에 대한 취약성과 함께 신뢰를 약화시켜 결국 많은 프로젝트에서 UI 테스트가 도움보다는 부담이 되게 할 수 있다.

이러한 모든 이유 때문에 대부분의 테스트를 단위 테스트 수준에 두는 것이 바람직하다. 그러나 이러한 테스트는 최종 사용자가 소프트웨어를 사용하고 경험하는 방식을 나타내지 않으므로, 서로 다른 종류의 테스트가 섞인 전체 구성의 일부일 뿐이다. 이를 염두에 두는 것이 중요하다. 그렇지 않으면 초점이 순수하게 기능적인 측면으로 이동하여 시각적 요소와 사용자 경험 같은 더 정성적인 측면을 무시할 수 있다.

이러한 테스트는 코드 안에 존재하고 코드로 정의되므로 개발자가 작성해야 한다. 이러한 테스트를 생성할 수 있는 인공지능 시스템은 비용을 줄이는 데 있어 큰 진전이 될 것이다. 이러한 이유로 인공지능이 이러한 테스트를 어떻게 생성할 수 있는지에 관한 많은 연구가 이루어졌다. 안타깝게도 시장에 출시할 수 있는 해결책을 갖기 위해서는 여전히 많은 과제를 해결해야 한다. 다음 절에서는 각 과제를 개별적으로 분석한다.

단위 테스트를 위한 테스트 오라클

앞서 이미 확인했듯이, 테스트 자동화를 생성할 때 일관성 오라클을 사용할 수 있다. 일관성 오라클은 소프트웨어가 동일한 소프트웨어의 이전 버전과 일관되어야 한다고 가정한다. 그러나 사용자의 관점에서 본 소프트웨어 동작에는 이것이 사실이지만, 그 동작의 저수준 구현 세부사항에서는 크게 다를 수 있다. 따라서 일관성 오라클을 어떻게 명세 하는지에 큰 주의를 기울여야 하며, 단위 테스트 자동화 영역에서는 보통 assertion 의 형태로 이를 수행한다. 견고 하면서 중복되지 않는 이상적인 단위 테스트는 테스트 대상 개별 단위의 단일 측면만 보장해야 하며, 그 밖의 것은 보장하지 않아야 한다. 그러나 코드 수준에서는 시스템 내부 상태의 대부분에 접근하거나 도달할 수 있으므로, 인공지능이 현재의 테스트 대상 단위나 현재 보장하려는 특정 측면에 초점을 맞추기 어렵다. 인공지능은 종종 의미가 없거나 관련이 없는 측면을 보장하게 된다. 특히 상태 변경이 어떤 연산의 원하는 결과일 때 이 문제가 분명해진다. 이러한 이유로 많은 학술 문헌은 변환과 계산처럼 상태가 없고 부작용이 없는 연산에 초점을 맞춘다.

문제를 설명하기 위해 소프트웨어에 대한 테스트를 생성한다고 가정해 보자. 코드에는 `user.setActive()`라는 연산이 있으며, 원하는 결과는 사용자의 상태가 'active'로 변경되는 것이다. 연산이 성공했는지를 확인하기 위해 사용자의 내부 상태에 직접 접근하는 것은 보통 바람직하지 않다. 인공지능은 무엇이 관련 있는지 알지 못하기 때문에 무작위로 생성된 비결정적 ID 나 특정 타임스탬프를 assertion 대상으로 선택할 수 있다. 사람이 만든 테스트라면 `user.getState()`와 같은 적절한 메서드를 사용하여 사용자 객체를 조회할 가능성이 높다. 사용자 영속성 단위는 아마 다른 많은 연산도 제공할 것이므로, 가능한 모든 선택지 중 해당 조회 연산을 선택하는 것은 어려운 문제이다. 물론 이러한 단순한 예는 변경되는 측면을 찾아내기 위해 연산을 여러 번 실행함으로써 해결할 수 있다. 그러나 일반적으로 데이터의 어떤 특정 측면을 문자 그대로 보장해야 하고 어떤 측면은 더 추상적인 방식으로 보장해야 하는지를 결정하는 것은 어렵고, 이 결정은 테스트의 플레이키니스와 변경에 대한 견고성 모두를 좌우한다.

UI 수준에서 흔히 사용되는 또 다른 오라클은 암묵적 견고성 오라클이다. 사용자가 무엇을 하든 소프트웨어는 절대로 충돌해서는 안 된다는 것이다. 코드 수준에서는 견고성이 일반적으로 인터페이스에서만 보장되기 때문에 예외와 충돌이 문제가 아닐 수 있다. 따라서 테스트 대상 단위에 null 참조와 같은 잘못된 데이터를 전달했을 때 예외를 발생시키고 충돌하는 것이 예상된 동작일 수도 있으며, 이 경우 이 오라클은 코드 수준의 테스트 생성에는 유효하지 않다.

사전조건과 파라미터

특정 단위에서 흥미롭거나 관련 있는 기능을 유발하기 위해 필요한 사전조건은 임의로 복잡할 수 있다. 예를 들어 코드가 이미 설정된 데이터베이스 연결의 존재나 특정 환경 변수의 정의 등을 기대할 수 있다. 최근의 모킹 기술(예: Mockito)의 발전과 실행 경로 분석을 결합하면 이 문제를 어느 정도 완화할 수 있다. 그러나 특정 단위나 테스트 대상 측면과 관련된 변경을 결정하는 앞서의 문제와 마찬가지로, 어떤 동작이 관련 있는지를 자동으로 결정하는 것은 여전히 어려운 문제이다. 너무 많이 모킹하면 자동화된 테스트는 결국 관련 기능이 아니라 모크(mock)를 테스트하게 된다. 충분히 모킹하지 않으면 관련 없는 사전조건까지 설정하게 되어 견고성이 낮아지고, 테스트가 실패했을 때 개발자가 디버깅하기 어려워진다. 본질적으로 테스트를 통합 테스트로 바꾸게 되는 것이다.

이 문제를 설명하기 위해 사용자 영속성 단위의 save(User) 연산에 대한 테스트로 다시 돌아가 보자. 원격 데이터베이스와의 연결을 설정하는 것은 어느 정도 비용이 들며, 데이터베이스 내부를 테스트하는 것이 목표는 아니다. 데이터베이스는 올바르게 동작한다고 안전하게 가정할 수 있다. 따라서 데이터베이스를 모킹하고 영속성 로직, 예를 들어 객체 검증과 SQL 문의 올바른 생성 및 발행만 테스트하는 것은 충분히 타당하다. 보통 인터페이스를 모킹하는 것이 좋은 생각이지만, 문제는 어떤 인터페이스를 모킹할지를 결정하는 것이다. 예를 들어 원격 연결의 통신 수준이나 기반 통신 프레임워크의 API 수준에서 이를 수행할 수 있다. 두 경우 모두 현재 구현 상태를 단지 ‘고정’할 뿐 정확성을 검증하지 않는다는 단점이 있다. 기반 SQL 스키마가 변경되면 테스트 대상 단위가 운영 환경에서 오작동하거나 완전히 실패할 수 있지만, 이 테스트에서는 이를 감지하지 못한다. 반면 사람이 작성한 테스트는 아마 인메모리 데이터베이스를 사용하도록 선택할 것이다. 이러한 데이터베이스는 원격 연결을 필요로 하지 않는다는 점에서 API와 비슷하게 동작하지만, SQL 문이 변경되지 않았는지를 검증하는 대신 현재 스키마에서 SQL 문이 동작하는지를 검증한다. 스키마 자체가 변경될 수 있더라도 마찬가지다.

많은 연산은 복잡한 파라미터와 함께 호출되기도 한다. 생성된 테스트에서는 이러한 파라미터를 현재 테스트하는 측면의 의도에 맞는 방식으로 만들어야 한다. 앞서 언급한 필요한 사전조건은

원문 PDF 111 페이지

파라미터로 바꿀 수도 있고 그 반대로도 할 수 있기 때문에, 이 과제를 구분하는지는 단지 코드가 어떻게 구성되어 있는지에 달려 있다. 복잡한 파라미터를 이야기할 때는 흔히 데이터와 그 유효성 및 구조가 강조된다.

이를 설명하기 위해 사용자 영속성 단위의 save(User) 연산이 전달된 복잡한 User 객체 안에 유효한 이메일 주소가 설정되어 있기를 요구한다고 하자. 이메일 주소의 유효성이 데이터베이스 수준에서 검증되는 것도 충분히 가능하다. 이 코드는 테스트 시스템의 범위 밖에 있고 실행 중 추가 분석을 위해 접근할 수도 없을 가능성이 높기 때문에, 테스트 생성 시스템이 문제를 ‘이해’하고 스스로 유효한 이메일 주소를 만들어내는 것은 매우 어렵다. 이 단순한 시나리오에서는 성공하고 올바른 실행을 관찰하고 학습할 수 있다면 개인정보 보호 문제에 유의하면서 문제를 쉽게 완화할 수 있다. 그러나 실제 세계에는 복잡한 의존성을 가진 많은 복잡한 시나리오가 있으며, 테스트 생성 시스템이 이를 ‘이해’하고 탐색하기는 어렵다.

프로토콜과 내부 상태

많은 실제 시나리오에서 테스트 대상 단위에는 관련 연산들, 이하에서는 메서드라고 부르는 것들이 호출되는 순서와 호출 시 사용되는 파라미터에 의해 영향을 받고 조작되는 내부 상태가 있다. 이 상태 변경이 특정한, 흔히 암묵적인 프로토콜에 따라 올바르게 유발되고 수행되는지를 검증하는 것이 바람직한 경우가 많다. 연결 설정 및 종료, 파일 열기 및 닫기 등을 생각해 볼 수 있다. 그러나 내부 상태의 변경이 중요한지 아닌지를 판단하는 것은 매우 어려운 문제이며, 본질적으로 정보와 잡음을 구분하는 문제로 귀결된다. 전체 선택지 수는 순열에 따라 곱해진다. 즉, 내부 상태가 10 개의 불리언 변수로 구성되어 있다면 해당 단위는 $2^{10} = 1024$ 개의 내부 상태를 가질 수 있다. 따라서 많은 실제 시스템에서는 의미 있는 구분만 고려하더라도 가능한 상태의 수가 이미 상당히 많다. 추상화 능력이 없다면 시스템의 상태 공간은 헤아릴 수 없을 정도로 크거나 심지어 무한할 수도 있다. 이러한 이유로 이 문제는 잘 알려진 ‘상태

공간 폭발 문제’(E. M. Clarke et al., 2012)라고 불린다. 이 때문에 가능한 모든 상태 변경에 대해 테스트를 생성하는 것은 사실상 불가능하다. 따라서 테스트 생성 시스템은 무엇이 관련 있는 상태인지, 어떤 상태 변경에 대한 테스트 생성을 우선해야 하는지를 결정하거나 우선순위를 정해야 한다.

원문 PDF 112 페이지

앞서의 사용자 연속성 단위를 다시 사용해 설명해 보자. User 객체를 연속화하고 로드하기 위해 해당 단위는 어떤 형태로든 데이터베이스에 연결할 것이다. 연속성 단위의 내부 상태를 분석할 때는 추상화 수준을 결정해야 한다. 이는 연결 객체를 이진적인 것으로 취급할 수도 있다. 즉, 단위 내부에 존재하거나 존재하지 않는 것으로 볼 수 있다. 또는 분석을 연결 객체 자체의 내부까지 확장할 수도 있다. 연결을 이진적으로 취급하는 것은 지나치게 거칠어 중요한 경우를 놓칠 수 있다. 예를 들어 연결 객체 자체는 존재하지만 기반 네트워크 연결이 아직 설정되지 않았거나 이미 종료되었을 수 있다. 하지만 분석을 연결 객체 내부로 확장하면 각 수준에서 추상화에 관한 동일한 결정을 재귀적으로 내려야 한다. 추적되는 모든 데이터 지점은 내부 상태를 구분할 수 있는 잠재적인 요소이다. 정확한 타임스탬프가 관련 있는가? 현재 시간과의 차이가 관련 있는가? 그렇다면 어느 정도의 세분성으로 보아야 하는가? 이것은 인간에게조차 이미 이해하기 어렵다. 그렇지 않았다면 애초에 테스트할 필요도 없었을 것이다. 결국 어떤 추상화가 타당한지를 결정하는 문제는 유명한 정지 문제(halting problem)의 한 사례로 귀결된다. 가능한 모든 프로그램에 대해 특정 상태 변경이 차이를 만드는지, 아니면 추상화해도 되는지를 확실하게 알 수 있는 유일한 방법은 프로그램을 실행하는 것이다.

테스트 코드 생성

단위 테스트 생성의 결과는 보통 코드 형태의 단위 테스트이다. 대부분의 개발자는 코드의 구조와 가독성이 매우 중요하다고 생각하며, 이는 소프트웨어 장인정신 선언(Manifesto for Software Craftsmanship, n.d.)과 클린 코드 이니셔티브(Clean Code Developer, n.d.) 같은 흐름과 운동, 그리고 ‘코드 스멜’을 검사하는 정적 분석 도구를 낳았다.¹ 이러한 태도는 코드가 작성되는 횟수보다 읽히는 횟수가 10 배 많다는 점(Martin, 2008)을 고려하면 이해할 수 있다. 그러나 ‘가독성’은 쉽고 명확한 개념이라기보다 하나의 예술에 가까워 엄격한 규칙으로 만들기 어렵다. 각 언어와 흔히 각 테스트 프레임워크마다 읽기 좋은 코드가 무엇인지에 대한 개념이 다르다는 점 때문에 문제는 더욱 복잡해진다.

명확한 초점과 의도

이 문제는 다른 몇 가지 과제와 겹친다. 앞서 논의했듯이, 이상적인 단위 테스트는 테스트 대상 개별 단위의 한 가지 측면만 보장해야 하며 그 밖의 것은 보장하지 않아야 한다. 인간이라면 이상적으로 특정 단위나 기능을 분석하고 의도된 사용 사례와 가능성이 있는 몇 가지 경계 사례를 모두 정의한 뒤 각각에 대한 테스트를 만들 것이다. 테스트 주도 개발은 실제 구현보다 먼저 이러한 테스트를 작성할 것을 제안한다. 이는 하향식 접근 방식이다.

원문 PDF 113 페이지

이해와 맥락이 부족한 테스트 생성 시스템은 대신 일반적으로 상향식 접근 방식을 적용한다. 코드를 분석하고 그 동작을 분석한 뒤 각각의 특정 경우에 대한 테스트를 생성하여 가능한 한 많은 코드를 커버하려고 한다. 두 접근 방식에

는 각각 장단점이 있다. 하향식 접근 방식은 어느 정도 더 구조적이고 테스트 케이스를 ‘논리적으로’ 구분할 수 있게 하지만, 상향식 접근 방식은 명세에 따르면 존재해서는 안 되지만 어떤 이유로 구현되어 버린 특정 경우를 감지할 수 있다. 이는 오라클 문제의 한 사례이다. 특정하고 구별되는 기존 동작의 경우가 의도된 것인지, 아니면 오류에 해당하는지를 결정하는 문제이다.

명백히 초점과 의도의 문제는 프로토콜 및 내부 상태의 문제, 즉 무엇이 관련 있는 상태이며 어떤 상태 변경에 대해 테스트를 생성해야 하는지와 매우 밀접하게 관련되어 있다. 그러나 지금까지 논의하지 않은 추가적인 측면은 전체적인 우선순위 문제이다. 시스템의 나머지 부분과 비교했을 때 특정 함수, 단위 또는 측면은 어느 시점에 ‘충분히’ 테스트되었다고 볼 수 있는가?

단위 테스트에서의 인공지능: 요약

앞서 본 것처럼 단위 수준에서 테스트를 생성하려고 할 때는 여러 과제가 있다. 그러나 기대되는 효율성 향상 때문에 이 분야에서는 계속 연구가 진행되고 있으며, 상용 도구와 무료로 사용할 수 있는 오픈소스 도구가 모두 등장하고 있다. 기존 코드를 위한 단위 테스트를 생성하는 오픈소스 도구 중 하나는 EvoSuite(Fraser, n.d.)이다.

UI 수준 테스트 자동화에서의 인공지능

단위 수준에서 테스트를 생성할 때 발생하는 많은 과제는 UI 수준이나 시스템 수준에서 테스트를 생성할 때 훨씬 작아지거나 전혀 발생하지 않는다(Gross et al., 2012).

UI 테스트를 위한 테스트 오라클

일관성 오라클과 암묵적 견고성 오라클은 단위 또는 코드 수준에서는 쉽게 적용되지 않거나 전혀 적용되지 않지만,

원문 PDF 114 페이지

사용자 인터페이스 수준에서는 둘 다 적용된다. 사용자는 한 버전에서 다음 버전으로 이어지는 일관성을 중요하게 여기므로, 사용자 인터페이스는 신중하고 의도적으로 변경하는 것이 매우 중요하다. 의도하지 않은 부작용과 승인되지 않은 변경은 모두 결함이다. 따라서 일관성 오라클을 사용하여 소프트웨어 반복 개발 과정에서 시스템 경계의 일관성을 검증할 수 있다. 소프트웨어 버전 간 그 경계에서 나타나는 차이는 의미가 있는 경향이 있으며, 소프트웨어가 결정론적이라면 어떤 경우에도 잡음의 비율이 훨씬 낮다.

견고성 오라클 역시 적용된다. 사용자의 관점에서 소프트웨어는 입력이 무엇이든 절대로 충돌해서는 안 된다. 따라서 유발할 수 있는 모든 충돌은 결함이다. 또한 소프트웨어는 일정한 시간 이내에 응답해야 한다.

흥미롭게도 위의 모든 관찰은 기술적인 인터페이스를 포함한 모든 시스템 인터페이스에도 동일하게 적용되며, API도 포함된다.

사전조건, 파라미터, 반복 가능성 및 상호의존성

UI 를 사용할 수 있으려면 일반적으로 전체 시스템이 시작되어야 한다. 대부분의 독립형 소프트웨어에서는 이것만으로 필요한 모든 사전조건이 생성된다. 즉, 설정을 로드하고 데이터베이스 연결을 설정하는 등의 작업을 수행하므로 앞서 단위 테스트에서 발생한 문제를 해결한다.

하지만 전체 소프트웨어를 테스트가 실행될 때마다 시작하거나 적어도 재설정해야 한다는 사실은 앞서 언급했듯이 이러한 종류의 테스트가 실행 주기가 길다는 문제를 더한다. 또한 특정 테스트 실행의 부작용은 데이터의 생성, 변경 또는 삭제이든 설정 변경이든 오래 지속되는 경향이 있다. 이 때문에 같은 데이터나 설정에 접근하는 테스트들이 서로 상호의존적인 경우가 많다. 더 나쁜 것은 이러한 상호의존성이 흔히 비의도적으로 만들어져 숨겨진 채 존재하다가 시간이 지나 하나 이상의 문제가 나타나면서 드러난다는 점이다.

테스트가 서로 상호의존적인 것은 여러 이유로 매우 바람직하지 않다. 앞서 설명했듯이 UI 테스트는 실행 주기가 긴 경향이 있다. 이를 쉽게 완화하는 방법은 병렬화와 우선순위화이지만, 상호의존적인 테스트에는 둘 다 적용할 수 없다. 또한 상호의존성은 이미 취약한 테스트를 더 쉽게 깨지게 하며, 원인-결과 분석의 복잡성과 그에 따른 유지보수 비용을 증가시킨다.

원문 PDF 115 페이지

(소프트웨어와 테스트가 모두 변경되는 경우) 가장 단순하지만 다행히도 가장 흔한 사례는 하나의 테스트가 실패하면 이에 의존하는 몇 개의 테스트도 함께 실패하여 원래 원인을 알아보기 어렵게 만드는 경우이다. 테스트가 더 미묘한 방식으로 서로 의존한다면 분석은 훨씬 어려워진다. 예를 들어 한 테스트가 어떤 데이터를, 심지어 비의도적이거나 암묵적으로 변경하고 다른 테스트가 그 데이터에 의존할 수 있다. 이는 수동 디버깅도 훨씬 어렵게 만들며, 누군가 디버깅하려고 하면 사라지는 문제인 ‘하이젠베르크 버그(Heisenberg Bugs)’ 또는 ‘하이젠버그(Heisenbugs)’를 발생시킨다.

앞서 논의한 또 다른 과제는 입력 파라미터의 생성이다. 코드 수준에서는 복잡한 데이터 객체부터 데이터베이스 연결까지 말 그대로 무엇이든 입력 파라미터가 될 수 있다. 반면 UI 를 위한 테스트를 생성할 때 입력 파라미터는 포인트 및 클릭 동작과 개별 입력 필드에 주로 생성되는 문자열 같은 기본형으로만 구성된다. 유효한 XML 과 같은 복잡한 입력 데이터는 테스트 대상 UI 의 유형에 따라 다르기는 하지만 매우 드물게 필요하다. 이 때문에 필요한 데이터를 무차별 대입 방식으로 생성하거나 개별 기록을 재사용할 수 있는 경우가 많다. 기존 프로토콜은 UI 설계에 의해 강제되거나 적어도 암시되는 경향이 있어 이 과정은 더욱 쉬워진다.

몽키 테스트

무한이 얼마나 광대한지 경이롭게 생각해 본 적이 있는가? 그것은 헤아릴 수 없다. 그리고 많은 흥미로운 사실 중 하나는 진정으로 무작위인 수의 연속, 즉 절대로 반복되지 않는 수열에는 다른 모든 수가 포함된다는 것이다. 이는 아르키메데스 상수 π 안에 성경이, 그리고 성경의 모든 판본과 모든 언어 버전이 포함되어 있다는 뜻이다.

이 논리를 따르면 무한 원숭이 정리는 원숭이가 무한한 시간 동안 타자기의 키를 무작위로 두드리면 결국 윌리엄 셰익스피어의 전집을 포함하여 어떤 텍스트라도 입력하게 된다고 말한다. ‘몽키 테스트’이라는 용어는 이 정리와 시스

템과 상호작용하는, 흔히 무의미한 방식 모두에서 유래했다. 몽키 정리에 따르면 충분한 시간이 주어질 경우 몽키 테스트는 이미 시스템에 가능한 모든 테스트를 생성하는 문제를 완전히 해결한다는 점에 주목해야 한다. 남은 유일한 문제는 대부분의 프로젝트에서 ‘무한’이라는 시간이 실용적이지 않다는 것이다. 따라서 몽키에 휴리스틱과 인공지능을 추가하는 것은 이를 더 효율적으로 만들기 위한 것이다. 즉, 몽키가 더 의미 있는 테스트에 더 빨리 도달하도록 하는 것이다.

원문 PDF 116 페이지

몽키 테스트는 테스트 실행을 생성하는 수단일 뿐이며, 생성된 실행에는 당연히 테스트 오라클이 없다. 따라서 전통적으로는 사용자가 무엇을 하더라도 시스템이 충돌해서는 안 된다는 암묵적 테스트 오라클과 함께 사용되어 왔다. 이러한 목적으로 몽키 테스트는 1980년대부터 시스템의 견고성과 안정성을 테스트하는 데 사용되어 왔다.

퍼즈 테스트는 (의사) 난수 생성기를 사용하여 테스트 데이터를 만드는 것이다. 어떤 의미에서는 클릭, 스와이프, 텍스트 입력과 같은 사용자 상호작용도 무작위로 생성된 특별한 종류의 테스트 데이터로 볼 수 있다. 따라서 몽키 테스트는 퍼즈 테스트의 특수한 경우이지만 생성되는 데이터 유형과 적용 가능한 인터페이스 유형이 제한된다.

이는 몽키 테스트에 퍼즈 테스트와 유사한 인공지능 접근 방식을 적용할 수 있음을 의미한다. 유전 알고리즘을 사용하여 기존의 기록된 테스트 케이스의 변형과 조합을 생성할 수 있다. 신경망도 사용할 수 있으며, 일반적인 사용자가 무엇을 할 것인지, 즉 어떤 필드와 버튼을 어떤 순서로 사용할지를 학습할 수 있다. 이는 본질적으로 그래픽 사용자 인터페이스(GUI)의 사용을 게임 플레이와 비교하는 것이다. 주어진 어떤 GUI 상태에서도 인공지능은 다음에 어떤 수를 둘지, 즉 어떤 사용자 상호작용을 수행할지를 결정해야 한다. 특히 흥미로운 점은 이러한 신경망에 인코딩되는 지식이 흔히 특정 소프트웨어에 한정되지 않는다는 것이다. 그 일부는 피츠의 법칙(Fitts' law)²과 같은 사용자 경험 설계의 일반 규칙을 나타낸다.

이러한 접근 방식을 결합할 수도 있다. 진화 알고리즘은 시드 개체군(seed population)에서 시작한다. 이 시드 개체군은 인간이 생성할 수도 있고, 예를 들어 사용자 스토리나 사용자 상호작용의 기록으로 만들 수 있으며, 무작위로 생성할 수도 있다. 시드 개체군이 좋을수록 결과가 더 좋고 더 빠르게 나온다. 따라서 시드 개체군이 일부라도 무작위로 생성된다면 신경망을 사용하여 사람과 더 유사한 테스트를 생성할 수 있다.

게임 플레이에서는 인공지능의 목표인 ‘승리’가 명확한 반면, 테스트에서는 목표가 그만큼 명확하지 않다. 목표는 인공지능이 완전히 자동화된 방식으로 측정할 수 있어야 하므로, 사람이 평가해야 하는 ‘정성적’ 기준은 사용할 수 없다. 단순한 목표는 최대 코드 커버리지일 수 있지만, 이는 사용자의 관점에서 비현실적이거나 무의미한 테스트를 만들어내는 경우가 많다. 더 타당한 목표는 입력 커버리지 또는 컨트롤 커버리지일 수 있다. 올바른 목표를 선택하는 것은 인공지능 알고리즘의 성능에 큰 영향을 준다.

원문 PDF 117 페이지

테스트 선택과 우선순위화

앞서 자세히 설명했듯이 UI 테스트는 실행 시간이 긴 경향이 있다. 이 때문에 모범 사례에서 요구하는 것처럼 커밋마다, 풀 리퀘스트마다, 또는 심지어 매일 밤 정기적으로 실행하기에는 자동화된 테스트가 너무 많아지는 상황이 빠르게 발생할 수 있다. 병렬화를 하더라도 필요한 계산 능력과 그에 따른 비용이 인식되는 이점에 비해 지나치게 클 수 있다. 게다가 취약성과 불안정성 때문에 많은 테스트가 실패하게 되며, 각각을 조사해야 하고 그중 많은 경우가 거짓 양성으로 밝혀질 수 있다. 이 모든 것은 테스트 선택과 우선순위가 필요하다는 강력한 근거이다.

테스트 우선순위는 코드 커버리지와 실행 경로의 테스트 유사성 같은 런타임 정보를 활용하며, 테스트 효과성, 즉 과거에 발견한 결함의 수와 심각도 같은 이력 정보를 이용하여 더욱 향상할 수 있다(Juergens et al., 2011).

객체 식별과 식별자 선택

객체 식별은 UI 테스트에 고유한 과제이며 다른 테스트 시나리오에서는 발생하지 않지만, 예를 들어 로보틱 프로세스 자동화(RPA)에서는 나타난다. 테스트 시스템은 사용자의 입력을 모사하고 예상 결과를 검증하기 위해 테스트 대상 소프트웨어와 상호작용해야 한다. 이를 위해 버튼, 텍스트 필드, 레이블과 같은 동일한 UI 요소를 신뢰성 있게 다시 식별할 수 있어야 한다. 더 이상 그렇게 할 수 없으면 테스트가 실패하며, 이는 흔히 거짓 양성을 나타낸다.

가장 조잡하고 취약한 방법이지만 때때로 마지막 수단이 되는 방법은 X/Y 좌표를 사용하는 것이다. 이는 해상도와 레이아웃이 변경되면 당연히 깨질 뿐만 아니라, 처음에 어떤 객체를 의도한 것인지에 대한 주석이 없다면 유지보수하기도 어렵다. 심지어 텍스트가 잘못된 필드에 입력되거나 잘못된 버튼이 클릭되는 등 원인을 식별하기 어려운 상황으로 이어질 수도 있다. 다소 나은 방법은 XPath 또는 이와 유사한 방법, 즉 GUI 내에서 요소가 가지는 내부의 고유 경로를 사용하는 것이지만 이 역시 변경될 수 있다. ID 나 이름과 같은 의미론적 식별 기준이 분명 더 바람직하지만, 이것들조차 변경될 수 있다.

인공지능은 다양한 식별 기준을 기반으로 올바른 객체를 식별하는 데 도움을 줄 수 있다.

원문 PDF 118 페이지

예를 들어 XPath, 레이블, ID, 클래스, X/Y 좌표를 사용할 수 있으며, 어떤 형태의 이미지 인식을 적용하여 객체의 ‘모양’을 기반으로 식별하거나, 과거에 가장 안정적이었던 단일 식별 기준을 선택할 수도 있다. 이러한 모든 접근 방식은 과거에 사용된 적이 있다.

시각적 테스트 자동화

시각적 테스트 자동화는 때때로 소프트웨어가 표시하는 정보 측면에서는 기술적으로 올바르지만 실제로는 사용할 수 없거나 ‘보기 흉한’ 상태가 되는 특정 문제를 다룬다. 이는 서로 다른 플랫폼에서의 시각적 차이, 즉 크로스 플랫폼 및 크로스 디바이스 테스트나 시각적 회귀만을 식별하는 것을 목표로 한다. 여기에는 GUI의 위치, 크기, 레이아웃, 글꼴, 색상 또는 그 밖의 보이는 속성에서 의도하지 않은 변경이 발생했는지, 어떤 객체가 보기 어렵거나 아예 접근할 수 없는지, 시각적 아티팩트나 다른 문제가 있는지 등이 포함된다. 시각적 테스트 자동화는 소프트웨어의 기능적 측면이나 비즈니스 측면을 테스트하는 것을 목표로 하지 않는다.

UI 수준 테스트 자동화에서의 인공지능: 요약

이제는 코드 수준에서 소프트웨어를 테스트하는 것보다 UI 수준에서 소프트웨어를 테스트하는 것이 훨씬 쉽다는 점이 분명해졌을 것이다. 물론 여전히 해결해야 할 많은 과제가 있다. 주요 장점 중 하나는 이 수준에서는 코딩 기술이 필요하지 않다는 점이며, 이로 인해 이 기술은 훨씬 더 많은 사람들이 접근할 수 있다.

소프트웨어 품질 보증의 다른 작업에 인공지능 적용

소프트웨어의 품질을 관리하고 보증하기 위해서는 단순히 소프트웨어를 테스트하는 것보다 훨씬 많은 작업이 필요하다. 그러나 작업의 범위가 너무 넓고 인공지능이 이를 지원할 수 있는 가능성의 수가 너무 많아서 거의 완전한 목록을 제공하는 것은 현실적이지 않다. 대신 가까운 미래에 인공지능의 지원을 받을 가능성이 가장 높은 작업들을 제시한다.

일부 작업, 예를 들어 테스트 데이터 생성, 테스트 최적화 또는 테스트 선택은 앞서 논의한 테스트 생성에 필요하므로 이미 위에서 다루었다는 점에 유의한다.

버그 트리아지

원문 PDF 119 페이지

수십만 명 이상의 사용자를 가진 대규모 프로젝트에서는 버그 보고서 관리에 상당한 노력이 들어간다. 특히 Microsoft Windows, Firefox 또는 이와 유사한 시스템처럼 자동화된 버그 보고 시스템이 있는 경우 더욱 그렇다. 버그 보고서의 최대 4분의 1은 중복일 수 있다. 클러스터링 알고리즘을 사용하여 중복을 제거하고, 버그 보고서의 큰 비율을 차지하는 특히 중요한 버그를 찾아낼 수 있다(Bettenburg et al., 2008).

이상 탐지

이상 탐지는 인공지능이 상당히 ‘자연스럽게’ 수행할 수 있는 작업이며 다양한 사용 사례에 활용할 수 있다. 예를 들어 운영 시스템이나 테스트 시스템을 실시간으로 모니터링하면서 인간 또는 시스템의 오류나 보안 침해까지 나타낼 수 있는 사건이나 ‘이상한 동작’을 찾는 데 사용할 수 있다. 모니터링할 수 있는 항목에는 네트워크 트래픽, 입출력 동작, 로그 메시지, API 호출 사용, 디스크 공간, 중앙처리장치와 메모리 사용량 등이 포함된다.

위험 예측과 결함 추정

소프트웨어 저장소에서 변경 패턴을 분석하고 버그 발생에 관한 과거 데이터와 비교하는 접근 방식은 지금까지 주로 과학 연구에서만 적용되어 왔다. 이러한 데이터를 아직 릴리스되지 않은 변경사항과 연관시키면 해당 변경의 위험성과 새로운 버그를 도입하거나 릴리스 이후 문제를 일으킬 가능성을 예측할 수 있다(Zimmermann et al., 2009).

이 접근 방식은 사용 기간이 지난 뒤 살충제가 곤충을 죽이지 못하게 되는 것처럼 시간이 지날수록 효과가 감소한다는 살충제 역설의 영향을 받는다는 점에 유의해야 한다.

수동 테스트를 위한 노력 추정

앞서 언급한 소프트웨어 저장소의 변경 패턴 분석은 가장 문제가 될 가능성이 높은 변경을 찾아내고, 더 표적화된 수동 테스트 및 검토 프로세스를 수행할 수 있게 한다. 이는 다시 관리자가 테스트에 필요한 노력을 어느 정도 조정할 수 있게 한다. 즉, 위험이 낮은 릴리스에는 더 적은 테스트가 필요하다.

결함 분석

원문 PDF 120 페이지

이 작업은 흔히 빠르게 수행해야 하며 보통 많은 노력이 필요하다. 또한 테스트 생성, 코드 분석, 실행 분석, 자동화된 프로그래밍과 같은 여러 흥미로운 연구 주제와 연결된다. 따라서 이 주제는 지속적으로 연구되고 있으며 많은 흥미로운 접근 방식이 개발되었다. 그러나 이 책을 집필하는 시점에는 구매할 수 있는 것이 없다. 특히 흥미로운 응용 중 하나는 기술적 오류에 대한 수정까지 자동으로 생성하는 SapFix 시스템이다(Jia et al., 2018).

결함은 매우 다양한 관점에서 분석할 수 있으며, 그 결과 수동 분석을 가속하거나 예를 들어 티켓을 가장 적합한 개발 리소스에 할당할 수 있는 다양한 방법이 생긴다. 여기에는 원인-결과 체인 분석을 위한 추가 테스트 생성, 스택 트레이스 분석, 실행 경로 분석 등이 포함된다.

다른 작업에 인공지능 적용: 요약

이러한 간단한 예는 인공지능이 개발과 품질 보증에서 인간을 지원하는 데 얼마나 다양한 방식으로 사용될 수 있는지를 보여준다. 앞으로 어떤 예상하지 못한 접근 방식이 나타날지 기대된다.

테스팅을 위한 도구 지원 평가

일반적인 프로젝트 환경에서는 앞서 언급한 기술을 직접 구현하는 것이 유용한 경우는 드물다. 대신 도구 공급업체나 플랫폼 제공업체가 도구를 제공하고, 사용자는 이를 사용할지 선택한다. 따라서 이 절은 대부분의 프로젝트에 가장 직접적으로 적용될 것이다. 주어진 접근 방식을 사용해야 하는지 어떻게 판단하며, 동일한 접근 방식을 구현한 여러 도구 중에서는 어떻게 선택해야 하는가?

인공지능 기반 테스트 접근 방식의 매트릭

원칙적으로 인공지능 기반 품질 보증 도구는 다른 모든 인공지능 시스템과 같은 인공지능 시스템인 동시에 다른 모든 품질 보증 도구와 같은 품질 보증 도구이므로 동일한 매트릭을 적용할 수 있다.

무임승차 없음 정리(no-free-lunch theorem)³에 대응하기 위해 인공지능 기반 시스템에 주어지는 작업의 요구사항 중 하나는 과거가 미래를 잘 예측해야 한다는 것이다. 그러나 많은 실제 시나리오에서 ‘잘’이라는 개념은 다소 모호하다. 예를 들어

원문 PDF 121 페이지

‘위험 예측과 결함 추정’에서는 살충제 역설과 자기실현적 예언의 효과를 모두 알아야 한다. 흥미롭게도 두 효과는 서로 반대 방향으로 작용한다. 시스템의 특정 부분에 테스트 노력을 집중하면 처음에는 해당 부분이 더 집중적으로 테

스트되기 때문에 더 많은 오류가 발견되고 더 높은 위험이 표시된다(자기실현적 예언). 시간이 지나 코드의 품질과 개발자 및 수동 테스터의 인식이 높아지면 살충제 역설이 나타나면서 예측의 효과가 감소한다.

인공지능 기반 접근 방식의 전형적인 메트릭에는 거짓 양성(type I 오류)과 거짓 음성(type II 오류)이 포함된다. 그러나 많은 실제 문제에서 후자는 측정하기 어렵다. 예를 들어 시스템에서 놓친 버그라는 type II 오류는 얼마나 많은 버그가 탐지되지 않았는지를 알기 어렵기 때문에 흔히 판단할 수 없다. 고려해야 할 또 다른 측면은 테스트 케이스가 찾아낸 결함의 심각도이며, 이는 충돌을 일으키는 경우에도 마찬가지이다. 도구가 원칙적으로는 높은 심각도를 가진 결함을 찾을 가능성이 있다. 충돌을 일으키기 때문이다. 그러나 어떤 고객도 그 결함을 유발하는 데 필요한 방식으로 소프트웨어를 사용하지 않을 수 있다. 이러한 결함을 보고하면 고객과 더 관련 있는 작업에서 초점을 돌리거나 티켓 시스템을 불필요한 보고로 채워 오히려 전체적으로 부정적인 효과를 낼 수도 있다. 따라서 중요한 것은 버그의 수가 아니라 관련 위험이나 고객에게 미치는 영향이며, 이는 측정하기가 훨씬 더 어렵다.

그러나 비정상적인 동작은 적대적 행위자가 시스템에 접근하려고 시도할 때 흔히 나타나는 방식이다. 따라서 테스트의 목표가 보안 테스트라면 잘 드러나지 않는 취약점까지도 찾는 것이 중요하다.

도구 공급업체의 주장 평가

많은 도구 공급업체는 매우 설득력 있는 이야기를 제공하도록 마케팅을 완성해 왔다. 그리고 인공지능이 유행하게 된 이후 이러한 이야기에는 흔히 어떤 형태로든 인공지능이 포함된다. 그러나 도구 자체를 위해 도구를 사고, 인공지능 자체를 위해 인공지능을 사서는 안 된다. 따라서 인공지능이 해결책의 일부인지 여부는 실제로 그리 중요하지 않으며, 많은 경우 실제로 인공지능이 포함되지 않기도 한다. 유럽연합 13 개국의 약 2,830 개 인공지능 스타트업을 조사한 MMC의 2019년 보고서에 따르면 40%의 사례에서는 인공지능을 사용한다는 증거를 찾지 못했다(MMC, 2019). 테스트 도구와 같은 소프트웨어 제품을 설명할 때도 'AI'를 정의하는 데 이와 비슷한 정도의 재량이 사용되고 있는 것이 합리적이다.

원문 PDF 122 페이지

공급업체의 주장을 검토할 때는 적절한 수준의 회의적인 태도가 필요하다. 도구가 공급업체가 약속한 대로 동작하는 한 인공지능이 실제로 적절한 해결책의 일부인지는 학술적인 문제에 더 가깝다. 그러나 바로 그 성능을 비판적으로 검토해야 한다. 인공지능 기반 테스트 도구에 대한 경영진의 기대도 서로 다를 수 있으며 비현실적일 수도 있다. 도구가 이러한 기대를 충족하지 못하면 도구를 지지하는 사람은 어려운 입장에 놓일 수 있다. 일반적으로 많은 도구는 작은 예제에서는 꽤 잘 동작하지만 큰 프로젝트에서는 확장성이 좋지 않으므로 평가할 때 이를 반드시 고려해야 한다.

안타깝게도 많은 프로젝트가 닭과 달걀의 문제에 직면한다. 도구를 현실적으로 평가하려면 도구를 적절하게 설정하고 구성하거나 심지어 학습시켜야 한다. 따라서 무엇이든 평가할 수 있기 전에 대부분의 노력을 먼저 투입해야 한다. 이는 결국 공급업체가 해당 제품을 이전에 구현한 사례를 살펴보고, 이상적으로는 기존 또는 과거 고객과 이야기해야 함을 의미한다.

투자수익률

궁극적으로 경영진은 일반적으로 어떤 형태로든 투자수익률(ROI) 계산을 보고 싶어 할 것이다. 이는 여러 이유로 어려울 수 있다.

(i) 적용된 프로세스에 따라 버그 수, 고객 요청 수, 다운타임과 같은 일부 메트릭은 얻기 쉽고 더 객관적이다. 추적 도구의 정확도에 따라 버그당 소요 시간과 같은 메트릭을 도출하고, 이를 기반으로 버그당 비용을 계산할 수 있다. 그러나 직관적으로 떠오르는 메트릭 대부분은 단지 관련이 없는 것이 아니라 잘못된 유형의 메트릭이며, 예를 들어 좋지 않은 평가로 인해 생기는 고객의 감정이나 판매 손실 같은 요소를 고려하지 않는다.

(ii) 선택한 메트릭에 따라 예상 감소량을 추정하기는 어렵다. 버그 수처럼 객관적으로 보이는 측정값도 그렇다. 하지만 결과는 미묘하거나 눈에 잘 띄지 않는 효과부터 새로운 기능을 추가하여 프로세스를 완전히 재설계하는 것까지 다양하므로 더 복잡하다. 일부 상황에서는 추가 가치가 시간이 지날수록 감소하여 추정이 더욱 어려워진다.

(iii) 추가적이고 예상하지 못한 비용도 발생할 가능성이 높다. 여기에는 특별한 지식과 경험의 필요, 시스템 유지보수 노력, 발견사항 검토 노력, 예를 들어 많은 거짓 양성을 정기적으로 수동 검토하는 일 등이 포함된다. 역설적으로 도구 자체가 추가적인 위험, 예를 들어 거짓 음성을 도입할 수도 있다.

원문 PDF 123 페이지

잠재적인 비용과 절감액을 최소한 대략적으로 추정하려면 몇 가지 질문을 고려하는 것이 합리적이다. 이 시스템은 기존 프로세스에 어떤 영향을 미칠 것인가? 예를 들어 기존 자동화 테스트 같은 산출물을 다시 만들거나 마이그레이션해야 하는가? 기존 테스트 및 보고 시스템, 지속적 통합/지속적 배포(CI/CD) 시스템, 릴리스 시스템, 문서화 시스템 또는 티켓 시스템 같은 기존 인프라와 잘 통합되는가? 직원의 행동을 바꾸게 하는가? 예를 들어 안일함이나 자동화로 인해 시스템을 맹목적으로 신뢰하게 되는가? 직원에게 추가 교육이 필요한가? 의도하지 않은 장기적 영향이 있을 수 있는가?

테스트 케이스 생성 도구 평가

테스트 케이스 생성 도구의 경우에는 몇 가지 구체적인 고려사항과 질문이 있다. 생성된 테스트는 전형적이거나 합리적인 사용을 나타낸다는 점에서 현실적인가? 핵심 비즈니스 위험을 커버하는가? 즉, 생성된 테스트 케이스가 관련 사용자 시나리오를 커버하는가? 사용되거나 생성된 데이터가 일반적인 데이터를 대표하는가? 추가 학습 없이 인공지능이 현실적인 테스트 케이스를 생성하는 것이 가능한가? 그렇지 않다면 그러한 학습은 어떻게 가능한가? 즉, 학습 데이터 측면에서 인공지능 시스템에 무엇을 입력하는가? 단지 테스트 케이스를 기록하는 것만으로 충분한가?

프로젝트에 테스트 케이스를 추가할 때 테스트 케이스 하나당 추가 가치는 추가되는 테스트 케이스 수가 늘어날수록 감소한다. 첫 번째 테스트 케이스는 천 번째 테스트 케이스보다 훨씬 더 가치가 크다. 테스트 케이스가 많아지면 유지보수 비용이 증가하고, 실행 주기가 느려지며, 수명주기 관리의 복잡성이 커지고, 그 밖의 많은 숨은 비용이 발생한다. 이러한 비용은 추가된 테스트 케이스가 가져오는 이점보다 작아야 하므로 한계 비용이 얼마인지 판단하는 것이 중요하다. 생성된 테스트 케이스는 보통 인간이 이해하기 어렵고, 예를 들어 특정 테스트 케이스의 목표를 파악하기 어렵기 때문에 유지보수하기도 어렵다. 때로는 작은 세부사항 하나가 테스트에 가치를 부여한다. 예를 들어 특정 옵션을 확인하여 커버리지를 확보하는 경우이다. 유지보수하는 사람이 다른 테스트와 비교했을 때 무엇이 해당 테스트

를 가치 있게 만드는지 알지 못하면, 가치 있는 테스트가 쉽게 쓸모없는 짐으로 바뀔 수 있다. 모든 것을 고려하면 생성된 테스트 케이스는 흔히 생성 비용은 낮지만 유지보수 비용은 높다. 생성된 테스트 케이스를 폐기하고 단순히 처음부터 다시 시작해야 하는 기준은 무엇인가?

원문 PDF 124 페이지

도구 지원 평가: 요약

품질 보증에서 도구 지원을 평가하려면 인공지능 기반인지 여부와 관계없이 많은 질문을 하고 답해야 한다는 것을 보았다. 안타깝게도 많은 질문의 올바른 답은 상황과 프로젝트에 따라 다르며, 결국 무엇이 실제로 ‘올바른 것’을 구성하는지를 판단하는 오라클 문제로 다시 돌아간다. 따라서 이 장의 목표는 도구를 평가할 때 어떤 측면을 살펴보고 무엇을 고려해야 하는지에 대한 아이디어를 제공하는 것이었다. 구체적인 답을 주는 것보다 이것이 장기적으로 훨씬 더 큰 가치를 제공하기를 바란다. 구체적인 답은 수명이 짧고 결국 대부분의 상황에서 잘못된 답이 될 것이기 때문이다.

인공지능에게 계속 어려울 가능성이 높은 작업

품질 보증 영역에는 앞으로도 인공지능에게 어려울 가능성이 높은 몇 가지 작업이 있다. 여기에는 명세와 가정을 수집하고 검토하는 일, 이해관계자와의 의사소통, 프로세스와 조직 개선 등이 포함된다. 이와 관련해 상당한 발전이 이루어진다면 일반적인 관리와 같은 다른 분야에서 더 큰 영향을 미칠 가능성이 높기 때문에 품질 보증에 미치는 영향은 그에 비해 작게 보일 것이다.

이 장의 시작에서 논의했듯이 테스트 오라클의 명세, 즉 소프트웨어 설계와 개발 프로세스 결과에 대한 최종 평가는 다른 모든 것이 자동화되더라도 수동 작업으로 남을 가능성이 높다. 여기에는 모호함을 명확히 하고 누락된 정보를 얻기 위해 이해관계자와 의사소통하는 작업이 포함된다. 자율주행 자동차 개발과 마찬가지로 품질 보증 분야의 인공지능에서도 갑작스러운 도약을 기대하기보다는, 이러한 도구가 일상적인 작업을 지원하는 방식이 꾸준히 개선될 것으로 기대해야 한다.

요약

인공지능은 품질 보증에 적용될 때 매우 다양한 방식으로 사용할 수 있다. 이 장에서는 단위, 인터페이스, 사용자 인터페이스 수준에 대한 적용을 살펴보았으며, 각각 고유한 과제와 해결책이 있다. 기술이 발전함에 따라 아직 예상하지 못한 훨씬 더 많은 접근 방식이 나타날 것으로 예상된다.

원문 PDF 125 페이지

그러나 이러한 도구를 구상하거나 개발하는 것은 테스트 실무자의 역할인 경우가 드물다. 따라서 이 장의 마지막 부분은 훨씬 더 가능성이 높은 역할, 즉 도구를 평가하고 선택한 뒤 실제로 적용하는 데 필요한 지식을 제공하는 것을 목적으로 했다.

많은 테스팅 전문가는 인공지능을 사용하는 것을 두려워하며, 장기적으로 자신의 일자리를 위협할 것이라고 우려한다. 우리의 관점에서는 인공지능이 소프트웨어 테스터를 대체하기보다는 소프트웨어 개발자를 적어도 부분적으로 대체할 가능성이 훨씬 높다는 사실이 위안이 될 수 있다. Microsoft Copilot 과 OpenAI Codex 를 통해 그 방향으로의 첫 번째 큰 걸음은 이미 이루어졌으며, 앞으로 분명 더 많은 진전이 뒤따를 것이다.

1 코드 스멜(code smell)은 대부분의 상황에서 좋지 않은 코딩 관행으로 간주되지만 정당한 예외가 존재하므로 절대적인 규칙은 없다. 2 피츠의 법칙(Fitts' law)은 목표물의 거리와 크기에 따라 인간이 클릭하기 전에 마우스를 가리키는 것과 같은 특정 움직임에 필요한 시간을 예측한다. 3 무임승차 없음 정리(no-free-lunch theorem)는 가능한 모든 문제에 걸쳐 성능을 평균하면 완전히 무작위인 알고리즘을 포함한 어떤 두 최적화 알고리즘도 동등하다고 말한다. 어떤 접근 방식, 예를 들어 인공지능이 더 우수하려면 과거가 미래를 잘 예측한다는 것과 같이 문제에 대한 특정 가정을 해야 한다.

원문 PDF 126 페이지

6 소프트웨어 테스팅을 위한 온톨로지

Joanna Isabelle Olszewska

인공지능에서 온톨로지는 특정 도메인에 대한 형식적 명세, 즉 관심 도메인에 대한 공유된 이해와 그 도메인의 형식적이고 기계가 이해할 수 있는 모델을 설명한다. 따라서 온톨로지는 소프트웨어 테스팅 지식을 체계적이고 상호운용 가능한 방식으로 포착하고, 공유하고, 표현하는 데 사용할 수 있다. 또한 온톨로지는 해당 지식에 대한 자동 추론을 지원하여, 인간이든 인공 시스템이든 지능형 에이전트(IA)가 소프트웨어와 시스템을 테스트하는 작업을 수행하도록 돕는다.

온톨로지에 대하여

온톨로지는 철학에서 등장했으며 ‘존재의 대상’을 연구하는 데 초점을 맞추었다. 정보기술(IT)에서는 온톨로지를 흔히 분류체계나 클래스 계층구조로 보지만, 온톨로지는 그보다 훨씬 더 많은 것을 포함한다.

인공지능, 소프트웨어 엔지니어링, 특히 소프트웨어 테스팅을 포함하는 컴퓨터 과학에서 온톨로지가 어떻게 사용되는지 더 자세히 살펴보자.

컴퓨터 과학에서의 온톨로지

30 년 전부터 온톨로지는 컴퓨터 과학에서 연구되기 시작했고, 개념화에 기반한 형식적으로 표현된 지식의 집합으로 온톨로지를 보는 시맨틱 웹의 한 축이 되었다.

원문 PDF 127 페이지

컴퓨터 과학에서 온톨로지는 개념과 그 개념들의 상호의존적인 모든 속성을 표현하는 것을 목표로 한다. 따라서 클래스와 그 인스턴스, 즉 개체(individual)를 온톨로지 안에 구성하고 정의된 속성을 통해 서로 연관시킬 수 있다.

온톨로지는 ‘개념화에 대한 명시적 명세’(Gruber, 1995).

온톨로지는 예제로부터의 프로그래밍(Gordon et al., 2010), 클라우드 컴퓨팅(Al Feel and Khafagy, 2011), 사이버보안(Simmons et al., 2014) 등 광범위한 컴퓨팅 응용을 위해 개발되어 왔다. 예를 들어 시맨틱 웹 검색 엔진인 Swoogle(Ding et al., 2004)을 통해 웹에서 검색할 수 있는 137,000 개 이상의 시맨틱 웹 문서 가운데 일부이다.

컴퓨터 과학에서 온톨로지가 이렇게 널리 사용되면서 온톨로지 엔지니어링(Gomez-Perez et al., 2004)이라는 분야가 등장했다. 이 분야는 온톨로지 개발 프로세스, 온톨로지 수명주기, 온톨로지를 구축하기 위한 방법과 방법론, 도구, 언어를 다룬다.

특히 온톨로지를 구축하기 위한 여러 방법론이 문헌에서 제안되었다. 예를 들어 METHONTOLOGY(Fernandez et al., 1997)는 온톨로지 개발과 통합을 지원하는 성숙한 프로세스이지만, 프로세스 초기 단계에서 기존 온톨로지를 재사용할 수 있는 가능성은 제공하지 않는다.

또 다른 예는 Toronto virtual enterprise(TOVE)(Gruninger and Fox, 1995)이다. 이 모델링 방법론은 온톨로지 개발의 전체 수명주기를 포함하지만, 보통 소규모 응용에는 지나치게 복잡하며 온톨로지의 재사용도 허용하지 않는다. 더 구체적으로 TOVE 접근 방식은 개념, 관계, 공리를 온톨로지의 기본 요소로 본다. 또한 TOVE는 한편으로 온톨로지가 올바르게 답할 수 있어야 하는 역량 질문(CQ)(Wisniewski et al., 2019)의 정식을 포함한다. 실제로 CQ는 온톨로지 요구사항의 집합으로 볼 수 있으며, 온톨로지에 표현되어야 할 대상 도메인의 범위를 설정하고 한정하는 방법으로도 볼 수 있다.

원문 PDF 128 페이지

다른 한편으로는 온톨로지의 공리화(Staab and Maedche, 2000)를 포함하여 공리를 식별하고 정의하도록 한다.

온톨로지는 이러한 CQ를 표현하고 해결하기 위해 필요충분한 공리 집합을 포함해야 하기 때문이다.

공리(Vasilecas et al., 2009)는 서로 다른 유형과 성격을 가질 수 있다는 점에 주목할 필요가 있다. 일반적으로 공리는 개념 간 복잡한 관계를 표현하고 제한하며, 그 관계가 의도된 방식으로 해석되도록 제약한다. 특히 도출 공리는 기존 지식으로부터 새로운 정보를 도출할 수 있게 한다. 인식론적 공리는 개념이 구조화되는 방식에 의해 부과되는 제약을 보여주기 위해 정의되며, 존재론적 공리는 도메인 의미의 제약을 설명한다. 또한 통합 공리는 관계가 일관되게 설정되기 위해 충족되어야 하는 제약을 일반적으로 부과한다.

또 다른 방법론은 기업 온톨로지(enterprise ontology, EO)(Dietz and Mulder, 2020)이다. 이는 전체 온톨로지 개발 수명주기를 다루며 사람이나 소프트웨어 에이전트 사이에서 정보 구조에 대한 공통 이해를 공유할 수 있게 한다.

이러한 방법론은 온톨로지 개발 프로세스의 수명주기 범위나 서로 다른 작업 및 단계의 정의 면에서 차이가 있다. 그러나 온톨로지 개발을 위한 몇 가지 일반적인 작업이 식별되어 있다(Olszewska, 2015). 주요 작업은 지식 선택, 분석, 기술이며, 일반적으로 온톨로지 명세, 개념화, 구현, 평가의 네 단계에서 수행된다.

또한 W3C(World Wide Web Consortium)는 온톨로지 도메인 모델링과 지식 표현을 위한 전용 온톨로지 프로그래밍 언어를 개발하였다. 이를 위해 Resource Description Framework(RDF)(2021)는 웹에서 데이터를 교환하기 위한 모델로 메타데이터와 설명을 생성하는 데 사용된다. Web Ontology Language(OWL)(2021)는 개념, 개념 집합, 개념 간 관계에 관한 풍부하고 복잡한 지식을 표현하도록 설계된 시맨틱 웹 언어이다. 또한 OWL은 계산 논리에 기반한 언어이므로 OWL로 표현된 지식은 컴퓨터 프로그램이 활용할 수 있다. 예를 들어 해당 지식의 일관성을 검증하거나 암묵적인 지식을 명시적으로 만들 수 있다.

원문 PDF 129 페이지

OWL 표준 언어에는 OWL-Lite, OWL-DL, OWL-Full이라는 세 가지 변형이 있다. 대부분의 응용 온톨로지(Ferreira de Souza et al., 2013)는 기술 논리(description logics, DL)(Baader et al., 2003)에 기반한 OWL 언어 변형인 OWL-DL을 채택한다. DL은 엄격하고 1차 술어 논리에 기반한 의미론과 표현력이 높은 개념적 지식을 구조화하고 인코딩하는 직관적인 방식을 결합한다는 점에 주목할 필요가 있다. 따라서 OWL-DL은 덜 사용되고 더 단순한 OWL-Lite보다 수학적 형식화와 더 높은 의미 표현력을 제공한다. 또한 OWL-DL 기반 온톨로지에 대해 자동 추론을 수행할 수 있다(Olszewska et al., 2014).

자동 추론은 추론을 논리의 형태로 컴퓨팅 시스템에 적용하여 자동으로 추론 결과를 도출하는 것과 관련된 컴퓨터 과학의 고급 분야이다(Wos et al., 2021).

자동 추론은 컴퓨터가 자동으로 추론할 수 있게 하는 프로그램을 만드는 것을 목표로 한다. 전체적인 목표는 다양한 형태의 추론을 기계화하는 것이지만, '자동 추론'이라는 용어는 주로 수학과 형식 논리에서 수행되는 타당한 연역 추론과 동일시되어 왔다(Portoraro, 2021).

따라서 OWL-DL은 추론기를 사용하여 추론된 온톨로지 클래스 계층구조를 계산하고 내부 일관성 검사를 수행할 수 있게 한다. 또한 OWL-DL의 추론기는 다항 시간에 동작한다는 점에서 계산 가능성이 있는 반면, OWL 구문과 RDF 데이터 표현을 결합한 변형인 OWL-Full에서는 자동 추론이 계산 가능하지 않다. 결과적으로 OWL-DL의 추론기는 OWL-Full의 추론기보다 계산적으로 더 효율적이다.

한편 온톨로지는 맥락 적응형 접근 방식(Holzinger et al., 2018)을 따를 수도 있다. 즉, 고전적인 논리 기반 온톨로지 접근 방식과 통계적 접근 방식을 결합하여 확률적 온톨로지를 구축할 수 있다.

원문 PDF 130 페이지

이와 함께 ODE(ontology-based software development environment)(de Almeida Falbo et al., 2003), WebODE(Arpirez et al., 2001) 같은 온톨로지 도구가 컴퓨터 과학자의 온톨로지 개발을 돕기 위해 만들어졌다. 특히 널리 확립된 오픈소스 온톨로지 편집기로 Protégé(Protégé, 2021)가 있다. Protégé는 풍부한 그래픽 메뉴를 갖춘 통합 지식 기반 편집 환경(Horridge, 2011)을 제공하는 독립 실행형 도구로, 온톨로지를 개념화하고 시각화하며 조작할 수 있게 한다. 또한 Protégé는 XML이나 Java 같은 서로 다른 프로그래밍 언어 플랫폼에서 접근할 수 있는 OWL 파일(Horridge and Bechhofer, 2011)을 생성하거나, 월드 와이드 웹(WWW)에 직접 게시할 수 있다. Protégé는 WebProtégé를 통해 클라우드에서도 접근할 수 있어 온톨로지를 공동 편집할 수 있다.

Protégé와 WebProtégé 모두 Twitter 의 @protegeproject 를 통해 접근할 수 있는 활발한 지원 커뮤니티의 도움을 받고 있다. 또한 2021 년 3 월 19 일 게시된 다음 트윗에 따르면 WebProtégé 는 온톨로지 개발에 가장 많이 사용되는 도구로 보인다.

‘20 년이 지난 뒤에도 @protegeproject 는 여전히 가장 많이 언급되는 도구이다!’

<https://twitter.com/i/web/status/1372856305560457225>

온톨로지와 인공지능

오늘날 온톨로지는 지식 표현(KR) 분야에 속하는 인공지능 기술로 간주된다. 온톨로지는 특정 현실을 설명하는 데 사용되는 특정 어휘와, 그 어휘의 의도된 의미에 관한 명시적 가정의 집합으로 구성되며, 인공지능이 해석할 수 있는 형식적인 방식으로 표현된다(Noy and McGuinness, 2001).

원문 PDF 131 페이지

지식 표현은 복잡한 작업을 해결하기 위해 컴퓨터 시스템이 사용할 수 있는 형태로 세계에 관한 정보를 표현하는 데 전념하는 인공지능 분야이다. 지식 표현은 (i) 인간이 문제를 해결하고 지식을 표현하는 방식에 관한 심리학의 연구 결과를 활용하여 복잡한 시스템을 더 쉽게 설계하고 구축할 수 있는 형식 체계를 설계하며, (ii) 규칙의 적용이나 집합과 부분집합의 관계 같은 여러 종류의 추론을 자동화하기 위해 논리학의 연구 결과를 활용한다(Davis et al., 1993). 따라서 지식 표현은 절차적 코드보다 복잡한 소프트웨어를 더 쉽게 정의하고 유지보수할 수 있게 하며, 전문가 시스템에 사용할 수 있다.

온톨로지 지식 표현은 온톨로지의 용어와 공리에 대한 자동 추론을 가능하게 한다. 이를 위해 HermiT(Glimm et al., 2014)나 FaCT++(Tsarkov, 2014) 같은 논리 기반 추론 엔진인 내장 추론기가 온톨로지에서 실행된다. 온톨로지는 일반적으로 도메인 어휘로 클래스와 속성을 설명하는 용어 상자(TBox)라는 공리 집합과, 해당 용어에 기반하여 개체에 관한 주장을 설명하는 주장 상자(ABox)라는 공리 집합을 가진다. 그러면 추론기는 개념의 충족 가능성에 관한 의도적 질의와 특정 개념의 인스턴스를 검색하는 것과 같은 외연적 질의 모두에 답할 수 있다.

온톨로지 지식 표현은 인공지능의 의사소통과 협업에 매우 중요한 상호운용성도 제공한다. 상호운용성(Uschold and Gruninger, 1996)은 에이전트가 특정 맥락이나 도메인 안에서 개념의 의도된 의미를 올바르게 자동으로 해석하고, 이를 자신의 로컬 응용에서 적절하게 사용할 수 있는 능력이다. 따라서 온톨로지는 컴퓨터 비전(Olszewska and McCluskey, 2011)부터 로봇틱스(Olszewska, 2018a)에 이르기까지 다양한 인공지능 응용에 유용한 지식 표현이며, 지능형 비전 시스템(IVS)(Olszewska, 2018b), 다중 에이전트 시스템(MAS)(Berrani et al., 2018), 클라우드 로봇틱스 시스템(CRS)(Pignaton de Freitas et al., 2020b), 인더스트리 4.0(I4.0)(Sampath Kumar et al., 2019)을 포함한다. 실제로 I4.0의 주요 측면 중 하나는

인간과 인공 소프트웨어 또는 하드웨어 에이전트를 포함하는 여러 지능형 시스템 사이의 의미론적 의사소통을 위해 일관된 접근 방식을 채택하는 것이다. 이를 위해 온톨로지는 스마트 제조 지식을 상호운용 가능한 방식으로 형식화함으로써 해결책을 제공할 수 있다(Sampath Kumar et al., 2019).

인더스트리 4.0은 디지털 데이터, 연결성, 사이버 시스템, 인공지능을 기반으로 한 최신 기술 발전이 이끄는 제 4차 산업혁명을 나타내기 위해 만들어진 용어이다. 그 결과 스마트 제품, 스마트 기계, 증강된 작업자가 등장하지만, 상호운용성과 같은 과제도 발생한다(Slee et al., 2021).

온톨로지의 또 다른 중요한 속성은 공유 가능성과 재사용 가능성이다. 이를 통해 이용 가능한 온톨로지 지식을 입력으로 사용하여 새로운 온톨로지를 생성할 수 있다(Katsumi and Gruninger, 2016). 이는 인공지능 기반 시스템처럼 복잡한 도메인이나 지능형 시스템을 위한 대규모 온톨로지를 개발할 때 특히 유용하다. 따라서 일부 온톨로지 개발 방법론은 기존 온톨로지를 재사용하여, 즉 결과 온톨로지의 일부가 되는 다른 온톨로지를 조합하고, 확장하고, 특수화하고, 적응시켜 온톨로지를 구축할 것을 제안한다(Pinto, 1999). 다른 접근 방식은 같은 주제에 관한 서로 다른 온톨로지를 하나로 병합하여 모두를 통합하는 단일 온톨로지를 구축할 수 있게 한다(Uschold et al., 1998).

온톨로지에는 다음과 같은 세 가지 주요 유형이 있다는 점도 주목할 필요가 있다(Wu and Hakansson, 2014). 첫째, 코어 온톨로지는 다른 개념을 이해하는 데 필요한 최소 개념만으로 구성된 기본적이고 최소한의 온톨로지이다. 예를 들어 자동화 시스템과 로봇틱스를 위한 IEEE 1872-2015 온톨로지 표준에 설명된 로봇틱스 및 자동화 코어 온톨로지(CORA)(Fiorini et al., 2017)는 로봇 시스템과 자동화의 기반을 이루는 핵심 개념을 정의한다. 둘째, 도메인 온톨로지는 세계의 특정 영역에 속하는 모든 또는 대부분의 개념을 표현하는 것을 목적으로 한다. 예를 들어 클라우드 로봇틱스 시스템 도메인 온톨로지(ROCO)(Pignaton de Freitas et al., 2020a), 시공간 시각 도메인 온톨로지(STVO)(Olszewska, 2011), 설명 가능한 인공지능(XAI) 온톨로지(Seeliger et al., 2019)가 있다. 셋째, 상위 온톨로지는

SUMO(suggested upper merged ontology)(2021), UFO(unified foundational ontology)(Guizzardi, 2005), DOLCE(descriptive ontology for linguistic and cognitive engineering)(2021)처럼 광범위한 도메인 온톨로지에 일반적으로 적용할 수 있는 공통 관계와 객체의 모델이다.

따라서 예를 들어 온톨로지의 재사용과 공유를 통해 클라우드 로봇틱스 시스템용 ROCO 도메인 온톨로지는 로봇틱스 코어 온톨로지(CORA)를 통합할 수 있으며, CORA는 다시 SUMO 상위 온톨로지에 기반한다. 이는 스마트 시스템(Slee et al., 2021)이나 시스템의 시스템(Yan et al., 2014)과 같은 복잡한 시스템의 모델링과 평가도 지원한다.

시스템의 시스템(System of Systems, SoS)은 독립적인 시스템들의 집합을 더 큰 시스템으로 통합하여 고유한 기능을 제공하는 것이다. 독립된 구성 시스템들은 각자 단독으로는 만들어낼 수 없는 전체적인 동작을 만들어내기 위해 협력한다(Boardman and Sauser, 2006).

소프트웨어 엔지니어링 맥락에서의 온톨로지

온톨로지는 본질적으로 지식을 모호하지 않게 표현하고, 정보를 상호운용 가능한 방식으로 공유하며, 자동 추론을 수행할 수 있게 하므로 여러 이해관계자(Murtazinam and Avdeenko, 2018)나 다수의 에이전트(Sugawara et al., 2011)가 소프트웨어 엔지니어링 도메인(Pop et al., 2014)에서 사용해 왔다. 이러한 온톨로지는 요구사항 엔지니어링 프로세스(Avdeenko and Pustovalova, 2016), 소프트웨어 개발(Olszewska and Allison, 2018), 소프트웨어 유지보수(Nor et al., 2012), 품질 보증 검사(Da Silva et al., 2015)를 지원하는 데 사용되어 왔다. 그렇다면 소프트웨어 테스트를 위한 온톨로지는 어떠한가? 이를 살펴보자.

소프트웨어 테스트를 위한 온톨로지 사용

소프트웨어 테스터는 10 년 넘게 소프트웨어 테스트 전용 온톨로지를 사용해 왔다(Chen and Xi, 2020).

원문 PDF 134 페이지

실제로 온톨로지는 테스트 개발(Landhausser and Genaid, 2012)과 테스트 자동화(Paydar and Kahani, 2010)를 지원하기 위해 개발되어 왔으며, 테스트 생성(Nasser et al., 2010), 테스트 실행(Yu et al., 2009), 테스트 관리(Sapna and Mohanty, 2011), 테스트 재사용(Lim and Zhang, 2012)을 포함한다.

또한 온톨로지 주도 소프트웨어 테스트는 다음과 같은 특정 테스트를 수행하는 데 적용되어 왔다(Klueck et al., 2018).

- 서로 다른 소프트웨어 테스트 실무: 예를 들어 요구사항 기반 테스트(Feldmann et al., 2014), 행위 주도 개발(BDD)(Rocha Silva et al., 2017)
- 서로 다른 소프트웨어 테스트 기법: 예를 들어 돌연변이 테스트(Wang et al., 2009)
- 서로 다른 소프트웨어 테스트 유형: 기능 테스트, 예를 들어 회귀 테스트(Campos et al., 2017), 그래픽 사용자 인터페이스 테스트(Li et al., 2009), 그리고 비기능 테스트, 예를 들어 신인성 테스트(Looker et al., 2005), 성능 테스트(Freitas and Vieira, 2014), 보안 테스트(Siqueira Bueno et al., 2018), 사용성 테스트(Robal et al., 2017)
- 서로 다른 소프트웨어 응용: API(Alqahtani et al., 2017), 웹 서비스(Wang et al., 2007)

그러나 소프트웨어 테스트 도메인의 개념 모델과 구현 모델을 모두 제시하는 소프트웨어 테스트 온톨로지의 사례는 소수에 불과하다. 따라서 온톨로지의 목적과 범위, 참조 지식체계(BoK)와 도메인 범위, 그리고 개발된 주요 온톨로지 개념의 관점에서 이를 분석해 보자.

사례 1. Software Testing Ontology for Web Service 온톨로지

소프트웨어 테스트를 위해 처음으로 확립된 온톨로지는 Software Testing Ontology for Web Service(STOWS)(Huo et al., 2003)라고 한다. 이 온톨로지는 웹 기반 테스트 서비스를 위한 다중 에이전트 통합 소프트웨어 환경이며, 테스트 작업과 테스터의 역량을 매칭하도록 돕는다.

한편 테스트 작업에는 각 테스트 활동과 그 활동을 어떻게 수행해야 하는지에 관한 관련 정보가 포함된다. 여기에는 맥락, 사용할 테스트 방법, 활동을 수행할 환경, 이용 가능한 자원, 테스트 결과에 대한 요구사항 등이 있다. 다른 한편 테스터의 역량은 테스터가 수행할 수 있는 활동과 함께, 에이전트가 활동을 수행하는 맥락, 사용되는 테스트 방법, 테스트 수행 환경, 필요한 자원 즉 입력, 그리고 테스터가 생성할 수 있는 출력으로 결정된다.

STOWS는 다음 여섯 가지 주요 개념을 중심으로 63 개의 용어를 제시한다.

- 테스터: 예를 들어 인간, 소프트웨어 에이전트, 팀
- 맥락: 예를 들어 단위 테스트, 통합 테스트, 시스템 테스트
- 활동: 예를 들어 테스트 계획, 테스트 케이스 생성, 테스트 실행, 테스트 결과 검증, 테스트 커버리지 측정, 테스트 보고서 생성
- 방법: 예를 들어 오류 기반 테스트, 결함 기반 테스트, 구조 기반 테스트
- 산출물: 예를 들어 중간 데이터, 테스트 결과, 테스트 계획, 테스트 스위트, 테스트 스크립트
- 환경: 예를 들어 하드웨어, 소프트웨어

따라서 STOWS는 테스트 활동, 테스트 산출물, 테스트 방법, 테스트 유형, 테스트 환경과 역량에 관련된 측면을 다루지만, 특정 참조 BoK를 준수하지 않으며 개발 과정에서 온톨로지 방법론을 따르지 않았다.

사례 2. OntoTest 온톨로지

OntoTest(Barbosa et al., 2006) 온톨로지는 테스트 지식의 획득, 조직화, 재사용, 공유를 지원하기 위해 소프트웨어 엔지니어링 도구를 위한 참조 아키텍처를 제공하도록 개발되었다. ISO/IEC 12207(2017) 표준에 기반하며 다음 여섯 개의 하위 온톨로지로 나뉜다.

- 테스트 프로세스: 예를 들어 테스트 수명주기 모델
- 테스트 단계: 예를 들어 단위 테스트, 통합 테스트, 회귀 테스트

- 테스트 산출물: 예를 들어 테스트 문서, 테스트 다이어그램, 테스트 케이스, 테스트 요구사항, 드라이버와 스텝, 테스트 대상 산출물
- 테스트 자원: 예를 들어 인적 자원, 하드웨어 자원, 테스트 도구를 포함한 소프트웨어 자원
- 테스트 절차: 예를 들어 테스트 방법, 테스트 지침, 테스트 기법
- 테스트 단계 작업: 예를 들어 테스트 계획, 테스트 케이스 설계, 테스트 실행, 테스트 분석

그 결과 총 52 개의 온톨로지 용어와 1 차 논리 공리가 정의되며, 기법과 기준, 인적 및 조직 자원, 자동화 도구 등 테스트 활동에 관련된 여러 측면을 다룬다.

사례 3. Software Test Ontology Integrated 온톨로지

Software Test Ontology Integrated(SwTOI)(Bezerra et al., 2009)는 Software Engineering Body of Knowledge(SWEBOK)(Bourque and Fairley, 2014)에서 소프트웨어 테스트 지식을 추출하여 유즈케이스 개념

을 위한 194 개의 용어와 공리를 형식화했다. 목적은 비용, 시간, 노력을 줄이면서 품질 높은 소프트웨어를 구축하도록 돕는 것이다.

특히 SwTOI 는 Linux 테스트에 초점을 두기 때문에 Linux 운영체제 도메인과 Linux 소프트웨어 테스트 도메인의 개념을 모두 포함한다. SwTOI 는 operating system ontology(OSOnto)와 software test ontology(SwTO)를 통합한다. OSOnto 는 Linux 운영체제 도메인의 개념을 표현하며 Basic Linux Ontology(BLO)(2021)를 재사용한다. SwTO 는 Linux 소프트웨어 테스트 도메인을 다루며, 스크립트, 도구, 3,000 개 이상의 테스트 케이스를 보유하지만 문서화가 부족한 저장소인 Linux Test Project(LTP)(2021)에 기반한다.

따라서 SwTOI 는 Linux 테스트 도메인의 형식적인 어휘를 제공하는 동시에 체계적인 테스트 작성 기준에 대한 의미적 기록과 테스트 설계자의 지식에 대한 의미적 기록을 제공하는 것을 목표로 한다.

사례 4. Reference Ontology on Software Testing 온톨로지

Reference Ontology on Software Testing(ROoST)(Ferreira de Souza et al., 2017)은 목적이 여러 가지인 성숙한 온톨로지이다. ROoST 는

원문 PDF 137 페이지

소프트웨어 테스트 도메인에 대한 공통 개념화를 확립하여 테스트에 참여하는 사람들을 위한 테스트 도메인의 공통 어휘를 정의하고, 테스트 지식 저장소를 구조화하며, 테스트 지식 항목에 주석을 달고, 관련 정보를 더 쉽게 검색하도록 한다.

또한 재사용을 위해 테스트 지식을 획득하고, 처리하고, 분석하고, 배포하는 작업에 컴퓨터화된 지원을 제공한다.

ROoST 는 ISO/IEC/IEEE 29119-2013 표준(1, 2, 3, 4 부), IEEE 829-2008 표준, SWEBOK(Bourque and Fairley, 2014)을 바탕으로 구축되었다. 1 차 논리로 45 개의 용어와 공리를 정의했으며 UFO 상위 온톨로지와 서로 연결된 도메인 관련 온톨로지 패턴을 사용했고, 그 개념을 테스트 도메인으로 확장했다. 따라서 ROoST 는 다음 네 개의 하위 온톨로지 구성된다.

- 테스트 프로세스와 활동: 예를 들어 테스트 계획, 테스트 케이스 설계, 테스트 코딩, 테스트 실행, 테스트 결과 분석(Black et al., 2012). PAE(process and activity execution) 패턴 재사용
- 테스트 산출물: 예를 들어 테스트 계획, 테스트 절차, 테스트 케이스, 테스트 결과. WPPA(work product participation) 패턴 재사용
- 테스트 기법: 예를 들어 블랙박스 테스트 기법, 화이트박스 테스트 기법, 결함 기반 테스트 기법, 모델 기반 테스트 기법. PRPA(procedure participation) 패턴 재사용
- 테스트 팀과 환경: 예를 들어 하드웨어 자원, 소프트웨어 자원, 인적 자원. HRP(human resource participation)와 RPA(resource participation) 패턴 재사용

사례 5. AI-T 온톨로지

인공지능 기반 시스템 테스트 온톨로지(AI-T)(Olszewska, 2020)는 특히 인공지능 기반 시스템의 테스트를 위해 만들어진 풍부한 소프트웨어 테스트 온톨로지이며, 현재까지 708 개의 용어와 706 개의 공리를 포함한다.

이 온톨로지는 소프트웨어 테스트 도메인과 인공지능 도메인을 모두 다루며 윤리적이고 설명 가능한 인공지능도 포함한다. AI-T의 지식체계에는

원문 PDF 138 페이지

ISO/IEC/IEEE 29119-2013 표준(1, 2, 3, 4 부), ISO/IEC/IEEE 24765:2017 표준, IEEE 7001(Winfield et al., 2021), 7010(2020) 등의 IEEE 70xx 표준과 SWEBOK(Bourque and Fairley, 2014), ISTQB(2021)가 포함된다.

AI-T 온톨로지에 인코딩된 개념과 속성의 일부를 그림 6.1에 제시한다.

AI-T 온톨로지의 주요 개념은 소프트웨어 테스트 전략, 소프트웨어 테스트 프로세스, 소프트웨어 테스트 문서화, 소프트웨어 테스트 수준, 소프트웨어 테스트 기법, 소프트웨어 테스트 유형, 소프트웨어 테스트 산출물, 소프트웨어 테스트 측정값, 소프트웨어 테스트 활동, 이해관계자, 환경으로 식별되었다.

온톨로지 주도 소프트웨어 테스트의 동향

이 장을 마무리하기 위해 온톨로지 기반 소프트웨어 테스트의 과거, 현재, 미래 동향을 논의하는 것이 중요하다.

온톨로지 요구사항

소프트웨어 테스트를 위한 온톨로지의 범위는 발전하고 있으며 요구사항도 함께 발전한다. 따라서 온톨로지의 요구사항 도출과 분석은 온톨로지 엔지니어링 프로세스에서 중요한 작업이다(Bayat et al., 2016). 이는 UML(Unified Modeling Language)(Kogut et al., 2002)을 사용하여 수행할 수 있다. 따라서 현재 소프트웨어 테스트를 위한 온톨로지의 주요 요구사항은 그림 6.2의 UML 유즈케이스 다이어그램과 같이 요약할 수 있다.

ROoST와 같은 초기 소프트웨어 테스트 온톨로지의 과거 요구사항은 다음과 같았다.

- 인간의 소프트웨어 테스트 프로세스 학습 지원
- 소프트웨어 테스트 관련 지식을 구조화하고 표현하기 위한 기반 제공
- 테스트 프로세스를 지원하는 소프트웨어 도구 통합을 위한 참조 모델 역할

원문 PDF 139 페이지

- 의미 기반 문서화 접근 방식에서 테스트와 자원에 주석을 달기 위한 참조 모델 역할

오늘날 AI-T와 같은 소프트웨어 테스트 온톨로지의 요구사항에는 다음이 포함된다.

- 인간 테스터가 소프트웨어 테스트를 관리하도록 지원
- 인간 테스터가 인공지능 기반 소프트웨어를 테스트하도록 지원

그림 6.1 소프트웨어 테스팅을 위한 AI-T 온톨로지의 화면

그림 6.1은 AI-T 온톨로지 편집 화면의 예를 보여주며, Software_Testing_Documentation, Software_Testing_Level, Software_Testing_Measure, Software_Testing_Process, Software_Testing_Strategy, Software_Testing_Technique, Dynamic_Test_Technique, Static_Test_Technique, Software_Testing_Type, Stakeholder 등의 개념과 이들 사이의 관계를 표시한다.

- 지능형 에이전트가 테스트 케이스를 생성하거나 재사용하도록 안내
- 지능형 에이전트가 소프트웨어 테스팅을 학습하도록 촉진
- 인간과 지능형 에이전트가 혼합된 협업 팀이 소프트웨어 에이전트를 테스트하도록 지원

시스템과 소프트웨어의 테스트가 더 복잡해짐에 따라 이러한 요구사항은 시간이 지나면서 확대될 수 있다 (Olszewska, 2019a).

그림 6.2 차세대 소프트웨어 테스팅 온톨로지의 UML 유즈케이스 다이어그램

원문 PDF 140 페이지

그림 6.2의 주요 항목:

- 소프트웨어 테스팅을 위한 온톨로지
- 지능형 에이전트
- 소프트웨어 테스터
- 테스트 에이전트
- 테스트 수행
- 테스트 적정성 측정
- 테스트 재사용
- 테스트 공유
- 테스트 자동화
- 보고서 생성
- 설명가능성 평가
- 신인성 판단
- 상호운용성 조사

온톨로지 설명가능성

온톨로지는 전통적으로 논리 공리를 사용하여 형식화되며, 논리 기반 형식 추론을 가능하게 한다. 따라서 이러한 논리 기반 온톨로지는 본질적으로 설명 가능하다. 그러므로 자율 시스템(Olszewska, 2018c)과 로봇틱스(Olivares-Alarcos et al., 2019)를 포함하는 응용에 적합한 투명한 인공지능 기술을 구성한다.

또한 이러한 논리적 온톨로지 접근 방식은 보통 도메인 전문가와 온톨로지 전문가가 수동으로 협업하여 구축한다 (Bermejo-Alonso et al., 2018). 예를 들어 ROSADev(ontological standard development life cycle) 프로세스(Olszewska et al., 2018)를 따르며, 2021년 IEEE Standards Association Award를 수상한 IEEE 7007-2021과 같은 온톨로지 표준으로 이어진다.

IEEE 7007-2021 은 윤리 주도 로봇틱스 및 자동화 시스템을 위한 온톨로지 표준이다(Olszewska et al., 2020).

온톨로지는 머신러닝과 함께 사용할 수도 있다. 실제로 클러스터링(Maedche and Staab, 2004) 같은 머신러닝 기법은 지식체계로부터 온톨로지 개념을 자동으로 추출하는 데 도움을 줄 수 있다. 더 최근에는 딥러닝(Petrucci et al., 2016)이 데이터셋에서 온톨로지 공리를 자동으로 식별하는 데 적용되었다.

한편 심층 재귀 신경망(Hohenecker and Lukasiewicz, 2020)은 온톨로지에 대한 자동 추론을 수행하는 데 사용되어 왔다. 논리 기반 형식 추론이 아니라 딥러닝에 기반한 이 새로운 온톨로지 추론 접근 방식은 유망한 성능을 보였다. 그러나 고전적인 논리 추론기에 의존하는 전통적 논리 기반 온톨로지는 자동 추론을 가능하게 하는 동시에 투명성(Dragoni et al., 2020)을 제공한다. 이 투명성은 시스템 및 소프트웨어 엔지니어, 설계자, 개발자, 테스터를 포함한 사용자와 전문가가 요구하는 특성이다.

온톨로지 응용

통합형, 분산형, 클라우드 기반 여부와 관계없이 모두 소프트웨어에 의존하는 지능형 시스템과 지능형 에이전트의 수와 복잡성이 증가함에 따라 이러한 시스템과 소프트웨어를 테스트하기 위한 새로운 기법이 필요하다(Olszewska, 2019b). 온톨로지는 이러한 복잡하고 이질적인 시스템의 소프트웨어 테스트에 해결책을 제공하는 인공지능 기술이다.

실제로 온톨로지는 인간 여부와 관계없이 지능형 에이전트 집단이 공유하는 도메인의 공통 지식 표현을 생성하는 것을 목표로 한다(Chari et al., 2020). 따라서 온톨로지는 이 공유 도메인에 대한 형식적 개념화를 생성하여 상호운용 가능한 시스템을 제공하고 그 위에서 자동 추론을 수행할 수 있게 한다(Tudorache, 2020).

예를 들어 인더스트리 4.0 에서 상호운용성은 제조 시스템 내에서 신뢰할 수 있는 환경을 구축하는 데 기여한다. 이 환경에서는 기계와 인간 사이에서 정보가 정확하고 신속하게 공유되며, 그 결과 비용을 절감하면서 생산성을 높일 수 있다(Koh et al., 2019). 따라서 인더스트리 4.0 을 위한 온톨로지는

(Sampath Kumar et al., 2019) 스마트 제조 도메인과 같은 최첨단 도메인을 포착하고, 해당 도메인 지식을 모호하지 않고 상호운용 가능하며 설명 가능하고 재사용 가능한 방식으로 표현한다. 이를 통해 자동화된 테스트를 수행하고 스마트 제조 디지털 트윈 모델링을 위한 기반을 마련하는 등의 작업이 가능하다.

특히 복잡하고 이질적이며 자율적인 시스템의 온톨로지 주도 테스트를 수행하기 위해 소프트웨어 테스터는 CORA 와 같은 로봇틱스 및 자동화 코어 온톨로지, SUMO 와 같은 세계 지식 표현에 관한 상위 온톨로지, 그리고 AI-T 소프트웨어 테스트 온톨로지처럼 소프트웨어 테스트 도메인을 형식화하는 더 전문화된 온톨로지를 함께 사용할 수 있다. 또한 테스트 대상 복잡 시스템이 스마트 환경에서 발전하는 경우, 스마트 응용, 예를 들어 스마트 제조, 스마트 시티 등을 위해 개발된 추가 도메인 온톨로지를 사물인터넷(IoT)(Ma et al., 2014) 전용 온톨로지와 같은 다른 이용 가능한 I4.0 온톨로지와 함께 재사용할 수 있다.

요약

요약하면 소프트웨어 테스트를 위한 온톨로지는 에이전트 중심 접근 방식을 제시한다. 한편으로는 소프트웨어 테스트를 자동화하도록 돕고, 다른 한편으로는 인공지능 기반 시스템을 테스트할 수 있게 한다.

또한 온톨로지는 설명 가능하고 상호운용 가능하다. 온톨로지는 인간, 자율 시스템, 지능형 에이전트, 시스템의 시스템과 같은 이질적인 지능형 에이전트 사이의 신뢰할 수 있는 협업을 가능하게 한다. 이는 특히 소프트웨어와 시스템 테스트의 맥락에서 유용하다. 실제로 이러한 온톨로지 접근 방식은 스마트 시스템 테스트를 수행하는 소프트웨어 테스터, 자가 수리를 시도하는 자율 시스템, 인간 참여형(human-in-the-loop) 테스트를 수행하는 지능형 시스템과 같은 지능형 에이전트가 사용할 수 있다.

또한 온톨로지는 소프트웨어 테스트를 위한 모듈식이고 재사용 가능한 접근 방식을 제공한다. OWL 2.0 온톨로지를 위한 Java API 인 OWL API 나 JSON 에서 Linked Data 를 직렬화하기 위한 경량 구문인 JSON-LD 등을 통해 다른 소스 코드에 통합할 수 있고, Testing-as-a-Service(TaaS) 플랫폼을 지원할 수도 있다.

원문 PDF 143 페이지

원문 PDF 144 페이지

7 디지털 트윈 테스트로 메타버스까지 시프트 라이트하기

Jonathon Wright

디지털 트윈 테스트를 통한 시프트 라이트는 분석과 자율화의 힘을 활용하여 현실 세계로부터 동적으로 학습하는 것을 의미한다. 변동성이 큰 시스템적 실패 모델에 직면한 디지털 인터페이스는 매우 견고해야 하고, 수많은 엔드포인트를 통해 동작하며, 막대한 양의 정형 및 비정형 데이터를 처리해야 한다. 지속적 전달과 테스트를 도입하면 민첩성을 확보하는 데 도움이 되지만, 자동화는 일반적으로 전통적인 스크립팅과 오케스트레이션을 사용하는 정적인 규칙에 의해 구동된다. 이러한 접근 방식은 변화하는 요구사항을 따라가기 위해 높은 유지보수 부담을 가진다. 최근 예측 분석과 AIOps 같은 인지 엔지니어링 기술이 등장하면서 변화하는 실제 상황 안에서 적응형 자동화를 자가 복구와 자가 구성 수준까지 발전시킬 가능성이 열렸다.

테스팅에 대한 시프트 라이트 접근 방식

정보 시스템 설계와 테스트 주기의 오른쪽에 더 초점을 이동하여 학습의 이점을 높이는 것.

원문 PDF 145 페이지

시프트 라이트가 무엇인지 더 잘 이해하기 위해 먼저 시프트 레프트가 무엇인지 살펴보자. 시프트 레프트는 워터폴/V-모델 방법론의 전통적인 순차 설계 기법, 즉 요구사항 분석, 설계, 테스트, 구현에서 최근의 반복적인 애자일 설계 원칙에 더 초점을 맞추는 것을 의미한다.

최근의 시스템 설계와 테스트 접근 방식도 수명주기 후반보다 설계 단계에서 문제를 해결하는 데 여전히 초점을 맞추는 경향이 있다. 예를 들어 근본원인 분석과 파레토 원칙을 사용하여 가장 많은 결함의 원인을 찾아 제거하는 방식이다(그림 7.1 참조). 목적은 이후의 변경에 따른 재작업에서 발생할 비용을 줄이는 것이다.

정보 시스템이나 소프트웨어를 설계하면서 가능한 모든 실패 방식을 테스트한 결과를 토대로 사용자에게 가장 좋은 것이 무엇인지 가능한 한 많이 파악하려고 하고, 이것이 초기 설계와 테스트 단계에 영향을 미친다면 이를 IT 시스템 또는 소프트웨어 개발 주기의 왼쪽으로 이동하는 것이라고 할 수 있다. 따라서 이러한 실무를 설명하는 데 시프트 레프트라는 용어를 사용한다.

시프트 레프트는 결함이 유입된 지점에 가능한 한 가까운 위치에서 결함을 찾아 제거하는 것과 관련된다. 이 접근 방식은 시간과 노력의 초점을 초기 설계 및 테스트 단계로 이동시키는 경향이 있다. 즉, 시스템 또는 소프트웨어 개발 주기의 왼쪽으로 이동시키는 것이다.

그림 7.1 시프트 레프트와 시프트 라이트

원문 PDF 146 페이지

참고: BA = 비즈니스 분석가, CI = 지속적 통합, DEV = 개발, NFT = 비기능 테스트, pre-prod = 사전 운영, QA = 품질 보증, SCM = 소스 코드 관리, UAT = 사용자 인수 테스트.

시프트 라이트와 시프트 레프트의 대비

시프트 라이트에서는 개발 및 테스트 단계 초기에 발생할 수 있는 문제를 식별하려고 하기보다, 테스트에 지나치게 많은 시간을 쓰지 않고 빠르게 개발하여 고객 피드백을 더 일찍 얻는다. 따라서 개발 및 테스트 주기의 오른쪽으로 더 이동한다.

시프트 라이트는 학습 측면과 시스템 설계 및 테스트의 지속적인 진화에 더 큰 비중을 둔다. 시스템이나 소프트웨어를 배포한 뒤 사용자 경험과 그 밖의 데이터를 피드백으로 사용한다. 이 피드백 루프는 시스템이나 소프트웨어가 무엇을 위해 설계되었는지를 실제 현실에서 일어나고 있는 일과 연결하여, 필요한 핵심 변경을 더 잘 이해하도록 한다.

시프트 라이트는 고객 피드백으로부터 빠르게 학습하는 데 초점을 맞추어 시스템을 반복적으로 개발하는 것과 관련된다. 이 접근 방식은 전체 테스트 시간을 최소화하기 위해 빠르게 개발하여 사용자나 포커스 그룹에 릴리스하는 데 초점을 맞춘다. 이러한 방법론은 개발 및 테스트 수명주기의 오른쪽으로 더 이동한다.

원문 PDF 147 페이지

인지 엔지니어링 원칙

인지 엔지니어링 소프트웨어 개발 원칙은 다음 세 요소의 결합으로 볼 수 있다.

- (i) 사고 단계. 어떤 것을 모델링하고 그것이 특정 방식으로 동작할 것으로 기대하는 단계.
- (ii) 생성 단계. 사고 측면에 따라 생성하는 단계.

(iii) 학습 단계. 운영 도구의 도움을 받아 생성된 제품이나 서비스가 (i) 단계에서 예측한 대로 제공되었는지를 매핑하여 학습하는 단계(그림 7.2).

이 전통적인 선형 프로세스에는 이제 현실 세계의 사건에서 얻은 데이터를 다시 공급하는 피드백 루프가 포함된다. 시프트 라이트 접근 방식을 사용하면 학습 측면을 더 일찍 달성할 수 있고, 개발자는 수명주기 훨씬 이른 시점부터 시스템 또는 소프트웨어 개발에 대한 피드백을 받기 시작할 수 있으므로 전체 개발과 테스트에 상당한 시간을 절약할 수 있다.

다시 말해 시프트 라이트는 정보 시스템이나 소프트웨어의 오른쪽, 즉 사용자 여정에서 학습하고 피드백 데이터를 사용하여 더 스마트한 개발 및 테스트 기법으로 발전시키는 프로세스이다. 이를 통해 개발자와 테스터는 발생하기 전에 잠재적인 문제를 이해할 수 있다.

그렇다면 테스터는 이렇게 개발된 시스템에서 발생할 수 있는 잠재적 문제를 어떻게 식별할 수 있는가? 단순히 설명하면 인공지능, 머신러닝, 딥러닝 기술의 도움을 받아 전통적인 DevOps 접근 방식보다 더 OpsDev 적인 접근 방식으로 바라보는 것이다.

그림 7.2 인지 엔지니어링(출처: J. Wright, Kinsbruner, 2020)

원문 PDF 148 페이지

우리는 모두 전통적인 소프트웨어 개발 주기와 DevOps 가 수행하는 역할에 익숙하다. 이는 실제로는 ‘개발 중심 운영 역량(Dev-centric Ops-capabilities)’을 가능하게 하는 것으로 가장 잘 설명할 수 있지만, 진정한 가치는 OpsDev, 즉 ‘운영 중심 개발 지원(Ops-centric Dev-enablement)’ 협업이어야 한다. 이는 운영 영역에 대한 더 나은 통찰을 제공함으로써 달성할 수 있으며, 이러한 도구를 운영 환경에만 제한해서는 안 된다. 시프트 라이트 원칙 아래에서 구축 중인 소프트웨어의 동작을 더 많이 학습하기 위해 유사하거나 더 나은 도구를 사용할 수 있다.

인지 엔지니어링 분야

인지 엔지니어링은 사용자가 기대하는 인지 과정을 지원하는 엔지니어링 시스템을 설계하고 개발하기 위해 인지행동심리학의 개념을 사용하는 연구 방법이다(Wilson et al., 2013). 인지 엔지니어링 분야는 테스트를 시프트 라이트하는 데 어떻게 도움을 줄 수 있는가? 다음과 같은 특정 휴리스틱을 활용할 수 있다.

- 성능 엔지니어링. 운영 부하를 모사하여 성능의 한계 또는 병목을 판단한다. 이러한 테스트는 높은 부하 조건에서 발생할 수 있는 문제를 찾아내기 위해 수행된다.
- 사이트 신뢰성 테스트(SRT). 운영 환경 안에서 소프트웨어를 현실적으로 테스트하는 데 초점을 맞춘다. 다시 말해 프로그래밍 가능한 인프라를 유지하고 그 위에서 실행되는 워크로드의 가용성을 최대화하는 실무이다.

원문 PDF 149 페이지

- 사이트 신뢰성 테스터는 IT 운영 관리(ITOM)에 소프트웨어 테스트의 사고방식을 적용하여 소프트웨어 개발 수명주기(SDLC) 내 테스트와 운영 우수성 사이의 경계를 흐리게 한다.
- 카오스 엔지니어링. 예측할 수 없는 조건을 견딜 수 있는 시스템의 능력에 대한 확신을 높이기 위해 운영 중인 소프트웨어 시스템에서 실험하는 분야이다.

그림 7.3의 '사고 단계'는 '실험'을 정의하고 동작에 대한 '가설'을 입증하며, 다음과 같은 '카오스 엔지니어링 원칙'(2019)을 공유한다.

- 정상 상태 동작을 중심으로 가설을 수립한다. 시스템 내부 속성보다 시스템의 측정 가능한 출력에 초점을 맞추고, 실험 중 시스템 동작 패턴에 초점을 맞춰 시스템이 어떻게 동작하는지를 확인하는 것이 아니라 실제로 동작하는지를 검증한다.
- 현실 세계의 사건을 도입한다. 카오스 변수는 현실 세계의 사건을 반영한다. 잠재적 영향이나 예상 빈도에 따라 사건의 우선순위를 정한다. 정상 상태를 방해할 수 있는 모든 사건은 실험의 잠재적인 변수가 된다.
- 실험을 자동화하여 지속적으로 실행한다. 관찰 가능성, 오케스트레이션, 분석을 구동할 수 있는 능력을 운영 시스템에 구축한다.

그림 7.3 인지적 사고: '디지털 선언문(The Digital Manifesto)'(출처: Wright, 2016b)

그림의 주요 흐름: Insight(통찰) - Interpreting(해석) - Modelling(모델링) - Challenging(도전) - Predicting(예측) - Act(행동) - Actionable(실행 가능) - Research(연구) - Experiment(실험) - Hypothesis(가설).

원문 PDF 150 페이지

카오스 엔지니어링은 격동적이고 예상하지 못한 조건을 견딜 수 있는 시스템 능력에 대한 확신을 높이기 위해 운영 중인 소프트웨어 시스템에서 실험하는 분야이다(Principles of Chaos Engineering, 2019).

이러한 분야를 인공지능, 머신러닝, 딥러닝을 사용하는 인지 플랫폼 개발에 적용할 수 있다. 인지 엔지니어링의 진정한 힘은 이러한 분야를 사용하여 가설을 빠르게 입증할 수 있다는 데 있다. 여기에서 모델링 뒤에 있는 실제 과학은 워크로드이다. 볼륨 모델은 현실 세계의 사용자 여정을 이해하도록 돕고, 목표로 미리 정의된 시스템 상태에 대해 실행할 수 있는 현실적인 부하 프로파일을 만들 수 있게 한다.

그림 7.4 인지 학습 - '혁명보다 디지털 진화(Digital Evolution, Over Revolution)'(출처: Wright, 2017)

그림의 주요 순환 흐름: Pivot(전환) - Experience(경험) - Evolve(진화), 그리고 Disrupt(교란), Interactions(상호작용), Behaviours(행동), Predicting(예측), Adapt(적응), Observe(관찰).

애플리케이션 성능 관리(APM) 플랫폼과 협업하는 인지 엔지니어링 분야를 지능형 운영(AIOps)과 결합하면 시스템 설계자가 시스템이 실제로 고장 나기 전에 언제 고장 날지를 식별하도록 도울 수 있다.

원문 PDF 151 페이지

SRT는 고도로 기준선화된 시스템의 관찰 가능성과 테스트 가능성을 제공한다. 즉, 시계열 데이터를 TICK 스택과 같은 스트리밍 데이터 서비스와 결합하는 것처럼 다차원 트랜잭션 데이터셋을 활용한다(그림 7.4). 이를 통해 현실적 이거나 합성으로 생성된 부하를 측정하고 시스템 또는 애플리케이션의 가설을 입증하는 데 사용할 수 있다.

시프트 라이트에서의 디지털 트윈 개념

현대 소프트웨어 개발의 복잡한 특성은 흔히 여러 추상화 계층을 만든다. 예를 들어 하드웨어 추상화 계층(HAL)은 하드웨어 설계의 복잡성을 소프트웨어 엔지니어로부터 숨긴다. 시스템의 시스템에서도 마찬가지다. 디지털 트윈 개념을 사용하면 시스템 사고를 추상화하여 비기술 분야 전문가도 설명을 이해할 수 있게 한다.

디지털 트윈은 물리적 또는 가상 객체를 나타내는 디지털 데이터를 생성하거나 수집하는 것이다. 디지털 트윈 개념은 엔지니어링과 엔지니어링 도면 또는 그래픽의 생성에 뿌리를 두고 있다. 디지털 트윈은 제품 설계와 엔지니어링 활동을 생성하는 과정의 지속적 개선에서 나온 결과이다.

NASA의 아폴로 13호 임무는 우주비행사를 지구로 안전하게 귀환시키기 위해 디지털 트윈 개념이 처음 사용된 사례로 볼 수 있다.

성공적으로 발사된 뒤 아폴로 13호는 서비스 모듈 분리 과정에서 산소 탱크가 폭발하면서 주 엔진에 손상을 입었다. 그 결과 우주비행사의 산소 공급 장치에 생명을 위협하는 누출이 발생했다.

미션 컨트롤은 우주비행사를 귀환시키기 위해 이용 가능한 모든 지상 자원을 신속하게 투입해야 했다.

원문 PDF 152 페이지

지상에서 시뮬레이터를 사용하는 다른 우주비행사들의 도움을 받아 NASA는 지구에서 거의 20만 마일 떨어진 우주에서 일어나고 있는 상황을 재현하고 우주비행사를 성공적으로 지구로 귀환시킬 해결책을 찾아냈다.

그들은 손상된 우주선의 가상 표현을 만들고 그 디지털 트윈에서 시뮬레이션을 실행하여 손상을 어떻게 해결할지를 더 잘 이해한 뒤, 시뮬레이션 환경에서 만든 변경을 실제 환경에 적용했다(Ritchson, 2021).

사례 연구: 팬데믹 동안 지역사회의 안전 유지 지원

그림 7.5는 'Splunk for Good' 팀의 구성원 Todd DeCapua가 제공한 2021년 2월 14일의 COVID-19 대시보드를 보여준다.

2020년 Rob Tiffany는 디지털 트윈이 COVID-19 팬데믹과 싸우는 가장 좋은 무기가 될 수 있다는 개념을 공유했다. 다음 YouTube에서 볼 수 있다. <https://youtu.be/XKINvqiTgxQ>

디지털 트윈 접근 방식은 GPS(Global Positioning System)와 GAEN(Google Apple Exposure Notifications) 접촉 추적 플랫폼의 테스트를 지원함으로써 팬데믹 동안 COVID-19 확산을 줄이는 데 도움을 주었다. Google 및 Apple과 긴밀하게 협력하여 MIT는 GPS 데이터에 의존하는 대신 Bluetooth 데이터를 포함할 수 있었고, 두 사람 또는 장치가 상호작용할 때 Bluetooth를 근접 센서로 사용할 수 있었다. 이러한 협업의 결과로 접촉 추적 테스트와 제를 해결하기 위해 디지털 트윈을 적용한다는 아이디어가 나왔다.

원문 PDF 153 페이지

2020년 초부터 나는 MIT와 직접 협력하여 COVID PathCheck Foundation(n.d.)의 품질 보증 및 테스트 활동을 이끌었다. 이는 당시 다른 보건 담당자들이 사용하던 수동 프로세스보다 더 효율적인 접촉 추적 해결책을 만드는 데

초점을 맞췄다. 수동 프로세스에서는 개인에게 지난 며칠 동안 어디에 있었는지, 누구를 만났는지 등을 묻는다. 이러한 수동 접근 방식은 개인이 방문했던 장소에 대한 기억에 크게 의존하므로 많은 결함이 발생하기 쉬웠다.

접촉 추적 프로젝트에서 이러한 개념을 사용한 동기는 거짓 양성 경보의 수를 줄여 앱의 정확성을 높이는 것이었다. 전 세계가 COVID-19 확산의 영향을 받았을 때 각국은 효과적인 접촉 추적 해결책이 절실히 필요했다. 앱을 사용할 수 있게 만들기 전에 가능한 모든 결과를 고려하는 데 많은 시간을 투입하는 것은 실현 가능한 해결책이 아니었다.

그림 7.5 COVID-19 패턴 및 동향, 2021(출처: Splunk for Good)

원문 PDF 154 페이지

영국 정부는 바이러스에 노출되었을 가능성을 사람들에게 알리는 모바일 앱을 개발하여 바이러스 확산을 줄이기 위해 작업을 시작했다. 정부는 실제 사람들이 스마트폰을 가지고 걸어 다니며 현실 세계의 상호작용을 모사하도록 하여 앱을 테스트했다. 이 시프트 레프트 접근 방식은 사전에 불균형할 정도로 많은 시간을 소모했고, 그 결과 보건 당국은 제때 동작 가능한 해결책을 제공하지 못했다(Ward, 2020). 이는 시프트 라이트 접근 방식이 시간 제약을 최소화하고 실용적이고 효율적인 접촉 추적 앱을 더 빠르게 개발할 수 있게 했을 수 있는 전형적인 사례이다.

우리의 시프트 라이트 접근 방식은 증상을 경험했거나 감염되었거나 회복한 사용자의 과거 위치 데이터를 표현하기 위해 디지털 트윈 의미론을 적용했다. 앱을 개발한 후 모바일 앱의 모델, 즉 그림 7.6의 왼쪽에 표시된 모델을 만들었고 실제 모바일 앱은 오른쪽에 표시했다.

이제 특정 날짜의 런던 사람들을 그림 7.7과 같이 핀으로 나타낸다고 해보자. 더 어두운 색으로 표시된 핀은 COVID-19에 감염된 사람을 나타낸다. 어두운 점은 비교적 적다. 이들 중 일부는 자동차를 운전하거나 버스를 타거나 집에서 자가격리 중일 수 있다. 자동차라는 밀폐 공간에서 운전하는 사람은 차에서 내려 밀폐된 공간으로 들어가기 전까지 차 밖의 사람을 감염시킬 위험이 없으므로 앱이 반드시 경보를 발생시킬 필요는 없다. 이러한 거짓 양성의 결과로 실제 감염 위험이 없는데도 사람들이 자가격리해야 할 수 있다.

여기에서 머신러닝을 활용하여 현실 세계의 시나리오를 더 잘 이해하고, 시속 6마일로 자동차를 운전하는 사람을 나타내는 어두운 점과 시속 3마일로 걷는 보행자, 그리고 다른 시민과 가까이 커피숍에 앉아 있는 사람을 구분할 수 있다.

2017년부터 기본적으로 모든 Android 휴대전화는 사실상 5분마다 사용자의 위치를 추적하고 기록해 왔으며, Google의 위치 서비스는 기본적으로 24시간마다 650개의 위치 데이터 지점을 수집하여 최대 14일의 '잠복기' 동안 9,100개의 데이터 지점을 만든다. 우리 프로젝트에서는 개인을 식별할 수 있는 모든 데이터를 제거하는 개인정보 보호 우선 접근 방식을 따르는 것이 중요했다(Collins, 2017). GCHQ와 NHSx 앱은

원문 PDF 155 페이지

아일오브와이트에서 모바일 앱을 현장 테스트하는 동안 비판을 받았다. 실제 사용자의 Google Firebase와 텔레메트리 데이터를 사용하고 있었기 때문에 데이터 개인정보 보호에 대한 우려가 부각되었다(Ward, 2020).

그림 7.6 MIT 프로젝트 Safe Paths 앱: 디지털 트윈 모델 화면

그림 7.7 MIT 프로젝트 Safe Places: 비식별화 도구(Redaction Tool)

그림 7.8 Google Takeout: 미국 매사추세츠주 보스턴 주변의 과거 데이터

원문 PDF 156 페이지

Google Takeout 에서 위치 데이터를 내보내 Android 장치의 과거 GPS 데이터, 즉 일반적으로 최대 10 년까지 이용 가능한 GPS 위치 데이터 로그를 얻을 수 있었다. 그림 7.8 의 왼쪽 아래에는 GPS 좌표가 표시되어 있다. Google Takeout 을 활용하여 Google 사용자의 기존 위치 데이터를 추출하고 현실 세계에서 무슨 일이 일어나고 있었는지를 알아냈다. 수백, 수천 명의 사용자에게서 GPX(GPS Exchange format) 데이터를 얻자 데이터 레이크를 만들 필요가 있을 만큼 충분한 빅데이터가 구성되었다.

인지 엔지니어링을 적용하여 이 데이터 레이크에 머신러닝, 인공지능, 딥러닝 기술을 통합했다. 스키마 기반 생성, 즉 무작위화 알고리즘과 신경망, 즉 자연어 처리와 생성 모델을 기반으로 정형 GPX 데이터를 처리하여 위치 데이터의 통계적 속성을 유지하고 현실적인 테스트 데이터 큐브를 합성했다. 합성으로 생성된 데이터를 사용하고 모델 기반 데이터 시나리오의 GPX 데이터셋을 수백 대의 실제 장치에 주입함으로써 각 장치의 노출 알림 상태 동작을 관찰할 수 있었다.

COVID-19 접촉 추적 플랫폼 테스트

디지털 경험(DX)은 사용자 행동이 디지털 상호작용과 결합된 것이다. 이 접근 방식은 정보 시스템과 애플리케이션의 사용자 경험을 최적화하기 위해 어떤 변경이 필요한지를 더 잘 이해하도록 했다. 이 책의 맥락에서 다른 방식으로 말하면 현실 세계, 즉 ‘오른쪽’, 이미 이용 가능한 애플리케이션이나 정보 시스템으로부터 학습하고 사용자 상호작용 정보를 모델, 즉 실제 애플리케이션의 디지털 트윈에 공급할 수 있다는 것이다.

원문 PDF 157 페이지

그 다음 MIT 프로젝트는 보건 담당자의 접촉 추적을 돕기 위해 Berkeley 에서 시작되었다(그림 7.9). 성공하려면 실제 사람의 데이터 입력이 필요했다. 과거 사용자 데이터를 모사할 수 없었기 때문이다. 이는 클라우드 테스트와 시프트 라이트 접근 방식을 결합하는 것이 운영 환경에서 해결책을 성공적으로 테스트하기 위한 최선의 선택이었던 사례이다.

사례 연구: 스마트 시티 데이터 익스체인지 - 메타버스에서의 테스트

Copenhagen Smart City Data Exchange 와 MIT 접촉 추적 프로젝트에서 사용된 접근 방식에는 여러 유사점이 있었다. 두 프로젝트에서 가장 큰 과제는 과거와 정확히 일치하는 수십억 개의 위치 데이터 지점을 합성으로 생성하고 위치 데이터를 GPS 에 실시간으로 주입할 수 있는 능력이었다.

코펜하겐의 Smart City Data Exchange 는 위치 데이터를 사용하여 디지털 시민들이 2020 년까지 탄소 중립을 달성하도록 돕고자 했다(그림 7.10). 스마트 시티 모바일 앱은 코펜하겐에 거주하고 일하는 38,000 명에게 배포되었다.

현실 세계의 테스트에 적용된 메타버스 개념

메타버스는 기술, 즉 소프트웨어와 물리적 하드웨어, 그리고 현실 세계 사이의 경계가 흐려지는 것이다. 이 개념은 1992 년 Neal Stephenson 의 소설 Snow Crash 까지 거슬러 올라간다.

2017 년 나의 인지 학습 TED 강연(그림 7.11)에서 보여주었듯이 Google, Apple, Meta 의 증강 및 혼합현실(AR/MR) 헤드셋은

원문 PDF 158 페이지

(Zuckerberg, 2021)(이전 Facebook) 메타데이터를 현실 세계 위에 겹쳐 표시함으로써 DX 상호작용이 실제 우리 세계를 그대로 반영하는 디지털 세계를 만들 수 있게 했고, 대체 불가능 토큰(NFT)을 통해 전례 없는 데이터 상호운용성과 교차 공유를 제공했다.

그림 7.9 MIT 프로젝트 Safe Places: 비식별화 도구(Redaction Tool)

그림 7.10 Copenhagen Smart City Data Exchange, 2015

원문 PDF 159 페이지

대체 불가능 토큰(non-fungible token)은 복사, 대체, 분할할 수 없는 고유한 디지털 식별자로, 블록체인에 기록되며 특정 디지털 자산과 이에 관련된 특정 권리의 진위성과 소유권을 증명하는 데 사용된다(Merriam-Webster, 2021).

그림 7.11 TED 강연: 인지 학습 - '혁명보다 디지털 진화(Digital Evolution, over Revolution)', 2017

원문 PDF 160 페이지

메타버스 안에서 테스트할 때의 과제는 시민이 물리적 세계에서 사용하는 여러 이동 수단을 식별할 수 있는 능력이다. 이동 수단에는 걷기(시속 2~3 마일), 자전거 타기(시속 10~12 마일), 여러 위치에서 정차하는 버스 이용, 기차 이용, 또는 한 위치에서 GPX 데이터 기록이 중단되었다가 다른 위치에서 다시 나타날 수 있는 지하철 이용 등이 있다. 각 이동 수단에는 사용한 교통수단 유형을 식별하기 위해 상호 참조해야 하는 고유한 흔적이 있다.

COVID-19 프로젝트 시나리오로 돌아가 보면, 우리는 사람들이 가장 많이 모일 가능성이 있는 장소와 사람들이 이동한 경로에 관한 모바일 장치의 과거 데이터를 이미 가지고 있다. 따라서 과거 데이터를 기반으로 사람들이 언제 어디에 있을지, 예를 들어 특정 지역 주변에 언제 모일지를 예측할 수 있다. 주말에는 해변으로 가는 사람이 많을 것이고, 평일에는 아침과 저녁 출퇴근 시간에 통근 서비스를 이용하는 사람이 많을 것이다. 이러한 방식으로 많은 사람의 행동 데이터를 기반으로 혼잡이 가장 발생하기 쉬운 위치를 예측할 수 있다.

이는 시계열 데이터를 활용한 과거 데이터를 사용하여 예측 기능을 가진 머신러닝 기술을 AIOps 안에 통합하는 것과 같은 아이디어이다.

원문 PDF 161 페이지

AIOps 는 인공지능을 사용하여 IT 운영 관리를 단순화하고 복잡한 현대 IT 환경에서 문제 해결을 가속하고 자동화한다. 이러한 운영 작업에는 자동화, 성능 모니터링, 이벤트 상관관계 등이 포함된다(IBM Cloud Education, 2020).

메타버스로 시프트 라이트하기

스마트폰, 태블릿 또는 어떤 전자 기기를 사용하든 장치가 제공하는 사용자 경험에 감탄하는 것은 드문 일이 아니다. 애플리케이션이나 정보 시스템도 다르지 않다. 우리 모두는 이러한 장치와 상호작용할 때 매우 사용자 친화적이고 풍부한 사용자 경험을 선호한다.

그러나 우리는 순수한 사용자 경험(UX)에서 완전히 몰입적인 디지털 경험(DX)으로 더 나아가고 있다. 예를 들어 관광을 하며 걷는 동안 휴대전화나 스마트 안경을 사용하면 촬영한 사진이 자동으로 메타버스에 업로드될 수 있다. 스마트 장치에는 GPS 위치, 걸음 수, 촬영된 대상의 식별 정보와 같은 관련 메타데이터가 있을 가능성이 높고, 디지털 미디어 사진, 비디오 또는 오디오를 NFT로 변환하여 블록체인 같은 디지털 원장의 디지털 마켓플레이스에서 거래하고 판매할 수도 있다. 이는 모두 컴퓨터 비전, 자연어 처리(NLP), 머신러닝과 같은 차세대 인지 엔지니어링 기능이 물리적인 엣지의 인공지능 지원 엔드포인트 장치에 내장되어 있기 때문에 가능하다.

이러한 정형 데이터가 사용자 경험을 풍부하게 하기 위해 플랫폼에서 어떻게 사용되었는지 몇 가지 예를 보면, Alexa에 질문할 때 우리는 음성 명령으로 장치와 상호작용하고 장치는 우리의 질문에 대한 해결책을 찾기 위해 여러 플랫폼과 통신하며 우리의 선호와 비선호를 기반으로 제안을 제공한다. 예를 들어 Amazon Prime과 Netflix 같은 비디오 스트리밍 서비스는 우리가 본 영화나 최근 관심을 가진 배우를 기반으로 영화를 추천할 수 있다.

원문 PDF 162 페이지

이 모든 일은 사용자가 직접 개입하지 않아도 사용자 경험을 개선하기 위해 일어난다. 이제 하루에 수억 대의 장치가 사람들에게 휴대되고 그 사용자 경험에서 수집되는 이러한 비정형 데이터의 양을 상상해 보자. 이러한 비정형 데이터는 추출, 변환, 적재(ETL)되어 정형 형식이나 관련 메타데이터로 바뀌기 전에는 쓸모없을 수 있다. 스키마 레지스트리나 지식 그래프 같은 것을 활용하면 더 많은 맥락을 인식하도록 만들 수 있다.

게이미피케이션, 순열, Pokémon Go에서 얻은 교훈

MIT COVID-19 프로젝트로 돌아가면 사용자 데이터는 실제 사용자 상호작용, 즉 현실 세계에서 스마트 장치를 가지고 이동하는 실제 사람들을 기반으로 한다. 팬데믹에서는 참여자가 감염될 위험이 있으므로 앱을 테스트하기 위해 실제 사람을 사용해 GPS와 Bluetooth 정보를 수집하는 것은 가장 안전한 생각이 아니다.

그림 7.12 Pokémon Go를 활용한 모델 기반 테스트(출처: Wright, 2016a)

그림 7.13 Agile Requirements Designer(ArD), 2016, 런던 지하철의 모델 기반 테스트(MBT)

원문 PDF 163 페이지

실제 사람을 사용하는 대신 위치 데이터를 사용하고 해당 위치를 주입하여 실제로 그곳에 가지 않고도 해당 장소를 방문하는 것을 모사할 수 있다. 이는 사람들이 GPS 위치 기반 모바일 게임 Pokémon Go에서 부정행위를 했던 방식과 유사하다. 이 게임은 증강현실, 게이미피케이션, 메타버스의 훌륭한 사례이다. 플레이어가 현실 세계의 장소로 실제 이동하여 위치와 시간에 따라 달라지는 업적을 해제하고 전 세계에 흩어진 Pokémon을 잡도록 설계되었다.

그림 7.12에서는 모델 기반 테스트(MBT)를 활용한 Pokémon 예를 볼 수 있다. 다시 말해 메타버스 테스트와 유사한 접근 방식을 사용하면 실제 어떤 장소에도 가지 않고 모든 물리적 위치에 마치 순간이동한 것처럼 나타날 수 있다. 이는 위치 데이터를 Google Street View와 결합하여 카메라 피드를 모사하고, 증강현실을 통해 메타버스 안의 제스처와 디지털 상호작용을 모사하여 Pokémon을 잡는 방식으로 달성했다.

현실 세계에서 A 지점에서 B 지점으로 이동하려고 한다고 가정하면 목적지로 가는 가능한 경로가 매우 많다. 런던 지하철에서 가능한 경로는 그림 7.13을 참조하라. 사람이 걷는지, 자동차를 운전하는지, 버스나 기차를 타는지와 같은 가능한 현실 세계의 변수와 그 순열을 최대한 많이 고려하면 물리적인 노력이나 위험 없이 거의 무한한 가능한 디지털 경험을 모사할 수 있다.

원문 PDF 164 페이지

혁명보다 진화

인류의 발전은 많은 혁명적 변화를 가져왔다. 첫 번째 세대의 진화인 증기 동력 혁명에서 시작하여, 전기와 화석연료로 동력을 얻는 두 번째 세대, 컴퓨터, 전자기기, 통신이 이끄는 세 번째 세대를 거쳐, 곧 인공지능이 동력을 제공할 네 번째 세대로 이어지고 있다. 이러한 혁명 속에서 컴퓨터 시스템과 소프트웨어 개발에 사용되는 설계 분야 역시 지난 수십 년 동안 많은 변화를 겪었다.

1939년 Hewlett과 Packard는 컴퓨터 시스템 개발에 사용되는 설계 원칙을 선도했다. 워터폴 방법은 매우 선형적인 프로세스이며, 소프트웨어와 정보 시스템 설계에 다시 사용되기 훨씬 전부터 하드웨어 제품의 제조를 조직하는데 사용되었다. 산업은 이후 더 반복적인 방법론으로 이동했다. 1960년대 EVO(Evolutionary Project Management의 약자)가 반복적 개발을 도입했으며, 1976년에 Tom Gilb가 처음 출판했다. 그 뒤 널리 채택된 Agile Manifesto가 등장했으며, 현재 많은 산업에서 가장 흔히 사용되는 방법이 되었다. 설계와 테스트 원칙은 발전했지만 학습 곡선은 대부분 '시프트 레프트'에 머물러 왔다.

반면 인지 학습은 시프트 라이트를 강조하며, LeanUX와 같은 전통적인 시프트 레프트 '설계 및 해결책 사고' 원칙을 보완한다.

또한 머신러닝, 인공지능, 딥러닝 같은 분야의 최근 발전을 크게 활용할 수 있다. 이를 현대의 소프트웨어 설계와 정보 시스템 테스트에 통합하여 메타버스와 같은 풍부한 디지털 경험 안에서 견고한 결과를 제공할 수 있다.

앞으로는 '시프트 라이트'와 같은 분야를 활용하여 현실 세계의 사용자 여정과 디지털 경험에서 '빠르게 실패하고 빠르게 학습하는' 피드백 루프를 도입하고, 머신러닝의 도움을 함께 사용함으로써 설계하고 테스트하는 소프트웨어와 시스템의 문제를 예측하는 데 도움을 받을 수 있다.

또한 시스템이나 소프트웨어의 디지털 트윈을 실시간으로 기반하여 설계를 빠르게 변경하고 검증할 수 있게 될 것이다. 현실 세계와 기술 사이의 경계를 흐리는 메타버스와 같은 디지털 경험 역시

원문 PDF 165 페이지

머신러닝과 인공지능이 가능하게 한 인지 학습 기능을 통해 예측된 결과를 기반으로 테스트할 수 있다.

요약

새로운 소프트웨어 시스템이나 해결책을 도입할 때 일반적으로 발생하는 막대한 사전 개발 작업을 줄이기 위해, 앞으로는 개념의 더 빠른 프로토타입과 이러한 아이디어의 진화를 선택할 수 있다. 현실 세계에서 일어나고 있는 일로부터 학습하고 시프트 라이트를 통해 이 피드백을 적용하는 것이 이 ‘혁명보다 진화(Evolution, Over Revolution)’를 이끌 것이다.

원문 PDF 166 페이지

참고문헌

[서지정보의 저자명, 문헌 제목, 학술지/학회명, URL 은 인용 정확성을 위해 원문 표기를 유지함]

ACLU (2019) ‘Facebook agrees to sweeping reforms to curb discriminatory ad targeting practices’. Available from: <https://www.aclu.org/press-releases/facebook-agrees-sweeping-reforms-curb-discriminatory-ad-targeting-practices>

Al Feel, H. T. and Khafagy, M. H. (2011) ‘OCSS: Ontology cloud storage system’. In Proceedings of the IEEE International Symposium on Network Cloud Computing and Applications. 9–13.

Ali, M., Sapiezynski, P., Bogen, M., Korolova, A., Mislove, A. and Rieke, A. (2019) ‘Discrimination through optimization: How Facebook’s ad delivery can lead to biased outcomes’. In Proceedings of the ACM on Human- Computer Interaction. 3. 1–30.

Alqahtani, S. S., Eghan, E. E. and Rilling, J. (2017) ‘Recovering semantic traceability links between APIs and security vulnerabilities: An ontological modeling approach’. In Proceedings of the IEEE International Conference on Software Testing, Verification and Validation. 80–91.

Alzantot, M., Sharma, Y., Elgohary, A., Ho, B.-J., Srivastava, M. and Chang, K.-W. (2018) ‘Generating natural language adversarial examples’. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. 2890–2896.

Alsudais, A. (2021) ‘Incorrect data in the widely used Inside Airbnb dataset’. Decision Support Systems, 141. 113453.

Angwin, J., Mattu, S. and Larson, J. (2015) ‘The tiger mom tax: Asians are nearly twice as likely to get a higher price from Princeton Review’. Available from: <https://www.propublica.org/article/asians-nearly-twice-as-likely-to-get-higher-price-from-princeton-review>

원문 PDF 167 페이지

princeton-review

- Angwin, J., Larson, J., Mattu, S. and Kirchner, L. (2016), 'Machine bias. There is software that is used across the country to predict future criminals. And it is biased against blacks'. Available from: <https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing>
- Arpirez, J. C., Corcho, O., Fernandez-Lopez, M. and Gomez-Perez, A. (2001) 'WebODE: A scalable ontological engineering workbench'. In Proceedings of the ACM International Conference on Knowledge Capture (K-CAP). 6–13.
- Arya, V., Bellamy, R. K. E., Chen, P.-Y., Dhurandhar, A., Hind, M., Hoffman, S. C., Houde, S., Liao, Q. V., Luss, R., Mojsilovic, A., Mourad, S., Pedemonte, P., Raghavendra, R., Richards, J., Sattigeri, P., Shanmugam, K., Singh, M., Varshney, K. R., Wei, D., Zhang, Y. (2019) 'One explanation does not fit all: A toolkit and taxonomy of AI explainability techniques'. Available from: <http://arxiv.org/abs/1909.03012>
- Avdeenko, T. V. and Pustovalova, N. V. (2016) 'The ontology-based approach to support the requirements engineering process'. In Proceedings of the IEEE International Scientific-Technical Conference on Actual Problems of Electronics Instrument Engineering. 513–518.
- Baader, F., Calvanese, D., McGuinness, D., Nardi, D. and Patel-Schneider, P. (eds) (2003) The Description Logic Handbook: Theory, Implementation, and Applications. Cambridge University Press.
- Barbosa, E. F., Nakagawa, E. Y. and Maldonado, J. C. (2006) 'Towards the establishment of an ontology of software testing'. In Proceedings of IEEE International Conference on Software Engineering and Knowledge Engineering (SEKE). 522–525.
- Basic Linux Ontology (BLO) (2021). Available from: <http://www.wis.win.tue.nl/~swale/blo>
- Bayat, B., Bermejo-Alonso, J., Carbonera, J. L., Facchinetti, T., Fiorini, S. R., Goncalves, P., Jorge, V., Habib, M., Khamis, A., Melo, K., Nguyen, B., Olszewska, J. I., Paull, L., Prestes, E., Ragavan, S. V., Saeedi, S., Sanz, R., Seto, M., Spencer, B., Trentini, M., Vosughi, A. and Li, H. (2016) 'Requirements for building an ontology for autonomous robots'. Industrial Robot: An International Journal, 43 (5). 469–480.
- BCS (2020a) 'The public don't trust computer algorithms to make decisions about them, survey finds'. Available from: <https://www.bcs.org/more/about-us/press-office/press-releases/the-public-don-t-trust-computer-algorithms-to-make-decisions-about-them-survey-finds/>

BCS (2020b) 'The exam question: How do we make algorithms do the right thing?' Available from: <https://www.bcs.org/media/6135/algorithms-report-2020.pdf>

Beizer, B. (1990) Software Testing Techniques. Van Nostrand Reinhold.

Bermejo-Alonso, J., Chibani, A., Goncalves, P., Li, H., Jordan, S., Olivares, A., Olszewska, J. I., Prestes, E.,

원문 PDF 168 페이지

Rama Fiorini, S. and Sanz, R. (2018) 'Collaboratively working towards ontology-based standards for robotics and automation'. In IEEE International Conference on Intelligent Robots and Systems (IROS). 79.

Berrani, S., Yachir, A., Djemaa, B. and Aissani, M. (2018) 'Extended multi-agent system based service composition in the Internet of Things'. In Proceedings of the IEEE International Conference on Pattern Analysis and Intelligent Systems. 1–8.

Bertrand, M. and Mullainathan, S. (2004) 'Are Emily and Greg more employable than Lakisha and Jamal? A field experiment on labor market discrimination'. The American Economic Review, 94. 991–1013.

Bettenburg, N., Premraj, R., Zimmermann, T. and Kim, S. (2008) 'Duplicate bug reports considered harmful ... really?' In Proceedings of the IEEE International Conference on Software Maintenance (ICSM). 337–345. DOI: 10.1109/ICSM.2008.4658082

Bezerra, D., Costa, A. and Okada, K. (2009) 'SwTOI (Software Test Ontology Integrated) and its application in Linux test'. In Proceedings of IEEE International Workshop on Ontology, Conceptualization and Epistemology for Information Systems, Software Engineering and Service Science. 25–36.

Bickel, P., Hammel, E. and O'Connell, J. (1975) 'Sex bias in graduate admissions: Data from Berkeley'. Science, 187 (4175). 398–404.

Binas, J., Neil, D., Liu, S.-C. and Delbruck, T. (2017) 'DDD17: End-to-end DAVIS driving dataset'. Available from: <https://arxiv.org/abs/1711.01458>

Black, R. (2015) 'Dimensions of test coverage'. RBCS. Available from: <https://rbcs-us.com/site/assets/files/1222/dimensions-of-test-coverage.pdf>

Black, R., Van Veenendaal, E. and Graham, D. (2012) Foundations of Software Testing – ISTQB Certification. CENGAGE Learning.

Boardman, J. and Sauser, B. (2006) ‘System of Systems: The meaning of’. In Proceedings of the IEEE/SMC International Conference on System of Systems Engineering. 118–123.

Bonferroni, C. E. (1936) ‘Teoria statistica delle classi e calcolo delle probabilità’. Pubblicazioni del R. Istituto Superiore di Scienze Economiche e Commerciali di Firenze, 8. 3–62.

Bourque, P. and Fairley, R. E. (2014) SWEBOK: Guide to Software Engineering Body of Knowledge, 3rd ed., IEEE.

Breck, E., Cai, S., Nielsen, E., Salib, M. and Sculley, D. (2017) ‘The ML test score: A rubric for ML production readiness and technical debt reduction’. In Proceedings of the IEEE International Conference on Big Data (Big Data). 1123–1132. DOI: 10.1109/BigData.2017.8258038

Caliskan, A., Bryson, J. and Narayanan, A. (2017) ‘Semantics derived automatically from language corpora contain human-like biases’. Science, 356. 183–186.

원문 PDF 169 페이지

Campos, H., Acacio, C., Braga, R., Araujo, M. A. P., David, J. M. N. and Campos, F. (2017) ‘Regression tests provenance data in the continuous software engineering context’. In Proceedings of the IEEE Brazilian Symposium on Systematic and Automated Software Testing. 1–6.

Chari, S., Seneviratne, O., Gruen, D. M., Foreman, M. A., Das, A. K. and McGuinness, D. L. (2020) ‘Explanation ontology: A model of explanations for user-centered AI’. In Proceedings of the International Semantic Web Conference (ISWC). 228–243.

Chaudhuri, A., Smith, A. L., Gardner, A., Gu, L., Salem, M. B. and Lévesque, M. (2018) ‘Regulatory frameworks relating to data privacy and algorithmic decision making in the context of emerging standards on algorithmic bias’. In Thirty-fifth Conference on Neural Information Processing Systems. 6.

Chen, P. and Xi, A. (2020) ‘Research on industrial software testing knowledge database based on ontology’. In Proceedings of the IEEE International Conference on Dependable Systems and Their Applications. 425–429.

Clarke, E. M., Klieber, W., Nováček, M. and Zuliani, P. (2012) ‘Model checking and the state explosion problem’. In B. Meyer and M. Nordio (eds). Tools for Practical Software Verification. LASER 2011. Lecture Notes in Computer Science, vol. 7682. Springer. DOI: 10.1007/978-3-642-35746-6_1

- Clarke, G. M., Anderson, C., Pettersson, F., Cardon, L., Morris, A. and Zondervan, K. (2011) 'Basic statistical analysis in genetic case-control studies'. *Nature Protocols*, 6 (2). 121–133.
- Clean Code Developer (n.d.) 'Clean code initiative'. Available from: <https://clean-code-developer.de>
- CMS Collaboration (2012) 'Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC'. *Physics Letters B*, 716 (1). 30–61.
- Cohn, M. (2009) *Succeeding with Agile*. Addison-Wesley Professional.
- Collins, K. (2017) 'Google collects Android users' locations even when location services are disabled'. *Quartz*. Available from: <https://qz.com/1131515/google-collects-android-users-locations-even-when-location-services-are-disabled/>
- Creel, K. A. (2020) 'Transparency in complex computational systems'. *Philosophy of Science*, 87 (4). 709–729.
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C. and Bontempi, G. (2015) 'Credit card fraud detection and concept-drift adaptation with delayed supervised information'. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*. 1–8. DOI: 10.1109/IJCNN.2015.7280527
- Da Silva, J. P. S., DallOglio, P., Coelho Da Silva Pinto, S. C., Bittencourt, I. I. and Sardi Mergen, S. L. (2015) 'OntoQAI: An ontology to support quality assurance inspections'. In *Proceedings of the IEEE Brazilian Symposium on Software Engineering*. 11–20.
- Davenport, J. (In press) Dataset for chapter 'Quality and Bias'. University of Bath Research Data Archive. DOI: 10.15125/BATH-01099
- 원문 PDF 170 페이지
- Davis, R., Shrobe, H. and Szolovits, P. (1993) 'What is a knowledge representation?' *AI Magazine*, 14 (1). 17–33.
- de Almeida Falbo, R., Cruz Natali, A. C., Gomes Mian, P., Bertollo, G. and Borges Ruy, F. (2003) 'ODE: Ontology-based software development environment'. *CACIC*. 1124–1135.
- Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE) (2021). Available from: www.loa.istc.cnr.it/dolce/overview.html
- Dietz, J. and Mulder, H. (2020) *Enterprise Ontology*. Springer.

Ding, L., Finin, T., Joshi, A., Pan, R., Cost, R. S., Peng, Y., Reddivari, P., Doshi, V. C. and Sachs, J. (2004) 'Swoogle: A semantic web search and metadata engine'. In Proceedings of the ACM Conference on Information and Knowledge Management. 652–659.

Dragoni, M., Donadello, I. and Eccher, C. (2020) 'Explainable AI meets persuasiveness: Translating reasoning results into behavioral change advice'. Artificial Intelligence in Medicine, 105. 1–17.

Dunn, O. J. (1961) 'Multiple comparisons among means'. Journal of the American Statistical Association, 56 (293). 52–64.

Edelman, B. G., Luca, M. and Svirsky, D. (2016) 'Racial discrimination in the sharing economy: Evidence from a field experiment.' Available from: <https://hbswk.hbs.edu/item/racial-discrimination-in-the-sharing-economy-evidence-from-a-field-experiment>

European Court of Justice (2011) Association Belge des Consommateurs Test-Achats ASBL and Others v Conseil des ministres. Available from: <http://curia.europa.eu/juris/liste.jsf?td=ALL&language=en&jur=C,T,F&parties=test%20achats>

European Parliament (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data. Available from: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

European Parliament (2021) Proposal for a Regulation laying down harmonised rules on artificial intelligence. Available from: <https://digital-strategy.ec.europa.eu/en/library/proposal-regulation-laying-down-harmonised-rules-artificial-intelligence>

Feldmann, S., Roeschm S., Legat, C. and Vogel-Heuser, B. (2014) 'Keeping requirements and test cases consistent: Towards an ontology-based approach'. In Proceedings of the IEEE International Conference on Industrial Informatics (INDIN). 726–732.

Fernandez, M., Gomez-Perez, A. and Juristo, N. (1997) 'Methontology: From ontological art towards ontological engineering'. In Proceedings of the AAAI Spring Symposium Series. 33–40.

원문 PDF 171 페이지

Ferreira de Souza, E., de Almeida Falbo, R. and Vijaykumar, N. L. (2013) 'Ontologies in software testing: A systematic literature review'. In Proceedings of the Seminar on Ontology Research in Brazil. 71–82.

Ferreira de Souza, E., de Almeida Falbo, R. and Vijaykumar, N. L. (2017) 'ROoST: Reference ontology on software testing'. Applied Ontology, 12 (1). 59–90.

- Fiorini, S., Bermejo-Alonso, J., Goncalves, P., Pignaton de Freitas, E., Olivares, A., Olszewska, J. I., Prestes, E., Schlenoff, C., Ragavan, S. V., Redfield, S., Spencer, B., and Li, H. (2017) 'A suite of ontologies for robotics and automation'. *IEEE Robotics and Automation Magazine*, 24 (1). 8–11.
- Fraser, G. (n.d.). Available from: <https://www.evosuite.org/>
- Freitas, A. and Vieira, R. (2014) 'An ontology for guiding performance testing'. In *Proceedings of the IEEE/WIC/ACM International Joint Conferences on Web Intelligence (WI) and Intelligent Agent Technologies (IAT)*. 400–407.
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H. and Crawford, K. (2021) 'Datasheets for datasets'. *Communications of the ACM*, 64 (12). 86–92.
- Gilvoich, T. and Griffin, D. (2010) 'Judgement and decision-making'. In S. T. Fiske, D. T. Gilbert and Lindzey, G. (eds). *Handbook of Social Psychology*, vol. 1. John Wiley & Sons. 542–589.
- Gladwell, M. (2021) 'I love you Waymo'. Revisionist History podcast. Available from: http://podcasts.pushkin.fm/revisionist-history-waymo?sid=mg&c=2_SiuQY8fGilo_E3KAHK-g&h=b8c24575b15cc6ec7
- Glimm, B., Horrocks, I., Motik, B., Stoilos, G. and Wang, Z. (2014) 'HermiT: An OWL 2 reasoner'. *Journal of Automated Reasoning*, 53 (3). 245–269.
- Gomez-Perez, A., Fernandez-Lopez, M. and Corcho, O. (2004) *Ontological Engineering*. Springer-Verlag.
- Google (2021) Machine learning glossary. Available from: <https://developers.google.com/machine-learning/glossary/#r>
- Gordon, P. M. K., Barker, K. and Sensen, C. W. (2010) 'Programming-by-example meets the semantic web: Using ontologies and web services to close the semantic gap'. In *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing*. 133–140.
- Griffor, E. R., Greer, C., Wollman, D. A. and Burns, M. J. (2017) *Framework for cyber-physical systems: Volume 2, Working group reports*. NIST SP 1500-202. National Institute of Standards and Technology. DOI: 10.6028/NIST.SP.1500-202
- Gross, F., Fraser, G. and Zeller, A. (2012) 'Search-based system testing: High coverage, no false alarms'. In *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA)*. 67–77. DOI: 10.1145/2338965.2336762

- Grosse, K., Papernot, N., Manoharan, P., Backes, M. and McDaniel, P. (2017) 'Adversarial examples for malware detection'. In Proceedings of the European Symposium on Research in Computer Security. Lecture Notes in Computer Science. 62–79. DOI: 10.1007/978-3-319-66399-9_4
- Gruber, T. (1995) 'Towards principles for the design of ontologies used for knowledge sharing'. International Journal of Human-Computer Studies, 43 (5–6). 907–928.
- Gruninger, M. and Fox, M. (1995) 'Methodologies for the design and evaluation of ontologies'. In Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI) Workshop. 1–10.
- Grzymek, V. and Puntschuh, M. (2019) 'What Europe knows and thinks about algorithms: Results of a representative survey'. Bertelsmann Stiftung. Available from: <https://www.bertelsmann-stiftung.de/fileadmin/files/BSt/Publikationen/GrauePublikationen/WhatEuropeKnowsAndThinkAboutAlgorithm.pdf>
- The Guardian (2004) 'Why are men worse drivers than women?' Available from: <https://www.theguardian.com/science/2004/may/13/thisweekssciencequestions1>
- Guizzardi, G. (2005) 'Ontological foundations for structural conceptual models'. Telematica Instituut Fundamental Research Series, No. 015 (TI/FRS/015). 1–416.
- Hao, K. (2019) 'There's an easy way to make lending fairer for women. Trouble is, it's illegal'. Available from: <https://www.technologyreview.com/2019/11/15/131935/theres-an-easy-way-to-make-lending-fairer-for-women-trouble-is-its-illegal/>
- Herbold, S. and Haar, T. (2022) 'Smoke testing for machine learning: Simple tests to discover severe defects'. Empirical Software Engineering, 27. 45. DOI: 10.1007/s10664-021-10073-7
- High-Level Expert Group on Artificial Intelligence (2020) 'Ethics guidelines for trustworthy AI'. Available from: <https://www.aepd.es/sites/default/files/2019-12/ai-ethics-guidelines.pdf>
- Hohenecker, P. and Lukasiewicz, T. (2020) 'Ontology reasoning with deep neural networks'. Journal of Artificial Intelligence Research, 68. 503–540.
- Holzinger, A., Kieseberg, P., Weippl, E. and Tjoa, A. M. (2018) 'Current advances, trends and challenges of machine learning and knowledge extraction: From machine learning to explainable AI'. In Proceedings of the International Cross-Domain Conference for Machine Learning and Knowledge Extraction. 1–8.

Horridge, M. (2011) 'A practical guide to building OWL ontologies using Protégé 4 and CO-ODE tools'. 1.3 ed. Available from: <http://owl.cs.manchester.ac.uk/research/co-ode/>

Horridge, M. and Bechhofer, S. (2011) 'The OWL API: A Java API for OWL ontologies'. *Semantic Web*, 2 (1). 11– 21.

원문 PDF 173 페이지

Huh, M., Agrawal, P. and Efros, A. A. (2016) 'What makes ImageNet good for transfer learning?' Available from: <http://arxiv.org/abs/1608.08614>

Huo, Q., Zhu, H. and Greenwood, S. (2003) 'A multi-agent software engineering environment for testing web- based applications'. In *Proceedings of IEEE International Computer Software and Applications Conference*. 210– 215.

IBM Cloud Education (2020) 'AIOps'. Available from: <https://www.ibm.com/uk-en/cloud/learn/aiops>

IEEE (2008) 'IEEE 829-2008 – IEEE Standard for software and system test documentation'. Available from: <https://standards.ieee.org/standard/829-2008.html>

IEEE (2015) 'IEEE 1872-2015 – IEEE Standard ontologies for robotics and automation'. Available from: <https://standards.ieee.org/standard/1872-2015.html>

IEEE (2020) 'IEEE 7010-2020 – IEEE Recommended practice for assessing the impact of autonomous and intelligent systems on human well-being'. Available from: <https://standards.ieee.org/standard/7010-2020.html>

Imana, B., Korolova, A. and Heidemann, J. (2021) 'Auditing for discrimination in algorithms delivering job ads'. In *Proceedings of the ACM Web Conference (WWW)*. 3767–3778. DOI: 10.1145/3442381.3450077

Institute and Faculty of Actuaries (2018) 'Continuous mortality investigation'. CMI 2017 Briefing note. Available from: <https://www.actuaries.org.uk/learn-and-develop/continuous-mortality-investigation/cmiworking-papers/mortality-projections/cmi-working-paper-105>

International Software Testing Qualifications Board (ISTQB) (2021). Available from: <https://www.istqb.org>

Ishikawa, F. and Yoshioka, N. (2019) 'How do engineers perceive difficulties in engineering of machine-learning systems? Questionnaire survey'. In *Proceedings of the IEEE/ACM Joint International Workshop on Conducting Empirical Studies in Industry (CESI) and the International*

Workshop on Software Engineering Research and Industrial Practice (SER&IP). 2-9. DOI:
10.1109/CESSER-IP.2019.00009

ISO (2005) 'ISO 9000:2005 Quality management systems – Fundamentals and vocabulary'. Available from: [https://www.iso.org/obp/ui/#iso:std: iso:9000:ed-3:v1:en](https://www.iso.org/obp/ui/#iso:std:iso:9000:ed-3:v1:en)

ISO/IEC (2011) 'ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE)'. Available from: <https://www.iso.org/standard/35733.html>

ISO/IEC (2017) 'ISO/IEC/IEEE 12207:2017 Systems and software engineering – Software life cycle processes'. Available from: <https://www.iso.org/standard/63712.html>

ISO/IEC (2020) 'ISO/IEC TR 29119-11:2020 Software and systems engineering – Software testing – Part 11: Guidelines on the testing of AI-based systems'. Available from:
<https://www.iso.org/standard/79016.html>

원문 PDF 174 페이지

ISO/IEC (2021a) 'ISO/IEC DIS 22989 – Information technology – Artificial intelligence – Artificial intelligence concepts and terminology'. Available from: <https://www.iso.org/standard/74296.html>

ISO/IEC (2021b) 'ISO/IEC TR 24027:2021 Information technology – Artificial intelligence (AI) – Bias in AI systems and AI aided decision making'. Available from: <https://www.iso.org/standard/77607.html>

ISO/IEC (2021c) 'ISO/IEC CD 25059: Software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Quality model for AI systems'. Available from: [https://www.iso.org/cms/render/live/en/sites/ isoorg/contents/data/standard/08/06/80655.html](https://www.iso.org/cms/render/live/en/sites/isoorg/contents/data/standard/08/06/80655.html)

ISO/IEC (2022) 'ISO/IEC DTR 24368 Information technology – Artificial intelligence – Overview of ethical and societal concerns'. Available from: <https://www.iso.org/standard/78507.html>

ISO/IEC/IEEE (2013a) 'ISO/IEC/IEEE 29119-1:2013 Software and systems engineering – Software testing – Part 1: Concepts and definitions'. Available from: <https://www.iso.org/standard/45142.html>

ISO/IEC/IEEE (2013b) 'ISO/IEC/IEEE 29119-2:2013 Software and systems engineering – Software testing – Part 2: Test processes'. Available from: <https://www.iso.org/standard/56736.html>

ISO/IEC/IEEE (2013c) 'ISO/IEC/IEEE 29119-3:2013 Software and systems engineering – Software testing – Part 3: Test documentation'. Available from: <https://www.iso.org/standard/56737.html>

ISO/IEC/IEEE (2015) 'ISO/IEC/IEEE 29119-4:2015 Software and systems engineering – Software testing – Part 4: Test techniques'. Available from: <https://www.iso.org/standard/60245.html>

ISO/IEC/IEEE (2017) 'ISO/IEC/IEEE 24765:2017 Systems and software engineering – Vocabulary'. Available from: <https://www.iso.org/standard/71952.html>

ISTQB (2021) 'Glossary'. Available from: <https://glossary.istqb.org/en/search/>

Jia, Y., Mao, K. and Harmon, M. (2018) 'Finding and fixing software bugs automatically with SapFix and Sapienz'. Available from: <https://engineering.fb.com/2018/09/13/developer-tools/finding-and-fixing-software-bugs-automatically-with-sapfix-and-sapienz/>

Juergens, E., Hummel, B., Deissenboeck, F., Feilkas, M., Schlögel, C. and Wübbeke, A. (2011) 'Regression test selection of manual system tests in practice'. In Proceedings of the IEEE European Conference on Software Maintenance and Reengineering (CSMR). 309–312. DOI: 10.1109/CSMR.2011.44

Katsumi, M. and Gruninger, M. (2016) 'What is ontology reuse?' In Proceedings of the International Conference on Formal Ontology in Information Systems (FOIS). 9–22.

Khan, J. H., Magnetti, S., Davis, E. and Zhang, J. (2000) 'Late outcomes of open heart surgery in patients 70 years old or later'. The Annals of Thoracic Surgery, 69 (1). 165–170. Available from: <https://www.annalsthoracicsurgery.org/article/>

원문 PDF 175 페이지

S0003-4975(99)01185-6/fulltext#relatedArticles

Kinsbruner, E. (2020) Accelerating Software Quality: Machine Learning and Artificial Intelligence in the Age of DevOps. Perforce.

Kloumann, I. and Tanner, J. (2021) 'How we're using Fairness Flow to help build AI that works better for everyone'. Facebook AI. Available from: <https://ai.facebook.com/blog/how-were-using-fairness-flow-to-help-build-ai-that-works-better-for-everyone/>

Klueck, F., Li, Y., Nica, M., Tao, J. and Wotawa, F. (2018) 'Using ontologies for test suites generation for automated and autonomous driving functions'. In Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops. 1–6.

Koch, C. (2016) 'How the computer beat the Go Master'. Scientific American. Available from: <https://www.scientificamerican.com/article/how-the-computer-beat-the-go-master/>

Kogut, P., Cranefield, S., Hart, L., Dutra, M., Baclawski, K., Kokar, M. and Smith, J. (2002) 'UML for ontology development'. The Knowledge Engineering Review, 17 (1). 61–64.

Koh, L., Orzes, G. and Jia, F. J. (2019) 'The fourth industrial revolution (Industry 4.0): Technologies disruption on operations and supply chain management'. *International Journal of Operations and Production Management*, 39 (6–8). 817–828.

Krisher (2019) 'Official: Safety lacking before Uber self-driving car crash'. Techxplore. Available from: <https://techxplore.com/news/2019-11-safety-lacking-uber-self-driving-car.html>

Kuleshov, A. (2018) 'Formalizing AI system parameters in standardization of AI'. In *Proceedings of the IEEE International Conference on Artificial Intelligence Applications and Innovations (IC-AIAI)*. 51–54.

Kurakin, A., Goodfellow, I. and Bengio, S. (2017) 'Adversarial machine learning at scale'. Available from: <http://arxiv.org/abs/1611.01236>

Kuwajima, H. and Ishikawa, F. (2019) 'Adapting SQuaRE for quality assessment of artificial intelligence systems'. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 13–18. DOI: 10.1109/ISSREW.2019.00035

Landhausser, M. and Genaid, A. (2012) 'Connecting user stories and code for test development'. In *Proceedings of the IEEE International Workshop on Recommendation Systems for Software Engineering*. 33–37.

Larson, J., Mattu, S., Kirchner, L. and Angwin, J. (2016) 'How we analyzed the COMPAS recidivism algorithm'. ProPublica. Available from: <https://www.propublica.org/article/how-we-analyzed-the-compas-recidivism-algorithm>

원문 PDF 176 페이지

Lenarduzzi, V., Lomio, F., Moreschini, S., Taibi, D. and Tamburri, D. A. (2021). 'Software quality for AI: Where we are now?' In D. Winkler, S. Biffl, D. Mendez, M. Wimmer and J. Bergsmann (eds). *Software Quality: Future Perspectives on Software Engineering Quality*, vol. 404. Springer International Publishing. 43–53. DOI: 10.1007/978-3-030-65854-0_4

Li, H., Chen, F., Yang, H., Guo, H., Chu, W. C.-C. and Yang, Y. (2009) 'An ontology-based approach for GUI testing'. In *Proceedings of the IEEE International Computer Software and Applications Conference*. 632–633.

Lim, X. and Zhang, W. (2012) 'Ontology-based testing platform for reusing'. In *Proceedings of the IEEE International Conference on Internet Computing for Science and Engineering*. 86–89.

Linux Test Project (LTP) (2021). Available from: <https://github.com/linux-test-project/ltp>

Looker, N., Gwynne, B., Xu, J. and Munro, M. (2005) 'An ontology-based approach for determining the dependability of service-oriented architectures'. In Proceedings of the Annual IEEE International Workshop on Object-Oriented Real-Time Dependable Systems. 1–8.

Lu, D. (2020) 'Uber and Lyft pricing algorithms charge more in non-white areas'. New Scientist. Available from: <https://www.newscientist.com/article/2246202-uber-and-lyft-pricing-algorithms-charge-more-in-non-white-areas/>

Ma, M., Wang, P. and Chu, C.-H. (2014) 'Ontology-based semantic modeling and evaluation for Internet of Things applications'. In Proceedings of the IEEE International Conference on Green Computing and Communications. 24–30.

Maedche, A. and Staab, S. (2004) 'Ontology learning'. In Handbook on Ontologies. Springer. 173–190.

Manifesto for Software Craftsmanship (n.d.). Available from: <https://manifesto.softwarecraftsmanship.org>

Martin, R. C. (2008) Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

McDonald, S. (2015) 'Indirect gender discrimination and the “Test-Achats Ruling”: An Examination of the UK motor insurance market'. Presentation to the Royal Economic Society. Available from: https://editorialexpress.com/cgi-bin/conference/download.cgi?db_name=RES2015&paper_id=791

Merriam-Webster (2021) 'Non-fungible token'. Available from: <https://www.merriam-webster.com/dictionary/non-fungible%20token>

Metz, C. (2019) 'We teach A.I. systems everything, including our biases'. New York Times. Available from: <https://www.nytimes.com/2019/11/11/technology/artificial-intelligence-bias.html>

MMC (2019) 'The State of AI: Divergence'. Available from: <https://www.stateofai2019.com/>

원문 PDF 177 페이지

Murtazinam, M. S. and Avdeenko, T. V. (2018) 'The ontology-driven approach to support the requirements engineering process in Scrum framework'. CEUR-WS 2212. 287–295.

Nasser, V. H., Du, W. and MacIsaac, D. (2010) 'An ontology-based software test generation framework'. In Proceedings of the IEEE International Conference on Software Engineering and Knowledge Engineering (SEKE). 192–197.

Newberry, C. (2019) 'The Facebook Pixel: What it is and how to use it'. Available from: <https://blog.hootsuite.com/facebook-pixel/>

- Nie, C. and Leung, H. (2011) 'A survey of combinatorial testing'. *ACM Computing Surveys*, 43 (2). 1–29. DOI: 10.1145/1883612.1883618
- Nor, M. Z. M., Abdullah, R., Selamat, M. H. and Murad, M. A. A. (2012) 'An agent-based knowledge management system for collaborative software maintenance environment design and evaluation'. In *Proceedings of the IEEE International Conference on Information Retrieval and Knowledge Management*. 115–120.
- Northcutt, C. G., Athalye, A. and Mueller, J. (2021) 'Pervasive label errors in test sets destabilize machine learning benchmarks'. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1 Pre-Proceedings (NeurIPS Datasets and Benchmarks)*. 1–13.
- Noy, N. and McGuinness, L. (2001) 'Ontology development 101: A guide to creating your first ontology'. Available from: http://protege.stanford.edu/publications/ontology_development/ontology101.pdf
- Olivares-Alarcos, A., Bessler, D., Khamis, A., Goncalves, P., Habib, M., Bermejo-Alonso, J., Barreto, M., Diab, M., Rosell, J., Quintas, J., Olszewska, J. I., Nakawala, H., Pignaton de Freitas, E., Gyrard, A., Borgo, S., Alenya, G., Beetz, M. and Li, H. (2019) 'A review and comparison of ontology-based approaches to robot autonomy'. *The Knowledge Engineering Review*, 34. 1–38.
- Olszewska, J. I. (2011) 'Spatio-temporal visual ontology'. In *EPSRC/BMVA Workshop on Vision and Language (VL)*. 1–2.
- Olszewska, J. I. (2015) 'UML activity diagrams for OWL ontology building'. In *Proceedings of the International Conference on Knowledge Engineering and Ontology Development*, 2. 370–374.
- Olszewska, J. I. (2018a) 'Ontologies for vision agents'. In *IEEE International Conference on Intelligent Robots and Systems (IROS)*.
- Olszewska, J. I. (2018b) 'Ontologies for intelligent vision systems'. In *International Conference on Knowledge Engineering and Ontology Development (KEOD)*. Available from: <http://www.keod.ic3k.org/Tutorials.aspx?y=2018>
- Olszewska, J. I. (2018c) 'Visual ontologies for intelligent robotics'. In *BMVA Symposium on Enabling Human- Level Understanding in Robots*.
- 원문 PDF 178 페이지
- Olszewska, J. I. (2019a) 'Designing transparent and autonomous intelligent vision systems'. In *Proceedings of the International Conference on Agents and Artificial Intelligence*. 850–856.

- Olszewska, J. I. (2019b) 'D7-R4: Software development life-cycle for intelligent vision systems'. In Proceedings of the International Conference on Knowledge Engineering and Ontology Development. 435–441.
- Olszewska, J. I. (2020) 'AI-T: Software testing ontology for AI-based systems'. In Proceedings of the International Conference on Knowledge Engineering and Ontology Development. 291–298.
- Olszewska, J. I. and Allison, I. K. (2018). 'ODYSSEY: Software development lifecycle ontology'. In Proceedings of the International Conference on Knowledge Engineering and Ontology Development. 303–311.
- Olszewska, J. I. and McCluskey, T. L. (2011) 'Ontology-coupled active contours for dynamic video scene understanding'. In Proceedings of the IEEE International Conference on Intelligent Engineering Systems. 369– 374.
- Olszewska, J. I., Houghtaling, M., Goncalves, P., Fabiano, N., Haidegger, T., Carbonera, J. L., Patterson, W. R., Ragavan, S. V., Fiorini, S. and Prestes, E. (2020) 'Robotic standard development life cycle in action'. Journal of Intelligent & Robotic Systems, 98 (1). 119–131.
- Olszewska, J. I., Houghtaling, M., Goncalves, P., Haidegger, T., Fabiano, N., Carbonera, J. L., Fiorini, S. and Prestes, E. (2018) 'Robotic ontological standard development life cycle'. In IEEE International Conference on Robotics and Automation (ICRA). 1–6.
- Olszewska, J. I., Simpson, R. M. and McCluskey, T. L. (2014) 'Dynamic OWL ontology design based on UML and BPMN'. In Proceedings of the International Conference on Knowledge Engineering and Ontology Development. 436–444.
- Pandey, A. and Caliskan, A. (2020) 'Iterative effect-size bias in ridehailing: Measuring social bias in dynamic pricing of 100 million rides'. Available from: <https://arxiv.org/abs/2006.04599>
- Pandey, C. and Caliskan, A. (2021) 'Disparate impact of artificial intelligence bias in ridehailing economy's price discrimination algorithms'. In Proceedings of the AAAI/ACM Conference on Artificial Intelligence, Ethics, and Society. 822–833.
- PathCheck Foundation (n.d.). Available from: <https://www.pathcheck.org/>
- Paydar, S. and Kahani, M. (2010) 'Ontology-based web application testing'. In Proceedings of IEEE Conference on Novel Algorithms and Techniques in Telecommunications and Networking. 23–27.

Perry, T. (2019) ‘San Diego’s connected street lights learn to spot bicycles’. Available from: <https://spectrum.ieee.org/view-from-the-valley/computing/software/san-diegos-streetlights-are-now-counting-bicycles>

Petrucci, G., Ghidini, C. and Rospocher, M. (2016) ‘Ontology learning in the deep’. In Proceedings of the

원문 PDF 179 페이지

European Knowledge Acquisition Workshop. 480–495.

Pignaton de Freitas, E., Bermejo-Alonso, J., Khamis, A., Li, H. and Olszewska, J. I. (2020a) ‘Ontologies for cloud robotics’. The Knowledge Engineering Review, 35. 1–19.

Pignaton de Freitas, E., Olszewska, J. I., Carbonera, J., Fiorini, S. R., Khamis, A., Ragavan, S. V., Barreto, M., Prestes, E., Habib, M. K., Redfield, S., Chibani, A., Goncalves, P., Bermejo-Alonso, J., Sanz, R., Tosello, E., Olivares-Alarcos, A., Konzen, A. A., Quintas, J. and Li, H. (2020b) ‘Ontological concepts for information sharing in cloud robotics’. Journal of Ambient Intelligence and Humanized Computing. 1–14. DOI: 10.1007/s12652-020-02150-4

Pinto, H. S. (1999) ‘Towards ontology reuse’. In Proceedings of AAAI Workshop on Ontology Management. 67– 73.

PK, M. A., Sheriff, M. R. and Chatterjee, D. (2021) ‘Measure of quality of finite-dimensional linear systems: A frame-theoretic view’. Systems & Control Letters, 151. 104911. DOI: 10.1016/j.sysconle.2021.104911

Pop, C., Moldovan, D., Antal, M., Valea, D., Cioara, T., Anghel, I. and Salomie, I. (2014) ‘M2O: A library for using ontologies in software engineering’. In Proceedings of the IEEE International Conference on Intelligent Computer Communication and Processing. 69–75.

Portoraro, F. (2021) ‘Automated reasoning’. In The Stanford Encyclopedia of Philosophy (Fall 2021 edition). Edward N. Zalta (ed.). Available from: <https://plato.stanford.edu/archives/fall2021/entries/reasoning-automated/>

Principles of Chaos Engineering (2019). Available from: <https://principlesofchaos.org/>

Protégé tool website (2021). Available from: <http://protege.stanford.edu/>

Resource Description Framework (RDF) (2021). Available from: <https://www.w3.org/RDF/>

Ritchson, M. (2021) ‘Teamwork, AI, and containerization (with NASA’s Michael Ritchson)’. The QA Lead Podcast. Available from: TheQALead.com/Podcasts

Robal, T., Marenkov, J. and Kalja, A. (2017) 'Ontology design for automatic evaluation of web user interface usability'. In Proceedings of the IEEE Portland International Conference on Management of Engineering and Technology. 1–8.

Rocha Silva, T., Hak, J.-L. and Winckler, M. (2017) 'A behavior-based ontology for supporting automated assessment of interactive systems'. In Proceedings of the IEEE International Conference on Semantic Computing. 250–257.

Rothstein, R. (2017) The Color of Law. Liveright.

Sampath Kumar, V. R., Khamis, A., Fiorini, S. R., Carbonera, J. L., Olivares-Alarcos, A., Habib, M., Goncalves, P., Li, H. and Olszewska, J. I. (2019) 'Ontologies for Industry 4.0'. The Knowledge Engineering Review, 34. 1–14.

원문 PDF 180 페이지

Sapna, P. G. and Mohanty, H. (2011) 'An ontology based approach for test scenario management'. In Proceedings of the International Conference on Information Intelligence, Systems, Technology and Management. 91–100.

Seeliger, A., Pfaff, M. and Krcmar, H. (2019) 'Semantic web technologies for explainable machine learning models: A literature review'. In International Semantic Web Conference (ISWC). 30–45.

Serrano, J. (2022) 'Airbnb will hide guests' first names in Oregon until bookings are confirmed to fight discrimination'. Available from: <https://gizmodo.com/airbnb-will-hide-guests-first-names-in-oregon-until-bo-1848294121>

Shankar, S., Halpern, Y., Breck, E., Atwood, J., Wilson, J. and Sculley, D. (2017) 'No classification without representation: Assessing geodiversity issues in open data sets for the developing world'. Available from: <https://arxiv.org/abs/1711.08536>

Šidák, Z. (1968) 'On multivariate normal probabilities of rectangles: Their dependence on correlations'. Annals of Mathematical Statistics, 39. 1425–1434.

Simmons, C. B., Shiva, S. G. and Simmons, L. L. (2014) 'A qualitative analysis of an ontology based issue resolution system for cyber attack management'. In Proceedings of the IEEE International Conference on Cyber Technology in Automation, Control and Intelligent. 323–329.

Simpson, E. H. (1951) 'The interpretation of interaction in contingency tables'. Journal of the Royal Statistical Society. Series B (Methodological), 13 (2). 238–241.

- Siqueira Bueno, P. M., de Franco Rosa, F., Jino, M. and Bonacin, R. (2018) 'A security testing process supported by an ontology environment: A conceptual proposal'. In Proceedings of the IEEE/ACS International Conference on Computer Systems and Applications. 1–8.
- Slee, D., Cain, S., Vichare, P. and Olszewska, J. I. (2021) 'Smart lifts: An ontological approach'. In Proceedings of the International Conference on Knowledge Engineering and Ontology Development. 210–219.
- Smith, A. (2018) 'Public attitudes toward computer algorithms'. Pew Research Center. Available from: <https://www.pewresearch.org/internet/2018/11/16/public-attitudes-toward-computer-algorithms/>
- Spocchia, G. and Morris, H. (2020) 'Revealed: The “daylight robbery” of school holiday price increases'. The Telegraph. Available from: <https://www.telegraph.co.uk/travel/news/half-term-holiday-prices/>
- Staab, S. and Maedche, A. (2000) 'Ontology engineering beyond the modeling of concepts and relations'. In Proceedings of the European Conference on Artificial Intelligence (ECAI) Workshop. 1–6.
- Steed, R. and Caliskan, A. (2021) 'Image representations learned with unsupervised pre-training contain human-like biases'. In ACM Conference on Fairness Accountability and Transparency (FAccT '21). 701–713. DOI: 10.1145/3442188.3445932
- 원문 PDF 181 페이지
- Stokel-Walker, C. (2021) 'Recruiters less likely to contact ethnic minority groups on Swiss site'. New Scientist. Available from: <https://www.newscientist.com/article/2265372-recruiters-less-likely-to-contact-ethnic-minority-groups-on-swiss-site/>
- Sugawara, K., Manabe, Y., Moulin, C. and Barthes, J.-P. (2011) 'Design assistant agents for supporting requirement specification definition in a distributed design team'. In Proceedings of the IEEE International Conference on Computer Supported Cooperative Work in Design. 329–334.
- Suggested Upper Merged Ontology (SUMO) (2021). Available from: www.ontologyportal.org/
- Thrun, S. (2018) 'Reputation in the age of artificial intelligence'. PRSA. Available from: https://apps.prsa.org/StrategiesTactics/Articles/view/12194/1155/Reputation_in_the_Age_of_Artificial_Intelligence#.YajjKJH7S2I

Tiffany, R. (2020) 'Fighting COVID-19 with digital twins'. YouTube, 9 April 2020. Available from: <https://youtu.be/XKINvqiTgxQ>

Tsarkov, D. (2014) 'Incremental and persistent reasoning in FaCT++'. In Proceedings of OWL Reasoner Evaluation Workshop. 16–22.

Tudorache, T. (2020) 'Ontology engineering: Current state, challenges, and future directions'. Semantic Web, 11 (1). 125–138.

Uschold, M. and Gruninger, M. (1996) 'Ontologies: Principles, methods and applications'. Knowledge Engineering Review, 11 (2). 93–136.

Uschold, M., Healy, M., Williamson, K., Clark, P. and Woods, S. (1998) 'Ontology reuse and application'. In Proceedings of the International Conference on Formal Ontology in Information Systems (FOIS). 179–192.

Vasilecas, O., Kalibatiene, D. and Guizzardi, G. (2009) 'Towards a formal method for the transformation of ontology axioms to application domain rules'. Information Technology and Control, 38 (4). 271–282.

Venkatadri, G., Andreou, A., Liu, Y., Mislove, A., Gummadi, K., Loiseau, P. and Goga, O. (2018) 'Privacy risks with Facebook's PII-based targeting: Auditing a data broker's advertising interface'. In Proceedings of the IEEE Symposium on Security and Privacy. 89–107.

Vincent, J. (2016) 'Twitter taught Microsoft's AI chatbot to be a racist asshole in less than a day'. The Verge. Available from: <https://www.theverge.com/2016/3/24/11297050/tay-microsoft-chatbot-racist>

Wang, X., Huang, N. and Wang, R. (2009) 'Mutation test based on OWL-S requirement model'. In Proceedings of the IEEE International Conference on Web Services. 1006–1007.

Wang, Y., Bai, X., Li, J. and Huang, R. (2007) 'Ontology-based test case generation for testing web services'. In

원문 PDF 182 페이지

Proceedings of the IEEE International Symposium on Autonomous Decentralized Systems. 43–50.

Ward, E. J. (2020) 'Abandoned NHS contact tracing app cost almost £12 million'. LBC. Available from: <https://www.lbc.co.uk/politics/abandoned-nhs-contact-tracing-app-cost-almost-12-million/>

Web Ontology Language (OWL) (2021). Available from: <https://www.w3.org/OWL/>

Wikipedia (2020) 'Trolley problem'. Available from: https://en.wikipedia.org/w/index.php?title=Trolley_problem&oldid=987726957

Wikipedia (2021) 'All-pairs testing'. Available from: https://en.wikipedia.org/wiki/All-pairs_testing

Wilson, K. M., Helton, W. S. and Wiggins, M. W. (2013) 'Cognitive engineering'. Wiley Interdisciplinary Reviews: Cognitive Science, 4 (1). 17–31.

Winfield, A. F. T., Booth, S., Dennis, L. A., Egawa, T., Hastie, H., Jacobs, N., Muttram, R., Olszewska, J. I., Rajabiyazdi, F., Theodorou, A., Underwood, M., Wortham, R. H. and Watson, E. (2021) 'IEEE P7001: A proposed standard on transparency'. Frontiers on Robotics and AI, 8. 1–16.

Wisniewski, D., Potoniec, J., Lawrynowicz, A. and Keet, C. M. (2019) 'Analysis of ontology competency questions and their formalizations in SPARQL-OWL'. Journal of Web Semantics, 59. 1–19.

Wos, L., Overbeck, R., Lusk, E. and Boyle, J. (2021) 'Automated reasoning: Introduction and applications'. U.S. Department of Energy – Office of Scientific and Technical Information. Available from: <https://www.osti.gov/biblio/6003867>

Wright, J. (2016a) 'STARWest - Think you can just “Test” that API? Think again'. YouTube. Available from: <https://youtu.be/Xu-rXUJ4IOQ>

Wright, J. (2016b) 'The Digital Manifesto'. Available from: <https://leanpub.com/digital>

Wright, J. (2017) 'Cognitive Learning – “Digital Evolution, Over Revolution”'. TEDxWilmingtonSalon. Available from: https://www.ted.com/talks/jonathon_wright_cognitive_learning_digital_evolution_over_revolution

Wu, D. and Hakansson, A. (2014) 'A method of identifying ontology domain'. Procedia Computer Science, 35. 504–513.

Yan, H., Jing, Z., Li-Qun, Y., Ze-Min, L. and Li-Jian, T. (2014) 'Based on ontology methodology to model and evaluate System of Systems (SoS)'. In Proceedings of the IEEE International Conference on System of Systems Engineering (SOSE). 101–106.

Yu, L., Zhang, L., Xiang, H., Su, Y., Zhao, W. and Zhu, J. (2009) 'A framework of testing as a service'. In Proceedings of the IEEE International Conference on Management and Service Science. 1–4.

원문 PDF 183 페이지

Zimmermann, T., Nagappan, N., Gall, H., Giger, E. and Murphy, B. (2009) 'Cross-project defect prediction: A large scale experiment on data vs. domain vs. process'. In Proceedings of the ACM Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering. 91–100. DOI: 10.1145/1595696.1595713

Zuckerberg, M. (2021) 'Introducing Meta: A social technology company'. Available from: <https://about.fb.com/news/2021/10/facebook-company-is-now-meta>

원문 PDF 184 페이지

색인

이탤릭체 페이지 번호는 그림과 표를 가리킨다. 페이지 번호는 원서 기준이다.

A/B 테스트 51

인수 테스트 41, 44, 85

적응성 17

적대적 공격 18, 51

적대적 테스트 51-2

애자일 선언문 103

AI(인공지능)

알고리즘 64

응용 74

‘서비스형’ 41, 42

동작 변경 2, 3, 4

편향 2, 4, 14, 15, 19

정의 8

개발 11

윤리 4, 5, 10, 19

협소 인공지능 17

품질 3, 8, 9, 10-22, 55

품질 문제 10-15

규제 20, 21, 22

연구 17

위험 19

강 인공지능 17

하위기호적 10

기호적 10, 21

기술 81, 82

원문 PDF 185 페이지

테스트 자동화 55-70

테스팅 1, 2, 22, 52-3, 67

신뢰할 수 있는 8-9, 13

AIOps 84, 89, 99, 100

알고리즘 8, 18, 20, 33, 63, 65, 67, 96

이상 탐지 65

애플리케이션 성능 관리(APM) 89

애플리케이션 프로그래밍 인터페이스(API) 41, 59, 65, 83

- 아르키메데스 상수 63
- 범용 인공지능(강 인공지능) 17
- 인공지능 기반 시스템 테스트 온톨로지(AI-T) 79, 80, 82
- 인공 신경망(ANN) 11
- 자동 추론 71, 73, 74, 76, 82
- 자동화 10-11, 40, 51, 53, 55-70, 75, 76, 82, 84, 100
- 자동화 편향 15, 44
- 자율 시스템 53, 81, 82, 83
- 공리 72, 74, 77, 78, 79, 81, 82
- 백투백 테스트 52
- 편향 2, 4, 6, 9, 14, 16, 17, 19, 21, 24-37, 43
 - 광고 29-30
 - 알고리즘 편향 53
 - 자동화 15, 44
 - 인지 편향 14, 16, 42
 - 확증 편향 34, 38
 - 데이터 편향 2, 14, 16-17, 31-2
 - 데이터 처리 33
 - 정의 24, 25-7
 - 탐지 44
 - 편향의 방향 26
 - 일상생활 27-8
 - 실험자 편향 42
 - 성별 편향 24, 25, 27, 28, 30, 36, 37
 - 역사적 편향 14, 32
 - 인간 편향 14, 33-4, 42
 - 부적절한 편향 53
 - 내재적 편향 28-9
 - 레이블링 편향 33
 - 측정 25
 - 자연적 편향 37
- 원문 PDF 186 페이지
 - 무응답 편향 43
 - 표본 편향 53
 - 샘플링 편향 31
 - 심슨의 역설 34-6, 34
 - 통계적 편향 14
 - 의도하지 않은 편향 31-4
- 블랙박스 테스트 40, 78
- Bonferroni-Dunn 보정 27
- 불리언 논리 47, 48, 49, 60
- 경계값 분석 4, 6, 50

‘버그 가정’ 4
버그 트리아지 65
사례 연구 90-100
카오스 엔지니어링 87, 88
챗봇 17
분류 16, 31, 45, 54
클러스터링 20, 65, 82
코드 생성 55, 60-1
코드 가독성 60, 61
인지
 편향 14, 16, 42
 엔지니어링 75, 84, 86, 87, 88, 89, 96, 100
 학습 89, 99, 103, 104
 사고 88
조합 테스트 4, 6, 47
완전성 3, 15, 16, 17, 18
데이터 복잡성 12
컴포넌트 테스트 39, 41, 42, 47
개념 드리프트 14, 44
혼동 행렬 46, 46
일관성 오라클 57, 58, 62
맥락 커버리지 17, 18, 20
제어 가능성 20
전통적 프로그래밍 8
코어 온톨로지 75
COVID-19 3, 14, 21, 29, 44, 90-6, 99, 100
데이터
 분석 28, 43, 43
 편향 2, 14, 16-17, 31-2
원문 PDF 187 페이지
 복잡성 12
 데이터셋 10, 12, 16, 17, 31, 41, 42, 43, 44, 51, 82, 89, 96
 데이터 주도 12
 흐름 6, 39, 42
 생성 55, 63, 65
 과거 데이터 66, 95, 99
 입력 11, 12, 18, 22, 42, 43, 44, 47, 50, 53, 63
 위치 31, 92, 96, 103
 메타데이터 72, 99, 100
 관찰 가능 데이터 11, 14, 17
 출력 12
 부분 데이터 43

개인정보 20
파이프라인 33, 39, 42
개인정보 보호 4, 92
처리 33
표현 73
정형 데이터 42, 43, 100
테스트 데이터 5, 12, 22, 47, 55, 63, 65, 96
데이터 테스트 16, 42, 43, 44, 50, 53
학습 데이터 11, 12, 19, 22, 38, 39, 42, 43, 44, 48, 51, 69
미관측 데이터 11, 16, 17, 44
비정형 데이터 84, 100
의사결정 10, 14, 15, 21
의사결정표 4
딥러닝 12, 82, 86, 88, 96, 103
결함 6, 38, 40, 47, 50, 53, 57, 64, 66, 67, 84
 결함 분석 66
기술 논리(DL) 73
차등 가격 책정 28-9
디지털 경험(DX) 85, 96, 103, 104
디지털 트윈 82, 104
 개념 89-90
 테스팅 84-104
도메인 온톨로지 75
노력 추정 66
기업 온톨로지(EO) 72
동등 분할 4, 6, 47
오류 유형 16
원문 PDF 188 페이지
윤리적 동작 3, 4, 14
EU AI 규정(제안) 21
EU AI 고위급 전문가 그룹 9
경험 기반 테스트 50
설명가능성 13, 18-19, 53, 54, 81
탐색적 데이터 분석(EDA) 43, 43
공정성 24
거짓 음성 16, 45, 67, 68
거짓 양성 16, 33, 45, 58, 64, 67, 68, 90, 92
결함 추정 66, 67
피츠의 법칙 63
유연성 17
기능 동작 5, 52, 53
기능 적합성 15-17

기능적 투명성 18-19
일반개인정보보호규정(GDPR) 20-2
그래픽 사용자 인터페이스(GUI) 63, 76
그라운드 트루스 12, 40, 51, 53
하드웨어 추상화 계층(HAL) 89
‘하이젠베르크 버그’ 62
역사적 편향 14, 32
인간 능력 13-14
인간 편향 14, 33-4, 42
하이퍼파라미터 38, 48
식별자 선택 64
암묵적 견고성 오라클 59, 62
인더스트리 4.0 75, 82
입력 데이터 테스트 42, 43, 44, 50, 53
입력 1, 3, 4, 5, 12, 13, 14, 17, 18, 40, 44, 46-8, 50, 53, 54, 96
통합 테스트 77, 85
지능형 에이전트(IA) 71, 82, 83
의도 56, 59, 61, 63, 70, 74
상호의존성 62
내부 상태 60, 61
국제표준화기구(ISO) 패러다임 24
국제소프트웨어테스팅자격위원회(ISTQB) 22
사물인터넷 6, 83
상호운용성 12, 74, 75, 81, 82, 99
원문 PDF 189 페이지
해석가능성 53, 54
ISO/IEC 25010 15, 17, 20
지식 표현(KR) 72, 74, 82
LIME(Local Interpretable Model-agnostic Explanations) 분석 54
논리 기반 형식 추론 81, 82
머신러닝(ML) 1, 10, 12, 82, 86, 88, 92, 96, 99, 100, 104
 알고리즘 11, 33, 39
 컴포넌트 41, 42
 정의 11
 프레임워크 49, 50
 모델 10, 11, 12, 38, 40, 41, 42, 43, 44, 45, 47, 49
 예측 42, 43
 테스팅 38-54
평균 절대 오차 16
평균 제곱 오차(MSE) 45, 46
AI 품질 측정 15-20
메타모픽 테스트 50-1

- 메타버스 84-104
- 메트릭 2, 22, 38, 44-6, 67, 68, 88
- 몽키 테스트 63-4
- 자연어 18, 54
- 자연어 처리(NLP) 13, 52, 96, 100
- 신경망 11, 12, 18, 19, 22, 47, 48, 49, 63, 64, 82, 96
- 뉴런 11, 49, 50
- 뉴런 커버리지 22, 47-50
- NFT(대체 불가능 토큰) 85, 99, 100
- 비결정론적 시스템 53
- 귀무가설 25, 26
- 객체 식별 64
- 온톨로지 표준 개발 수명주기(ROSADev) 82
- 온톨로지 71-83
- OntoTest 77
- 오라클
 - 일관성 57, 58, 62
 - 암시적 50
 - 문제 5, 40, 51, 52, 55, 56, 57, 61, 69
- 원문 PDF 190 페이지
 - 의사 오라클 41, 52
 - 견고성 59, 62
 - 테스트 오라클 5, 10, 41, 47, 58, 62, 63, 69
- 출력 14, 16, 17, 18, 38, 39, 40, 44, 46, 51, 53, 54
- 페어와이즈 테스트 47
- 성능 5, 16, 17, 18, 21, 24, 25, 44, 45, 52, 68, 76, 82, 87, 89, 100
- 성능 엔지니어링 87
- Pokémon Go 100, 101, 103
- 예측 실패 43
- 우선순위화 61, 62, 64
- 확률적 동작 5
- 확률적 시스템 40, 41
- Protégé 73, 74
- 품질 9, 10-22, 24-37
 - 보증 10, 55-6, 57, 65-6, 67, 69, 70, 76, 85, 90
 - 관리 15, 21, 22
 - 문제 10-15
 - 규제 20-2
- 추론 54, 63, 71, 73, 74, 76, 81, 82
- 소프트웨어 테스트 참조 온톨로지(ROoST) 78, 79
- 회귀 16, 45, 52, 53, 56, 65, 76, 77
- 회귀 테스트 52, 53, 56, 76, 77

- AI 품질 규제 20
- 강화학습 13, 17
- 위험 2, 3, 17, 21, 22, 28, 31, 41, 68
 - 위험 기반 접근 방식 46
 - 완화 19-20
 - 예측 66, 67
- 로보틱 프로세스 자동화(RPA) 64
- 견고성 18, 59, 62, 63
- ROI(투자수익률) 5, 68
- 실행 투명성 18, 19
- 자가 최적화 13
- 자가 최적화 시스템 52-3
- SHAP(Shapley Additive exPlanations) 분석 54
- 시프트 레프트 6, 84, 92, 103
- 시프트 라이트 84, 85, 86, 87, 92, 96, 103, 104
- 원문 PDF 191 페이지
- 심슨의 역설 34-6, 34
- 사이트 신뢰성 테스트(SRT) 87, 89
- Smart City Data Exchange 96-9
- 스마트 시스템 75
- 소셜 미디어 30
- 소프트웨어
 - 인지 엔지니어링 86, 87
 - 전통적 15, 41, 42, 44, 47, 50, 87
 - 개발 55, 56, 73, 76, 86, 89
 - 개발 주기 84, 87
 - 엔지니어링 6, 45, 47, 71, 76, 77
 - 테스팅 1, 6, 71-83, 87
 - 테스팅 온톨로지 76-9
- Software Testing Ontology for Web Service(STOWS) 76
- 상태 다이어그램 4
- ‘상태 공간 폭발 문제’ 60
- 확률적 온톨로지 73
- 구조적 투명성 18, 19
- 하위호적 인공지능 시스템 10
- 기호적 인공지능 시스템 10
- 시스템 복잡성 12-13
- 시스템의 시스템 6, 75
- 시스템 테스트 10, 38-54, 77, 79, 83
- 표적 광고 29-30
- 테스트
 - 자동화 5, 55-70, 76

케이스 생성 68-9, 77
케이스 47, 50, 51, 61, 63, 67, 68, 69, 77, 78, 81
코드 생성 60-1
커버리지 3, 47, 49, 77
데이터 5, 12, 22, 47, 55, 63, 65, 96
데이터 생성 55, 65
설계 4, 5, 6, 47, 52
생성 55, 59, 60, 61, 66, 76
수준 39, 41, 42, 43, 44, 58
메트릭 2
오라클 5, 10, 40, 41, 47, 51, 52, 58, 62, 63, 69
피라미드 57, 57, 58
선택 64, 65

원문 PDF 192 페이지

UI 58, 62, 64
단위 55, 57-61, 62

테스팅

A/B 51
적대적 51-2
AI 특화 52-4
자동화 47, 82
백투백 52
블랙박스 40, 78
과제 2-6, 90
규정 준수 4
컴포넌트 39, 41, 42, 47
디지털 트윈 84-104
경험 기반 50
퍼즈 63
입력 데이터 42, 43, 44, 50, 53
수준 41-4, 79
Linux 78
유지보수 56
메타모픽 50-1
메트릭 44-6
ML 시스템 38-54
모델 42-3, 43, 44, 53
모델 기반(MBT) 101, 103
몽키 63-4
목표 3, 22
페어와이즈 47
성능 44

문제 40, 41
회귀 52, 53, 56, 76, 77
테스터의 역할 38-40
시프트 라이트 84
사이트 신뢰성 테스트(SRT) 87, 89
소프트웨어 1, 6, 71-83, 87
시스템 10, 38-54, 77, 79, 83
기법 3, 40, 46-52, 76, 77, 78, 79, 86
도구 지원 66-9
단위 57-61, 77
사용자 인수 44, 85
화이트박스 40, 51, 55, 78

원문 PDF 193 페이지

Testing-as-a-Service(TaaS) 83
Toronto virtual enterprise(TOVE) 72
학습 데이터 11, 12, 19, 22, 38, 39, 42, 43, 44, 48, 51, 69
투명성 13, 18-19, 53, 82
참 음성 45
참 양성 45
신뢰성 8-9
통합 모델링 언어(UML) 79, 81
의도하지 않은 편향 31-4
알 수 없는 정답 11-12
상위 온톨로지 75, 78, 82
사용자 인수 테스트 44, 85
사용자 경험(UX) 58, 63, 86, 96, 100
확인(validation) 42, 43, 59
검증(verification) 41, 77
시각적 테스트 자동화 65
Web Ontology Language(OWL) 72, 73, 83
화이트박스 테스트 40, 51, 55, 78
XPath 64

원문 PDF 194 페이지